

Formal Modeling and Automatic Analysis of
Complex Systems
Draft

Stefan Ratschan

May 8, 2026

Contents

I	Logical Basis	9
1	Basics of Practical Logic	11
1.1	How to Form Logical Expressions (Syntax)	11
1.2	Abbreviations, Syntactic Sugar etc.	12
1.3	Further Notions	13
1.4	Meaning of Logical Formulas (Semantics)	14
1.5	Logic in Practice	15
1.5.1	Basic Equivalences	15
1.5.2	Equivalences with Negation	15
1.5.3	Equivalences with Quantifiers	16
1.5.4	Equivalence rules	16
1.5.5	Implication and Equivalence Signs	16
1.6	Proofs	17
1.7	Extending the System of Rules	17
1.8	Equality	17
1.9	Usage of Predicate Logic	17
1.10	A Complete Proof Calculus	18
1.10.1	Basic Proof System	18
1.10.2	Equalities and Equivalences	22
1.10.3	Lemmata	23
1.10.4	Example 1	24
1.10.5	Example 2	24
1.10.6	Substitutions: Avoiding Name Clashes	25
1.10.7	Unsuccessful Proofs, Counter-Examples	25
1.10.8	Completeness	26
1.10.9	Alternative Methods, Literature	26
2	Logical Theories	27
2.1	Introductory Example	27
2.2	Modeling of Data Structures	29
2.2.1	Axiomatization of Lists	29
2.2.2	Axiomatization of Arrays	30
2.2.3	Axiomatization of Parametric Data Types	31
2.2.4	Axiomatization of Natural Numbers	31
2.2.5	Set Theory	32
2.2.6	Decision Procedures	34
2.2.7	Limitations	34
2.2.8	Discussion	35

2.3	Example Proofs	35
2.3.1	Proof in the Theory of Lists	35
2.3.2	Proof in the Theory of Arrays	36
2.3.3	Proof in the Theory of Arrays	37
2.3.4	Proof in the Theory of Arrays	37
2.3.5	Proof in the Joined Theory of Arrays and Integers	38
II	Systems Theory	41
3	System Models	43
3.1	Example 1	43
3.2	Example 2	44
4	Discrete Time System Models	47
4.1	System Communication	47
4.2	Black Box Description of Reactive Systems	47
4.3	Systems Properties	50
4.4	White Box Description of Reactive Systems	52
4.4.1	Properties of Automata	55
4.5	Variables: Extended State Machines	55
4.6	Hierarchical Automata	57
4.7	Conclusion	58
5	Interacting System Components	59
5.1	Cascade Composition	59
5.2	General Composition—Synchronous Reactive Models	62
5.3	Simulation	66
5.4	Tee and Delay as Automata	67
5.5	General Composition of Automata	69
5.6	Special Cases	72
5.7	Deconstructing Automata	73
5.8	Conclusion	74
6	Temporal Logic	77
6.1	Introduction	77
6.2	Basic Automata Model	78
6.3	Specifying Temporal Properties	79
6.4	Properties of LTL	84
6.5	Counter-Examples	85
6.6	Application of Temporal Logic	86
7	Unbounded Model Checking	89
7.1	The Model Checking Problem	89
7.2	Model Checking G ok	90
7.2.1	Certificates	90
7.2.2	Explicit State Model Checking	93
7.2.3	Symbolic Model Checking	95
7.2.4	Backward Computation	96
7.3	Model Checking F goal	97

7.3.1	Certificates	97
7.3.2	Explicit State Model Checking	99
7.3.3	Relationship to the Literature	100
7.4	Arbitrary LTL formulas	101
7.5	Infinite Number of States	101
7.6	Relationship to Mathematical Induction	102
7.7	Conclusion	102
8	Testing, Bounded Model Checking	103
8.1	Motivation	103
8.2	Bounded Semantics of LTL	104
8.2.1	Bounded Semantics of G	104
8.2.2	Bounded Semantics of F	104
8.2.3	Bounded Semantics of LTL	105
8.2.4	Example 1	106
8.2.5	Example 2	106
8.3	Testing Transition Systems	107
8.4	Bounded Model Checking	108
8.4.1	Example 1	108
8.4.2	Special Case	109
8.4.3	Example 2	109
8.4.4	BMC(ϕ, n) and System Correctness	109
8.5	Temporal Logic for Planning	110
8.6	Bounded vs. Unbounded Model Checking	112
8.7	Efficient Checking of BMC(ϕ, n)	112
9	Boolean Satisfiability (SAT)	115
9.1	Boolean Encoding of BMC	116
9.2	Problem Statement and Applications	117
9.3	SAT Solving	119
9.4	Conflict Driven Clause Learning (CDCL)	123
9.4.1	Basic Idea	123
9.4.2	Details	124
9.5	Implementation Techniques	125
9.6	Local Search	126
9.7	SAT Modulo Theory	126
9.8	Improvements for Bounded Model Checking	126
9.9	Checking the Safety Verification Conditions	127
9.10	General Usage of SAT Algorithms	128
9.11	Example applications	128
10	Timed Automata	129
10.1	State and Evolution of Timed Automata	131
10.2	Simulation	133
10.3	Specification	134
10.4	Testing	134
10.5	Symbolic Simulation	135
10.5.1	Computation of Symbolic Delay Transitions	136
10.5.2	Computation of Symbolic Action Transitions	138
10.6	Unbounded Model Checking	139

10.7	Conclusions	140
11	Petri Nets	141
11.1	Motivation	141
11.2	Examples	141
11.3	Formal Definitions	143
11.4	Properties of Petri Nets	144
11.4.1	Reachability	144
11.4.2	Boundedness	144
11.4.3	Liveness	145
11.5	Automatic Analysis of Petri Net Properties	145
11.5.1	Petri Net Transformations	149
11.6	Translation to Transition System	149
11.7	Translation from (Finite) Transition System	153
11.8	Invariant Computation	153
11.8.1	T -Invariants	153
11.8.2	P -Invariants	154
11.8.3	Summary	155
11.8.4	Example of T -Invariant Computation	155
11.8.5	Example of P -Invariant computation	156
11.8.6	Solution of Equations	157
11.9	Conclusion	157
12	Probabilistic Models	159
12.1	Specification of Properties of Probabilistic Transition Systems . .	161
12.2	Model Checking	161
12.2.1	Naive Model Checking of $\mathbf{F}p$	161
12.2.2	Behavior from a Certain State	162
12.2.3	Model Checking of $\mathbf{F}p$	164
12.2.4	Model Checking of Temporal Logic Formulas: Plan	164
12.2.5	LTL Model Checking: Formulas without Temporal Oper- ators Properties	165
12.2.6	LTL Model Checking: $\mathbf{X}\phi$	166
12.2.7	LTL Model Checking: Negation	166
12.2.8	LTL Model Checking: $\mathbf{F}\phi$	167
12.2.9	Combination	167
12.2.10	Overall Probability	168
12.3	Nondeterminism versus Probability	169
12.4	Markov Decision Processes	169
12.5	Costs and Rewards	169
12.6	Further Possibilities	170
12.7	Conclusion	170
13	Modeling of Physical Environment	171
13.1	Introduction	171
13.2	Continuous Modeling of Time	171
13.3	Continuous Component Models	172
13.4	System Properties	174
13.5	Modeling Real-World Phenomena	174
13.6	Relationship to Timed Automata	178

13.7 Non-Determinism	178
13.8 Input and Output	179
13.9 Choice of Model	181
13.10 Conclusion	182
III Programs: Systems with Data-Structures	185
14 Program Specification	187
14.1 Input/Output Specifications	187
14.2 Specification Languages	188
14.3 Examples	189
14.3.1 Example 1: String Matching	189
14.3.2 Example 2: String Search	190
14.3.3 Example 3	190
14.4 Shortcuts	191
15 Correctness of Programs Without Control Structures	193
15.1 Program Correctness and Assertions	193
15.2 Program Correctness: Assignments	194
15.3 Program Correctness: Array Assignments	197
15.4 Program Correctness: User Input	197
15.5 Correctness of Programs without Control Structures	198
15.6 Programs with Several Assertions @	201
15.7 Proving Verification Conditions	202
15.8 Verification Conditions: Alternative Form	202
15.9 Summary	204
16 Correctness of Programs with Control Structures	205
16.1 if-then-else	205
16.2 Loops	208
16.3 General Program Linearization	211
16.4 Verification Conditions vs. Assertion Failures	212
16.5 Loop Invariants: Example	213
16.6 Loop Invariant Positions	215
16.7 Automation	217
16.8 Conclusion	217
16.9 Further Examples	218
16.9.1 Example 1	218
16.9.2 Example 2	220
16.9.3 Example 3	222
17 Functions, Procedures	225
17.1 Correctness of Functions	225
17.2 What Is a Function?	228
17.3 Recursive Function Calls	229
17.4 Correctness of Procedures	231
17.5 Example: Recursive Binary Search	234
17.6 Sorting Algorithm: Basic Definitions	236
17.7 Conclusion	237

18 Data Structures and OO Programming	239
18.1 Object-Oriented Programming and Logic	239
18.2 Specification of Abstract Data Types	241
18.3 Implementation	242
18.4 Logical Terminology	245
18.5 Theory Extensions	246
18.6 Definition of Abstract Data Types by Extension	248
18.7 Methods for Specification of Data Types	252
18.8 Conclusion	252
19 Program Termination	253
19.1 The Problem of Program Termination	253
19.2 Termination Proofs	254
19.3 Connections to Different Areas	258
19.4 Termination of Recursion	259
19.5 Automation	260
19.6 A Useful Well-Founded Relation	261
19.7 Conclusion	262
20 Symbolic Execution	263
21 Bounded Software Model Checking	269
21.1 Consequences	273
21.2 Application: Equivalence Checking	273
21.3 Further Application: Combination with Testing	274
21.4 Literature and Tools	274
22 Decision Procedures	275
22.1 Prelude: Validity vs. Satisfiability	275
22.2 Abstraction	276
22.3 Program Abstraction	277
22.4 Decision Procedures	278
22.4.1 Solvers for Propositional Logic	278
22.4.2 Equality and Function Calls	278
22.4.3 General Formulas: Disjunctions, Quantifiers	281
22.4.4 Application	281
22.4.5 Arrays	282
22.4.6 Witnesses and Their Usage	283
22.4.7 Real Numbers	284
22.5 Summary and Conclusion	284
23 Automated Formal Software Verification	285
23.1 Weakest Preconditions	285
23.2 Strongest Postconditions	288
23.3 Pre- or Postcondition Based Software Verification	290
23.4 Computing Pre- and Postconditions for Programs with Loops	290
23.5 Conclusion	296

Part I

Logical Basis

Chapter 1

Basics of Practical Logic

1.1 How to Form Logical Expressions (Syntax)

A *term* is built from

- variables, and
- function symbols of a certain arity (i.e., number of arguments).

Examples of terms are: $2x + 3$, `first(rest(l))`, $a(i)$

Functions symbols of arity 0 are called *constants*.

We usually assign some meaning to function symbols. For example:

- $+$ usually means addition
- `rest` usually means the rest of a list etc.

Moreover we often assign to function symbols certain types, for example:

- $+: \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$
- `num_elem : List  $\rightarrow \mathbb{N}_0$ .`

There is a variant of logic (so-called "many-sorted logic"), that describes the handling of types precisely. However, as computer scientist, we are already have a very good intuitive understanding of types, and hence we will not go into more details.

An *atomic formula* is one of the following:

- the Boolean constant $\top$ (called "true", "tautology") or $\perp$ (called "false", "contradiction")
- a predicate symbol of a certain arity, with terms as arguments, especially the predicate symbol $=$ that has arity 2.

Note that there are many alternative ways of writing Boolean constants, for example: **T** and **F**, or 1 and 0.

An example of an atomic formula is $2x + 3 > 0$.

A *formula* is built from

- atomic formulas

- quantifiers: $\forall, \exists$
- logical connectives: $\vee, \wedge, \neg$

Examples of logical formulas are $\exists x. 2x + 3 > 0 \wedge x^2 \leq 0$ and $\text{empty}(a(i))$.

Here one has to be careful about various syntactical conventions. For example:

- The connective $\neg$ usually has higher precedence than $\vee$ and $\wedge$.
- Without further information, the formula $\exists x. P(x) \wedge Q(x)$ can either be read as $\exists x. [P(x) \wedge Q(x)]$ or as $[\exists x. P(x)] \wedge Q(x)$. In this text, we always choose the first possibility, that is, quantifiers bind less than logical connectives.

There are many alternative notations for quantifiers. For example, many people instead of $\forall x. P(x) \vee Q(x)$, write

- $(\forall x)(P(x) \vee Q(x))$, or
- $P(x) \vee Q(x) \quad \forall x$

We usually assign some meaning to predicate symbols, for example, $<$ usually means "less than".

Moreover, we often assign some types to predicate symbols, for example:

- $<$ is a predicate on $\mathbb{R} \times \mathbb{R}$,
- `is_empty_list` is a predicate on lists

When writing formulas, we have to take care that

- all function and predicate symbols always have the correct number of arguments, and that
- the types of those arguments match.

In the case of programming languages this usually is done by the compiler (with the exception of JavaScript, LISP, Prolog, ...) In other cases we have to be careful (from your average newspaper: "Obama is the better president").

Throughout the text we assume that we have an infinite set of predicate and function symbols available.

1.2 Abbreviations, Syntactic Sugar etc.

- The comma $,$ often denotes conjunction, that is, it stands for the symbol $\wedge$
- A block of quantifiers $\exists x, y, z$ means $\exists x \exists y \exists z$
- $\bigwedge_{i \in \{1, \dots, n\}} \phi_i$ means $\phi_1 \wedge \dots \wedge \phi_n$

Often logical symbols are used in prose: "For all x such that ..."

Mathematical texts usually contain many further forms of syntactic sugar, variants, dialects etc. (e.g., $7!$, $f' = xyz$).

Moreover, in mathematical texts, certain operators are assigned a higher operator precedence than others. For example, the term $xy + z$ is usually read as $(xy) + z$.

The syntactical structure of logical formulas can be made explicit using a *syntax tree*. For example, Figure 1.1 shows a syntax tree for the formula $\exists x. 2x + 3 > 0 \wedge x^2 \leq 0$. Note that here we have several possibilities of how to parse the formula. For example, we interpreted the term x^2 as the application of the square function to the variable x . It would also be possible to interpret this as the application of the 2-ary power function to the first argument x , and the second argument 2. A third possibility would be to interpret it as the multiplication of the variable x with itself.

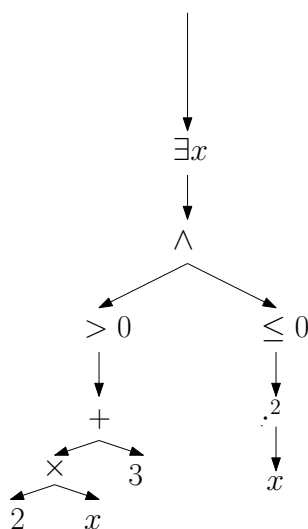


Figure 1.1: Syntax Tree

1.3 Further Notions

A further important notion is the notion of a *free* and of a *bound* variable. There are several ways of defining this. For example, translating J. Velebil¹ (emphasis is mine) we have:

The occurrence of variable x in a formula is called *bound* if on the path from this leaf to the root of the corresponding syntax tree there is a node marked with the quantifier $\forall x$ or $\exists x$. In the opposite case we call this occurrence *free*. A quantifier $\forall x$ or $\exists x$ *binds* all occurrences of the variable x , that occur in the syntactic tree below this quantifier.

Moreover, we will write $A[x \leftarrow t]$ for the result of substituting every *free* occurrence of the variable x with the term t in expression A . For example, the

¹“Velmi jemný úvod do matematické logiky” (ČVUT FEL)

substitution

$$[x < 0 \wedge \forall x . x^2 + 1 \geq 0][x \leftarrow u + 1]$$

results in the formula

$$u + 1 < 0 \wedge \forall x . x^2 + 1 \geq 0.$$

Here, one has to be careful to avoid name clashes. Hence, most authors explicitly forbid substitutions such as $[\exists x . p(y)][y \leftarrow x]$. Moreover, we will assume that substitution binds stronger than logical symbols. So the formula $A(x) \wedge B(x)[x \leftarrow f(x)]$ should be read as $A(x) \wedge [B(x)[x \leftarrow f(x)]]$, and not as $[A(x) \wedge B(x)][x \leftarrow f(x)]$, and the result of the substitution is $A(x) \wedge B(f(x))$.

Note that substitution is *not* the same thing as renaming of variables! For example, renaming x to y in $\forall x . f(x)$ results in $\forall y . f(y)$. But $[\forall x . f(x)][x \leftarrow y]$ is $\forall x . f(x)$, since the variable x does not occur freely in the formula $\forall x . f(x)$.

Based on substitution, we can introduce one more abbreviation: $\exists^1$ is a shortcut for the formula $\exists x . \phi \wedge \neg \exists y . \neg[x = y] \wedge \phi[x \leftarrow y]$. In other words, it means “there is precisely one”. Often this is also written using an exclamation mark: $\exists!$.

As an example, take the formula $\exists^1 x . \text{president}(x)$ which stands for

$$\exists x . \text{president}(x) \wedge \neg \exists y . \neg[x = y] \wedge \text{president}(y)$$

and expresses that there is precisely one president.

1.4 Meaning of Logical Formulas (Semantics)

An *interpretation* gives a certain meaning to every predicate and function symbol. Here, arity and types must match. A (*variable*) *assignment* gives a certain value to every variable. Then, for a certain interpretation $\mathcal{I}$, variable assignment σ , and term t we can define the corresponding value of the term. For example, the term $1 + 1$ has the value 2 under the usual interpretation in $\mathbb{Z}$, but the value 0 under the interpretation in $\mathbb{Z}_2$.

Based on that, for a certain interpretation $\mathcal{I}$, variable assignment σ , formula ϕ , the corresponding meaning is denoted by:

$$\mathcal{I}, \sigma \models \phi$$

For example, denoting the usual interpretation of symbols in the real numbers by $\mathbb{R}$ and in the complex numbers by $\mathbb{C}$, we have $\mathbb{C} \models \exists x . x^2 = -1$, but not $\mathbb{R} \models \exists x . x^2 = -1$. Here, we did not provide any variable assignment since the formula $\exists x . x^2 = -1$ does not contain any free variables. For any formula that does contain free variables, we do not a variable assignment. For example, $\mathbb{R}, \{y \mapsto 1\} \models \exists x . x^2 = y$ and $\mathbb{R}, \{y \mapsto -1\} \not\models \exists x . x^2 = y$.

All of this can be precisely defined. For details see any textbook on formal logic.

The notation $\models \phi$ means that the formula ϕ is *valid* (holds independently of a certain interpretation/variable assignment, that is for all interpretations/variable assignments). For example, any formula of the form $P(x) \vee \neg P(x)$, with $P(x)$ standing for any formula with a free variable x , is valid. And especially, the tautology $\top$ is valid. In a similar way one calls a formula ϕ *satisfiable* iff there

is an interpretation $\mathcal{I}$ and variable assignment σ such that $\mathcal{I}, \sigma \models \phi$. For example, any formula of the form $P(x) \wedge \neg P(x)$, with $P(x)$ standing for any formula with a free variable x , is not satisfiable, that is, unsatisfiable. And especially, the formula $\perp$ is unsatisfiable. The formula $P \wedge Q$ (now P and Q are 0-ary predicate symbols) is satisfiable, but not valid.

1.5 Logic in Practice

In practice it is usually sufficient to know semantics intuitively. But it is important to know rules for handling logical formulas.

For two formulas that have the same meaning, we will say that one formula *is equivalent* to the other one. If two formulas are equivalent, we can replace each of them by the other one. We have (A, B, C, P are placeholders for arbitrary formulas):

1.5.1 Basic Equivalences

- $A \wedge \top$ is equivalent to A
- $A \wedge \perp$ is equivalent to $\perp$
- $A \vee \top$ is equivalent to $\top$
- $A \vee \perp$ is equivalent to A
- $A \wedge B$ is equivalent to $B \wedge A$
- $A \vee B$ is equivalent to $B \vee A$
- $A \wedge [B \vee C]$ is equivalent to $[A \wedge B] \vee [A \wedge C]$
- $A \vee [B \wedge C]$ is equivalent to $[A \vee B] \wedge [A \vee C]$

1.5.2 Equivalences with Negation

- $\neg \top$ is equivalent to $\perp$
- $\top$ is equivalent to $\neg \perp$
- $\neg \neg A$ is equivalent to A
- $A \wedge \neg A$ is equivalent to $\perp$
- $A \vee \neg A$ is equivalent to $\top$
- $\neg[A \wedge B]$ is equivalent to $\neg A \vee \neg B$
- $\neg[A \vee B]$ is equivalent to $\neg A \wedge \neg B$

1.5.3 Equivalences with Quantifiers

- $\exists x \exists y . A$ is equivalent to $\exists y \exists x . A$
- $\forall x \forall y . A$ is equivalent to $\forall y \forall x . A$
- $\neg \forall x . A$ is equivalent to $\exists x . \neg A$
- $\neg \exists x . A$ is equivalent to $\forall x . \neg A$
- $\forall x . A \wedge B$ is equivalent to $[\forall x . A] \wedge [\forall x . B]$
- $\exists x . A \vee B$ is equivalent to $[\exists x . A] \vee [\exists x . B]$
- $\forall x . A \vee B$ is equivalent to $[\forall x . A] \vee B$, if B does not contain x
- $\exists x . A \wedge B$ is equivalent to $[\exists x . A] \wedge B$, if B does not contain x
- $\exists x . P$ is equivalent to P if x does not occur freely in P
- $\forall x . P$ is equivalent to P if x does not occur freely in P
- A is equivalent to B , if A and B are the same up to (renaming of bound variables without name clashes) α -conversion)

1.5.4 Equivalence rules

- A is equivalent to A (*reflexivity*)
- If A is equivalent to B , then B is equivalent to A (*symmetry*)
- If A is equivalent to B and B is equivalent to C , then A is equivalent to C (*transitivity*)

1.5.5 Implication and Equivalence Signs

- $A \Rightarrow B$ can be viewed as a short-cut for $\neg A \vee B$
- $A \Leftrightarrow B$ can be viewed as a short-cut for $A \Rightarrow B \wedge B \Rightarrow A$

Attention: Various different conventions regarding operator precedence!

The above definition of implication and equivalence has the following consequences:

- $A \Rightarrow \top$ is equivalent to $\top$
- $\perp \Rightarrow B$ is equivalent to $\top$
- $[A \wedge B] \Rightarrow A$ is equivalent to $\top$
- $A \Rightarrow [A \vee B]$ is equivalent to $\top$
- $A \Rightarrow B$ is equivalent to $\neg B \Rightarrow \neg A$
- $A \Leftrightarrow B$ is equivalent to $\neg B \Leftrightarrow \neg A$

Further (*case distinction*):

- A is equivalent to $[B \Rightarrow A] \wedge [\neg B \Rightarrow A]$
- A is equivalent to $[B_1 \Rightarrow A] \wedge \dots \wedge [B_n \Rightarrow A]$,
if $B_1 \vee \dots \vee B_n$ is equivalent to $\top$

1.6 Proofs

A proof is a certificate that a certain formula ϕ is valid ($\models \phi$). One simple way of proving that $A \Leftrightarrow B$ is to prove that A is equivalent to B .

For example, to prove $[A \vee B] \Leftrightarrow A \vee [B \wedge \neg A]$ it suffices to find a sequence of equivalent formulas such as the following:

$$\begin{aligned} & A \vee B \\ & [A \vee B] \wedge [A \vee \neg A] \\ & A \vee [B \wedge \neg A] \end{aligned}$$

Due to the transivity of equivalence, this means that the first element of the sequence is equivalent to the last one, which proves the needed equivalence.

Moreover, we can prove a formula A by proving that A is equivalent to $\top$. But, in general, this is not enough—there are formulas that cannot be proved that way. See Section 1.10 for a more general way of proving formulas.

1.7 Extending the System of Rules

As soon as you have a proof of an equivalence, you can add it as a further “private” rule. It is important to keep in mind that this is only your own rule, that other people do not know. So whenever you face the question “why does this hold?” you should be able to provide the original proof of the equivalence.

1.8 Equality

Usually we have available the special predicate symbol $=$, for which we know (x, y, z can stand for any term):

- $x = x$ (*reflexivity*)
- If $x = y$, then $y = x$ (*symmetry*)
- If $x = y$ and $y = z$, then $x = z$ (*transitivity*)

If we know that a certain equality holds, we can always replace the left-hand side by the right-hand side and vice versa.

A widely used notation is $x \neq y$ which means $\neg[x = y]$.

1.9 Usage of Predicate Logic

Predicate logic is usually hidden in prose. For example, consider Theorem 2.3 from a classical textbook on automata theory [25]:

“Let r be a regular expression. Then there exists an NFA with ϵ -transitions that accepts $L(r)$ ”

This sentence corresponds to the following predicate logical formula

$$\forall r . \text{regexp}(r) \Rightarrow \exists a . \text{NFAeps}(a) \wedge \text{accepts}(a, L(r))$$

Here, the predicates *regexp*, *NFAeps*, *accepts* and the function symbol *L* correspond to notions have been defined in the book before this theorem.

As soon as this theorem is proved, one can use it. Especially, if one has a certain regular expression *r*, for example *01**, then one can conclude from the theorem that there exists a NFA with ϵ -transitions that accepts $L(01^*)$.

In general, if we know that a formula with a leading universal quantifier holds, we can substitute concrete values for the corresponding variables. This is a special case of the proof calculus that we will introduce in the next section.

1.10 A Complete Proof Calculus

We already know that one can prove $A \Leftrightarrow B$ by proving that *A* is equivalent to *B*. But, unfortunately, this is not enough, since this method is not complete, that is, this method cannot prove every true formula. In this section we will describe a method for proving formulas that is indeed complete (cf. Section 1.10.8).

1.10.1 Basic Proof System

For proving something, we keep a list of facts that we already know, and

- add further facts (using phrases such as “we assume that”, “we conclude that”), and
- simplify the formula that we want to prove, until

either

- what we want to prove is known, or
- we have a contradiction, that is, a situation when a formula and its negation are both known.

The following proof rules depend on the outer-most symbol of the given formula, that is, to the root of its syntactic tree. Here, the symbols *A* and *B* are so-called meta-variables, that is, they represent arbitrary formulas. We will first list the proof rules, and then explain their usage on examples.

Proof rules for simplifying the formula that we want to prove, according to the outer-most symbol of this formula:

outermost symbol	proof rule
$A \wedge B$	Separately prove <i>A</i> and <i>B</i>
$A \vee B$	Assume $\neg A$ and prove <i>B</i> (or vice versa)
$A \Rightarrow B$	Assume <i>A</i> and prove <i>B</i> .
$\neg A$	Assume <i>A</i> and try to find a contradiction
$\exists x. A$	Choose (by intuition, creativity, or any other special connection with god) a term <i>t</i> , and prove $A[x \leftarrow t]$
$\forall x. A$	Choose a new constant, e.g., <i>a</i> , write/say “let <i>a</i> be arbitrary but fixed” and prove $A[x \leftarrow a]$, instead

Proof rules for adding further facts from given facts, according to the out-

ermost symbol of a given fact:

outermost symbol	proof rule
$A \wedge B$	Conclude both A and B
$A \vee B$	Do a case distinction: first write "Case A :", assume A and finish the proof, then write "Case B :", assume B , and finish the proof.
$A \Rightarrow B$	If we also know A then we conclude B .
$\exists x . A$	Choose a new constant a , write/say "let a be such that $A[x \leftarrow a]$ ", and add $A[x \leftarrow a]$ to our knowledge base
$\forall x . A$	Choose (by intuition, creativity, or any other special connection with god) a term t and conclude $A[x \leftarrow t]$

Note that the rule for proving conjunctions and using a disjunctive fact start two *separate* sub-proofs. We have to finish *both* sub-proofs to finish the overall proof. Especially, a contradiction occurring in a sub-proof only finishes these sub-proofs, but not the overall proof.

You might be surprised to see proof rules for both disjunction and implication. Indeed, since we introduced implication as a shortcut for a disjunction, no separate proof rule would be necessary. However, the redundant rules for implications allows us to write proofs with implications in a more direct form than first expanding the implication to a disjunction and then applying the proof rules for disjunction. Note also, that the table does not have proof rules for the equivalence symbols $\Leftrightarrow$, which we indeed handle as a short-cut for two implications, as described above.

Our first example will only use the proof rules for implication and conjunction. We want to prove

$$[p \Rightarrow q] \Rightarrow [[p \wedge r] \Rightarrow [q \wedge r]]$$

Proof. The outer-most symbol is an implication whose left-hand side is the sub-formula $p \Rightarrow q$ and right-hand side is $[p \wedge r] \Rightarrow [q \wedge r]$. Applying the rule for proving an implication we assume $p \Rightarrow q$ and prove $[p \wedge r] \Rightarrow [q \wedge r]$. For proving the formula $[p \wedge r] \Rightarrow [q \wedge r]$ we again apply the rule for implication, assume $p \wedge r$ and prove $q \wedge r$. Now the outer-most symbol of the formula to prove is a conjunction. Applying the corresponding rule, for proving $q \wedge r$ we prove both q and r :

- Proof of q : We apply the rule for conjunction in the known facts to the assumption $p \wedge r$. According to the rule we have the new assumptions p and r . From the assumptions p and $p \Rightarrow q$, using the rule for implication, we get the assumption q which finishes this part of the proof.
- Proof of r : We already discovered that from the assumption $p \wedge r$ we know r . This finishes also this part of the proof.

■

Remember that a contradiction, that is, a situation when a formula and its negation are both known, also finishes a proof. To understand, how such a proof

by contradiction works, consider the example of the formula

$$[p \wedge \neg p] \Rightarrow q.$$

The left-hand side of the implication in the formula is a contradiction. Hence the formula holds. The following proof is based precisely on this argument:

Proof. We assume $p \wedge \neg p$ and prove q . From the assumption $p \wedge \neg p$ we know both p and $\neg p$. This is a contradiction, which successfully finishes the proof. ■

Here, it is important to realize that a contradiction finishes the proof of the *original* formula. So in the example, we just proved that the original formula $[p \wedge \neg p] \Rightarrow q$ holds.

As a further example, consider the formula $p \Rightarrow [\neg p \Rightarrow q]$, and observe that if p holds, then the left-hand side of the implication in the sub-formula $\neg p \Rightarrow q$ is false, and hence the formula holds. If p does not hold, then the left-hand side of the outer implication is false which again makes the formula false. Hence the formula holds independent of how q looks like. A proof using our method against proves such a formula by contradiction: It first adds p then $\neg p$ as an assumption which creates a contradiction and proves the formula without even touching the sub-formula q .

But also remember that the rule for proving conjunctions and using a disjunctive fact start two *separate* sub-proofs. A contradiction occurring in a sub-proof only finishes these sub-proofs, but not the overall proof.

Our method also allows us to prove negations by contradiction. The reader may remember the famous proof that the square root of 2 is not a rational number by assuming that it is rational and deriving a contradiction. Here is an example of such an indirect proof:

$$[[p \Rightarrow q] \wedge [p \Rightarrow \neg q]] \Rightarrow \neg p$$

Proof. We assume $[p \Rightarrow q] \wedge [p \Rightarrow \neg q]$ and try to prove $\neg p$. For proving the negated formula $\neg p$ we assume p and try to find a contradiction.

From the assumption $[p \Rightarrow q] \wedge [p \Rightarrow \neg q]$ we can infer both the assumption $p \Rightarrow q$ and $p \Rightarrow \neg q$. From the assumptions p and $p \Rightarrow q$ we infer q , and from the assumption p and $p \Rightarrow \neg q$ we infer $\neg q$. Hence we have both formulas q and $\neg q$ are assumptions. This is a contradiction which successfully finishes the proof. ■

Note that there is no rule for generating new known facts from a formula whose outermost symbol is the negation $\neg$. We will later see 1.10.3 how to use a known formula whose outermost symbol is a negation.

Let us now have a look at an example with an existential quantifier:

$$\exists x \in \mathbb{N} . x \geq 4 \wedge 2 \nmid x$$

For proving that something exists that fulfills a certain property, we just have to find one object that fulfills this property. For example, here such a number: 7. If we substitute this number into the sub-formula $x \geq 4 \wedge 2 \nmid x$, we get $7 \geq 4 \wedge 2 \nmid 7$ which clearly holds. But of, course, we could also have chosen different number, for example the number 5.

An example with a universal quantifier is the following:

$$\forall x \in \mathbb{Z}. \quad \begin{aligned} f(x) \geq 0 &\Rightarrow x - f(x) - 1 < x - |f(x)| \wedge \\ \neg f(x) \geq 0 &\Rightarrow x + f(x) - 1 < x - |f(x)| \end{aligned}$$

Proof. We want to prove the formula for an arbitrary integer, that is, an integer, about which we do not know anything. So we introduce a new constant a , and prove

$$\begin{aligned} f(a) \geq 0 &\Rightarrow a - f(a) - 1 < a - |f(a)| \wedge \\ \neg f(a) \geq 0 &\Rightarrow a + f(a) - 1 < a - |f(a)| \end{aligned}$$

To prove the conjunction we prove both $f(a) \geq 0 \Rightarrow a - f(a) - 1 < a - |f(a)|$ and $\neg f(a) \geq 0 \Rightarrow a + f(a) - 1 < a - |f(a)|$. To prove $f(a) \geq 0 \Rightarrow a - f(a) - 1 < a - |f(a)|$ we assume $f(a) \geq 0$ and prove $a - f(a) - 1 < a - |f(a)|$ which clearly holds under this assumption. The proof of $\neg f(a) \geq 0 \Rightarrow a + f(a) - 1 < a - |f(a)|$ is similar. ■

The following example, in addition, uses the proof rule for universal quantifiers in assumptions:

$$[\forall x. P(x)] \Rightarrow [\forall x. P(f(x))]$$

Proof. We assume $\forall x. P(x)$ and prove $\forall x. P(f(x))$. To prove the latter, we assume an arbitrary, but fixed a , and prove $P(f(a))$. Due to the assumption, $P(x)$ holds for every x , especially, it holds for $f(a)$, hence we can assume $P(f(a))$, which is, what we wanted to prove. ■

Note the order in which we used the proof rules. If we would have used them in the opposite order, we would have get stuck: If our first step would have been to assume $P(f(a))$, then a is not a new constant any more. Hence we would *not* be allowed to reduce the proof of $\forall x. P(f(x))$ to the proof of $P(f(a))$. We would still be allowed to assume an arbitrary, but fixed, let us say b , proving $P(f(b))$, and continuing as above, but this does not use the first step of assuming $P(f(a))$ at all.

Usually, it makes sense to first apply the proof rules that substitute new constants, and only later the rules that require the construction of a term to substitute: In the first case, the choice of the concrete name of the constant is not important, the only important thing is, that the constant is *new*. In the second case, the choice of the used term is very important, and if we postpone this choice as much as possible, we already have enough information on which choice to use.

Here is another example with quantifiers:

$$[(\forall x. P(x) \Rightarrow Q(x)) \wedge (\exists x. P(f(x)))] \Rightarrow [\exists x. Q(x)]$$

Proof. We assume $[\forall x. P(x) \Rightarrow Q(x)] \wedge [\exists x. P(f(x))]$, and so we assume both $\forall x. P(x) \Rightarrow Q(x)$ and $\exists x. P(f(x))$. We want to prove $\exists x. Q(x)$. Due to the assumption $\exists x. P(f(x))$ we can choose a new constant a such that $P(f(a))$. Due to the assumption $\forall x. P(x) \Rightarrow Q(x)$ we can conclude $P(f(a)) \Rightarrow Q(f(a))$. For proving $\exists x. Q(x)$ we choose for x the term $f(a)$ and now it suffices to prove $Q(f(a))$ which follows from the assumptions $P(f(a))$ and $P(f(a)) \Rightarrow Q(f(a))$. This finishes the proof. ■

1.10.2 Equalities and Equivalences

For handling equality, we have the two following rules:

- For every term t we can assume $t = t$.
- If we know $t_1 = t_2$, then we can always replace t_1 by t_2 and vice versa.

Here, the replacement must not change bound variables or introduce new bound variables. For example, if we know $\exists x . P(x, y)$ and $y = f(x)$, then we are not allowed to replace y by $f(x)$ in $\exists x . P(x, y)$ since the result $\exists x . P(x, f(x))$ has a new bound variable.

Moreover, throughout proofs, we may always replace equivalent things by equivalent ones according to the rules of Section 1.5. Strictly necessary is only the equivalence between ϕ and $\neg\neg\phi$, all other equivalences can be derived using our proof method (ignoring the Boolean constants $\perp$, $\top$).

The equivalence rule for double negation can especially be useful when applied as follows: When proving A , we can

1. replace A by $\neg\neg A$,
2. apply the rule for proving negations, and,
3. add $\neg A$ as additional knowledge.

The following example combines the techniques we have seen so far: Assume that we want to prove the correctness of the following, trivial program

Input: x
return $6x$

wrt. the specification:

- Input: $x \in \mathbb{N}$, x is divisible by 2
- Output: $x \in \mathbb{N}$, x is divisible by 4

We can do this, by proving

$$\forall x \in \mathbb{N} . x \text{ is divisible by } 2 \Rightarrow 6x \text{ is divisible by } 4$$

Here we need to prove

$$x \text{ is divisible by } 2 \Rightarrow 6x \text{ is divisible by } 4$$

for an arbitrary natural number x . The word “arbitrary” requires that for this number we do not know anything whatsoever. For ensuring this, we introduce a new constant, say a and prove

$$a \text{ is divisible by } 2 \Rightarrow 6a \text{ is divisible by } 4$$

and so, due to the prove rule for implication we assume a is divisible by 2 and prove $6a$ is divisible by 4.

The definition of divisibility is: $\forall x, y . x \text{ is divisible by } y :\Leftrightarrow$

$$\exists k \in \mathbb{N} . yk = x.$$

The rule for universal quantifiers allows us to apply this definition to the cases that we have here, arriving at

- a is divisible by 2 $\Leftrightarrow \exists k \in \mathbb{N} . 2k = a$
- $6a$ is divisible by 4 $\Leftrightarrow \exists k \in \mathbb{N} . 4k = 6a$

Now we can replace the formulas in the proof by equivalent ones. So, from the assumption

$$\exists k \in \mathbb{N} . 2k = a$$

we will prove

$$\exists k \in \mathbb{N} . 4k = 6a.$$

The proof rule for existential quantification allows us eliminate the existential quantifier in the assumption, giving it a new name, for example k' . Then we know the new formula

$$2k' = a.$$

Now we observe that the formula that we have to prove has a similar form. To bring it even closer to this form, we multiply the equality by 6, obtaining

$$12k' = 6a$$

which is equivalent to

$$4 \cdot 3 k' = 6a$$

Now it is clear that for proving the above formula, we can choose for the existentially quantified variable k the term $3k'$. After substituting this into the formula under the quantifier we obtain

$$4 \cdot 3 k' = 6a$$

which we already know. Hence this finishes the proof.

1.10.3 Lemmata

In our proof method we always have exactly one formula that we are trying to prove, but we can have several known formulas in our set of assumptions. Now the question arises what to do if we would like to prove some additional, intermediate formula. In this case, we can prove a so-called lemma: Prove the desired formula separately, using the known assumptions. As soon as this additional proof is finished, we can add the proved formula as an additional assumption. We can even have recursive lemmas, that is, a lemma can contain further lemmas.

As an example, we prove

$$\left[p \wedge [p \vee q] \Rightarrow r \right] \Rightarrow r$$

Proof. We assume $p, [p \vee q] \Rightarrow r$ and prove r . In this situation it would be convenient to know $p \vee q$. We prove this formula using a lemma:

Proof of the lemma $p \vee q$ from our known facts: For proving $p \vee q$ we assume $\neg q$ and prove p . The formula p is already a known fact and hence the proof of the lemma is finished.

So we add the formula $p \vee q$ to our known facts. Now we can deduce r from $[p \vee q] \Rightarrow r$ and $p \vee q$. This finishes the main proof. ■

Here are example situations that motivate the usage of lemmas:

- If we know $\neg A$ (A is a place-holder for an arbitrary formula) we can try to separately prove A , calling this proof a lemma. If we succeed in proving this lemma, we can add A as an additional known fact which results in a contradiction and hence successfully finishes the main proof.
- If we know $A \Rightarrow B$, then we can use a lemma to prove A . If the prove of this lemma succeeds, we can add A to our set of known formulas, and then—using the proof rule for implication in known facts—we can infer from A and $A \Rightarrow B$ the new knowledge B .
- The formula $A \vee \neg A$ (case distinction) can always be proven without any assumptions. Hence we can always add it to our known facts.

But be careful: A lemma is a separate proof! Except for the formula that we prove in the lemma, this proof may not influence the main proof in any way.

1.10.4 Example 1

We will now combine the techniques from above in a slightly bigger example, a proof of the formula

$$[\neg \forall x . P(f(g(x)))] \Rightarrow [\exists x . \neg P(x)].$$

For proving this formula we assume $\neg \forall x . P(f(g(x)))$ and prove $\exists x . \neg P(x)$. For proving an existential quantifier, we would need a term to substitute, but we do not know anything useful. So we add a double negation, resulting in $\neg \neg \exists x . \neg P(x)$, and apply the rule for proving negations which adds $\neg \exists x . \neg P(x)$ as additional knowledge.

Now we have two formulas with negations as knowledge. If we would know $\forall x . P(f(g(x)))$, we would arrive at a contradiction. Hence we prove this formula in a lemma.

Lemma: Proof of $\forall x . P(f(g(x)))$: We apply the proof rule for universal quantifiers, and prove $P(f(g(a)))$ for a new, arbitrary, but fixed constant a . Now we do not have any proof rule left to apply. So we again add a double negation, and then add $\neg P(f(g(a)))$ as additional knowledge.

At this point, we are still proving the lemma. We know $\neg \exists x . \neg P(x)$ and $\neg P(f(g(a)))$. The latter formula contains the term $f(g(a))$ that we may use to prove an existential quantifier. Hence, we prove $\exists x . \neg P(x)$ using a another lemma within the current lemma.

The proof of this second lemma immediately succeeds by choosing $f(g(a))$ for x . Hence we can add $\exists x . \neg P(x)$ to the knowledge in the original lemma. This creates a contradiction which proves this original lemma. As a consequence, we can add $\forall x . P(f(g(x)))$ to the main proof. This again creates a contradiction which finishes the proof.

1.10.5 Example 2

$$[[\forall x . P(x)] \Rightarrow S] \Rightarrow [\exists x . P(x) \Rightarrow S]$$

We assume $[\forall x . P(x)] \Rightarrow S$ and want to prove $\exists x . P(x) \Rightarrow S$. It is important to realize that the outermost symbol in the latter formula is the

quantifier and *not* the implication. The problem is that for proving an existential quantifier we need to come up with a term for which to prove the formula under the quantifier. However, at this point, we do not have any term that we could use here. In any case, how would such a term ideally look like? Well, the implication $P(x) \Rightarrow S$ can be made true by making the left-hand side false. Hence, in the case where there would be some element for which P is false, we would know how to do the proof. So let us look at this case, and try the other case later. In other words, we do a case distinction $[\exists x . \neg P(x)] \vee \neg[\exists x . \neg P(x)]$.

In the case $\exists x . \neg P(x)$ we have an existential quantifier in our assumptions, and hence we can give a name to such an x , let us say a . So we know $\neg P(a)$ now. To prove $\exists x . P(x) \Rightarrow S$ we choose a for x , so we are left to prove $P(a) \Rightarrow S$. For this we assume $P(a)$ which is in contradiction with $\neg P(a)$, and so this case is proven.

Now we are left with the case where we know $\neg \exists x . \neg P(x)$. Here, we will prove $\forall x . P(x)$ in a lemma. To prove this, let a be arbitrary, but fixed (we did use a before, but that was in a different branch of the proof, so a is new here). So we have to prove $P(a)$ which is equivalent to $\neg \neg P(a)$. Using the proof rules for negation, we assume $\neg P(a)$. Based on this assumption, we can prove the lemma $\exists x . P(x)$ (i.e., a sub-lemma of the current one). Based on this, we can add this formula to our assumptions, which creates a contradiction to $\neg \exists x . \neg P(x)$ which finishes the proof of the second case.

This proof looks unnecessarily complicated: Everybody knows that $\neg \exists x . \neg P(x)$ is equivalent to $\forall x . P(x)$. However here we used two nested lemmas to prove it! That is why, in practice, everybody uses equivalence rules in such situations. However, equivalence rules are not complete, and hence, in general, they do not suffice as a proof technique.

1.10.6 Substitutions: Avoiding Name Clashes

When using proof rules we must not use substitutions that result in new bound variables. For example, if we want to prove

$$\exists x \forall y . p(x, z)$$

we cannot choose $f(y)$ for the variable x , because the result of the substitution $p(x, y)[x \leftarrow f(z)]$ is $p(f(y), z)$, which—in the context of the quantifier $\forall y$ —would result in a new bound variable.

In a similar way, if we know both $\exists x . P(x, y)$ and $y = f(x)$, then we are not allowed to substitute $f(x)$ for y in $\exists x . P(x, y)$ since the result $\exists x . P(x, f(x))$ would have a new bound variable.

One can easily get around such problems by renaming the bound variable. For example, after renaming the bound variable x in $\exists x . P(x, y)$ to z , which results in $\exists z . P(z, y)$, substituting $f(x)$ for y is not problem any more. We will always allow such a renaming and consider the resulting formula equivalent to the original one.

1.10.7 Unsuccessful Proofs, Counter-Examples

Now assume that we cannot prove a certain formula. Does this mean that the formula does not hold? Certainly not! Rather we are in one of the two following situations: The formula either

- holds and we were not able to prove it, or it
- does not hold.

For showing that a formula does not hold we may

- find a counter-example, or
- try to prove the negation of the formula.

For example, the formula

$$p \vee [\neg p \wedge q]$$

does not hold and hence is not provable. It has the counter-example $\{p \mapsto \perp, q \mapsto \perp\}$. The negation

$$\neg[p \vee [\neg p \wedge q]]$$

also does not hold and has the counter-example $\{p \mapsto \top, q \mapsto \perp\}$. The formula

$$p \wedge \neg p$$

not only has a counter-example, but we can even prove its negation. The formula

$$\forall x . P(x)$$

also does not hold, but in this case we need a more complicated counter-example. For example, we can interpret the formula over the integers, and $P(x)$ as “ x is even”. Note that also in the case of this formula, its negation $\neg \forall x . P(x)$ does not hold.

1.10.8 Completeness

Kurt Gödel (1906, Brno — 1978, Princeton) in his Ph.D. thesis (that had only 33 pages) proved that for every formula ϕ such that $\models \phi$, there exists a corresponding proof. He used a different proof system [22] but this completeness property is also fulfilled by our proof system (if we allow usage of the equivalence between A and $\neg\neg A$).

1.10.9 Alternative Methods, Literature

Our method is an informal version of a proof system called sequent calculus [19], which is usually introduced as a formal proof system with formally defined deduction rules. There many variants of this proof system. However, the most frequent variants of the sequent calculus usually do not work with one formula to be proved, but several ones.

Chapter 2

Logical Theories

2.1 Introductory Example

Consider an example specification where both the input p and the corresponding output p' is a pair of integers. Assume that the input has to fulfill a predicate $I(p)$ and that the corresponding outputs a predicate $O(p, p')$. Assume that those predicates are defined as follows:

- $\forall p . I(p) :\Leftrightarrow \text{snd}(p) = 1$
- $\forall p, p' . O(p, p') :\Leftrightarrow \text{snd}(p') = 2\text{fst}(p)$

Now consider the algorithm that for an input pair p contains just the following line:

return pair(snd(p), 2fst(p))

We first observe that the algorithm does not use the fact that the input p fulfills $\text{snd}(p) = 1$. This is no problem.

Now let us try to prove the correctness of the algorithm. Denoting the output for the input p by $f(p)$, we have to prove

$$\forall p \forall p' . [I(p) \wedge p' = f(p)] \Rightarrow O(p, p')$$

which, in our concrete case, is

$$\forall p \forall p' . [\text{snd}(p) = 1 \wedge p' = \text{pair}(\text{snd}(p), 2\text{fst}(p))] \Rightarrow \text{snd}(p') = 2\text{fst}(p).$$

In general, assume an I/O-specification given by a predicate $I(x)$ modeling that x is allowed input, and a predicate $O(x, x')$ modeling that x' is allowed output for input x ¹. Moreover, assume an algorithm that for inputs x consists of the line

return $f(x)$

where f is a term like x^2 , $a[2i]$, $\text{rest}(x)$. Then correctness of the algorithm follows from

$$\forall x \forall x' . [I(x) \wedge x' = f(x)] \Rightarrow O(x, x').$$

¹Here, both x and x' may stand for several variables.

We will now try to prove the correctness of the algorithm above. We want to prove

$$\forall p \forall p'. [\text{snd}(p) = 1 \wedge p' = \text{pair}(\text{snd}(p), 2\text{fst}(p))] \Rightarrow \text{snd}(p') = 2\text{fst}(p)$$

Let p, p' be arbitrary, but fixed pairs². We assume $\text{snd}(p) = 1$ and $p' = \text{pair}(\text{snd}(p), 2\text{fst}(p))$, and we prove $\text{snd}(p') = 2\text{fst}(p)$. Due to the assumption $p' = \text{pair}(\text{snd}(p), 2\text{fst}(p))$ we can substitute $\text{pair}(\text{snd}(p), 2\text{fst}(p))$ for p' , and hence it suffices to prove $\text{snd}(\text{pair}(\text{snd}(p), 2\text{fst}(p))) = 2\text{fst}(p)$.

Now we are stuck. It seems that the formula $\text{snd}(\text{pair}(\text{snd}(p), 2\text{fst}(p))) = 2\text{fst}(p)$ should hold obviously! Yes, indeed. So let us write down more general formulas that describe what we consider to hold obviously:

- $\forall x, y. \text{fst}(\text{pair}(x, y)) = x$
- $\forall x, y. \text{snd}(\text{pair}(x, y)) = y$
- $\forall p. \text{pair}(\text{fst}(p), \text{snd}(p)) = p$

Such formulas are called *axioms*, and we can add such axioms beforehand as known facts. When doing this for proving a certain formula ϕ , we say that we *prove that ϕ holds in the theory of pairs*, or that we *prove ϕ in the theory of pairs*. Intuitively this means that ϕ follows from the axioms. Formally, a theory is a set of axioms, and a formula holds in a theory iff it is a logical consequence of the theory.

Now we continue with the proof. We were stuck in the situation when we wanted to prove the formula:

$$\text{snd}(\text{pair}(\text{snd}(p), 2\text{fst}(p))) = 2\text{fst}(p).$$

From the axiom

$$\forall x, y. \text{snd}(\text{pair}(x, y)) = y,$$

using the prove rule for universal quantifiers in known facts, after choosing $x \leftarrow \text{snd}(p)$, $y \leftarrow 2\text{fst}(p)$ we know that

$$\text{snd}(\text{pair}(\text{snd}(p), 2\text{fst}(p))) = 2\text{fst}(p),$$

which is what we wanted to prove.

Now let us summarize the theory of pairs. It uses a type that we denote by $\mathcal{P}$. The elements are of an arbitrary type that we denote by $\mathcal{T}$. Sometimes, we will also write $\mathcal{P}[\mathcal{T}]$ for the type of pairs with elements from $\mathcal{T}$.

The function symbols of the theory of pairs are:

- $\text{pair}: \mathcal{T} \times \mathcal{T} \rightarrow \mathcal{P}[\mathcal{T}]$
- $\text{fst}: \mathcal{P}[\mathcal{T}] \rightarrow \mathcal{T}$
- $\text{snd}: \mathcal{P}[\mathcal{T}] \rightarrow \mathcal{T}$

²The original proof rules require us to introduce new variable names. However, since neither p nor p' have been used anywhere else in the proof so far, we can leave their names unchanged, and simply remove the quantifiers.

The axioms are $(x, y \in \mathcal{T}, p \in \mathcal{P}[\mathcal{T}])$:

- $\forall x, y. \text{fst}(\text{pair}(x, y)) = x$
- $\forall x, y. \text{snd}(\text{pair}(x, y)) = y$
- $\forall p. \text{pair}(\text{fst}(p), \text{snd}(p)) = p$

2.2 Modeling of Data Structures

Definition 1 A signature is a set of types and of function and predicate symbols over those types. A logical theory is a pair consisting of a signature and a set of axioms.

Based on this, we can now model data structures. The general approach is the following: We first agree on a description of the behavior of the data structure in the form of a certain logical theory. Then we can prove theorems in the corresponding theory, that is, we assume the axioms from the beginning as known facts. Further examples of such proofs can be found in Section 2.3.

The process of describing the behavior of formal objects using axioms is also called *axiomatization* which has been a fundamental mathematical technique throughout the history of mathematics. Classical examples of such axiomatizations in mathematics are algebras such as groups, rings, fields.

We will now describe how several further data structures can be axiomatized [8].

2.2.1 Axiomatization of Lists

We will denote the type of lists by $\mathcal{L}$, and assume that the elements be of an arbitrary type that we denote by $\mathcal{T}$. For the type of lists with elements from the type $\mathcal{T}$ we also use the notation $\mathcal{L}[\mathcal{T}]$. We first have to agree on which predicates and function symbols to use, and how many and which arguments they allow. The list of types, function, and predicate symbols of a theory is also called its *signature*.

In the case of lists we have the following function symbols:

- $\text{cons} : \mathcal{T} \times \mathcal{L}[\mathcal{T}] \rightarrow \mathcal{L}[\mathcal{T}]$
- $\text{first} : \mathcal{L}[\mathcal{T}] \rightarrow \mathcal{T}$
- $\text{rest} : \mathcal{L}[\mathcal{T}] \rightarrow \mathcal{L}[\mathcal{T}]$
- $\text{empty} : \rightarrow \mathcal{L}[\mathcal{T}]$

Very often, a notation such as $\langle x_1, \dots, x_n \rangle$ is used. This abbreviation is nothing else than a short way of writing the list

$$\text{cons}(x_1, \text{cons}(x_2, \dots, \text{cons}(x_n, \text{empty}()))).$$

We have the following axioms ($l \in \mathcal{L}[\mathcal{T}], x \in \mathcal{T}$):

- $\forall l, x. \text{first}(\text{cons}(x, l)) = x$

- $\forall l, x. \text{rest}(\text{cons}(x, l)) = l$
- $\forall l. \neg[l = \text{empty}()] \Rightarrow \text{cons}(\text{first}(l), \text{rest}(l)) = l$
- $\forall l, x. \neg[\text{cons}(x, l) = \text{empty}()]$

Here, the behavior of $\text{first}(\text{empty}())$, $\text{rest}(\text{empty}())$ is not specified. There are several ways of handling such a situation. For example, one could introduce a constant `error` such that $\text{first}(\text{empty}()) = \text{error}$ and so on. Here, we will ignore this issue.

See Section 2.3.1 for an example of a proof in the theory of lists.

2.2.2 Axiomatization of Arrays

If an array has a known fixed size n , it is often enough to use n variables, otherwise we need to use a specific array type, that we will denote by $\mathcal{A}$. Again, we assume that the elements have an arbitrary type, that we denote by $\mathcal{T}$. For the type of arrays with elements in $\mathcal{T}$ we also use the notation $\mathcal{A}[\mathcal{T}]$.

The function symbols of the theory of arrays are:

- $\cdot[\cdot] : \mathcal{A}[\mathcal{T}] \times \mathcal{N} \rightarrow \mathcal{T}$
- $\text{write} : \mathcal{A}[\mathcal{T}] \times \mathcal{N} \times \mathcal{T} \rightarrow \mathcal{A}[\mathcal{T}]$

The first of those function symbols uses the usual notation for reading from arrays (the two dots denote the two arguments). For example, for an array a , $a[7]$ is the result of reading the element at position 7 from the array a . The second function symbol writes into an array. However, in logic—in the same way as in functional programming—there is no notion of memory. And especially, there is no way of changing memory. Hence the write function takes an array, an index, and the new element to store at the position of the index, and returns the array that is the result of the write operation. Also note that our arrays have infinite length: We can read from and write to a position given by an arbitrary natural number index.

When working with arrays one often uses notation such as

7	5		6	8
---	---	--	---	---

 to denote an array

$$\text{write}(\text{write}(\text{write}(\text{write}(a, 0, 7), 1, 5), 3, 6), 4, 8).$$

Here, the order of write operations is irrelevant, and hence one can write the same arrays equivalently as $\text{write}(\text{write}(\text{write}(\text{write}(a, 1, 5), 3, 6), 0, 7), 4, 8)$ or using any other order of the write operations. Here a is an arbitrary array for which we did not specify any properties before. Such a construction makes the $\text{new}()$ operation used by many programming languages superfluous.

We have the following array axioms ($a, b \in \mathcal{A}[\mathcal{T}]$, $v \in \mathcal{T}$, $i, j \in \mathcal{N}$)

- $\forall a, v, i, j. i = j \Rightarrow \text{write}(a, i, v)[j] = v$
- $\forall a, v, i, j. \neg[i = j] \Rightarrow \text{write}(a, i, v)[j] = a[j]$
- $\forall a, b. [\forall i. a[i] = b[i]] \Leftrightarrow a = b$

The third axiom says that two arrays are the same iff they contain the same elements. Thus, unlike the behavior of arrays in many programming languages, if we write an element into an array that is different from the old element at the given place, then the result is a new, *different* array. We can use the third axiom to prove that two arrays are the same by substituting the two arrays for the universally quantified variables in the axiom. Then we can use the resulting equivalence to replace the equality on the right-hand side by the left-hand side.

The shown axiomatization does not model the fact that in programming languages, arrays have a certain finite length. This is not a problem, since array overflows can be checked separately by modeling the length of each array by an integer variable.

There are two possibilities for modeling multi-dimensional arrays:

- arrays of arrays
- arrays with index set $\mathcal{N} \times \mathcal{N}$ etc.

For the second case, observe that the array axioms do not assume anything about the index set except for the possibility to check equality. Hence the axioms above can be used for various index sets.

2.2.3 Axiomatization of Parametric Data Types

For example, we might have a list where we want to store integers, rational numbers, arrays etc. Here, parametrization can be dynamic (i.e., sub-typing) or static (i.e., templates). It suffices to axiomatize the required behavior of allowed types. For example, one could require that the elements of a certain list will belong to a type, that is a group. In this case, we can use the axioms of group theory:

- Associativity: $\forall x, y, z. x + (y + z) = (x + y) + z$
- 0 is neutral element: $\forall x. 0 + x = x \wedge x + 0 = x$
- Existence of inverse: $\forall x \exists v. x + v = 0 \wedge v + x = 0$

2.2.4 Axiomatization of Natural Numbers

We will denote the type of natural numbers by $\mathcal{N}$. The function Symbols are :

- $0 : \rightarrow \mathcal{N}$
- $1 : \rightarrow \mathcal{N}$
- $+: \mathcal{N} \times \mathcal{N} \rightarrow \mathcal{N}$
- $\cdot : \mathcal{N} \times \mathcal{N} \rightarrow \mathcal{N}$

The most common axiomatization of the natural numbers is so-called Peano arithmetic [35]:

- $\forall x. \neg[x + 1 = 0]$
- $\forall x, y. x + 1 = y + 1 \Rightarrow x = y$

- $\forall x . x + 0 = x$
- $\forall x, y . x + (y + 1) = (x + y) + 1$
- $\forall x . x0 = 0$
- $\forall x, y . x(y + 1) = xy + x$

Finally, in addition to those six axioms, we have for each formula F with precisely one free variables x , an induction axiom

$$\left[F[x \leftarrow 0] \wedge [\forall x . F \Rightarrow F[x \leftarrow x + 1]] \right] \Rightarrow \forall x . F$$

For example, for the formula $0x = 0$, this results in the following axiom

$$\left[00 = 0 \wedge [\forall x . 0x = 0 \Rightarrow 0(x + 1) = 0] \right] \Rightarrow \forall x . 0x = 0$$

The usual usage of this axiom is to prove the left-hand side using a lemma which implies the right-hand side.

Unlike the theories that we have seen so far, here the number of axioms is infinite. Unfortunately, there is no finite axiomatization

It is easy to extend the theory of natural numbers with various elements that are convenient for everyday usage. Of course, one wants to use the constants 2, 3, $\dots$. Formally, they are short-cuts for the expressions $1 + 1$, $1 + 1 + 1$, $\dots$.

Another question is how to introduce the inequality symbol $\leq$? It can be easily defined by

$$\forall x, y \in \mathcal{N} . x \leq y :\Leftrightarrow \exists k \in \mathcal{N} . x + k = y.$$

The integers can be introduced by viewing each integer variable as the difference of two natural numbers. Many further theories can be built from this, e.g., the theory of real numbers.

2.2.5 Set Theory

Basics

It is possible to write down axioms for set theory, for example the axioms of Zermelo-Fraenkel set theory (ZFC) (C: axiom of choice). However, this is too complicated for our purposes. Instead, we will provide a few rules for everyday usage.

We form sets as follows: $\{x \mid A\}$, where A is a logical formula.

For sets S and T we have the operations

- $S \cap T \doteq \{x \mid x \in S \wedge x \in T\}$
- $S \cup T \doteq \{x \mid x \in S \vee x \in T\}$
- $S \setminus T \doteq \{x \mid x \in S \wedge \neg[x \in T]\}$

The *empty set* is a set $\emptyset$ s.t. $\neg \exists x . x \in \emptyset$, and hence $\exists x . x \in \emptyset$ is equivalent to $\perp$.

We have the following predicates for sets (S and T are sets):

- $S \subseteq T$ is defined as $\forall x. x \in S \Rightarrow x \in T$,
- $S = T$ is defined as $\forall x. x \in S \Leftrightarrow x \in T$,
- $a \in \{x \mid A\}$ is defined as $A[x \leftarrow a]$.

Moreover we have the rule that for a set S , $\{x \mid x \in S\} = S$.
Two further operations on sets are:

- Cartesian product: $S \times T \doteq \{(x, y) \mid x \in S, y \in T\}$
- Power set: $\mathcal{P}(T) \doteq \{S \mid S \subseteq T\}$ (sometimes also 2^S)

Very often the following notation is used:

- $\{a\}$ for $\{x \mid x = a\}$
- $\{a_1, \dots, a_n\}$ for $\{x \mid x = a_1 \vee \dots \vee x = a_n\}$
- $\forall x \in S. A$ for $\forall x. x \in S \Rightarrow A$
- $\exists x \in S. A$ for $\exists x. x \in S \wedge A$
- $x \notin A$ for $\neg[x \in A]$
- $\cap, \cup, \dots$

Proofs in Set Theory

In some cases it is possible to prove properties in set theory using simple equivalence transformations (see Section 1.6). For example, one can prove $A \setminus (B \cap A) \subseteq A$ as follows:

$$\begin{aligned}
 & A \setminus (B \cap A) \subseteq A \\
 & \forall x. [x \in A \setminus (B \cap A)] \Rightarrow x \in A \\
 & \forall x. [x \in \{x \mid x \in A \wedge \neg x \in (B \cap A)\}] \Rightarrow x \in A \\
 & \forall x. [x \in A \wedge \neg x \in (B \cap A)] \Rightarrow x \in A \\
 & \quad \forall x. \top \\
 & \quad \top
 \end{aligned}$$

Here, each line results from the previous one by the application of one equivalence rules from Section 1.5.

The Role of Set Theory in Mathematics

First-order predicate logic + set theory suffices for building all of mathematics! For example, pairs and operations on them can be constructed in set theory as follows [27]:

- $\forall x, y. \text{pair}(x, y) := \{\{x\}, \{x, y\}\}$
- $\forall z, p. z = \text{fst}(p) :\Leftrightarrow \forall \alpha \in p. z \in \alpha$
- $\forall z, p. z = \text{snd}(p) :\Leftrightarrow \exists^1 \alpha \in p. z \in \alpha$

Further examples are:

- A relation on sets $S_1, \dots, S_n$ is a subset of the Cartesian product $S_1 \times \dots \times S_n$. For example, the relation on the sets $\mathbb{N}$ and $\mathbb{N}$ that formalizes the property “is a prime factor of” is the (infinite) set

$$\{(2, 2), (3, 3), (2, 4), (5, 5), (2, 6), (3, 6), \dots\}.$$

- A function from a set S to a set T is a relation F on S and T such that for every $s \in S$ there is exactly one $t \in T$ with $F(s, t)$. The function length (of a string) is the set

$$\{("ab", 2), ("cdx", 3), ("", 0), \dots\}$$

- The natural numbers can be constructed by defining zero as the empty set, and addition of one to a given number x as $\{x\}$.
- Arrays can be formalized as functions from the natural numbers.
- Lists: either the empty set, or a pair consisting of an element and the rest of the list

In practice, predicate logic + set theory is enough for building all of mathematics (i.e., everything that can be formalized). Due to this, one might ask the question why to use different theories at all? One reason for this are decision procedures.

2.2.6 Decision Procedures

Some logical theories are *decidable*, that is, there is an algorithm with

- Input: formula ϕ with symbols from $\mathcal{T}$
- Output: $\top$, if ϕ holds in the theory $\mathcal{T}$, $\perp$ otherwise.

In such cases we can automatically prove the correctness of an algorithm that simply returns the result of evaluating a term, such as the introductory example from Section 2.1. In practice, software often checks satisfiability instead of validity, and also returns an explanation for the result in the form of a counter-example.

But: Peano arithmetic and set theory are undecidable [12, 37]. Still, Presburger arithmetic is decidable.

Another issue is that programs usually do not use only integers, or only lists, arrays etc. but various data structures that interact. Here, combination procedures are useful. Those are algorithms that can in certain cases decide sentences from a combination of various theories

2.2.7 Limitations

In Section 1.10.7 we discussed the possibilities that we have when we cannot prove a certain formula. For showing that a formula does not hold we may

- find a counter-example, or
- try to prove the negation of the formula.

However, the second strategy does not always work. For example, the formulas $p \vee [\neg p \wedge q]$ and $\forall x . P(x)$ both have a counter-example. But also their negation has a counter-example, and hence cannot be proved:

formula	counter-example
$p \vee [\neg p \wedge q]$	$\{p \mapsto \perp, q \mapsto \perp\}$
$\neg[p \vee [\neg p \wedge q]]$	$\{p \mapsto \top, q \mapsto \perp\}$
$\forall x . P(x)$	$\{P \mapsto \perp\}$
$\neg\forall x . P(x)$	$\{P \mapsto \top\}$

For so-called *complete* logical theories this cannot occur. Roughly speaking, in a complete theory, for every sentence ϕ either the sentence ϕ itself or its negation $\neg\phi$ follows from the axioms, and hence the the second strategy mentioned above can always be successful. An example of such a theory is Presburger arithmetic, that is, Peano arithmetic without multiplication. For illustration, consider the formula $\exists x . x \leq 5 \wedge x \geq 10$. This formula does not follow from the axioms of Presburger arithmetic, and hence we can be sure that its negation $\neg\exists x . x \leq 5 \wedge x \geq 10$ does.

Here one should be careful not to confuse terminology: (in)completeness of a theory is not the same thing as (in)completeness of a logical calculus!

At the beginning of the 20th century, there was the expectation that all of mathematics can be built up using deductions from certain basic axioms. Kurt Gödel, whom we already mentioned in the previous section, dealt a huge blow to this by proving his famous first incompleteness theorem: *Every theory that is strong enough to be able to express natural number arithmetic is incomplete* [23].

This theorem shocked the mathematical world in those days. However, it later turned out, that in practice Gödel's first incompleteness theorem is no problem. All of mathematics, and later, theoretical computer science, can indeed be built on top of first-order predicate logic and set theory, and sentences ϕ for which neither ϕ nor $\neg\phi$ follows from the axioms have turned out to not be relevant in practice.

2.2.8 Discussion

Why do we need infinite types, such as integers, real numbers, or dynamical data structures at all? Machine integers are only a finite set, floating point numbers, too, and in general, computer only have finite memory!

Each individual computer only has a finite number of bits, that is, an algorithm has only finite behavior, we only would have to test *all* of it! In hardware verification this is sometimes done (see SAT), but in the case of software, the data structures used are usually too big, and so we usually do not have a chance with this. Due to this, in the case of software verification it is usually easier to work with infinite types than with finite types.

2.3 Example Proofs

2.3.1 Proof in the Theory of Lists

For proving

$$< 1 > \neq < 2 >$$

we first observe that this is a way of writing

$$\neg \text{cons}(1, \text{empty}()) = \text{cons}(2, \text{empty}()).$$

Using the prove rule for negation, we assume $\langle 1 \rangle = \langle 2 \rangle$ and try to find a contradiction.

Let us look at the first elements of both sides of this equality by using the axiom

$$\forall l, x. \text{first}(\text{cons}(x, l)) = x.$$

Using the choices $l \leftarrow \langle \rangle, x \leftarrow 1$ and $l \leftarrow \langle \rangle, x \leftarrow 2$ respectively, we get

$$\text{first}(\langle 1 \rangle) = 1 \text{ and } \text{first}(\langle 2 \rangle) = 2.$$

Since $\langle 1 \rangle = \langle 2 \rangle$, this means that also

$$\text{first}(\langle 1 \rangle) = 2,$$

and since $\text{first}(\langle 1 \rangle) = 1$, this means that

$$1 = 2,$$

a contradiction.

2.3.2 Proof in the Theory of Arrays

We will prove the formula

$$\forall x \exists a. a[0] = x$$

where a is an array.

Let x be arbitrary, but fixed. The outer-most symbol of the rest of the formula, $\exists a. a[0] = x$ is an existential quantifier. To prove it, we need to substitute a term for a that has the value x at position zero. Such a term can be easily constructed from another array, let us say b , as $\text{write}(b, 0, x)$. Substituting this for a , we get $\text{write}(b, 0, x)[0] = x$. To prove this, we can use the array axiom $\forall a, v, i, j. i = j \Rightarrow \text{write}(a, i, v)[j] = v$. The array axioms belong to our assumptions, and hence we are allowed to substitute arbitrary terms for the universal quantifier $\forall a, v, i, j$. In order to apply the axiom to our situation, we use the substitution $a \leftarrow \text{write}(b, 0, x), v \leftarrow x, i \leftarrow 0, j \leftarrow 0$. Here, b is another array that we do not know anything about. Unlike most programming language, where one first would need to allocate memory for fresh arrays, this is no problem in logic, where we can write into arrays without restrictions dictated by memory.

The result of the substitution is $0 = 0 \Rightarrow \text{write}(b, 0, x)[0] = x$. Since $0 = 0$ holds (actually, in the theory of integers), we also know $\text{write}(b, 0, x)[0] = x$ which is, what we needed to prove.

In practice, one will, of course, not use such a detailed proof. Especially, instead of substituting terms for the universal quantifiers of axioms, one will usually just say something like “ $\text{write}(b, 0, x)[0] = x$ holds due to the first array axiom”.

2.3.3 Proof in the Theory of Arrays

We prove

$$\forall a \in \mathcal{A}, s \in \mathcal{N}, t \in \mathcal{N}, x, y.$$

$$s \neq t \Rightarrow \text{write}(\text{write}(a, s, x), t, y) = \text{write}(\text{write}(a, t, y), s, x)$$

Let $\forall a \in \mathcal{A}, s \in \mathcal{N}, t \in \mathcal{N}, x, y$ be arbitrary, but fixed with $s \neq t$. We prove

$$\text{write}(\text{write}(a, s, x), t, y) = \text{write}(\text{write}(a, t, y), s, x).$$

This means that we have to prove the equality of two arrays, and so we use the array axiom

$$\forall a, b. [\forall i. a[i] = b[i]] \Leftrightarrow a = b$$

Substituting $\text{write}(\text{write}(a, s, x), t, y)$ for a and $\text{write}(\text{write}(a, t, y), s, x)$ for b , we get

$$\begin{aligned} \forall i. \text{write}(\text{write}(a, s, x), t, y)[i] &= \text{write}(\text{write}(a, t, y), s, x)[i] \Leftrightarrow \\ &\text{write}(\text{write}(a, s, x), t, y) = \text{write}(\text{write}(a, t, y), s, x) \end{aligned}$$

So, instead of proving the equality above, we prove

$$\forall i. \text{write}(\text{write}(a, s, x), t, y)[i] = \text{write}(\text{write}(a, t, y), s, x)[i]$$

which means for an arbitrary, but fixed i , to prove

$$\text{write}(\text{write}(a, s, x), t, y)[i] = \text{write}(\text{write}(a, t, y), s, x)[i]$$

Now we have three cases:

- $i \neq s, i \neq t$:
 $\text{write}(\text{write}(a, s, x), t, y)[i] = a[i]$
 $\text{write}(\text{write}(a, t, y), s, x)[i] = a[i]$
- $i = s, i \neq t$:
 $\text{write}(\text{write}(a, s, x), t, y)[i] = \text{write}(a, s, x)[i] = x$
 $\text{write}(\text{write}(a, t, y), s, x)[i] = x$
- $i \neq s, i = t$:
 $\text{write}(\text{write}(a, s, x), t, y)[i] = y$
 $\text{write}(\text{write}(a, t, y), s, x)[i] = \text{write}(a, t, y) = y$

In all cases both sides are the same which finishes the proof.

2.3.4 Proof in the Theory of Arrays

We prove

$$\forall a \in \mathcal{A}, i, j, k \in \mathcal{N}, e, f \in \mathcal{T}.$$

$$[\text{write}(\text{write}(a, i, e), j, f)[k] = g \wedge j \neq k \wedge i = j] \Rightarrow a[k] = g$$

in the theory of arrays.

We let $a \in \mathcal{A}, i, j, k \in \mathcal{N}, e, f \in \mathcal{T}$ be arbitrary but fixed, assume

$$\text{write}(\text{write}(a, i, e), j, f)[k] = g, j \neq k, i = j,$$

and try to prove

$$a[k] = g.$$

Seeing that in $\text{write}(\text{write}(a, i, e), j, f)[k] = g$ we read from array element k which is different from the index j of the previous write we realize that we can use the array axiom

$$\forall a, v, i, j . i \neq j \Rightarrow \text{write}(a, i, v)[j] = a[j]$$

Choosing $\text{write}(a, i, e)$ for the universally quantified variable a , f for v , j for i , and k for j , we get

$$j \neq k \Rightarrow \text{write}(\text{write}(a, i, e), j, f)[k] = \text{write}(a, i, e)[k],$$

and since $j \neq k$ we also know

$$\text{write}(\text{write}(a, i, e), j, f)[k] = \text{write}(a, i, e)[k].$$

Hence we can replace the left-hand side of this equality by the right-hand side in our known fact $\text{write}(\text{write}(a, i, e), j, f)[k] = g$, arriving at

$$\text{write}(a, i, e)[k] = g.$$

To prove this, from $j \neq k$ and $i = j$ we conclude that also $i \neq k$ (we can prove this by assuming $i = k$, and using transitivity to conclude $j = k$, which is a contradiction).

Now, since $i \neq k$ we can use the same array axiom once more to conclude that $\text{write}(a, i, e)[k] = a[k]$. So we can replace the left-hand side of $\text{write}(a, i, e)[k] = g$ by $a[k]$ arriving at

$$a[k] = g$$

which is what we wanted to prove.

2.3.5 Proof in the Joined Theory of Arrays and Integers

We prove

$$\exists a \in \mathcal{A} . a[1] + 0 \neq 0,$$

in the theory of arrays with integer elements, using the Peano axioms for the integers.

We choose for a the array $\text{write}(b, 1, 1 + 1 + 1)$, with $b \in \mathcal{A}$ and prove that

$$\text{write}(b, 1, 1 + 1 + 1)[1] + 0 \neq 0.$$

We see that here we read from the same array element as we write. Hence we decide to use the array axiom

$$\forall a, v, i, j . i = j \Rightarrow \text{write}(a, i, v)[j] = v,$$

arriving at the knowledge

$$\text{write}(b, 1, 1 + 1 + 1)[1] = 1 + 1 + 1.$$

Based on this we can replace `write(b, 1, 1 + 1 + 1)[1]` by `1 + 1 + 1` above, which leaves us to prove

$$(1 + 1 + 1) + 0 \neq 0.$$

Now we can use the Peano axiom

$$\forall x . x + 0 = x,$$

and hence we only have to prove $1 + 1 + 1 \neq 0$ which holds due to the Peano axiom $\forall x . x + 1 \neq 0$.

Part II

Systems Theory

Chapter 3

System Models

The following two sections contain examples of system models of various types. Most examples are informal, being based on natural language instead of the language of mathematics. In later chapters, we will then introduce more formal classes of models.

3.1 Example 1

Consider the task of designing a parcel delivery system based on quadcopters. A first static model just lists the necessary components:

- n quadcopters
- scheduler
- jobs (parcels to deliver)

Now assume that we would like to design a scheduler that assigns jobs to quadcopters. To avoid assigning a job to a quadcopter that is busy with another job, we equip the quadcopters with state. The simplest model has just two states: wait and work. This already adds a dynamic aspect to the model. A scheduler can now simply keep a queue of jobs, always assigning the oldest jobs in the queue to the next available (i.e., waiting) quadcopter.

As a next task, we would like to check whether a certain number of quadcopters will be sufficient for executing certain jobs within a certain time limit. For this we need to model the execution time of jobs. The basic information necessary for this, is the coordinates of the sender and the destination of the parcel. Assuming that the quadcopters execute each task by starting at the base station, flying to the sender address, delivering the parcel at its destination, and then returning to the base station, we can now model the time needed using the sum of the Euclidean distances of those three parts of a route. If we want to keep discrete time steps, we can even round the resulting time to an integer. It is an easy task to write a program that simulates the resulting model for a given sequence of jobs and to check whether delivery is finished by a certain time limit.

Now assume that we also want to send acknowledgement messages after a parcel was picked up, and after it was delivered. For this, we can extend the

quadcopter model to four states: wait, flying to sender, flying to destination, returning to base.

Now assume that instead of a global deadline, each job comes with its own deadline. Now it does not suffice for the scheduler to schedule on a first come, first serve basis. Instead, the schedule should now ensure that all deadlines are kept. If a job comes in that cannot be finished within its deadline, we should immediately raise an alarm. To ensure that we do not miss alarms, but also do not raise false alarms, we might need a more precise model of the needed delivery time.

We might also want to minimize energy usage. This is a so-called constrained optimal control problem: minimize energy usage of the quadcopters while keeping all deadlines. The more realistic the used model of the quadcopter is, the better are our possibilities for optimization. A precise model would include the speed of the four rotors, resulting in certain energy usage, and would use differential equations to model the resulting flight trajectory of the quadcopter.

3.2 Example 2

As an example system we take a concert of a jazz band. Consider the task for the sound engineer to decide, which microphones to prepare for the concert. For this task, a simple list of instruments is often enough, for example:

- Trumpet

- Saxophone

- Piano

- Bass

- Drums

This model is static since it does not model time. Usually, this level of abstraction is appropriate for the given task. But even for a sound engineer, there are cases where such a level of abstraction is too high: Better paid musicians often have special requirements, for example, they want a certain piano, certain microphones to be used etc. This often result in a more detailed model of the required sound system, consisting of a pages-long document.

Now consider the task for the piano player to play at the concert. For this task, a model such as the following is useful (which is called a *chord chart*):



This is a dynamic model: It describes how the chords of a certain song evolve in time. There is some room for interpretation, though: the same sequence of chords can be played fast or slowly, and there are various different ways of playing the same chord. The level of abstraction of this model may be useful for a jazz piano player who improvises based on the chords of a song. But a classical piano player usually needs more detailed notes, that is, a lower level of abstraction. But even notes for classical piano players do not describe all details. For example, the word "allegro" may represent any tempo between approximately 110 and 170 beats per minute.

Now consider another task: Designing a piano sound module to be played at the concert. Here, the following model, describing the motion of a piano string [11], might be useful

$$\frac{\partial^2 y}{\partial t^2} = c^2 \frac{\partial^2 y}{\partial x^2} - \epsilon c^2 L^2 \frac{\partial^4 y}{\partial x^4} - 2b_1 \frac{\partial y}{\partial t} + 2b_3 \frac{\partial^3 y}{\partial t^3} + f(x, x_0, t)$$

This is a partial differential equation and hence the model is dynamic (note the time variable t). While this model can be useful for designing a sound module, it certainly does not help the piano player (or any other musician) to achieve a good performance.

We can arrange such models in a hierarchy where the upper levels hide details, and the lower levels add more and more details. An example of such a hierarchy is the following:

- list of instruments
- chord chart
- detailed score
- mathematical model of instrument sounds

Chapter 4

Discrete Time System Models

In Chapter 3, we introduced several examples of system models. Those models were to a large extent informal, described in the English language. This is imprecise, and can hence lead to misunderstandings. Moreover, we want to simulate and analyze system models on computers, which makes it necessary to describe them in a more precise form. For this, we will now discuss how to describe system models formally.

4.1 System Communication

When we look at various examples of complex systems (e.g., cells, organizations, computer networks), then we observe that such systems usually consist of components that influence each other. For example, the quadcopter system discussed in Section 3.1, contains components in the form of quadcopters and a scheduler that have to exchange information in order to achieve their goals.

Usually, each component can again be viewed as a system, and might again consist of sub-components. And those sub-components again influence each other by some form of communication.

For modeling this, we first fix an alphabet for communication. This will be an arbitrary set S whose elements we call *symbols*. These symbols then form the basis for communication:

Definition 2 A (discrete time) signal over S is an infinite sequence of elements of symbols (i.e., $\mathbb{N}_0 \rightarrow S$).

We write Σ_S for the set of signals over S . For example, the sequence $(free, free, free, free, free, scheduled, mission, \dots)$ is a signal from the set of signals $\Sigma_{\{free, scheduled, mission\}}$.

4.2 Black Box Description of Reactive Systems

The next question is how to describe the input/output behavior of a system. For each input signal, we have certain possible output signals. In general, we

have a relation between input signals and output signals:

Definition 3 A (discrete time) system with input set I and output set O is a relation between signals over I and signals over O , that is a subset of $\Sigma_I \times \Sigma_O$.

Every pair (i, o) of input signal i and output signal o that is in the relation (and hence an element in this subset) is called a behavior of the system.

Since this describes a system purely by its input/output behavior, without taking into account its internal structure, this is some form of black box description of systems.

Note that the notion of system defined here differs from our usage of the term system in the previous chapter: In the previous chapter, a system was some part of the real world. Definition 3, however, defines a mathematical way of describing such a part of the real world, that is, it defines a certain type of *model*. In order to avoid confusion, we will also use the term “real-world system” for the notion from the previous chapter, distinguishing it from the notion of discrete time system defined here.

A first example of a discrete time system is based on the quadcopter example from Chapter 3.1. We will first describe the system informally and later formalize the example.

Example 1

- *Input signal: an arbitrary signal from $\Sigma_{\{nil, start\}}$*
- *Output signal: a signal from $\Sigma_{\{free, scheduled, mission\}}$ s.t.
input start results in output mission within at most 10 steps*

In this example, one input signal allows several corresponding output signals. For example, for the input signal

$$(nil, nil, start, nil, nil, nil, nil, \dots),$$

both

$$(free, free, free, scheduled, scheduled, scheduled, mission, \dots)$$

and

$$(free, free, free, free, free, free, mission, \dots)$$

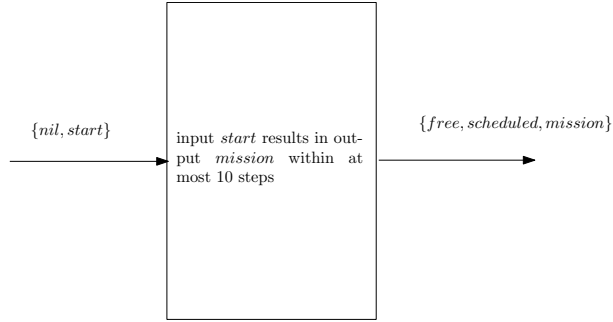
are corresponding output signals, since for both output signals, the input *start* results in output *mission* within 10 steps.

Example 2 The behavior of the UNIX device `/dev/null` can be modeled as the following system

- *Input signal: arbitrary string*
- *Output signal: EOF EOF EOF ...*

Such a system description can also be represented graphically. For example, Figure 4.2 shows a graphical representation of Example 1.

In the two examples above, we defined the discrete time system informally. Here are corresponding formal definitions:



Example 3 *Quadcopter:*

$$\{(i, o) \in \Sigma_{\{nil, start\}} \times \Sigma_{\{free, scheduled, mission\}} \mid \forall t \in \mathbb{N}_0 . i(t) = start \Rightarrow \exists d \in \{0, \dots, 10\} . o(t + d) = mission\}$$

Example 4 */dev/null*

$$\{(i, o) \in \Sigma_{\{0, \dots, 255\}} \times \Sigma_{\{0, \dots, 255, EOF\}} \mid \forall t \in \mathbb{N}_0 . o(t) = EOF\}$$

In Example 3, the quadcopter is allowed to behave in various different ways. For example, we may have a quadcopter that starts to fly precisely 7 steps after receiving a start symbol. Such a quadcopter does not contradict the specification given in Example 3, but it does not show all the behaviors that Example 3 allows. One may also make the specification more specific, allowing, for example, only a delay of 1 or 2 steps instead of up to 10 steps:

Example 5

$$\{(i, o) \in \Sigma_{\{nil, start\}} \times \Sigma_{\{free, scheduled, mission\}} \mid \forall t \in \mathbb{N}_0 . i(t) = start \Rightarrow \exists d \in \{1, 2\} . o(t + d) = mission\}$$

In general, one can restrict system behavior as follows:

Definition 4 A system $\mathcal{S}_1$ is a refinement of a system $\mathcal{S}_2$ iff $\mathcal{S}_1 \subseteq \mathcal{S}_2$

The system from Example 5 is a refinement of the system from Example 3. Here are some further examples of systems:

- In lazy functional programming languages, input/output is sometimes in the form of streams, that is signals in $\Sigma_{\{0, \dots, 255\}}$. A program in such a language can then be viewed as a discrete time systems with input set $\{0, \dots, 255\}$ and output set $\{0, \dots, 255\}$.
- In exact real arithmetic, real numbers are computed as streams. As a first idea, this may be signals from $\Sigma_{\{0, \dots, 9\}}$ (there is some complication involved in this).
- In a computer network the computational elements of the network can communicate using byte streams, that is, signals from $\Sigma_{\{0, \dots, 255\}}$. Hence those elements can be viewed as discrete time systems.

We observe that also the halting problem can be encoded as a system:

$$\left\{ (i, o) \in \Sigma_{\{0, \dots, 255\}} \times \Sigma_{\{0, 1\}} \mid \begin{array}{l} \bullet i \text{ is a computer program of length } l \\ \text{padded with blanks,} \\ \bullet o(l+1) = 1, \text{ if program from input} \\ \text{always terminates, 0, otherwise.} \end{array} \right\}$$

Since the halting problem is undecidable, we conclude from this that in general, there are discrete-time systems that cannot be implemented on computers.

Finally, consider the system

$$\{(i, o) \mid o(0) = 1 \text{ iff } \mathbf{P} = \mathbf{NP}\}.$$

In contrast to the system encoding the halting problem, here a computer program that implements this system exists! Does this mean that the famous **P** versus **NP** problem has been solved? No! The point is, that we know such a program exists, but we do *not* know, how it looks like: Consider a program *A* that writes 0 as its first output symbol, and a program *B* that writes 1 as its first output symbol. We know that one of the two programs implements the above system, but we do not know, which of the two it is.

Let us now compare discrete time systems with classical algorithms. Also for a classical algorithm, I/O specifications relate input to output. But traditional algorithms have input and output of finite length. Here we have reactive system, and hence input and output of infinite length (i.e., signals).

4.3 Systems Properties

Let us assume a discrete time system $\mathcal{S}$ with input set I and output set O . Observe that, according to the definition, this system $\mathcal{S}$ is a subset of $\Sigma_I \times \Sigma_O$. Can it even be the empty set? Yes, the empty set is a system that does not show any behavior, and hence never produces any output. A system that, for each input reacts with some output looks as follows:

Definition 5 $\mathcal{S}$ is receptive iff for all $i \in \Sigma_I$ there exists $o \in \Sigma_O$ s.t. $(i, o) \in \mathcal{S}$.

This also includes the possibility that a system allows more than one output for a given input.

Definition 6 $\mathcal{S}$ is deterministic iff for all $i \in \Sigma_I$ there exists precisely one $o \in \Sigma_O$ s.t. $(i, o) \in \mathcal{S}$.

In such a case the system can be viewed as a function (instead of a relation). Using nondeterminism one can model under-specified systems (systems whose behavior is partially left open as in Example 3) or uncertain systems (systems whose behavior is not yet precisely known).

What about a system, that produces the output from the input, but from elements of the input that come from the future? An example is the system $\mathcal{S} = \{(i, o) \mid \forall t \in \mathbb{N}_0 . o(t) = i(t+1)\}$.

Definition 7 $\mathcal{S}$ is causal iff for all $i_1, i_2 \in \Sigma_I$, $x \in O$, $t \in \mathbb{N}_0$ such that for all $t' \in \{0, \dots, t\}$, $i_1(t') = i_2(t')$,

$$\begin{aligned} & \text{there exists an } o \in \Sigma_O \text{ s.t. } (i_1, o) \in \mathcal{S}, o(t) = x \\ & \quad \text{iff} \\ & \text{there exists an } o \in \Sigma_O \text{ s.t. } (i_2, o) \in \mathcal{S}, o(t) = x. \end{aligned}$$

Intuitively, this definition says that the output at a given time is always determined by input up to that time. Interestingly, there are applications, where non-causal systems are very useful. For example, when processing audio signals that is already fully available (i.e., not produced in real time), one can also use the part of the signal lying in the future for creating the output signal at a certain point in time.

Definition 8 $\mathcal{S}$ is memory-less iff there exists $R \subseteq I \times O$ s.t. for all $(i, o) \in \Sigma_I \times \Sigma_O$, $(i, o) \in \mathcal{S}$ iff for all $t \in \mathbb{N}_0$, $(i(t), o(t)) \in R$.

Here, the intuition is that output at given time is determined only by input at this time. For example, the system

$$\{(i, o) \mid \forall t \in \mathbb{N}_0 . o(t) \text{ is a prime divisor of } i(t)\}$$

is memory-less. Clearly every memory-less system is causal.

These definitions open up certain philosophical questions, relating the formal definitions to real-world systems.

- Do non-causal systems exist in the real world? In other words, can the present depend on the future?
- Do non-deterministic systems exist in the real world? In other words, is the randomness in the real world?

Consider the weather forecast: Its input signal consists of the measurements of certain weather stations, and the corresponding output signal consists of the weather forecast computed from this input data. Can such a system be non-deterministic? Can it be non-causal?

A non-deterministic weather forecast means that for one and the same measurement data, it may produce different forecasts. This is certainly a realistic scenario: The algorithms used for compute weather forecasts often uses random simulations, and different random number generators might result in different weather forecasts—under the assumption that a real random number generator is used, and not just a pseudorandom number generator.

The situation is different for causality: non-causality of such a weather forecast would mean that for measurement data i_1 and i_2 that are the same up to the present (but may differ in the future), there is a weather forecast x allowed by data i_1 and but not by i_2 . This does not make sense: The decision whether x is a valid forecast is only based on the input data up to the present, but certainly not on data that will be available in the future. A system that uses future data for producing its output, would be some kind of oracle—something most people do not believe in. This does not exclude the possibility of prediction, because predictions just use *past* data for producing a prediction, not future data.

In the mathematical world such systems do exist, and non-deterministic and non-causal models are very useful. For example, classical physics is a deterministic model, quantum physics a non-deterministic one. In the real world the answer to this question is unknown, since the definitions refer to discrete time systems in the mathematical sense and not to the real world. For applying the definitions to real world systems we have two possibilities:

1. Model the real world system using a discrete time systems, and then apply the definition to the model.
2. Directly apply the definition to the real world system.

In the first case, the answer to the question depends on the model, and hence is not unique. In the second case we would have to check the system for all inputs which is impossible.

For us:

- Non-determinism will be very useful
- We will use certain memory-less systems
- Models of the real world will always be receptive and causal, if not, this is a bug.

4.4 White Box Description of Reactive Systems

Up to now, we have modeled systems as black-boxes that produce output from input. However, since we want to analyze models, for example with the goal of finding design bugs, we want to implement discrete time systems on computers.

One option is to use full-fledged programming languages for this, for example, a C program that reads from stdin and writes to stdout. There are special programming languages for this (*synchronous reactive programming*), for example Esterel and Lustre, that are part of SCADE system (<http://www.esterel-technologies.com/>).

Such an approach has some advantages, but also some disadvantages:

- We have to decide on data structures, memory management, fight with complex syntax.
- Automatic analysis of such programs is in full generality impossible, while manual testing is unreliable and costly.

In order to avoid these problems, we want a programming language for reactive systems, that is as simple as possible. For this we can use automata as a basis:

Definition 9 A (discrete time) automaton is a quintuple (S, S_0, I, O, R) , where

- S is a set (state space) whose elements we call states
- $S_0 \subseteq S$ (set of initial states), such that $S_0 \neq \emptyset$
- I is a set whose elements we call inputs

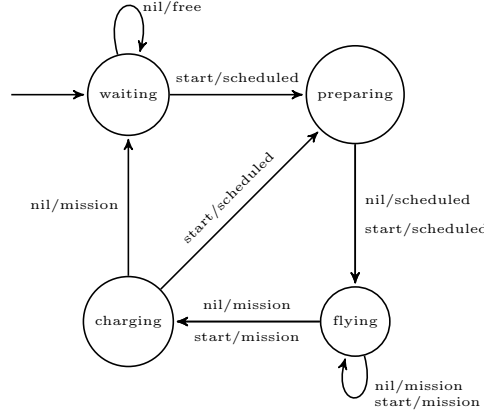


Figure 4.1: Example: Quadcopter

- O is a set whose element we call outputs
- $R \subseteq I \times S \times S \times O$ (transition relation) s.t. for all $i \in I$, $s \in S$, there exists $s' \in S$, $o \in O$ s.t. $(i, s, s', o) \in R$

The sets in this definition do not necessarily have to be finite, but often they are. In such a case, the given automaton is often also called a *finite state automaton*. The condition that for every input and state, there must be a corresponding transition is called *left totality*. This means that the transition relation of an automaton must be left total.

Note, that unlike classical automata, we do not have any terminal states here. There are various equivalent variants (e.g., transitions based on sets, separate output function/relation), extensions, and names (transducers, I/O automata)

For representing such automata, often graphical notation is used, where the nodes represent the states and the edges represent the transitions. The edges are labeled with the input and output of the corresponding transition. For example, Figure 4.1 contains a graphical representation of an automaton implementing the quadcopter system from Example 3. The corresponding automaton is the quintuple (S, S_0, I, O, R) with

- $S = \{\text{waiting}, \text{preparing}, \text{flying}, \text{charging}\}$
- $S_0 = \{\text{waiting}\}$
- $I = \{\text{nil}, \text{start}\}$
- $O = \{\text{free}, \text{scheduled}, \text{mission}\}$
- $R = \left\{ \begin{array}{l} (\text{nil}, \text{waiting}, \text{waiting}, \text{free}), \\ (\text{start}, \text{waiting}, \text{preparing}, \text{scheduled}), \\ (\text{start}, \text{preparing}, \text{flying}, \text{scheduled}), \\ \dots \end{array} \right\}$

In a similar way as discrete time systems, discrete time automata allow certain input-output behavior. However, now this behavior depends on the internal state.

Definition 10 Given an automaton (S, S_0, I, O, R) the pair (i, o) of signals $i \in \Sigma_I$, $o \in \Sigma_O$, is a behavior of the automaton iff there exists an $s \in \Sigma_S$ s.t.

- $s(0) \in S_0$,
- for all $t \in \{0, 1, \dots\}$, $(i(t), s(t), s(t+1), o(t)) \in R$.

In this case we also write $s(0) \xrightarrow{i(0)/o(0)} s(1) \xrightarrow{i(1)/o(1)} s(2) \dots$

Definition 11 An automaton T represents the system

$$\llbracket T \rrbracket := \{(i, o) \in \Sigma_I \times \Sigma_O \mid (i, o) \text{ is a behavior of } T\}.$$

Let us now analyze our example: Does the automaton in Figure 4.1 represent the specification given by the system from Example 3? According to Definition 11, we have to check whether the system $\llbracket S \rrbracket$ is equal to the system from Example 3. Here we have to check equality of two sets. Two sets are equal, if they contain the same elements. Do both systems contain the same behaviors?

Let us first check whether every behavior of the system $\llbracket S \rrbracket$, that is every behavior of the automaton S , is a behavior of the system from Example 3. We have to analyze the edges whose input label is *start*, and check whether all possible evolutions from such an edge reach an edge with output label *mission* in not more than 10 steps. For those edges with input *start* where the corresponding output is already *mission* this holds trivially. For the two other edges with input *start*, the automaton must evolve to an edge without output *mission*.

Now let us check the opposite direction: Is every behavior of the system from Example 3 also a behavior of the automaton S ? Clearly not. The specification allows the number of steps between an input symbol *start* and output symbol *mission* to be up to 10. However, the automaton never takes as many steps. So it has strictly less behaviors than the specification. In other words, it does not represent the system from Example 3 but it represents a refinement of this system.

Another question is, whether for a given system, there is an automaton representing it. Certainly this is not the case for systems that are not causal (i.e., acausal): the state of an automaton only depends on the past, but the output of an acausal system depends on the future. The following example shows, that even causal systems might not be representable as automata.

$$\{(i, o) \in \Sigma_{\{a,b,c\}} \times \Sigma_{\mathbb{N}_0} \mid \forall t. o(t) = |\{t' \mid t' \leq t, i(t') = a\}|\}$$

This system counts the number times the symbol *a* has occurred at the input which no finite automaton can implement.

Another question is uniqueness. For example, consider the system

$$\{(i, o) \in \Sigma_{\{nil\}} \times \Sigma_{\{\heartsuit\}} \mid \forall t. o(t) = \heartsuit\}.$$

This system can be represented by two different automata (see Figure 4.2).

Definition 12 Two automata are equivalent iff they represent the same system.

Since several equivalent automata may represent the same system, one may ask, whether there exists a minimal automaton representing a given system. Details can be found in any textbook on automata theory, and for systems with outputs [31].

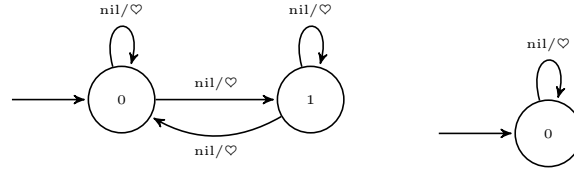


Figure 4.2: Equivalent Automata

4.4.1 Properties of Automata

In Section 4.3 we discussed some properties of discrete time systems. Since every automaton represents a discrete time system, it is a natural question to ask, which properties this system fulfills. Clearly, due to the way automata are defined, the system an automaton represents is always receptive and causal. However, in the case of determinism, the situation is more subtle. The system represented by an automaton is deterministic, if the automaton, for a given input, always has a unique corresponding output. Analyzing the automaton in Figure 4.1, we see that from vertex *flying* upon input *nil* we can either stay in this vertex, or continue to vertex *charging*. The same can happen upon input *start*. Due to this, one input signal has several corresponding output signals. Hence the system this automaton represents is not deterministic.

Is it possible to check somehow, whether a given automaton represents a deterministic system? Clearly this system is deterministic if the automaton fulfills the following property.

Definition 13 *An automaton is deterministic iff*

- $|S_0| = 1$
- for all $i \in I$, $s \in S$, there exists precisely one $s' \in S$, $o \in O$ s.t. $(i, s, s', o) \in R$

However, the implication does not hold in the opposite direction. To see this, consider the left-hand side of Figure 4.2 which shows a non-deterministic automaton that represents a deterministic system.

In a similar way, a simple test whether an automaton represents a memory-less system is the fact that every automaton that has only one state represents a memory-less system. And again, this test is not complete: there are automata with more than one state that also represent a memory-less systems.

4.5 Variables: Extended State Machines

Consider the following example of traffic lights. The lights are connected to a button. After this button is pressed, the lights wait for 30 seconds, and then switch to green..

This can be modeled by the following automaton:

- State space: $\{(s, x) \mid s \in \{default, count\}, x \in \{0, \dots, 30\}\}$
- Initial states: $\{(default, 0)\}$

- Inputs: $\{nil, button\}$
- Output: $\{green, red\}$
- Transitions:

$$\left\{ \begin{array}{l} (nil, (default, 0), (default, 0), red) \\ (button, (default, 0), (count, 30), red) \\ (nil, (count, 30), (count, 29), red) \\ (nil, (count, 29), (count, 28), red), \\ \dots \end{array} \right\}$$

The automaton has many transitions, far too many to write down each of them. A more compact way of representing all transitions is

$$\left\{ (pressed, (l, x), (l', x'), light) \mid \begin{array}{l} [l = default \wedge l' = default \wedge pressed = nil \wedge light = red] \vee \\ [l = default \wedge l' = count \wedge pressed = button \wedge light = red \wedge x' = 30] \vee \\ [l = count \wedge l' = count \wedge x > 0 \wedge light = red \wedge x' = x - 1] \vee \\ [l = count \wedge l' = default \wedge x = 0 \wedge light = green] \end{array} \right\}$$

This is the basis for a widely-used graphical notation called *extended state machine*. The extended state machine representing the above automaton can be seen in Figure 4.3. Here, one first declares some input, output and state variables. In our example, we have precisely one variable in each case. In general, extended state machines allow several of those variables. Here, n variables correspond to an n -tuple in the corresponding automaton.

- Input variables: $pressed \in \{nil, button\}$
- Output variables: $light \in \{green, red\}$
- State variables: $x \in \{0, \dots, 30\}$

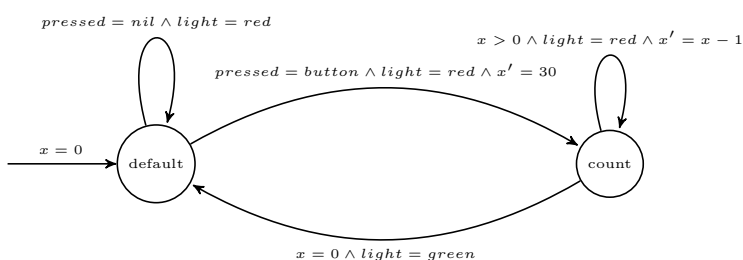


Figure 4.3: Extended State Machine

Variants of state machines use various different types of notation, see for example the variant in Figure 4.4.

In general, the vertices of such a graph are also called *locations*. Note that the set of states of an extended state machine is *not* equal to the set of locations, since the state also contains state variables. Conditions on input variables and state variables are usually called *guards* and changes on state variables *updates*.

- Inputs: $pressed \in \{nil, button\}$
- Variables: $x \in \{0, \dots, 30\}$

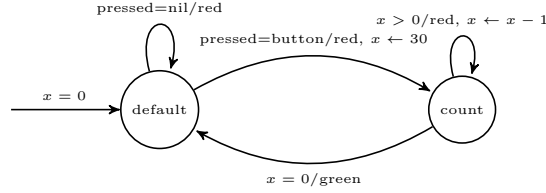


Figure 4.4: Extended State Machine (Alternative Notation)

In Figure 4.3 those updates are equalities on state variables and their primed versions, whereas in Figure 4.4 they are represented as assignments. Variables that do not occur in such an assignment stay unchanged.

4.6 Hierarchical Automata

Consider a pocket calculator like the one shown in Figure 4.5. When the user presses the button OFF, the calculator is switched off, independently from the state it is currently in. When the user presses the button C, the calculator returns to its initial state, also independently from the state it is currently in. Hence, a corresponding automaton will have many transitions of a similar kind.

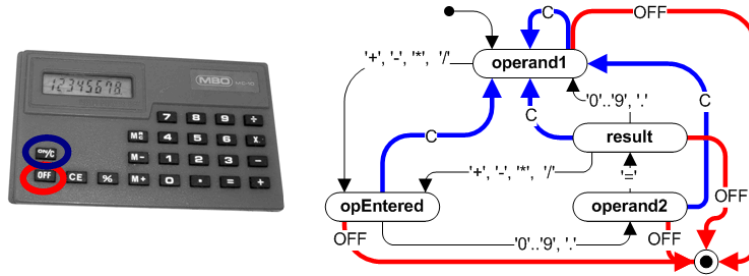


Figure 4.5: Pocket Calculator

Hierarchical automata allow us to factor out such similar behavior by collecting states into super-states, in the case of our example, into a super-state **on** comprising all states from which one can clear or switch off the calculator (see Figure 4.6).

Such super-states can be viewed as an additional way for recursively hiding of modeling details. So, a super-state can be viewed as a form of abstraction that hides the sub-states of the super-state.

Such hierarchical automata form the basis for the formalism Statecharts [24]. There are various variants and extensions. Especially, UML State Machines are built on this formalism.

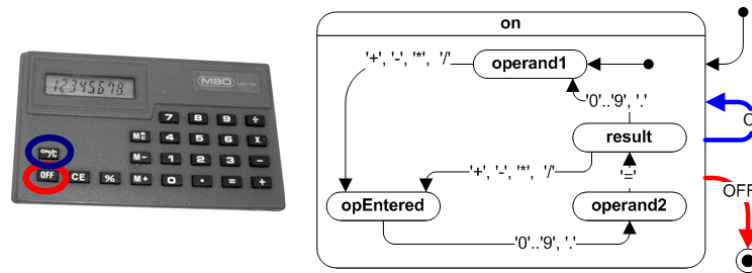


Figure 4.6: Pocket Calculator

4.7 Conclusion

Systems consist of communicating sub-systems

We can describe them

- from the outside (black box): Which inputs result in which outputs?
- from the inside (white box): How are outputs produced from inputs?

Corresponding main definitions:

- discrete-time system (a model)
- automaton

Comparison: classical algorithms

- I/O specification (relation between inputs and outputs):
- Implementation in a programming language

Behind all of this there is a general theory of automata with inputs of infinite length: the theory of ω -automata.

Chapter 5

Interacting System Components

In the previous chapter we have observed that complex systems usually consist of components that influence each other. We have also discussed possibilities of how to describe such components using the notion of a discrete-time system. In this chapter, we will discuss how to describe the interaction between such components.

5.1 Cascade Composition

It is often the case that certain behavior of a component may result in a reaction of other components. For modeling this, we will connect the output of one discrete-time system to the input of the other system in a similar way as UNIX pipes connect the output of one process with the input of another process (e.g., `tail -0f logfile | grep login`).

This can be represented graphically. For this, we continue with the quadcopter example from the previous chapter. We assume that the drone is controlled by a device that gets as input the information whether the battery of the drone is charged and, based on this, decides on when to start for another mission. The graphical representation of this can be seen in Figure 5.1. Note the intermediate signal that the controller sends to the drone. This intermediate signal is essential for defining cascade composition formally. In the following definition it is represented by the existentially quantified variable $\dot{i}o$.

Definition 14 Consider systems $\mathcal{S}^1, \mathcal{S}^2$ with input set I^1, I^2 and output set O^1, O^2 , respectively such that $O^1 \subseteq I^2$. Then the cascade composition of $\mathcal{S}^1$ and $\mathcal{S}^2$, denoted by $\mathcal{S}^1 \rightsquigarrow \mathcal{S}^2$, is the system

$$\mathcal{S}^1 \rightsquigarrow \mathcal{S}^2 := \{(i_1, o_2) \mid \exists \dot{i}o . (i_1, \dot{i}o) \in \mathcal{S}^1, (\dot{i}o, o_2) \in \mathcal{S}^2\}$$

with input set I^1 and output set O^2 .

Here, the condition $O^1 \subseteq I^2$ ensures that the second system understands all commands that it receives from the first one.

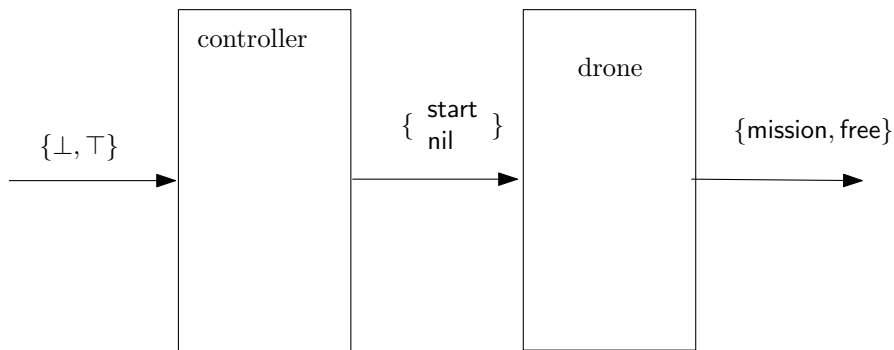


Figure 5.1: Example of Cascade Composition

Also note the similarity to the join operation on databases: Both discrete-time systems and relational databases are relations, and cascade composition is the analogous operation to the join operation of two relational databases followed by a project on the input and output of the composed system. The main difference is that discrete-time systems are relations on *infinite* objects (signals), whereas relational databases are relations on finite objects.

As an example, consider the two systems

$$\mathcal{S}^1 = \{((i_1, i_2, \dots), (o_1, o_2, \dots)) \in \Sigma_{\mathbb{Z}} \times \Sigma_{\mathbb{Z}} \mid \forall k \in \mathbb{N} . o_k = \sum_{l=1}^k i_l\}$$

and

$$\mathcal{S}^2 = \{((i_1, i_2, \dots), (o_1, o_2, \dots)) \in \Sigma_{\mathbb{Z}} \times \Sigma_{\mathbb{Z}} \mid \forall k \in \mathbb{N} . o_k = \max\{0, i_k\}\}.$$

Then

$$((1, -2, 2, 3, \dots), (1, 0, 1, 4, \dots)) \in \mathcal{S}^1 \rightsquigarrow \mathcal{S}^2$$

because

$$((1, -2, 2, 3, \dots), (1, -1, 1, 4, \dots)) \in \mathcal{S}^1$$

and

$$((1, -1, 1, 4, \dots), (1, 0, 1, 4, \dots)) \in \mathcal{S}^2.$$

This is straightforward, but in Chapter 4, we already saw that while the systems formalism is very general and powerful, it also has certain disadvantages: Due to its generality, formal models based on this formalism may be difficult to simulate and to analyze. This holds all the more in cases, where a system is the result of the composition of systems that are difficult to simulate and analyze by themselves.

To avoid this problem, we also define a corresponding operation on automata:

Definition 15 For automata $(S^1, S_0^1, I^1, O^1, R^1)$ and $(S^2, S_0^2, I^2, O^2, R^2)$ with $O^1 \subseteq I^2$ their synchronous cascade composition

$$(S^1, S_0^1, I^1, O^1, R^1) \rightsquigarrow (S^2, S_0^2, I^2, O^2, R^2)$$

is defined as

$$(S^1 \times S^2, S_0^1 \times S_0^2, I^1, O^2, \{t \mid IsTrans(t)\})$$

where

$$\begin{aligned} IsTrans(i^1, (s^1, s^2), (s'^1, s'^2), o^2) : \iff \\ \exists o_1, i_2 . (i^1, s^1, s'^1, o_1) \in R^1, (i_2, s^2, s'^2, o^2) \in R^2, o_1 = i_2 \end{aligned}$$

The state space of the combined automaton is formed by the Cartesian product of the state spaces of the individual automata since the two automata run in parallel. However, the transitions are restricted in the such way that each transition of the composed automaton is formed by transitions corresponding to one transition of the first automaton R^1 and one transition of the second automaton R^2 for which the output o_1 of the first transition is equal to the input i_2 of the second transition. Note that the condition $IsTrans$ could also be written equivalently as

$$\exists x . (i^1, s^1, s'^1, x) \in R^1, (x, s^2, s'^2, o^2) \in R^2$$

which gives the same name x to the intermediate symbol connecting a transition from the first automaton with a transition from the second one.

If the two automata satisfy the condition $O^1 \subseteq I^2$, the second automaton is always able to react to a transition of the first one. Hence the composed automaton will be again have a well-formed transition relation satisfying the requirement from Definition 9 that the transition relation needs to be left-total, that is, for every input and current state, it must have a corresponding transition.

As an example, consider the two automata from Figure 5.2 that implement the systems from Figure 5.1. The second automaton is a simplified version of the quadcopter model from Figure 4.1, and the first one an implementation of a controller that decides on when to start the drone for the next mission based on the information whether the battery of the quadcopter is charged or not.

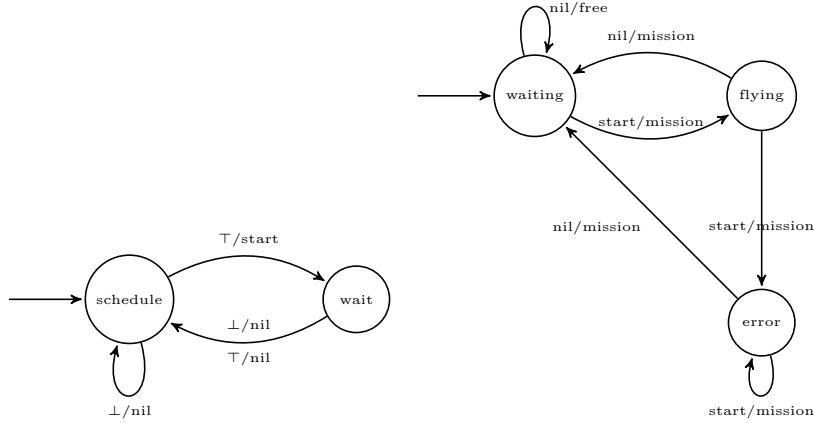


Figure 5.2: Automata: Controller and Drone

Since each of the two automata has its own state, the set of states of the combined automaton is the Cartesian product of the set of states of the individual automata. In the example, the first automaton has two states, the second

one three, and hence the Cartesian product—the set of pairs with first elements from the first and second elements from the second set—has six elements. Since each individual automaton has one initial state, the combined automaton also has one initial state. And the transition of the combined automaton are given by a combination of transitions of the individual automata such that the output of the first automaton is equal to the input of the second one. Figure 5.3 shows some examples, where the transitions the left-hand side are taken from the first automaton, the transitions on the right-hand side are taken from the second automaton, and the resulting combinations is shown in the center. Note that the output of the transitions on the left-hand side is always the same as the input of the respective transitions on the right-hand side.

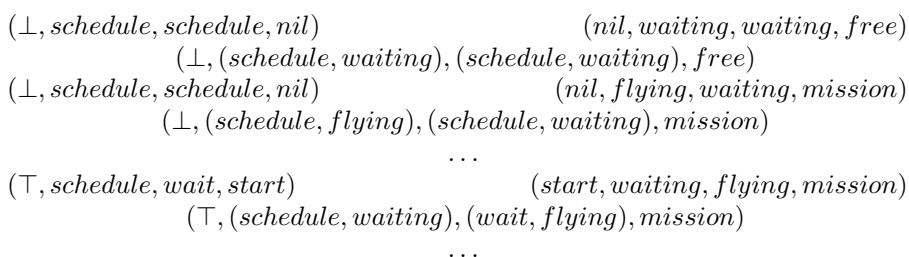


Figure 5.3: Combining Transitions

The result of forming the cascade composition of those two automata can be seen in Figure 5.4, and after a re-arrangement of the states for better visibility, in 5.5 (for saving space, the states of the first automaton are abbreviated to just the letters *s* and *w*, and a slash is used instead of classical pair notation). As one can see, in the resulting automaton two states do not have incoming transitions. For example, all transitions of the controller entering the state **wait** (there is just one such transition) produce the output symbols **start**, and no transitions of the drone with input **start** enters the state **waiting**. Hence the state **(wait, waiting)** is not reachable in the combined automaton. Often it makes sense to simplify the automaton by removing such unreachable states.

Composition of automata is compatible with composition of the represented systems:

Property 1

$$\llbracket \mathcal{A}_1 \rightsquigarrow \mathcal{A}_2 \rrbracket = \llbracket \mathcal{A}_1 \rrbracket \rightsquigarrow \llbracket \mathcal{A}_2 \rrbracket.$$

This means that first composing two automata, and then taking the system that the resulting composed automaton represents (the top-right two arrows in Figure 5.6) has the same result as first taking the two individual systems represented by the two automata and only then composing those systems (the bottom-left two arrows in Figure 5.6).

5.2 General Composition—Synchronous Reactive Models

The controller on the left-hand side of Figure 5.2 takes as input information on whether the battery of the drone is charged. However, it assumes that missions

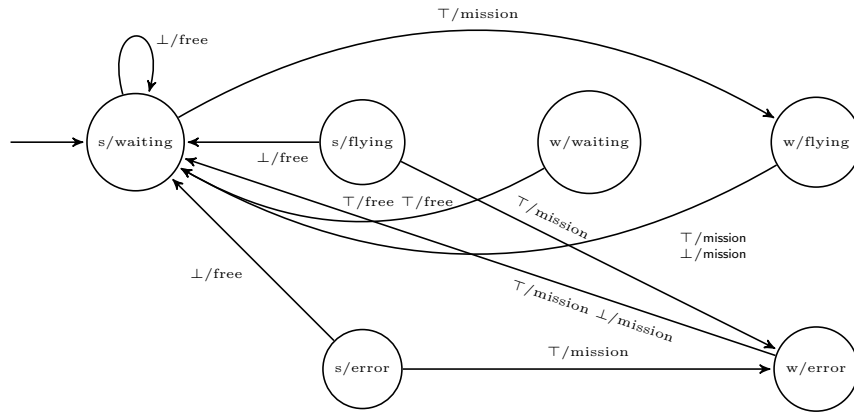


Figure 5.4: Cascade Composition

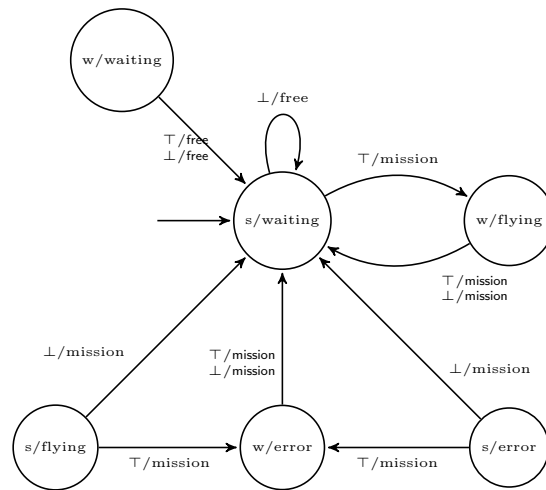


Figure 5.5: Cascade Composition: Example

$$\begin{array}{ccc}
\mathcal{A}_1, \mathcal{A}_2 & \xrightarrow{\sim} & \mathcal{A}_1 \rightsquigarrow \mathcal{A}_2 \\
\downarrow \llbracket \cdot \rrbracket & & \downarrow \llbracket \cdot \rrbracket \\
\llbracket \mathcal{A}_1 \rrbracket, \llbracket \mathcal{A}_2 \rrbracket & \xrightarrow{\sim} & \llbracket \mathcal{A}_1 \rrbracket \rightsquigarrow \llbracket \mathcal{A}_2 \rrbracket = \llbracket \mathcal{A}_1 \rightsquigarrow \mathcal{A}_2 \rrbracket
\end{array}$$

Figure 5.6: Compatibility of Cascade Composition

are that short that after starting a mission the drone is already available again in the next step. Now assume that missions may have various lengths. For modeling this, we add another transition to the drone model that keeps the automaton in the **flying** state (see Figure 5.7). This makes the automaton non-deterministic, allowing two different transitions after receiving the input **nil** in the **flying** state.

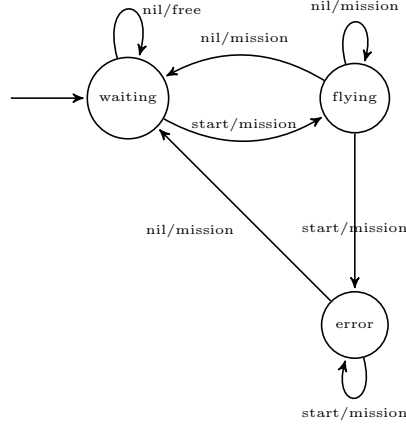


Figure 5.7: Modified Drone Automaton

Now missions do not have a fixed duration any more, and can be arbitrarily long, depending on how often this transition is taken. The controller needs to take this into account and needs the information from the drone, when the mission has ended and the drone is free again. For providing this information to the controller, we create a *feedback loop*, see Figure 5.8.

The figure contains the following new phenomena:

- Arbitrary connection of inputs and outputs:
- Systems with several inputs/outputs: $S \subseteq \Sigma_{I_1 \times \dots \times I_r} \times \Sigma_{O_1 \times \dots \times O_s}$
- Tee: $\{(i, (o_1, o_2)) \mid o_1 = i, o_2 = i\}$

Such system models are also called *synchronous reactive models* and each input or output of such a network or one of its components is called a *port*.

Now observe that formally, systems with several inputs and several outputs take tuples as inputs and outputs. For being able to formalize the behavior of

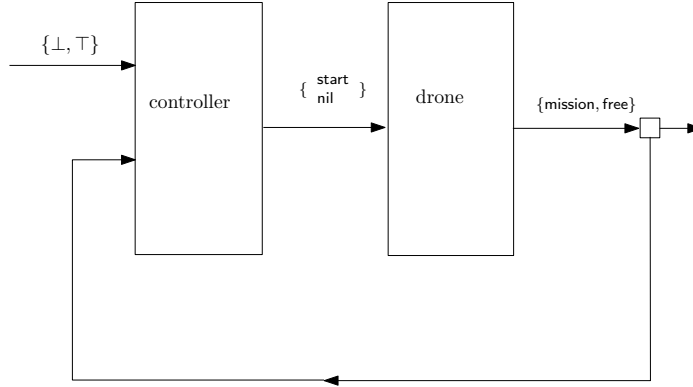


Figure 5.8: Feedback Loop

networks containing such systems as components we will need an operation that allows us to bundle several signals (corresponding to several connections in the network) into one signal of such tuples.

$$\text{zip}((s_1^1, s_2^1, \dots), (s_1^2, s_2^2, \dots), \dots, (s_1^r, s_2^r, \dots)) := ((s_1^1, s_1^2, \dots, s_1^r), (s_2^1, s_2^2, \dots, s_2^r), \dots)$$

So, for any $r \in \mathbb{N}$ this *zip*-operation takes r signals and bundles them into one signal of r -tuples that can then form the input or output of a system component.

The special case $r = 1$ deserves discussion: Strictly speaking, in this case, *zip* takes one signal, and wraps each of its elements into a 1-tuple. This creates input and output signals of system components that are sequences of 1-tuples. In practice, however, one usually does not make a difference between an element x and the 1-tuple (x) containing this element.

Now let us try to investigate which system the feedback loop in Figure 5.8 represents. In other words, we want to define its semantics. The figure shows a network with

- one input and one output
- three components
- ten ports
- five connections between ports

We start by giving a name to each port of the network. The result can be seen in Figure 5.9. The network represents a system $N \subseteq \Sigma_{\{\perp, \top\}} \times \Sigma_{\{\text{mission}, \text{free}\}}$, where $(i_N, o_N) \in N$ iff there are signals $i_c^1, i_c^2, o_c, i_d, o_d, i_t, o_t^1, o_t^2$ s.t.

- $(\text{zip}(i_c^1, i_c^2), o_c) \in \text{controller}$, $(i_d, o_d) \in \text{drone}$, $(i_t, \text{zip}(o_t^1, o_t^2)) \in \text{tee}$,
- $i_N = i_c^1, o_c = i_d, o_d = i_t, o_t^2 = i_c^2, o_t^1 = o_N$.

In other words, a pair (i_N, o_N) is a signal of the system if and only if the signals corresponding to a component satisfy the behavior of this component, and the signals of two ports of the same connection are the same.

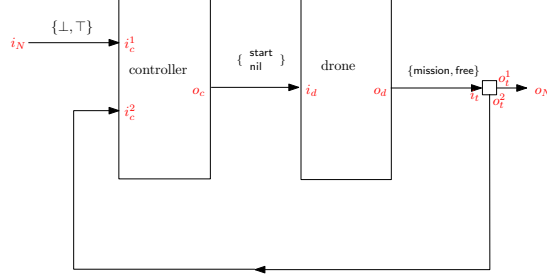


Figure 5.9: Feedback Loop with Named Ports

Definition 16 Given a network of components N that has r inputs, and s outputs, $(i, o) \in N$ iff there exist a corresponding signal for every port of the network such that

- for every component S with corresponding input signals $(i_1^C, \dots, i_{r_C}^C)$ and output signals $(o_1^C, \dots, o_{r_C}^C)$, $(\text{zip}(i_1^C, \dots, i_{r_C}^C), \text{zip}(o_1^C, \dots, o_{r_C}^C)) \in S$, and
- for every connection between two ports, the corresponding signals are identical.

Here, every port can correspond to

- a network input from i ,
- a network output from o ,
- an input of a component, or
- an output of a component.

In the case of cascade composition, we have seen that the condition $O_1 \subseteq I_2$, that ensures that each output port only sends symbols that the connected input port understands, also ensures that also the composed system can react with some output for every input. However, for general composition, this is not the case, in general, as we will see in the next section.

5.3 Simulation

Our feedback loop from Figure 5.8 still has a problem: There is a circular dependence of the three components in the loop. Most software packages for modelling such feedback loops will return an error message when simulating such a loop, since this would require them to know the output of each component to be able to compute the input of the same component at the *same time*. This corresponds to the fact mentioned at the end of the previous section that the composed system may not be able to react with an output for every input.

For solving this problem, we observe that in reality, the components in the loop do not react at the very same point in time, but there is a slight delay in the communication between those components. For example, when the drone sends the message that it is free, this message will not arrive at the controller immediately, but after a (very small) delay. Hence we will introduce another element that allows us to model such delays.

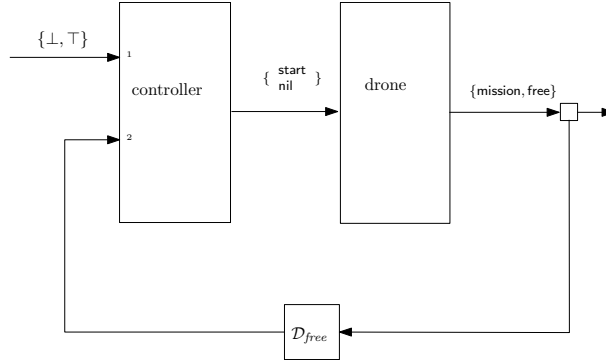


Figure 5.10: Feedback Loop with Delay

Definition 17 For sets S_0 , I and O s.t. $I \subseteq O$, $S_0 \subseteq O$, the delay with set of initial states S_0 , inputs I , and outputs O is

$$\{(i, o) \mid o(0) \in S_0, \forall k \in \mathbb{N}, o(k) = i(k-1)\}.$$

We will also denote such a delay by $\mathcal{D}_{S_0, I, O}$, and if I and O are clear from the context, by $\mathcal{D}_{S_0}$.

Here are some examples of how such a delay behaves:

- $((18, 22, 18, 22, \dots), (0, 18, 22, 18, 22, \dots)) \in \mathcal{D}_{\{0,1\}}$
- $((18, 22, 18, 22, \dots), (1, 18, 22, 18, 22, \dots)) \in \mathcal{D}_{\{0,1\}}$
- $((18, 22, 18, 22, \dots), (18, 22, 18, 22, \dots)) \notin \mathcal{D}_{\{0,1\}}$
- $((18, 22, 18, 22, \dots), (0, 22, 18, 22, 18, \dots)) \notin \mathcal{D}_{\{0,1\}}$

Now we can add such a delay into our feedback loop, for example, as shown by Figure 5.10. Now, the input of each component still depends on the output of the same component, however, on the output of the same component at the *previous* point in time. Hence it is now no problem to simulate such loops.

In the example, the information whether the drone is free comes from output of the drone at the previous time step. In other words, the delay functions as a memory element that remembers the value at a certain link from one clock tick to the next.

Now we can provide an algorithm (Figure 5.11) that simulates a given network N for given inputs $i_1, \dots, i_r$, computing corresponding outputs.

5.4 Tee and Delay as Automata

Examples such as the one from Figure 5.10, also contain elements for which we do not yet know how to represent them as automata. This is what we will study now.

Property 2 For input set I , the automaton,

$$(\{\perp\}, \{\perp\}, I, (I, I), \{(i, \perp, \perp, (o_1, o_2)) \mid o_1 = i, o_2 = i\})$$

```

for  $t \leftarrow 0 \dots$  do
  for each delay  $\mathcal{D}_{S_0}$  in  $N$  and corresponding
    input signal  $i_{\mathcal{D}_{S_0}}$  and
    output signal  $o_{\mathcal{D}_{S_0}}$ 
    let  $o_{\mathcal{D}_{S_0}}(t)$  be
      if  $t = 0$  then  $s_0$ , for an arbitrary  $s_0 \in S_0$ ,
      else  $i_{\mathcal{D}_{S_0}}(t - 1)$ 
    while there is a network element  $S$  with known  $i_s(t)$  and unknown  $o_s(t)$ 
      compute  $o_s(t)$  from  $i_s(t)$ 

```

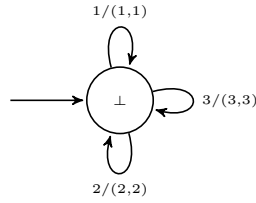
Figure 5.11: Simulating Component Networks

or, equivalently, the automaton

$$(\{\perp\}, \{\perp\}, I, (I, I), \{(i, \perp, \perp, (i, i))\})$$

represents a tee with input set I .

For example, a tee with input set $I = \{1, 2, 3\}$ is represented by the automaton



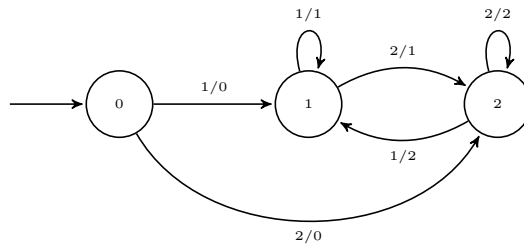
Note that this automaton has only one state which reflects the fact that tees are memory-less systems.

Property 3 *The automaton*

$$(I \cup S_0, S_0, I, O, \{(i, o, i, o) \mid i \in I, o \in O\})$$

represents the delay $\mathcal{D}_{S_0, I, O}$.

For example, the delay $\mathcal{D}_{\{0\}, \{1, 2\}, \{0, 1, 2\}}$ is represented by the automaton



5.5 General Composition of Automata

In the case of cascade composition, we not only showed how to compose systems (Definition 14), but also how to compose automata (Definition 15). Also for general composition we will—in addition to composition of systems (Definition 16)—also introduce general composition of automata.

Let us first have a look at our example. The automaton implementing the original controller (left-hand side of Figure 5.2), did not yet have any input reacting to the output of the drone. Instead, it had a second internal state representing the fact that it is necessary for a mission to finish. Now we change the controller in such a way, that it reacts to the output of the drone instead. The result can be seen in Figure 5.12.

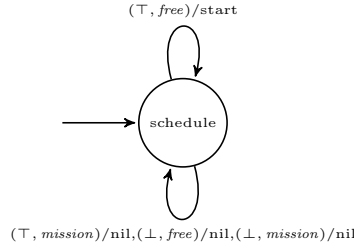


Figure 5.12: Controller with Additional Input

Our goal is now to create one single automaton, that behaves in the same way as the whole feedback loop. Figure 5.13 contains four automata, each corresponding to one network element. We have seen the two automata on the top already before, and at the bottom, the figure shows an automaton corresponding to the delay and an automaton corresponding to the tee. For computing the composition, we can use the fact that the cycle contains a delay, for which the output of each transition only depends on its current state, but *not* on the current input of the delay. Hence, we can compute the output of the delay without knowing its input, use this output as an input to the controller to compute for every four-tuple of current states of the individual automata their next states.

The resulting transition relation can be seen in the following table:

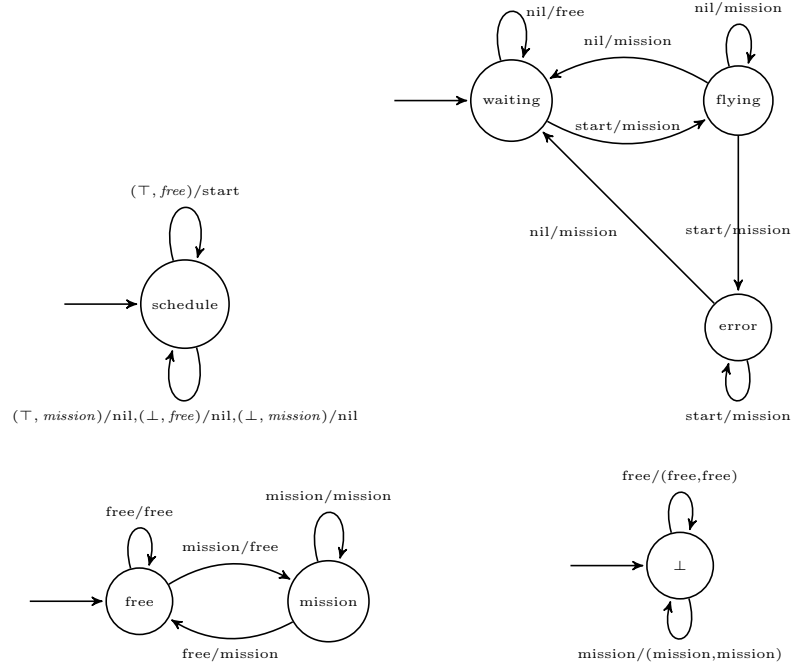


Figure 5.13: Automata Constituting Network

i	s	s'	o
$\perp$	(free, schedule, waiting, $\perp$)	(free, schedule, waiting, $\perp$)	free
T	(free, schedule, waiting, $\perp$)	(mission, schedule, flying, $\perp$),	mission
$\perp$	(mission, schedule, waiting, $\perp$)	(free, schedule, waiting, $\perp$)	free
T	(mission, schedule, waiting, $\perp$)	(free, schedule, waiting, $\perp$),	free
$\perp$	(free, schedule, flying, $\perp$)	(mission, schedule, flying, $\perp$)	mission
$\perp$	(free, schedule, flying, $\perp$)	(mission, schedule, waiting, $\perp$)	mission
T	(free, schedule, flying, $\perp$)	(mission, schedule, error, $\perp$),	mission
$\perp$	(mission, schedule, flying, $\perp$)	(mission, schedule, flying, $\perp$)	mission
$\perp$	(mission, schedule, flying, $\perp$)	(mission, schedule, waiting, $\perp$)	mission
T	(mission, schedule, flying, $\perp$)	(mission, schedule, flying, $\perp$),	mission
T	(mission, schedule, flying, $\perp$)	(mission, schedule, waiting, $\perp$),	mission
$\perp$	(free, schedule, error, $\perp$)	(mission, schedule, waiting, $\perp$)	mission
T	(free, schedule, error, $\perp$)	(mission, schedule, error, $\perp$),	mission
$\perp$	(mission, schedule, error, $\perp$)	(mission, schedule, waiting, $\perp$)	mission
T	(mission, schedule, error, $\perp$)	(mission, schedule, waiting, $\perp$),	mission

This corresponds to the automaton in Figure 5.14. At first sight, it is difficult to understand, how this automaton behaves. Observe however, that only a few states are reachable from the initial state. After deleting the transitions that are not reachable, the situation is much easier to understand, as can be seen in Figure 5.15. Especially, it is now clear that the composed automaton cannot reach any error state—a state corresponding to an error state of the drone.

In general, general composition can be defined as follows:

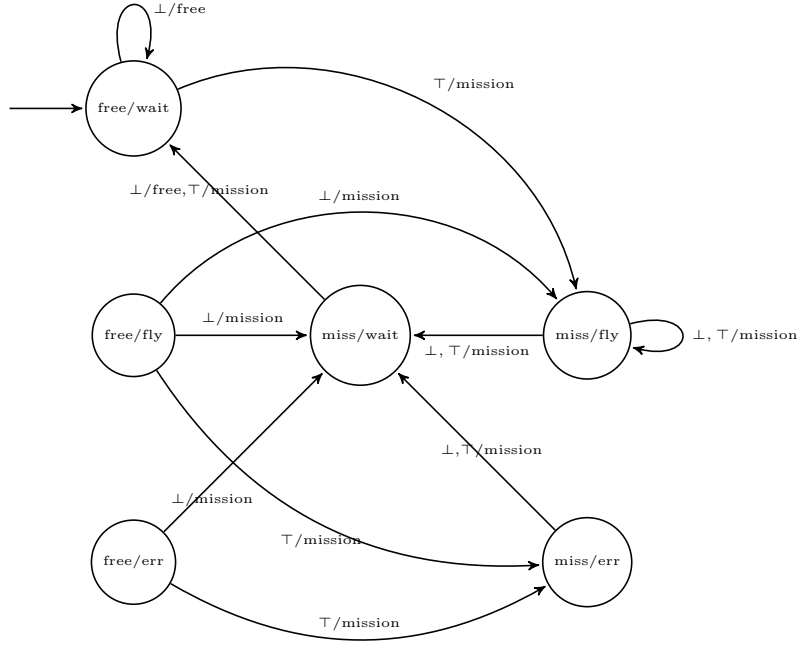


Figure 5.14: Composed Automaton

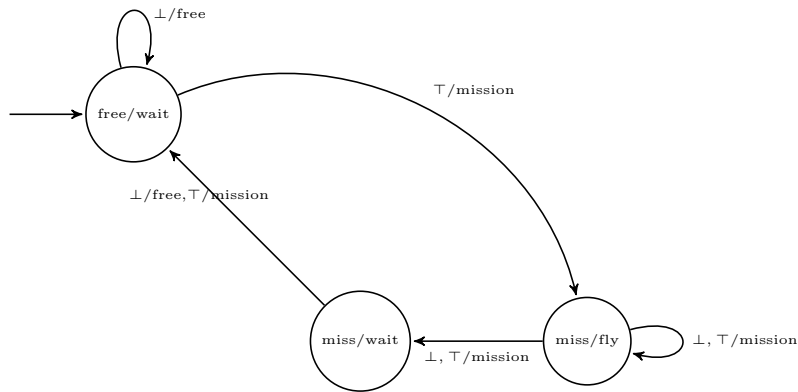


Figure 5.15: Reachable Transitions of Composed Automaton

Definition 18 For a given network of automata $(S^1, S_0^1, I^1, O^1, R^1), \dots, (S^n, S_0^n, I^n, O^n, R^n)$ their composition is $(S^1 \times \dots \times S^n, S_0^1 \times \dots \times S_0^n, I, O, T)$, where

- I : Cartesian product of input sets of input ports of the network
- O : Cartesian product of output sets of output ports of the network
- $(i_N, (s^1, \dots, s^n), (s'^1, \dots, s'^n), o_N) \in T$ iff there are component inputs $c_I^1, \dots, c_I^n$ and outputs $c_O^1, \dots, c_O^n$ s.t.
 - $(c_I^1, s^1, s'^1, c_O^1) \in R^1, \dots, (c_I^n, s^n, s'^n, c_O^n) \in R^n$, and
 - the symbols on ports belonging to the same connection are identical.

If the network contains a cycle without a delay, the constraints defining the composed transition relation also contain a cycle, which makes it difficult to compute the composed transition relation. However, having a delay in each cycle, avoids this problem, since the output of a delay does not depend on its current input (only on its previous input).

The question is whether composition preserves left-totality of the transition relation which is necessary for making the resulting automata well-formed. We have: If

- the transition relation of each component automaton is left-total, and
- for each connection from automaton i to automaton j , $I_i \subseteq O_j$

then the transition relation of the composed automaton is again left-total.

5.6 Special Cases

Note that general composition can also be used to combine independent components as can, for example, be seen in Figure 5.16. This is often also called synchronous parallel composition. Here, the word synchronous refers to the fact that the two components are synchronized in the sense that they take their respective steps at the same time. Also note that cascade composition is nothing

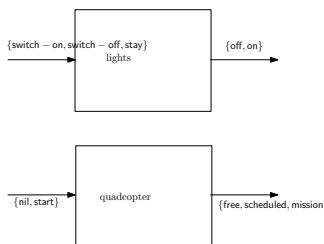


Figure 5.16: Parallel Composition

else than a special case of general composition.

5.7 Deconstructing Automata

In a similar way as for cascade composition (Definition 15), we could now come up with an analogous way of composing automata. However, instead we will think of a way of representing automata by composing even simpler elements. Recalling the definition of behavior of automaton, we realize that automata have two main tasks:

- Remember the current state
- Translate current input and state to output and next state

For the first task, we can use the delay element that we just introduced. For the second task we can use the following:

Definition 19 For a certain relation $R \subseteq I \times O$, the table lookup in R (which we will often denote by $\mathcal{L}_R$) is a system with input set I and output set O s.t.

$$(i, o) \in \mathcal{L}_R \text{ iff for all } k \in \mathbb{N}_0, (i(k), o(k)) \in R$$

Examples:

- $R = \{(x, 2x) \mid x \in \mathbb{Z}\}$.
Then $((1, 2, 3, 4, \dots), (2, 4, 6, 8, \dots)) \in \mathcal{L}_R$
- $R = \{(1, a), (1, b)\}$.
Then $((1, 1, 1, \dots), (a, b, a, b, \dots)) \in \mathcal{L}_R$
- $R = \{(x, y, \max\{x, y\}) \mid x, y \in \mathbb{R}\}$
Then $((5, 5, 5, \dots), (2, 7, 3, 5, 8, \dots), (5, 7, 5, 5, 8, \dots)) \in \mathcal{L}_R$

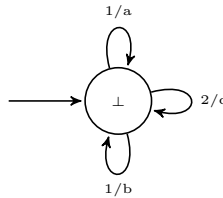
Clearly, table lookup is a memory-less system. Depending on the relation used, it can also be non-deterministic, and non-receptive.

Property 4 The automaton

$$(\{\perp\}, \{\perp\}, I, O, \{(i, \perp, \perp, o) \mid (i, o) \in R\})$$

represents the table lookup $\mathcal{L}_R$.

For example, the table lookup $\mathcal{L}_{\{(1,a),(1,b),(2,c)\}}$ is represented by the automaton



Using delay, table lookup, and general composition with loops, we can build arbitrary discrete time automata. The reason is, that using those elements one can build the feedback loop from Figure 5.17 that implements an arbitrary automaton.

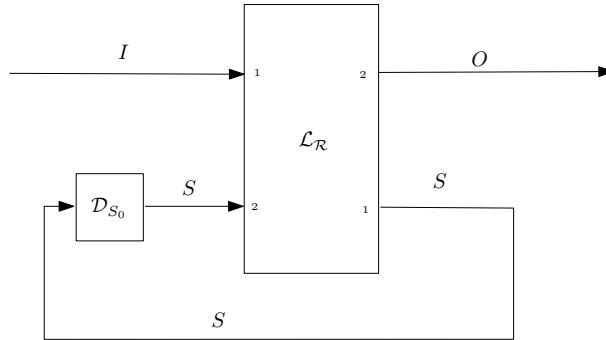


Figure 5.17: Automaton Implemented Using Feedback Loop

5.8 Conclusion

In this chapter we have seen how to compose systems and automata to create new, more complex, systems and automata. The reader might remember similar operations on automata in automata theory and theory of formal languages. However, the main motivation there is compiler construction. In compiler construction, automata represent languages (i.e., sets of strings). Operations on automata implement language operations (union, intersection etc.). In contrast to that, here operations model how systems interact in the real-world

In practice, synchronous reactive models are used in many applications. For example, IBM stream computing/InfoSphere Streams¹ is based on such principles. The original motivation for this product was gathering and analyzing security information from all across US after 9/11. But nowadays, it is used in various contexts for example, the online analysis for companies, transportation information, banking security, intelligent energy networks.

Various further tools used in industry are based on similar principles. In addition to delay and table lookup, they often provide whole libraries of further elements. Examples of such tools are:

- Matlab/Simulink/Stateflow
- Scilab/Xcos
- UML/SysML based tools
- ...

Especially, synchronous reactive programming languages extend this to full programming languages (e.g., Scade/Lustre)

The synchronous reactive model of communication by far is not the only one that is available. Communication can also be asynchronous, and there are many further possibilities. Examples of further models of communication are.

- actors
- dataflow

¹<http://www-01.ibm.com/software/data/infosphere/stream-computing/overview.html>

- process networks, process algebras
- functional reactive programming

Chapter 6

Temporal Logic

6.1 Introduction

In the previous chapter, we saw that even the composition of very simple systems/automata can show highly complex behavior. For example, the behavior of Figure 5.14 is not easy to understand at first sight, although it is the result of the composition of extremely simple automata. In general, the composition of n automata of each k states results in an automaton with k^n states. So the number of states of the resulting automaton grows exponentially in the number of components. It may be very difficult to see whether the resulting model behaves as wanted. In this chapter we will introduce a way of precisely specifying how such a system should behave. Later we will then discuss computer algorithms that can automatically check whether a given automaton satisfies its specification.

Actually, we have already seen a possibility of specifying system behavior, Example 1 and its formalized version in Example 3 define a certain system, let us say S . For every automaton A that should implement this system, we can ask the question, whether every behavior of the implementing automaton A is a behavior of the specifying system S . This is equivalent to asking whether the system $\llbracket A \rrbracket$ represented by the automaton is a refinement of the system S , formally $\llbracket A \rrbracket \subseteq S$.

Using general system as specifications, however, has a big disadvantage: If we write the specifying system in natural language, as in Example 1, it may lack in precision and be difficult to understand by computers. If we write it in the first-order predicate language, as in Example 3, the specification is precise, but the language is too general to allow efficient handling by computer algorithms. Hence, we will introduce a language that is more restrictive, but can precisely define the temporal behavior of systems.

Here are some examples of sentences that one might want to formalize:

- “The reactor is not going to overheat”.
- “If the elevator is called, it will show up eventually”.
- “Tomorrow the weather will be nice”.
- “Central locking of a car opens immediately after a crash”.

- “The **airbag** **only** **inflates** **if** a **car crash happens**”.
- “The **server acknowledge** has to be **preceded** by a **request**”.

All of this can be formalized in predicate logic. However, predicate logic is too general, and hence often difficult to handle for both people and computers. For coming up with a more specific, but still precise language, let us analyze the above examples in more detail. It can be seen that there are two specific ways the sentences specify behavior:

- **Properties**
- **Temporal specification**

Here we use color to highlight the corresponding parts of the sentence above. Since the first of these two items is already present in classical input/output specifications of algorithms, we will not discuss it further here, and concentrate on temporal specification, instead.

Here we will also want to combine several such temporal specifications and properties, as in the example “**every time** the **drone** will be **ordered**, **it will show up eventually** and **immediately afterwards** **hand over a pizza**”. We will achieve this by working with a recursively defined language, as we will see later.

6.2 Basic Automata Model

For discussing specification of temporal properties of systems, the input/output structure of systems is irrelevant, and hence we will work with an automata model that is as simple as possible:

Definition 20 A transition system is a triple (S, S_0, R) whose elements have the following roles:

- a set of states (state space) S
- a non-empty set $S_0 \subseteq S$ of initial states
- a transition relation $R \subseteq S \times S$ s.t. for all $s \in S$ there exists $s' \in S$ s.t. $(s, s') \in R$.

Note that the set of states S can even be an infinite set (e.g., $\mathbb{N}$, $\mathbb{R}$, lists of integers). If it is finite, we can visualize the transition systems using graphs, in a similar way as for automata. See Figure 6.1 for an example.

Definition 21 A path of a transition system (S, S_0, R) is a sequence $(s_0, s_1, \dots) \in \Sigma_S$, for which $s_0 \in S_0$, $(s_0, s_1) \in R$, $(s_1, s_2) \in R$, ...

We will use the following notion for denoting paths:

$$s_0 \rightarrow s_1 \rightarrow \dots$$

Note that one transition system can, in general, have many paths.

As an example of a transition system with an infinite set of states, we will show that how a computer program can be viewed as a transition system:

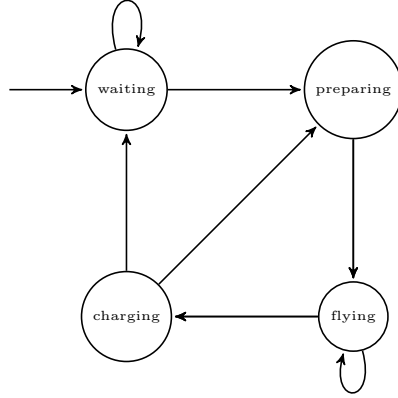


Figure 6.1: Transition System

```

i, k ∈ ℕ; r ∈ {prime, nonprime}
1: r ← prime
2: i ← 2
3: while i < k and r = prime do
4:   if i divides k then r ← nonprime
5:   i ← i + 1
6: done

```

This can be translated to a transition systems, where the state space is formed by program counter and the types of all variables, and where the transitions are given by the above program. So we have a transition system (S, S_0, R) , where

- $S = \{1, \dots, 6\} \times \mathbb{N} \times \mathbb{N} \times \{\text{prime}, \text{nonprime}\}$
- $S_0 = \{(1, i, k, r) \mid i \in \mathbb{N}, k \in \mathbb{N}, r \in \{\text{prime}, \text{nonprime}\}\}$
- $R = R_{1,2} \cup R_{2,3} \cup R_{3,4} \cup R_{3,6} \cup R_{4,5} \cup R_{5,3}$ where
 - $R_{1,2} = \{((1, i, k, r), (2, i', k', r')) \mid i' = i \wedge k' = k \wedge r' = \text{prime}\}$
 - $R_{2,3} = \{((2, i, k, r), (3, i', k', r')) \mid i' = 2 \wedge k' = k \wedge r' = r\}$
 - $R_{3,4} = \{((3, i, k, r), (4, i', k', r')) \mid i' = i \wedge k' = k \wedge r' = r \wedge i < k \wedge r = \text{prime}\}$
 - $R_{3,6} = \{((3, i, k, r), (6, i', k', r')) \mid i' = i \wedge k' = k \wedge r' = r \wedge \neg(i < k \wedge r = \text{prime})\}$
 - ...

6.3 Specifying Temporal Properties

We will now explain, how temporal properties of transition systems can be specified using the language of the temporal logic LTL. We will introduce the

language and its semantics in the following steps:

1. properties of states (i.e., no time)
2. properties of paths (i.e. temporal specification)
3. properties of transition systems (i.e., several paths)

When specifying properties of individual states, time does not play any role. Here are some examples:

- The state space of the transition system given in Figure 6.1 is the set $\{\text{waiting, preparing, flying, charging}\}$. The fact, that the quadcopter is on the ground can be described by the set $\{\text{waiting, preparing, charging}\}$.
- The state space of the transition system might be $S = \mathbb{R}$, modeling temperature. Then the fact that the system overheats might be given by the set $\{T \in S \mid T > 800\}$.
- For the state space $S = \{\text{rain, sunshine}\} \times \mathbb{R}$, the fact that weather is nice, might be specified by the set $\{(w, T) \in S \mid w = \text{sunshine} \wedge T \geq 18 \wedge T \leq 26\}$.
- For the state space $S = \mathbb{N}$, the fact that a number is not prime might be specified by the set $\{x \in S \mid \exists p \exists q \in \mathbb{N} : p \geq 2 \wedge p < x \wedge q \geq 2 \wedge q < x \wedge pq = x\}$.

So we describe each such property by a subset of the whole state space S for which it holds. We assume a function $\mathcal{I}$ (*interpretation*), that assigns to each *state property*, as denoted by a certain name, a set of states. For example, we might denote the second property above by the name **overheat**, and define $\mathcal{I}(\text{overheat}) = \{T \in S \mid T > 800\}$.

Definition 22 A state property p holds on a state s ($s \models p$) iff $s \in \mathcal{I}(p)$

For example, $922 \models \text{overheat}$ iff $922 \in \mathcal{I}(\text{overheat})$ iff $922 \in \{T \in S \mid T > 800\}$ iff $922 > 800$. So the property holds for the state 922.

Now we continue with describing properties of paths. For this we need the following notation:

For a path π of the form $(s_0, s_1, \dots)$, we denote by

- π^i (the i -th *suffix* of π), the path $(s_i, s_{i+1}, \dots)$, and by
- $\pi(i)$ the element s_i .

For understanding how properties of whole paths can be specified, we will now analyze examples of natural language sentences and try to formalize them:

“The weather is nice”

This property states that the weather is nice *now*. When talking about paths, this is the first element of the path. Hence a state property p will hold on a path π (we will denote this by $\pi \models p$) iff it holds on first element of the path, that is, $\pi(0) \models p$. Note that here the second usage of the symbol $\models$ follows Definition 22, while the first usage is something new that we will formally define below.

Here is an example:

$$\begin{aligned} &(\text{flying}, \text{charging}, \text{waiting}, \text{waiting}, \text{waiting}, \dots) \models \text{onground} \\ &(\text{flying}, \text{charging}, \text{waiting}, \text{waiting}, \text{waiting}, \dots)(0) \models \text{onground} \\ &\text{flying} \models \text{onground} \\ &\text{flying} \in \mathcal{I}(\text{onground}) \\ &\text{flying} \in \{\text{waiting}, \text{preparing}, \text{charging}\} \end{aligned}$$

Of course, the final line does not hold and hence the property does not hold. The usage of the symbol $\models$ on the first line has a path on its left-hand side, but further usages have a state on their left-hand side, and hence refer to Definition 22.

The sentence

“Tomorrow the weather will be nice”

specifies what will happen at the next step. In terms of paths, we will define this in such a way that the property holds on the first suffix of the path: $\pi^1 \models p$. We will denote this by $\pi \models \mathbf{X}p$, where $\mathbf{X}$ stands for “next”. Note that here we do *not* define this as $\pi(1) \models p$! The reason is that later, we will allow p to stand for more complex properties, not only state properties. When p is a state property, $\pi^1 \models p$ is defined as $\pi^1(0) \models p$ due to the previous situation, and $\pi^1(0) \models p$ is indeed $\pi(1) \models p$.

The sentence

“The train eventually reaches full speed”

specifies that at some point in the future the property holds. In terms of paths, this means that there exists $k \geq 0$ s.t. $\pi^k \models p$. We will denote this by $\pi \models \mathbf{F}p$, where $\mathbf{F}$ stands for “future”.

The sentence

“The number of motor rotations always stays in safe area”

says that at *all* points in time the property holds. In terms of paths, this means that for all $k \geq 0$, $\pi^k \models p$. We will denote this by $\pi \models \mathbf{G}p$, where $\mathbf{G}$ stands for “globally”.

We will use two more operators, as illustrated by the two following examples:

“The train eventually stops and until then the doors remain closed”

there exists i s.t.

$$\begin{aligned} &\pi^i \models q, \text{ and} \\ &\text{for all } j < i, \pi^j \models p \end{aligned} \quad (\pi \models p\mathbf{U}q: \text{“until”})$$

*“As long as the plane does not reach full height
the fasten seat belts sign is on”*

$$\text{for all } j \geq 0, \left[\begin{array}{l} \text{if for all } i < j, \pi^i \not\models p \\ \text{then } \pi^j \models q \end{array} \right] \quad (\pi \models p\mathbf{R}q: \text{“release”})$$

The elements that we have seen up to now, can also be combined:

“The train will never move with open doors”

$$\mathbf{G}\neg[p \wedge q]$$

“Whenever the elevator is called, it will eventually show up”

$$\mathbf{G}[p \Rightarrow \mathbf{F}q]$$

So, in addition to the temporal operators introduced above, one can also use the Boolean connectives $\wedge$, $\vee$, $\neg$, $\Rightarrow$. One can even recursively combine the temporal operators themselves. For example:

- $\mathbf{FG}p$: Eventually the property p will hold forever.
- $\mathbf{GF}p$: Always eventually p will hold.

Whenever I come to the station, soon or later a train will come

The result of combine all of this is the temporal logic *Linear Temporal Logic* (*LTL*). Summarizing, formulas will have the following syntax:

Definition 23

- Every state property is an LTL formula
- If ϕ and ψ are LTL formulas then also $\mathbf{X}\phi$, $\phi\mathbf{U}\psi$, $\phi\mathbf{R}\psi$, $\mathbf{F}\phi$, $\mathbf{G}\phi$, and $\phi\mathbf{U}\psi$, $\neg\phi$, $\phi \vee \psi$, $\phi \wedge \psi$, are LTL formulas.

Here, the priority of operators is as follows:

- $\neg$, $\mathbf{X}$, $\mathbf{F}$, $\mathbf{G}$
- $\mathbf{U}$, $\mathbf{R}$
- $\wedge$, $\vee$, $\Rightarrow$, $\Leftrightarrow$

A remaining question is whether to read $\phi\mathbf{U}\psi\mathbf{U}\chi$ as $[\phi\mathbf{U}\psi]\mathbf{U}\chi$ or as $\phi\mathbf{U}[\psi\mathbf{U}\chi]$. In the literature, usually the second possibility is used, that is, the operators $\mathbf{U}$ and $\mathbf{R}$ are usually used in a right-associative way. But there are exceptions. For example, the language PROMELA uses them in a left-associative way.

The definition of the semantics of formulas follows the examples above, and combines them in a recursive way:

Definition 24 For a path π LTL formulas ϕ , ψ , and state property p ,

- $\pi \models p$ iff $\pi(0) \models p$
- $\pi \models \mathbf{X}\phi$ iff $\pi^1 \models \phi$
- $\pi \models \mathbf{F}\phi$ iff there exists k s.t. $\pi^k \models \phi$
- $\pi \models \mathbf{G}\phi$ iff for all k , $\pi^k \models \phi$
- $\pi \models \phi\mathbf{U}\psi$ iff there exists i s.t. $\pi^i \models \psi$ and for all $j < i$, $\pi^j \models \phi$.
- $\pi \models \phi\mathbf{R}\psi$ iff for all j , if for all $i < j$, $\pi^i \not\models \phi$ then $\pi^j \models \psi$
- $\pi \models \neg\phi$ iff not $\pi \models \phi$
- $\pi \models \phi \wedge \psi$ iff $\pi \models \phi$ and $\pi \models \psi$

- $\pi \models \phi \vee \psi$ iff $\pi \models \phi$ or $\pi \models \psi$

Here, the variables i , j , and k range over $\mathbb{N}_0$. The base case of the recursion is the first item. Note that, on the right-hand side of this item the symbol $\models$ is not used recursively, but refers to Definition 22.

Consider the following example: Assume a path π of the form $(a, b, c, d, a, b, c, d, \dots) \in \Sigma_{\{a,b,c,d\}}$, and a state property $\mathbf{p}$ with interpretation $\mathcal{I}(\mathbf{p}) = \{c, d\}$. We want to check whether $\pi \models \mathbf{Fp}$. For this we expand the expression by applying one definition after the other:

- there exists k s.t. $\pi^k \models \mathbf{p}$
- there exists k s.t. $\pi^k(0) \models \mathbf{p}$
- there exists k s.t. $\pi^k(0) \in \mathcal{I}(\mathbf{p})$ there exists k s.t. $\pi^k(0) \in \{c, d\}$
- there exists k s.t. $(a, b, c, d, a, b, c, d, \dots)^k(0) \in \{c, d\}$
- there exists k s.t. $(a, b, c, d, a, b, c, d, \dots)^k(0) \in \{c, d\}$

Now the outermost symbol is an existential quantifier. To check whether it holds, we try various choices for k :

- First attempt $k = 0$:
 $(a, b, c, d, a, b, c, d, \dots)^0(0) \in \{c, d\}$
 $(a, b, c, d, a, b, c, d, \dots)(0) \in \{c, d\}$
 $a \in \{c, d\}$
 $\perp$
- Second attempt $k = 1$:
 $(a, b, c, d, a, b, c, d, \dots)^1(0) \in \{c, d\}$
 $(b, c, d, a, b, c, d, \dots)(0) \models \{c, d\}$
 $b \in \{c, d\}$
 $\perp$
- Third attempt $k = 2$:
 $(a, b, c, d, a, b, c, d, \dots)^2(0) \in \{c, d\}$
 $(c, d, a, b, c, d, \dots)(0) \in \{c, d\}$
 $c \in \{c, d\}$
 $\top$

So we can conclude that indeed $\pi \models \mathbf{Fp}$!

A further example combines two operators. We want to check whether $\pi \models \mathbf{GFp}$. From the definition we know: $\pi \models \mathbf{GFp}$ iff for all k , $\pi^k \models \mathbf{p}$. Hence the original question is equivalent to the question whether for all k , $\pi^k \models \mathbf{Fp}$.

From the definition we also know : $\pi \models \mathbf{Fp}$ iff there exists $k \geq 0$ s.t. $\pi^k \models \mathbf{p}$. Since here also the variable k appears, in order to avoid a name clash, we rename the variable to l , resulting in for all l , $\pi^l \models \mathbf{Fp}$. Using this definition we arrive at the question whether

- for all l ,
there exists k s.t. $(\pi^l)^k \models \mathbf{p}$

This is equivalent to:

for all l ,
there exists k s.t. $\pi^{l+k} \models p$

We can now check this against whatever path π we are looking at.

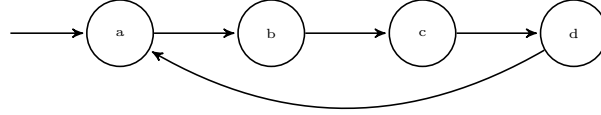
Now finally we can extend our definition to full transition systems:

Definition 25 For a given transition system (S, S_0, R) , $(S, S_0, R) \models \phi$ iff for all paths π of the transition system (S, S_0, R) ,

$$\pi \models \phi.$$

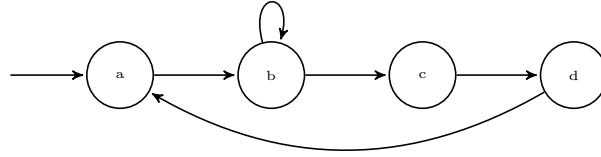
If the transition system is clear from the context, one often write just $\models \phi$ instead of $(S, S_0, R) \models \phi$.

For illustrating the definition, consider the transition system



which has only one path: $(a, b, c, d, a, b, c, d, \dots)$. For checking whether $\mathbf{F}p$, with $\mathcal{I}(p) = \{d\}$ holds on this transition system, we have to check just this single path. Clearly, $\mathbf{F}p$ holds.

However, after adding a transition from b to b , arriving at



the transition system has more (even infinitely many paths, for example $(a, b, b, b, \dots)$ or $a, b, c, d, a, b, b, b, \dots$). The formula $\mathbf{F}p$ holds iff it holds on *all* paths. Clearly this is not the case.

From Definition 25 it also clear that $\models \neg\phi$ is not equivalent to $\not\models \phi$. The first formula expresses that on all paths, ϕ does not hold, while the second formula expresses that not for all paths it is the case that ϕ holds, which is equivalent to the existence of a path on which ϕ does not hold. So $\models \neg\phi$ is a stronger statement than $\not\models \phi$.

6.4 Properties of LTL

It is not difficult to see that certain formulas in LTL are equivalent to others. For example, for arbitrary LTL formulas ϕ and ψ ,

- $\neg\mathbf{G}\phi$ is equivalent to $\mathbf{F}\neg\phi$
- $\neg\mathbf{F}\phi$ is equivalent to $\mathbf{G}\neg\phi$
- $\neg\mathbf{X}\phi$ is equivalent to $\mathbf{X}\neg\phi$
- $\mathbf{G}\phi$ is equivalent to $\phi \wedge \mathbf{XG}\phi$

- $\mathbf{F}\phi$ is equivalent to $\phi \vee \mathbf{X}\mathbf{F}\phi$
- $\mathbf{G}\mathbf{G}\phi$ is equivalent to $\mathbf{G}\phi$
- $\mathbf{F}\mathbf{F}\phi$ is equivalent to $\mathbf{F}\phi$
- $\mathbf{F}\mathbf{G}\mathbf{F}\phi$ is equivalent to $\mathbf{G}\mathbf{F}\phi$
- $\mathbf{G}\mathbf{F}\mathbf{G}\phi$ is equivalent to $\mathbf{F}\mathbf{G}\phi$
- $\mathbf{F}[\phi \vee \psi]$ is equivalent to $\mathbf{F}\phi \vee \mathbf{F}\psi$
- $\mathbf{G}[\phi \wedge \psi]$ is equivalent to $\mathbf{G}\phi \wedge \mathbf{G}\psi$
- $\mathbf{F}\phi$ is equivalent to $\top \mathbf{U}\phi$
- $\mathbf{G}\phi$ is equivalent to $\perp \mathbf{R}\phi$
- $\neg[\phi \mathbf{U}\psi]$ is equivalent to $\neg\phi \mathbf{R}\neg\psi$
- $\neg[\phi \mathbf{R}\psi]$ is equivalent to $\neg\phi \mathbf{U}\neg\psi$

Due to such equivalences, a subset of all operators suffices to express all LTL formulas:

Theorem 1 *Every LTL formula can be expressed in terms of $\vee, \neg, \mathbf{X}, \mathbf{U}$ only.*

Proof.

- $\phi \wedge \psi \equiv \neg[\neg\phi \vee \neg\psi]$
- $\mathbf{F}\phi \equiv \top \mathbf{U}\phi$
- $\mathbf{G}\phi \equiv \neg\mathbf{F}\neg\phi$
- $\phi \mathbf{R}\psi \equiv \neg[\neg\phi \mathbf{U}\neg\psi]$

■

6.5 Counter-Examples

Recall that $(S, S_0, R) \models \phi$ holds iff for all paths π of (S, S_0, R) , $\pi \models \phi$. If however, $(S, S_0, R) \models \phi$ does not hold, we would like to have some information for debugging:

Definition 26 *A path π of a transition system (S, S_0, R) s.t. $\pi \not\models \phi$ is called counter-example of the formula ϕ for the transition system (s, s_0, r) .*

For finding counter-examples one can use the fact that $\pi \not\models \phi$ is equivalent to $\pi \models \neg\phi$. Note that this just speaks about *one* path, unlike the case of a property of a transition system where the symbol $\models$ means that the property holds *for all* paths which does *not* commute with negation.

Now consider the system in Figure 6.2 and the property $\mathbf{p} \mathbf{R} \mathbf{q}$, where $\mathcal{I}(\mathbf{p}) = \{c\}$, $\mathcal{I}(\mathbf{q}) = \{a\}$. For checking whether this property holds, we first extend the

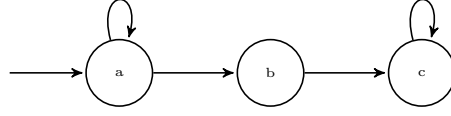


Figure 6.2: Transition System

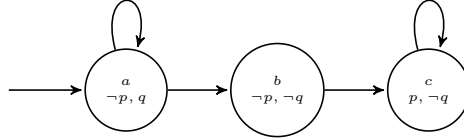


Figure 6.3: Transition System with State Properties

figure by labeling each state with information about which state property holds in the state. The result can be seen in Figure 6.3

The definition says that $\mathbf{p R q}$ holds for a transition system, if it holds for all paths of the transition system. Looking up the definition of the temporal operator $\mathbf{R}$ we see that we have to check whether

for every path π
for all j , [if for all $i < j$, $\pi^i \not\models \mathbf{p}$, then $\pi^j \models \mathbf{q}$].

Let us analyze some example paths. One path of the transition system is $(a, a, a, \dots)$. Looking at the label of state a , we see that the state property $\mathbf{p}$ never holds in a . So, independent of the value of j , the left-hand of the implication, “if for all $i < j$, $\pi^i \not\models \mathbf{p}$ ” always holds. Hence the right-hand side $\pi^j \models \mathbf{q}$ has to hold, which it indeed does, as long as we stay in state a .

Now we analyze the path $(a, b, c, \dots)$. Here the state b looks problematic. Before reaching it, $\neg \mathbf{p}$ always holds, but $\mathbf{q}$ does *not* hold. Hence this path is a counter-example.

Instead of trying to check whether the property holds, one can also, alternatively, directly search for a counter-example, that is, a path π , for which $\pi \not\models \mathbf{p R q}$, which means that $\pi \models \neg[\mathbf{p R q}]$. Using the equivalence rules from above it follows that this is equivalent to $\pi \models \neg \mathbf{p U} \neg \mathbf{q}$. Again we can apply the definition, which says that this is equivalent to

there exists i s.t. $\pi^i \models \neg \mathbf{q}$ and for all $j < i$, $\pi^j \models \neg \mathbf{p}$.

We immediately see that this holds for $(a, b, c, \dots)$ and hence this path is a counter-example to the original property $\mathbf{p R q}$.

6.6 Application of Temporal Logic

We use a discrete time mode. Here, one time step might correspond, for example, to

- a fixed time unit (e.g., clock cycle, 1sec)
- events (e.g., key pressed)

In many applications this is useful, but in other applications a more flexible time model is needed. Also, there are temporal logics that allow specification of different temporal properties (CTL, CTL*,...).

All of these logics are defined using first-order predicate logic expressions over paths. Why not directly use those expressions instead of LTL formulas?

- By using temporal logic one has to understand only a few basic operators, and one can then form complex properties by combining them.
- Efficient algorithms (“model checking”) for automatic checking temporal logic properties on finite state systems.

LTL is the basis for various specification languages that are used in industry, for example, the Property Specification Language (PSL), and the SystemVerilog Assertion Language (SVA). Most specification languages also allow inputs and outputs which can be simply added to LTL by allowing state properties to also refer to such inputs and outputs.

Finally, we compare our definition of LTL with the literature: We give state properties a certain interpretation $\mathcal{I}$. In the literature (e.g., [14]), often predicates are added to transition system (instead of being handled by logic). The literature calls the result “*Kripke structure*”.

Also, the literature often uses the following alternative notation:

- $\bigcirc$: **X**
- $\diamond$: **F**
- $\square$: **G**

Chapter 7

Unbounded Model Checking

7.1 The Model Checking Problem

We already observed that systems usually consist of many components that we can again consider as systems. After creating such a model, we want to understand its behavior. For example, we might want to know whether it can reach an error state or—on the contrary—always stays in states that are OK, as formalized by the LTL formula $\mathbf{Gok}$.

For systems consisting of just two simple automata, one can directly answer the question by looking at the state graph. However, in practice we have hundreds, or even thousands of interacting components, and when trying to represent the state graph on a computer, it might not even fit into memory.

In general, we want to check whether a system model represented by a transition system fulfills its specification given as a temporal logical formula. For example, we might have given the transition system from Figure 7.1, and want to check whether $\models \mathbf{Fp}$, where $\mathcal{I}(\mathbf{p}) = \{d\}$. In order to do this, we have

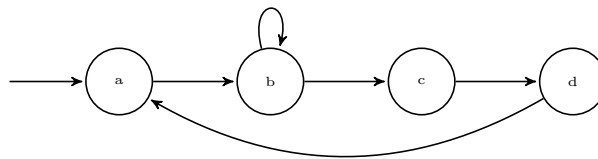


Figure 7.1: Transition System

to look up the corresponding definition, which is Definition 25. So we have to check, whether for all paths π of the transition system, $\pi \models \mathbf{Fp}$. If $\not\models \mathbf{Fp}$, of course, we will want to understand why. For this, we will want to know a *counter-example*, that is, a path π s.t. $\pi \not\models \mathbf{Fp}$. However, also in the case where $\models \mathbf{Fp}$ it will be very useful to receive some information that demonstrates that this is the case. Such an object is called a *certificate*. We will later see, how such a certificate may look like.

To summarize, we have the following so-called *model checking problem*:

- Given: a transition system and an LTL formula ϕ
- Output:
 - yes, and ideally, in addition, a certificate,
if $\models \phi$ (i.e., for all paths π , $\pi \models \phi$),
 - no, and ideally, in addition, a counter-example (a path π s.t. $\pi \not\models \phi$),
otherwise.

The difficulty here is, that a transition system may have infinitely many paths of infinite length. In the case of the example above, we would have to check, whether for all paths π of the transition system, $\pi \models \mathbf{Fp}$. So we would have to check this for π being equal to

$(a, b, c, d, a, b, c, d, a, \dots)$
 $(a, b, b, c, d, a, b, c, d, a, \dots)$
 $(a, b, b, b, c, d, a, b, c, d, a, \dots)$
 $(a, b, b, b, b, c, d, a, b, c, d, a, \dots)$
 $(a, b, b, b, b, b, c, d, a, b, c, d, a, \dots)$
 $\dots$

and many further paths.

Nonetheless we will try to solve this problem directly for a certain part of LTL. This is also called *unbounded model checking*. In the next chapter, we will instead replace the general problem by a simpler one, resulting in methods for *testing* and *bounded model checking*.

7.2 Model Checking G ok

7.2.1 Certificates

We will first consider the case of the property $\mathbf{G ok}$, where $\mathbf{ok}$ is an arbitrary state property. In practice, the state property $\mathbf{ok}$ usually models the set of states of a systems that do not correspond to a bug. The property $\mathbf{G ok}$ then expresses that we always want to stay in the set of states that are not buggy. Checking this property is called *safety verification*. Assume a transition system (S, S_0, R) with the graphical representation from Figure 7.2, and $\mathcal{I}(\mathbf{ok}) = \{A, B, C, D\}$.

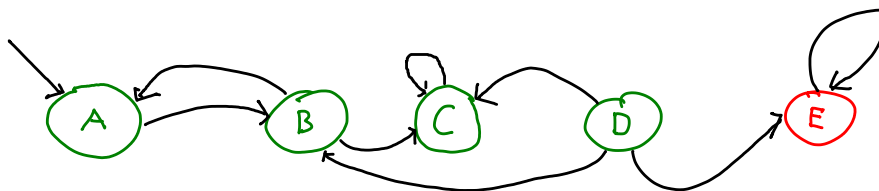


Figure 7.2: Transition System

This system, of course, fulfills this property, that is, it is safe. Now assume that you want to convince somebody else that this system is safe. For doing

so, you will probably explain to this person how to follow the transitions of the system to realize that the state E is not reachable, which means that the system will always stay in the set of safe states $\{A, B, C, D\}$. Now try to do the same thing for the transition system in Figure 7.3. This system has one initial state (in blue), and two states (in red) that are not safe. However, this system is so big, that one cannot even see the arrows of the individual transitions! So you will certainly not ask the person to follow the transitions, as before. Instead, everybody sees immediately, that it is not possible to reach the red states from the blue state! The reason is, of course that the graph consists of two separate clusters (connected components in the terminology of graph theory) with the cluster containing the initial state containing only safe states.

Indeed, Figure 7.4 shows that the same reasoning can even be used when the two clusters are connected, but with the transitions connecting them only leading *into* the cluster containing the initial state, and no transition leading out of this cluster. To sum up, to convince somebody else that a given transition system is safe, one just needs to demonstrate that the system contains such a cluster.

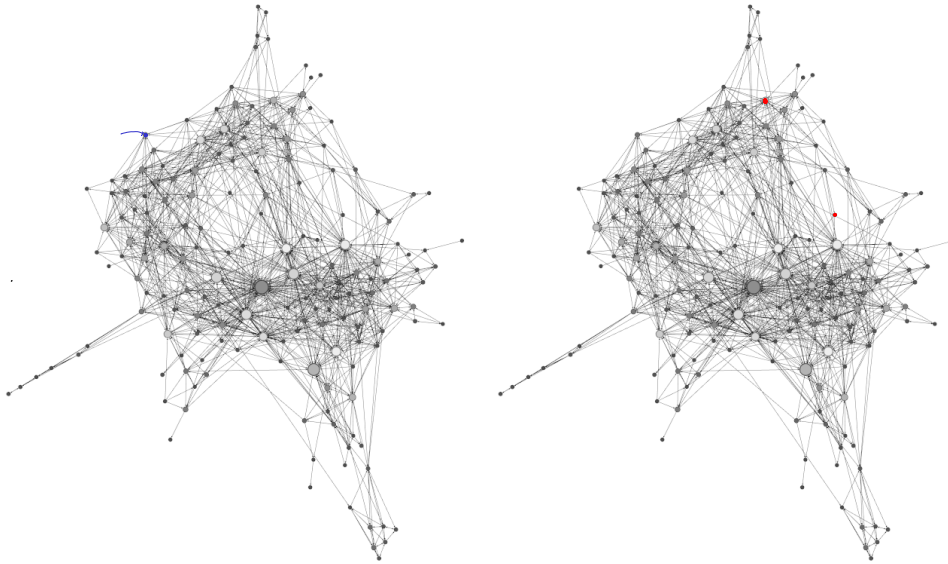


Figure 7.3: Big Transition System

The following definition makes this precise.

Definition 27 *A set of states V is a safety certificate of a transition system (S, S_0, R) and a set of safe states $\mathcal{I}(ok)$ iff*

- *V contains every initial state: $S_0 \subseteq V$*
- *No transition leads out of V : $Post_R(V) \subseteq V$ where $Post_R(V) := \{x' \mid x \in V, (x, x') \in R\}$*
- *V contains only safe (i.e., no unsafe) states: $V \subseteq \mathcal{I}(ok)$*

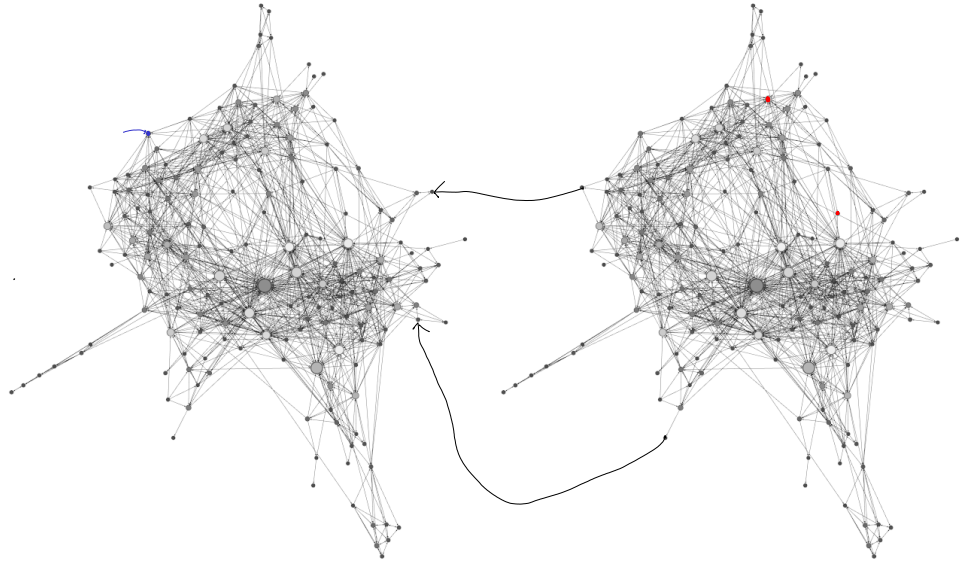


Figure 7.4: Big Transition System—Connected

For example, the set $\{A, B, C\}$ is a safety certificate for the transition system and safety property shown in Figure 7.2.

We call the three conditions from Definition 27 *safety verification conditions*, the first two conditions *inductivity conditions*, and a set V satisfying the inductivity conditions an *inductive invariant*. Of course, every safety certificate is an inductive invariant, and hence the set $\{A, B, C\}$ is also an inductive invariant for Figure 7.2. However, the set $\{A, B, C, D, E\}$ is also an inductive invariant, although it is certainly not a safety certificate.

Clearly, every inductive invariant is an invariant, that is, a set that contains all reachable states of a transition system. Adding the third condition ensures that all reachable states are safe. The set of all reachable states of a transition system is also called the *reach set* of the transition system.

It is important to have a good understanding of those terms. For example, one might ask whether every invariant is an inductive invariant. This question is the same as the question whether every set that contains all reachable states fulfills the inductivity conditions. The answer is no, as can be seen from the transition system in Figure 7.2 from the beginning of this chapter. The reach set of this transition system is $\{A, B, C\}$. The set $\{A, B, C, D\}$ is a superset of $\{A, B, C\}$ and hence an invariant. But due to the transition from D to E it is not inductive.

Another question is whether every transition system has an inductive invariant? Here the answer is yes: The reach set clearly is always an inductive invariant. The same holds for the set of all states of the given transition system. In general, these two inductive invariants are different, but they are the same if the all states of the transition system are reachable.

Figure 7.5 summarizes the relationship between the terms we just introduced.

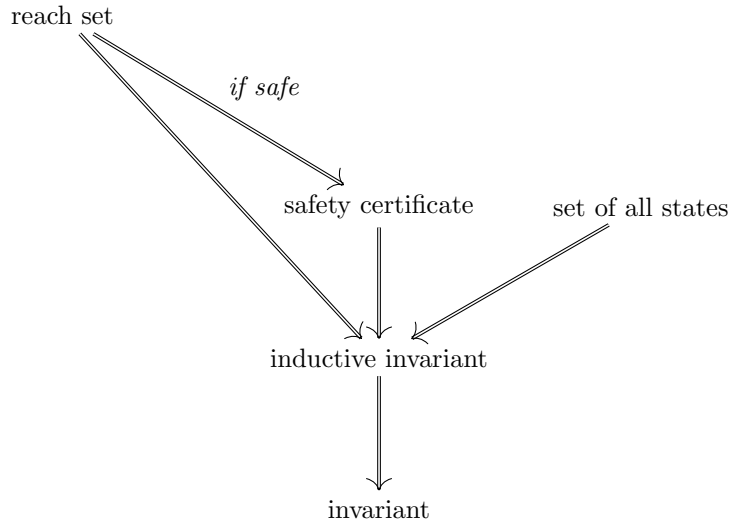


Figure 7.5: Relationship between Terms

7.2.2 Explicit State Model Checking

Now let us assume a safe transition system (S_0, S, R) and discuss how to compute a safety certificate for this system. For this, we view the transition system as a directed graph (S, R) and apply graph algorithms. For example, depth-first search easily detects that the system in Figure 7.6 is not safe. On the other

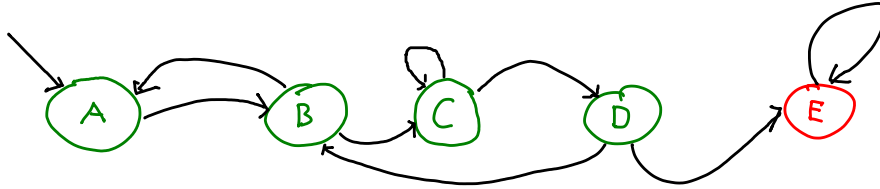


Figure 7.6: Unsafe System

hand, it will be unable to find the state E for the original system in Figure 7.2, thus declaring it safe.

For defining an algorithm based on this idea, we use the notation $\langle s_0, \dots, s_{n-1} \rangle$ to denote sequences (which we can, equivalently, view as lists). Moreover we need some computer representation of paths, since those are infinite objects.

Definition 28 A finite path of a transition system (S_0, S, R) is a sequence $\langle s_0, s_1, \dots, s_{n-1} \rangle \in \Sigma_S$, s.t. $s_0 \in S_0$, $(s_0, s_1) \in R$, $\dots$, $(s_{n-1}, s_{n-2}) \in R$.

Then, for a list of states l , $path(l)$ is a path that

- starts with the elements of l , and
- then continues arbitrarily according to the transition relation R .

```

search( $\langle \pi_0, \pi_1, \dots, \pi_{n-1} \rangle, V$ )
In:     $\langle \pi_0, \pi_1, \dots, \pi_{n-1} \rangle$ : a finite path of  $(S_0, S, R)$ 
         $V$  s.t.  $\{\pi_0, \pi_1, \dots, \pi_{n-2}\} \subseteq V \subseteq \mathcal{I}(\text{ok})$ ,  $\pi_{n-1} \notin V$ 
Out:  either a finite path  $\pi'$  s.t.  $\text{path}(\pi') \not\models \mathbf{G} \text{ ok}$ ,
        or a set of states  $V'$  s.t.
            •  $V \subseteq V' \subseteq \mathcal{I}(\text{ok})$ 
            •  $V'$  contains  $\pi_{n-1}$  and
              every state that can be reached from it
              without reaching a state in  $V$ 

if  $\pi_{n-1} \notin \mathcal{I}(\text{ok})$  then return  $\langle \pi_0, \pi_1, \dots, \pi_{n-1} \rangle$     // counter-example found
 $V \leftarrow V \cup \{\pi_{n-1}\}$ 
while there is a state  $s \in S$  s.t.  $(\pi_{n-1}, s) \in R$ ,  $s \notin V$  do
     $r \leftarrow \text{search}(\langle \pi_0, \pi_1, \dots, \pi_{n-1}, s \rangle, V)$ 
    if  $r$  is a path then return  $r$     // counter-example from recursive call
     $V \leftarrow r$ 
return  $V$ 

Initial call:
 $V \leftarrow \emptyset$ 
while  $V$  is not a path and there is an  $s \in S_0$ ,  $s \notin V$  do
     $V \leftarrow \text{search}(\langle s \rangle, V \cup \{s\})$ 

```

Figure 7.7: Safety Verification Based on Depth-First Search

When using this notation, the transition system defining the transition relation R will usually be clear from the context.

Figure 7.7 contains a resulting algorithm for safety verification based on depth-first search. The final value of the variable V is either a safety certificate or a finite path representing a counter-example.

The algorithm is an example of an *explicit state model checking* algorithm since it checks the correctness of a transition systems by explicitly going through all its individual states. The basis for this are classical graph algorithms, in this case, depth-first search. Thus, the algorithm inherits the basic properties of depth-first search. Especially,

- the resulting counter-examples tend to be long,
- the algorithm may get lost in irrelevant parts of graph, and it
- does not work for systems with infinitely many states.

Those problem can be avoided using breadth-first or best-first search¹, instead. However, this will result in different disadvantages, especially in high memory requirements, because now it is necessary to maintain a queue of states

¹an extension of breadth-first search that uses heuristics to choose the next graph vertex to visit

to inspect. Still, the use of best-first search (*directed model checking*) can be useful in many cases, especially when

- there are good heuristics, and when
- the goal is mainly to find counter-examples.

7.2.3 Symbolic Model Checking

Especially for systems with a huge number of states, the algorithms mentioned so-far often do not work well. Instead, one can apply algorithms that work on sets of states, using special representations of those sets. For introducing such algorithm let us have a look once more at the safety verification conditions:

- $S_0 \subseteq V$
- $Post_R(V) \subseteq V$
- $V \subseteq \mathcal{I}(\text{ok})$

The basic idea is to start with the set of initial states S_0 , to then continue to add addition states until the second condition holds, and finally, to check the third condition on the result. The result is the algorithm from Figure 7.8. If the resulting set V is a subset of $\mathcal{I}(\text{ok})$ then it fulfills the verification conditions.

```

V ← S0
while there is a transition  $(x, x')$  such that  $x \in V, x' \notin V$  do
    V ← V ∪ {x' | (x, x') ∈ R, x ∈ V} // V ∪ PostR(V)
return V

```

Figure 7.8:

Analyzing this algorithm in more detail, we see that the resulting set V is always an inductive invariant, and hence contains all reachable states. The question is whether it will also fulfill the third verification condition $V \subseteq \mathcal{I}(\text{ok})$ for every safe system? Observe that in the k -th cycle, V is the set of states reachable in k steps. Moreover, during algorithm execution V never contains unreachable states. So the algorithm computes the set of reachable states. Hence we conclude that if the property **Gp** holds, then the result of this algorithm will always be a safety certificate. In addition, it will always terminate for finite transition systems, since in each iteration it adds further states to the set V , and cannot do so infinitely often if the set of states S is finite.

However, in practice, convergence may be slow. In addition, for systems with infinitely many states, it does not necessarily terminate at all. And finally, the sets that are constructed during algorithm execution maybe quite complex, making their representation costly. Due to this, the modified algorithm from Figure 7.9 is often used. This algorithm always compute over-approximations of the state operations. So we have:

- If the algorithm terminates, then its result is an inductive invariant
- In the k -th cycle, V is a superset of the set of states reachable in k steps.

```

let  $V$  be a superset of  $S_0$ 
while there is a transition  $(x, x')$  such that  $x \in V, x' \notin V$  do
    let  $V$  be a superset of  $V \cup \{x' \mid (x, x') \in R, x \in V\}$ 
return  $V$ 

```

Figure 7.9: Set Based Model Checking **Gok**

So, in general, the result is a superset of the reach set.

Of course, when to decide how much to over-approximate in this algorithm, one needs to find a good compromise:

- Too much overapproximation might result in violation of $V \subseteq \mathcal{I}(\text{ok})$
- Too small overapproximation might result in termination problems (in the infinite state case) or the representation of the set V blowing up.

There are special data structures for compact set representation. Especially Binary Decision Diagrams (BDDs) do not represent each and every element explicitly, but use a symbolic representation, instead. Hence, such algorithms are usually called algorithms for *symbolic model checking*.

7.2.4 Backward Computation

Instead of a set that fulfills the safety verification conditions we can also compute its complement. For example, for the transition system in Figure 7.2 this is the set $\{D, E\}$. This is the set of states that leads to an unsafe state, the *backward reach set*. If the result does not contain any initial state, safety is proven.

For computing the backward reach set, one can simply use versions of the above algorithms that work in opposite direction, backwards from set of unsafe states to set of initial states. However, one does not need to re-implement those algorithms to apply them in backward direction! Instead, one can simply use the reversed transition system (S, S_0^-, R^-) where

- $S_0^- = S \setminus \mathcal{I}(\text{ok}) = \{s \mid s \models \neg \text{ok}\}$
- $R^- = \{(x', x) \mid (x, x') \in R\}$

For example, the transition system in Figure 7.10 is the reverse of the transition system of Figure 7.2.



Figure 7.10: Reversed Transition System

One can then check $\models \mathbf{G}$ on the original transition system, by checking $\models \mathbf{G} \neg \text{ok}^-$, where $\mathcal{I}(\text{ok}^-) = S_0$.

7.3 Model Checking $\mathbf{F}$ goal

7.3.1 Certificates

Now let us turn to the property $\mathbf{F}$ goal. Again, **goal** is an arbitrary state property. In practice, it usually models a set of states that we want to reach along all paths of the system.

How is it possible to convince somebody else that $\mathbf{F}$ goal holds for a given transition system? For example, for the transition system from Figure 7.11 it is clear that every path has to lead to state E . Due to this, the property $\mathbf{F}$ goal with $\mathcal{I}(\text{goal}) = \{E\}$ holds.

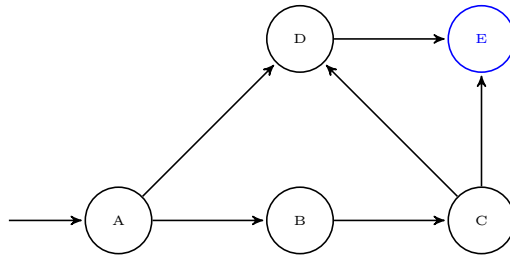


Figure 7.11: Transition System

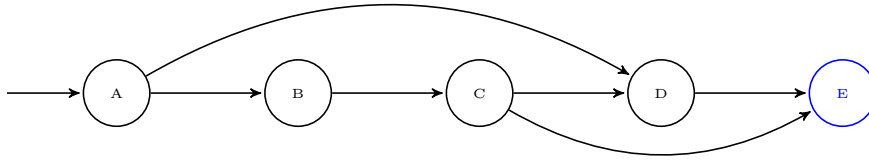


Figure 7.12: Transition System

The question of how to find a certificate for this becomes clearer, when changing the layout of the graph to the one in Figure 7.12. In this case, the situation is clear: For a system with a strict linear (total) order on all states, the absence of cycles forces the system to reach the final state wrt. this order. Such an order can be computed using topological sorting, and we can represent the resulting order using a list of states. In the case of our example, this is the list $\langle A, B, C, D, E \rangle$.

However, this does not always work. For example, the transition system in Figure 7.13 fulfills the property $\mathbf{F}$ goal where $\mathcal{I}(\text{goal}) = \{C, E\}$, but there does not exist any total order due to the loop cycling the states D, B, C . However, this loop does not violate our property because it involves the goal state C . Also consider the transition system in Figure 7.14 that contains further cycles. However, the only way to reach these cycles is via a goal state, and hence they do not violate the property $\mathbf{F}$ goal.

Hence we must weaken the requirement of a linear order on *all* states. Instead, it suffices to use such a requirement on a sub-set of states in such a way

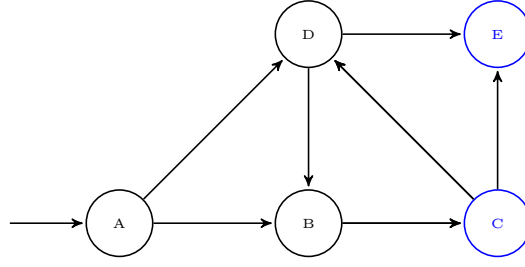


Figure 7.13: Transition System without Total Order

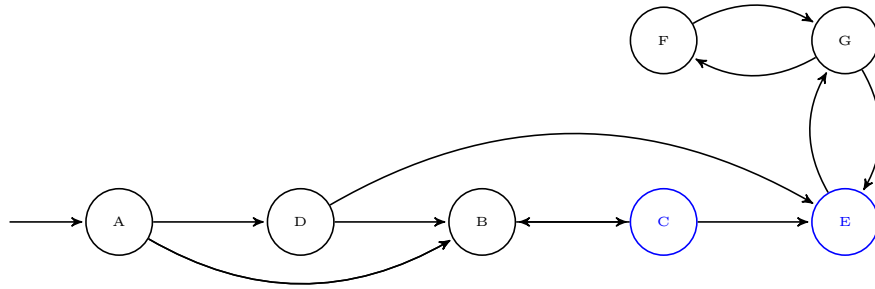


Figure 7.14: Transition System with Unreachable Cycle

that this subset is not left by any transition before reaching a goal state. We formalize this, as follows:

Definition 29 *A list of states V is a progress certificate iff*

- *V contains all non-goal initial states: $S_0 \setminus \mathcal{I}(\text{goal}) \subseteq V$*
- *transitions leave V only through goal states: $\text{Post}(V) \subseteq V \cup \mathcal{I}(\text{goal})$*
- *transitions in V make progress toward the goal:
for all $(s, s') \in R \cap (V \times V)$, $s \prec_V s'$*

where

- *set operations applied to lists denote
the corresponding operation on the set of their elements*
- *for all $i, j \in \{0, \dots, n-1\}$, $t_i \prec_{\langle t_0, \dots, t_{n-1} \rangle} t_j$ iff $i < j$*

Now let us apply the definition to the transition system from Figure 7.14. The list $\langle A \rangle$ only containing the initial state A violates the second condition, since the transition from A to B leads to a state that is not contained in this list. However, the list $\langle A, D, B \rangle$ does not contain any such transition. It also does not contain any backward transition, and hence it proves that the property **F goal** holds for the given transition system.

7.3.2 Explicit State Model Checking

We will design an algorithm with the interface:

In: a transition system (S_0, S, R)

Out: either a counter-example to **F** goal or a progress certificate V

where we will implicitly assume a certain interpretation $\mathcal{I}(\text{goal})$.

This interface needs a way of representing counter-examples, which—in the case of the property **F** goal—are paths that cycle in an infinite loop without reaching the goal. Again, we use lists for this purpose:

Definition 30 For a finite path $\langle s_0, \dots, s_{k-1} \rangle$, $\text{cycle}(\langle s_0, \dots, s_{k-1} \rangle)$ iff there is precisely one $i \in \{0, \dots, k-2\}$ s.t. $s_i = s_{k-1}$. Moreover, for every path $\langle s_0, \dots, s_{k-1} \rangle$ with $\text{cycle}(\langle s_0, \dots, s_{k-1} \rangle)$ and $i \in \{0, \dots, k-2\}$ s.t. $s_i = s_{k-1}$,

$$\text{path_loop}(\langle s_0, \dots, s_{k-1} \rangle) := \langle s_0, \dots, s_{k-1}, s_{i+1}, \dots, s_{k-1}, s_{i+1}, \dots, s_{k-1}, \dots \rangle.$$

For example, for $l = \langle A, B, E, C, D, E \rangle$, $\text{cycle}(l)$ holds and $\text{path_loop}(l) = \langle A, B, E, C, D, E, C, D, E, C, D, E, \dots \rangle$.

Now we will again solve the model checking problem by depth-first search, this time for the property **F** goal. The result can be seen in Figure 7.15. Here the notation $s :: \langle t_0, \dots, t_{k-1} \rangle$ means adding the element s to the front of the list, and hence the result of this operation is $\langle s, t_0, \dots, t_{k-1} \rangle$. The algorithm again traverses the graph defined by the transition relation using depth-first search. However, unlike the safety verification algorithm, it does not add elements to the list V at the beginning, but only at the end of analyzing the given state. The intuition is to add states to V in the moment when it is clear that every path from this state *must* reach a goal state. Adding the state to the front of the list V ensures that all transitions between the added element π_{n-1} and elements of V will be forward transitions. If the recursive calls find a counter-example, we return the counter-example. If not, the recursive calls added all successor states of π_{n-1} into V ensuring that we must reach the goal from all of them. Hence we also must reach the goal from π_{n-1} , and so we can add π_{n-1} to V , as well.

Property 5 Throughout the algorithm, the list V satisfies the second and third condition of Definition 29.

The initial call just calls the recursive function on all initial states, again adding them in front of V , if not counter-example is found. Upon the termination of the initial call, all initial states that are no goal states are in V , and hence it also satisfies the third condition of Definition 29. So the algorithm computes a progress certificate.

Let us now have a look at an example. Here we will use the transition system from Figure 7.16 with $\mathcal{I}(\text{goal}) = \{E\}$. The following table documents

```

search( $\langle \pi_0, \pi_1, \dots, \pi_{n-1} \rangle, V$ )
    • a finite path  $\langle \pi_0, \pi_1, \dots, \pi_{n-1} \rangle$  of  $(S_0, S, R)$  not containing goal states
In:
    •  $\{\pi_0, \pi_1, \dots, \pi_{n-1}\} \cap V = \emptyset, \{\pi_0, \pi_1, \dots, \pi_{n-1}\} \cap \mathcal{I}(\text{goal}) = \emptyset$ 

Out: either a finite path  $\pi'$  s.t.  $\text{path\_loop}(\pi') \not\models \mathbf{F} \text{goal}$ 
        or a list of states  $V'$  s.t.
            •  $V \cup \{\pi_{n-1}\} \subseteq V'$ , and
            •  $V'$  contains every state that can be reached from  $\pi_{n-1}$ 
              without reaching a state in  $V$  or  $\mathcal{I}(\text{goal})$ 

if  $\text{cycle}(\langle \pi_0, \dots, \pi_{n-1} \rangle)$  then return  $\langle \pi_0, \dots, \pi_{n-1} \rangle$  // counter-example
while there is a state  $s'$  s.t.  $(\pi_{n-1}, s') \in R, s' \notin V, s' \notin \mathcal{I}(\text{goal})$  do
     $V \leftarrow \text{search}(\langle \pi_0, \dots, \pi_{n-1}, s' \rangle, V)$ 
    if  $\text{cycle}(V)$  then return  $r$  // counter-example from recursive call
return  $\pi_{n-1} :: V$ 

```

Initial call:

```

 $V \leftarrow \langle \rangle$ 
while [not  $\text{cycle}(V)$ ] and [there is an  $s \in S_0 \setminus \mathcal{I}(\text{goal}), s \notin V$ ] do
     $V \leftarrow s :: \text{search}(\langle s \rangle, V)$ 

```

Figure 7.15: Model Checking $\mathbf{Fgoal}$ Using Depth-First Search

the recursive calls of the algorithm and their return values:

$\langle \pi, \dots, \pi_{n-1} \rangle :$	V	$return$
$\langle A \rangle$	$\langle \rangle$	
$\langle A, D \rangle$	$\langle \rangle$	$\langle D \rangle$
$\langle A \rangle$	$\langle D \rangle$	
$\langle A, B \rangle$	$\langle D \rangle$	
$\langle A, B, C \rangle$	$\langle D \rangle$	$\langle C, D \rangle$
$\langle A, B \rangle$	$\langle C, D \rangle$	$\langle B, C, D \rangle$
$\langle A \rangle$	$\langle B, C, D \rangle$	$\langle A, B, C, D \rangle$

As we can see, the final return value is the list $\langle A, B, C, D \rangle$, which represents a progress certificate, and hence certifies the property $\mathbf{F} \text{goal}$.

7.3.3 Relationship to the Literature

While the terms we defined gggin the context of safety verification are widespread in the literature, the notion of progress certificate is non-standard. The literature uses objects that a similar, but slightly more complicated. For example, for proving termination of loops in computer programs using similar objects (ranking functions, variants) that assign a natural number to the program state in such a way that this number decreases with every loop iteration. Since this number cannot decrease infinitely often, this ensures termination of the loop.

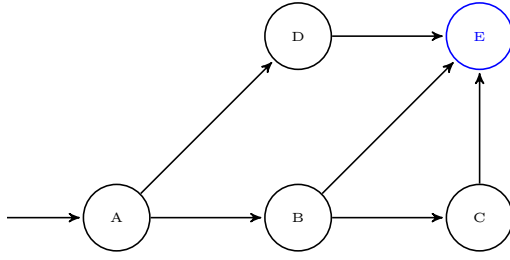


Figure 7.16: Example Transition System

We would get such a function from a progress certificate by measuring the distance of a state to the end of the list V . This is a natural number, and it must decrease, since transitions that do not reach the goal may only lead to states that are later in the list. Also control engineers also use similar objects (Lyapunov functions) to prove stability of continuous systems.

7.4 Arbitrary LTL formulas

In the case of a finite number of states there are methods for checking arbitrary LTL (CTL) formulas. Many different methods exist, here. For example, some are explicit state model checking methods based on graph algorithms, and some are symbolic based on optimized representations of sets of states.

There are also many tools, for example:

- SPIN: <http://spinroot.com>
- SAL: <http://sal.csl.sri.com>
- NuSMV: <http://nusmv.fbk.eu>

7.5 Infinite Number of States

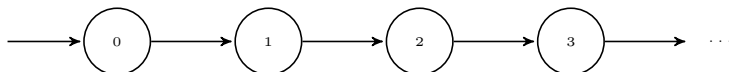
The same principles also work for systems with infinite state space, for example, for computer programs over integers, where the state space is state space $\{1, \dots, loc\} \times \mathbb{N}^k$, where k is number of program variables, loc is the number of program lines. Here, and in several other cases of systems with infinitely many states, safety verification is undecidable. But still, there are many methods that often work in practice, see the area of “software model checking”.² Of course, the semantics of computer programs in most programming languages does *not* use the infinite set of all integers, but a finite domain, for example the set $\{0, \dots, 2^{64} - 1\}$. However, this set is so big, that it is often easier to replace it with an infinite set in model checking.

We will encounter further examples of finite state systems in the rest of these lecture notes, for example, in Chapter 10.

²and the course MI-FME

7.6 Relationship to Mathematical Induction

Consider the transition system $(\mathbb{N}, \{0\}, \{(x, x') \mid x' = x + 1\})$.



Now assume that we want to prove some property p on the natural numbers. This corresponds to a proof of $\mathbf{G}p$ for this transition system.

Example: $\mathcal{I}(p) = \{n \mid P\}$, where $P \equiv 0 + 1 + \dots + n = \frac{n(n+1)}{2}$

Now take the safety verification conditions from Definition 27 and plug in the property P for the certificate V . The result is:

- $\forall x . S_0(x) \Rightarrow P(x)$
- $\forall x \forall x' . [P(x) \wedge R(x, x')] \Rightarrow P(x')$
- $\forall x . P(x) \Rightarrow P(x)$

We first observe that the third condition holds trivially. And moreover, the first two conditions are nothing else than classical mathematical induction!

So one might ask the question why not to choose V as $\mathcal{I}(p)$, in general? The reason for this can already be seen in the transition system of Figure 7.2, where the set of safe states $\{A, B, C, D\}$ is not inductive. This cannot happen for standard mathematical induction on natural numbers, but it certainly can happen in different mathematical structures. And in any case, when proving a certain property P , it is often easier to prove a different property P' by induction that implies the desired property P .

7.7 Conclusion

It is possible to check an infinite number of paths of infinite length using principles related to mathematical induction. The explanation can be represented in the form of a certificate. Similar principles are used all over computer science/mathematics hardly every explained explicitly.

Chapter 8

Testing, Bounded Model Checking

8.1 Motivation

We have seen that for solving the model checking problem one has to check transition systems with infinitely many paths of infinite length which is, of course, very difficult. To make the problem easier, we will now look at how the problem of correctness has been solved traditionally for various types of systems, especially for computer programs. Here, instead of checking correctness for all inputs, one usually checks correctness for *some* inputs. These inputs are usually called *test cases*.

Let us try the same approach for testing transition systems against LTL specifications. Here, testing would mean the following:

- Given: a transition system and an LTL formula ϕ
- Output:
 - o.k., if for all test cases π , $\pi \models \phi$, and
 - a path π s.t. $\pi \not\models \phi$, otherwise.

Of course, in the same way as for testing of software, here the output o.k. does not mean that the system is correct. It just means that the system works for the set of test cases, nothing more.

Usually the set of test cases is finite, and hence, we have reduced the problem of having to check infinitely many paths to a problem, where we only have to check finitely many paths. However, for a test case given by a certain path π , we still have to check $\pi \models \phi$, where the path π has infinite length. In other words, we have reduced the problem of checking infinitely many paths of infinite length to the problem of checking finitely many paths of infinite length. In order to avoid this second kind of infinity, we introduce the following definition.

Definition 31 A finite path of length n of a transition system (S, S_0, R) is a finite sequence of states $s_0 s_1 s_2 \dots s_{n-1}$ s.t.

- $s_0 \in S_0$,

- for all $i \in \{0, 1, \dots, n-2\}$, $(s_i, s_{i+1}) \in R$.

Now the question is, how to replace the test $\pi \models \phi$ by a test $t \models \phi$, where t is a finite path.

8.2 Bounded Semantics of LTL

8.2.1 Bounded Semantics of G

We will first analyze the situation of an LTL formula of the form $\mathbf{G} \text{ok}$, where ok is a state property. According to Definition 24, $\pi \models \mathbf{G} \text{ok}$ iff for all $k \geq 0$, $\pi(k) \models \text{ok}$. Here the quantifier ranges over the infinitely many elements of the path π . Since we want to work with finite paths now, we restrict the quantifier to the elements of such a finite path. So, for a finite path t of length n , we define $t \models \mathbf{G} \text{ok}$ as

$$\text{for all } k \in \{0, \dots, n-1\}, t(k) \models \text{ok}.$$

Note that here, we overload the symbol $\models$ with yet another meaning—in the case where on its left-hand side there is a finite, instead of an infinite path. The intuition here is that for an LTL formula ϕ and a finite path t , $t \models \phi$ if and only if the test of the formula ϕ using the finite path t is successful in the sense that it does not find any bug.

As intended for a test, this is a necessary condition for correctness. So, if the property holds for the given system, then the test succeeds, and equivalently, if the test fails, then the system is incorrect. More formally:

Property 6 *For any finite path t of the given transition system, if $t \models \mathbf{G} \text{ok}$, then $t \models \mathbf{G} \text{ok}$. Equivalently, $t \not\models \mathbf{G} \text{ok}$ implies $t \not\models \mathbf{G} \text{ok}$.*

As an example, we ask the question, whether for the transition system from Figure 7.1, $\models \mathbf{G} \text{ok}$, where $\mathcal{I}(\text{ok}) = \{a, b\}$. For testing this, we might use the finite paths (a, b, b) , and (a, b, c) . Clearly, $(a, b, b) \models \mathbf{G} \text{ok}$, $(a, b, c) \not\models \mathbf{G} \text{ok}$ which, according to the property above, shows that the system does not fulfill the property. Any path that starts with (a, b, c) , and then continues arbitrarily, is a counter-example. In general, any continuation of a finite path t s.t. $t \not\models \mathbf{G} \text{ok}$ is a counter-example to this property.

8.2.2 Bounded Semantics of F

Now we will turn to the situation of an LTL formula of the form $\mathbf{F} \text{goal}$, where goal is a state property. According to Definition 24, $\pi \models \mathbf{F} \text{goal}$ if and only if there exists a $k \geq 0$ such that $\pi(k) \models \text{goal}$. Again we cannot check all $k \geq 0$, and the question is whether it suffices to test finitely many elements. Figure 8.2.2 contains a slightly changed version of the transition system of Figure 7.1. For this transition system, the formula $\mathbf{F} \text{goal}$, with $\mathcal{I}(\text{goal}) = \{d\}$ holds. But the finite path (a, b, c) does not reach the goal state d . Hence, a restriction of k to the set $\{0, \dots, n-1\}$ as in the case of the $\mathbf{G}$ operator, would result in a false alarm, reporting a bug, although there is none.

If we return to the original transition system from Figure 7.1, then $\mathbf{F} \text{goal}$ does not hold any more, but the finite path (a, b, b, b, c, d) does reach the goal.

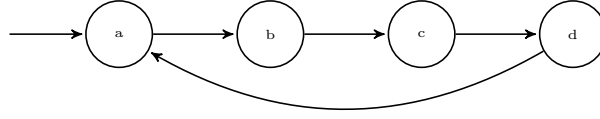


Figure 8.1: Modified Transition System

Still, one can detect that the transition system does not fulfill the property, since the finite path has a loop from b to b and hence can stay in b forever without ever reaching the goal state d . Indeed, the shorter finite path (a, b, b) already detects this loop.

So, it seems that for detecting bugs for the $\mathbf{F}$ operator, one has to detect loops. So we might define for a finite path t of length n , $t \models \mathbf{F} \text{goal}$ if and only if the finite path does not contain a loop. However, this still has a problem. And this problem can be seen in Figure 8.2, for which $\mathbf{F} \text{goal}$ holds, but the finite path (a, b, c, d, e, e) contains a loop. Hence, such a definition would report a bug, although there is none.

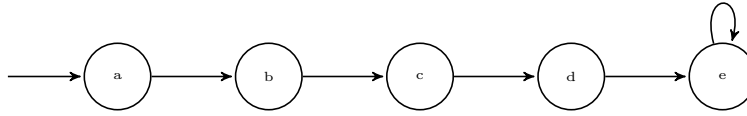


Figure 8.2: Transition System with Loop

The problem is, that here the goal is reached *before* the loop. Hence, the loop is no problem. So we modify the definition in such a way that for a finite path t of length n $t \models \mathbf{F} \text{goal}$ if and only if the fact that the finite path t has a loop implies that the goal is reached. More precisely, $t \models \mathbf{F} \text{goal}$ if and only if the fact that there exists an $l \in \{0, \dots, n-2\}$ s.t. $t(l) = t(n-1)$ implies that there exists a $k \in \{0, \dots, n-1\}$ such that $t(k) \models \text{goal}$. Note that this only checks for loops that contain the final element of the finite path, which is $t(n-1)$. Again, such a definition is a necessary condition for correctness, that is, Property 6 holds analogically, for the $\mathbf{F}$ operator, as well.

Now, if we found a finite path t s.t. $t \not\models \mathbf{F} \text{goal}$, it is easy to reconstruct a counter-example, that is, an infinite path π with $\pi \not\models \mathbf{F} \text{goal}$: Simply take the part of t leading to a loop and the repeat the loop infinitely often.

8.2.3 Bounded Semantics of LTL

Now we can combine the special cases above into one recursive definition of the semantics of LTL for finite paths.

Definition 32 For a finite path t of length n and LTL formulas ϕ, ψ ,

- $t \models \phi$ if t has length 0, and otherwise,
- $t \models p$ where p is a state property iff $t(0) \models p$

- $t \models \neg p$ where p is a state property iff $t(0) \not\models p$
- $t \models X\phi$ iff $t^1 \models \phi$
- $t \models F\phi$ iff
 $\text{there exists } l \in \{0, \dots, n-2\} \text{ s.t. } t(l) = t(n-1)$
 implies
 $\text{there exists } k \in \{0, \dots, n-1\} \text{ s.t. } t^k \models \phi$
- $t \models G\phi$ iff for all $k \in \{0, \dots, n-1\}$, $t^k \models \phi$
- $t \models \phi \wedge \psi$ iff $t \models \phi$ and $t \models \psi$
- $t \models \phi \vee \psi$ iff $t \models \phi$ or $t \models \psi$

Note that the base case of this recursive definition is given by the first three items. The items corresponding to the operators **F** and **G** correspond to the definitions from the discussion above. Unlike the standard semantics of LTL from Definition 25, here we allow negation only directly before a state property. If we want to apply the definition to a formula such as $\neg G p$, where the negation does not appear before a state property, one can simply push negation down using equivalence rules. For the given example, this results in the equivalent formula $F \neg p$, to which the definition applies.

Based on this definition, Property 6 generalizes to arbitrary LTL formulas with negation only occurring directly before a state property.

Property 7 *Let ϕ be an LTL formula with negation only occurring directly before a state property. For a finite path t of the given transition system, if $t \models \phi$, then $t \models \phi$. Equivalently, $t \not\models \phi$ implies $t \not\models \phi$.*

8.2.4 Example 1

We will apply Definition 32 to check whether $(a, b, c, d, e, c, d) \models GFp$, where $\mathcal{I}(p) = \{b\}$. According to the item corresponding to **G** in the definition, we have to check whether for all $k \in \{0, \dots, 6\}$, $(a, b, c, d, e, c, d)^k \models Fp$. For $k = 0$, this results in the check $(a, b, c, d, e, c, d) \models Fp$, which certainly holds, because the finite path contains the goal state b , and hence the right-hand side of the implication holds. Due to the same argument the case $k = 1$, which corresponds to the check $(b, c, d, e, c, d) \models Fp$, holds.

In the case $k = 2$, we have to check $(c, d, e, c, d) \models Fp$. Here we have the same cycle as before. But unlike before, we do not reach the goal. Hence this is false, which makes the whole formula false. A corresponding counter-example is $(a, b, c, d, e, c, d, e, c, d, e, \dots)$.

8.2.5 Example 2

Now we want to check $(a, b, c, d, e) \models FGp$, where $\mathcal{I}(p) = \{e\}$. After plugging in the item corresponding to the **F** operator in Definition 32, we get: If there exists an $l \in \{0, \dots, 3\}$ s.t. $t(l) = t(4)$ then there exists a $k \in \{0, \dots, 4\}$ s.t. $(a, b, c, d, e)^k \models Gp$. The finite path does not contain any loop, and hence this holds trivially.

Now we slightly modify the path and check $(a, b, e, d, e) \models FGp$, where $\mathcal{I}(p) = \{e\}$. This does contain a cycle. Still the formula holds, since there exists

a $k \in \{0, \dots, 4\}$ s.t. $(a, b, e, d, e)^k \models \mathbf{Gp}$. More concretely, (e) is a suffix of (a, b, c, d, e) , and $(e) \models \mathbf{Gp}$.

Still, if we would extend the finite path to an infinite path by continuing with the given cycle, we get $(a, b, e, d, e, d, e, d, e, \dots) \not\models \mathbf{FGp}$! In other words, the transition system corresponding to the finite path (a, b, e, d, e) certainly does *not* fulfill the given property. But our definition does not detect this. We detect the bug using our definition only if we add one more element from this cycle, and get $(a, b, e, d, e, d) \not\models \mathbf{FGp}$.

This is the reason why, in the literature, one can find stronger tests, that are able to detect the problem already for the shorter path (a, b, e, d, e) [28, 7]. However, those tests are more complicated, and hence we do not work with them, here.

8.3 Testing Transition Systems

To summarize, instead of checking correctness for *all* paths, we can test a given transition system by checking correctness for *some finite* paths. So we can define

$$\text{Test}(\phi, T) :\Leftrightarrow \text{ for all finite paths } t \in T, t \models \phi$$

and view testing as the following task:

- Given:
 - a transition system,
 - an LTL formula ϕ , and
 - a set of finite paths T
- Output:
 - o.k., if $\text{Test}(\phi, T)$
 - a path π s.t. $\pi \not\models \phi$, otherwise.

The process of generating finite paths is also called *simulation* which is extremely simple for finite transition systems. But later we will meet systems for which simulation is more difficult.

Again we have the following property:

Property 8

- $\models \phi$ *implies* $\text{Test}(\phi, T)$
- $\neg \text{Test}(\phi, T)$ *implies* $\not\models \phi$

Since transition systems can have infinitely many paths of infinite length, we need systematic methods for choosing finite paths for testing, for example black-box testing, white-box testing, coverage criteria, etc. This is a field on its own, see MI-TSP (testing and reliability), ...

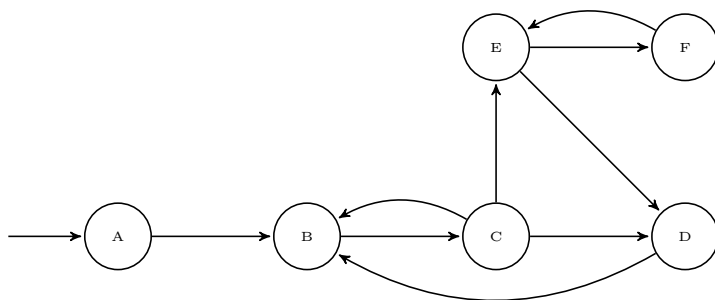


Figure 8.3:

8.4 Bounded Model Checking

Problems of testing:

- Can easily miss bugs
- Not reliable enough for many safety critical systems

This became an especially urgent problem when the Intel FDIV Bug (1994) occurred, that cost the company 1/2 billion dollars, or when the cost of testing for companies such as Airbus became so high, that they already exceeded the cost of airplane design itself. Due to this, industry has more and more interest in methods for *proving* correctness of systems. We already saw such a method in the previous chapter on unbounded model checking. However, unbounded model checking usually not efficient enough to be applicable to very large systems.

In this section, we will discuss a technique that can provide higher correctness guarantees than traditional testing, but is often more efficient than unbounded model checking. The technique is based on the observation that a transition system with finitely many states only has finitely many paths of a given finite length. Due to this we can replace the original problem of checking infinitely many paths of infinite length, with the simpler problem of checking *all* finite paths of a certain given length n . For this, we define

$$\begin{aligned} \text{BMC}(\phi, n) &: \Leftrightarrow \text{for all finite paths } t \text{ of length } n, t \models \phi \\ &\Leftrightarrow \text{Test}(\phi, \{t \mid t \text{ is a finite path of length } n\}) \end{aligned}$$

8.4.1 Example 1

We will see how to use bounded model checking for the LTL formula $\mathbf{G\,ok}$. A system fulfills this formula iff for every path π , for all $k \geq 0$, $\pi(k) \models \text{ok}$. Bounded model checking replaces this property by a check whether all finite paths of a certain length fulfill the formula. More precisely, $\text{BMC}(\mathbf{G\,ok}, n)$ iff for every finite path t of length n , for all $k \in \{0, \dots, n-1\}$, $t(k) \models \text{ok}$.

Now consider the transition system from Figure 8.3, and $\mathcal{I}(\text{ok}) = \{A, B, C, D, E\}$. Here, $\text{BMC}(\mathbf{G\,ok}, n)$ holds for $n \leq 4$, but does not hold for $n > 4$.

We already know:

- $\models \mathbf{G\ ok}$ implies $\text{BMC}(\mathbf{G\ ok}, n)$, i.e.,
- $\neg \text{BMC}(\mathbf{G\ ok}, n)$ implies $\not\models \mathbf{G\ ok}$

But, in general, $\text{BMC}(\mathbf{G\ ok}, n)$ does not imply $\models \mathbf{G\ ok}$. $\text{BMC}(\mathbf{G\ ok}, n)$ only ensures correctness within n steps, i.e., non-existence of a counter-example of length n

8.4.2 Special Case

Let us analyze the special case with $n = 0$. According to the definition, $\text{BMC}(\mathbf{G\ ok}, 0)$ iff for all finite paths t of length 0, for all $k \in \{0, \dots, -1\}$, $t(k) \models \text{ok}$ which is equivalent to:

- for all finite paths t of length 0, for all $k \in \emptyset$, $t(k) \models \text{ok}$
- for all finite paths t of length 0, for all $k . k \in \emptyset \Rightarrow t(k) \models \text{ok}$
- for all finite paths t of length 0, for all $k . \perp \Rightarrow t(k) \models \text{ok}$
- for all finite paths t of length 0, for all $k . \neg \perp \vee t(k) \models \text{ok}$
- for all finite paths t of length 0, for all $k \in \emptyset$, $t(k) \models \text{ok}$

which is obviously true.

8.4.3 Example 2

Let us check $\text{BMC}(\mathbf{F\ goal}, 4)$ for the transition system from Figure 8.3, and $\mathcal{I}(\text{goal}) = \{D, E\}$? We need to check whether for every finite path t of length 4, if there exists an $l \in \{0, \dots, 2\}$ s.t. $t(l) = t(3)$ then there exists a $k \in \{0, \dots, 3\}$ s.t. $t(k) \models \text{goal}$.

The finite paths of length 4 are $\{(A, B, C, D), (A, B, C, E), (A, B, C, B)\}$. The first two of those finite paths do not contain a loop, hence they fulfill the property trivially. The third one contains a loop and does not reach the goal, and hence proves that the property does not hold. It is important to observe that also $\text{BMC}(\mathbf{F\ goal}, 5)$, $\text{BMC}(\mathbf{F\ goal}, 6)$, $\dots$ will find the same cycle. The corresponding finite paths are (A, B, C, B, C) , (A, B, C, B, C, B) , and so on.

$\text{BMC}(\mathbf{F\ goal}, 0)$, $\text{BMC}(\mathbf{F\ goal}, 1)$: similar to the case $\text{BMC}(\mathbf{G\ ok}, 0)$

8.4.4 $\text{BMC}(\phi, n)$ and System Correctness

In general we have that for every n and n' with $n' \leq n$,

- $\models \phi$ implies $\text{BMC}(\phi, n)$ implies $\text{BMC}(\phi, n')$ i.e.,
- $\neg \text{BMC}(\phi, n')$ implies $\neg \text{BMC}(\phi, n)$ implies $\not\models \phi$.

Note that the two items are equivalent. Intuitively, if BMC finds a bug, then it will also find a bug for higher n and the system really is incorrect.

But $\text{BMC}(\phi, n)$ does not imply $\models \phi$, in general. Still one can, in some cases, really use bounded model checking to prove the correctness of the given

system. For this, we make the observation that for a finite transition system, the transition relation forms a graph, and for big n , the paths of length n reach all states that are reachable in this graph from an initial state!

General Observation:

- Every finite path that is longer than $|S|$ contains a cycle.
- If there exists a counter-example for $\mathbf{G ok}$ that contains a cycle, then there also exists a counter-example that is shorter.
- Hence: If there exists a counter-example for $\mathbf{G ok}$ that is longer than $|S|$, then there also exists a counter-example that is shorter.
- Hence: $\models \mathbf{G ok}$ iff $\text{BMC}(\mathbf{G ok}, |S|)$.

This observation can be generalized to LTL as follows:

Theorem 2 ([6]) *for all finite transition system, for all LTL formulas ϕ there exists a bound n s.t. for all $n' \geq n$, $\text{BMC}(\phi, n')$ iff $\models \phi$*

This holds independently of which method one uses for checking *BMC*. But unfortunately, the bound is often too big for being practically useful.

8.5 Temporal Logic for Planning

Consider the traditional puzzle where a farmer wants to bring a wolf, a goat, and some cabbage across a river in a boat that is so small that the farmer himself and one of the three items fit into the boat. However, he cannot leave the wolf and the goat, or the goat and the cabbage alone on one side of the river since one of the two would eat the other. This situation can be modeled as a transition system. One way of doing so is the following: We use a 4-tuple to model the physical position of farmer, wolf and cabbage, where -1 means left river bank, 0 means boat, and 1 means right river bank. The initial state would be the state where all four items are on the left river bank. The transition relation is defined in such a way that farmer and his three belongings move maximally by one position. Formally, this is a transition system (S, S_0, R) where

$$\begin{cases} S &= \{-1, 0, 1\}^4 \\ S_0 &= \{(-1, -1, -1, -1)\} \\ R &= \{(s, s') \in S \times S \mid \exists d \in \{-1, 0, 1\}^4 . s' = s + d\}. \end{cases}$$

The goal is to arrive at a state where all four items are on the other side of the river while not risking the cabbage or the goat being eaten, ensuring that the belongings of the farmer do not enter the boat without the farmer, and that the maximum capacity of the boat is not exceeded. Hence we want to find a path π for this transition system with π s.t. $\pi \models \mathbf{safeUgoal}$ where

$$\begin{aligned} \mathcal{I}(\mathbf{goal}) &= \{(1, 1, 1, 1)\} \\ \mathcal{I}(\mathbf{safe}) &= \left\{ (F, C, G, W) \mid \begin{array}{l} \neg \text{cabbage_goat}(F, C, G, W) \wedge \\ \neg \text{wolf_goat}(F, C, G, W) \wedge \\ [C = 0 \Rightarrow F = 0] \wedge \\ [G = 0 \Rightarrow F = 0] \wedge \\ [W = 0 \Rightarrow F = 0] \wedge \\ \text{boat_max2}(F, C, G, W) \end{array} \right\}, \end{aligned}$$

and *cabbage_goat*, *wolve_goat*, and *boat_max2* are some sub-formulas defined in an appropriate way.

Such a model uses non-determinism in a different way than seen so far. Up to now, non-determinism has corresponded to things beyond our control, such as unknown weather conditions or details we do not want to model. But now, non-determinism corresponds to our decisions. For example, from the initial state in our puzzle, we can either move the wolve, goat, or cabbage into the boat, which corresponds to transitions from the initial state to one of the state $(0, 0, -1, -1)$, $(0, -1, 0, -1)$, or $(0, -1, -1, 0)$.

In such a case, the problem we want to solve, is different from the one that we discussed up to now. Given

- a transition system $(S, \{s_0\}, R)$, where s_0 is the current state of the system, and
- an LTL formula ϕ ,

we want to find a path π , s.t. $\pi \models \phi$. This path corresponds to a plan what to do from the current state s_0 .

Ideally, we would like to use the methods that we have seen up to now for solving this problem. The idea is to obtain such a path as a counter-example to the negated input formula $\neg\phi$. Hence we can find plans by applying a method for checking $\models \neg\phi$.

In our case, we have the property **Fgoal**. A plan is a counter-example to $\neg\mathbf{Fgoal}$ which is equivalent to **G¬goal**. A model checking method will then produce a counter-example to **G¬goal**, that is, a path, that will reach $\mathcal{I}(\mathbf{goal})$. This path is the desired plan, that will move the wolve, the goat, and the cabbage to other side of the river.

As a further example, consider the case of the property **Gok**, with $\mathcal{I}(ok) = \{A, B, C, E, F\}$, current state C , and the transitions system from Figure 8.4.

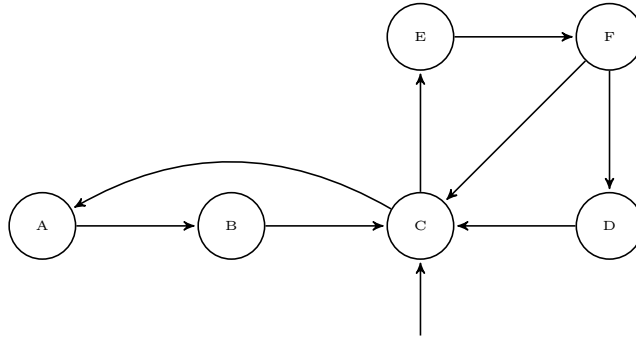


Figure 8.4: Example Transition System

Our goal is to find a plan, that is a path π s.t. $\pi \models \mathbf{Gok}$. We do this by finding a counter-example to $\neg\mathbf{Gok}$. This formula is equivalent to **F¬ok**. It is easy to see that the paths $(C, A, B, C, \dots)$, $(C, E, F, C, \dots)$ are counter-examples to this property, both being successful plans that keep us in the set of safe states forever.

In general, the result is a path, that will cycle outside of $\{s \mid s \models \neg \text{ok}\}$, in $\mathcal{I}(\text{ok})$.

In Chapter 6.5 we discussed that one can reduce the problem of checking $\models \mathbf{pRq}$ to the search for a counter-example to its negation. More concretely, the problem of checking the property $\mathbf{pRq}$ is equivalent to the search for a path π s.t. $\pi \not\models \mathbf{pRq}$, which is equivalent to $\pi \models \neg[\mathbf{pRq}]$, and $\pi \models \neg \mathbf{pU}\neg \mathbf{q}$. This was the opposite direction to what we did here.

So, in general, we have two situations:

- Model checking problem:
Either return that $\models \phi$, or a counter-example ($\pi \not\models \phi$)
- Planning problem:
Find π s.t. $\pi \models \phi$ or return “does not exist” (i.e., $\models \neg \phi$)

Each of the two problems is reducible to the other one.

Note that BMC, in general, does not prove $\models \phi$, but is a weaker. We will see the consequences of this in the following example.

Consider once more the transition system from Figure 8.4. Assume that we want to ensure $\mathbf{Gok}$, where $\mathcal{I}(\text{ok}) = \{A, B, C, E, F\}$ and assume that the current state is C . Then the paths of length 2 are: (C, A) , (C, E) , and hence $\text{BMC}(\mathbf{F}\neg\text{ok}, 2)$ does not find any counter-example, that can serve as a plan. The paths of length 3 are (C, A, B) and (C, E, F) , and again $\text{BMC}(\mathbf{F}\neg\text{ok}, 3)$ fails to find a plan. The paths of length 4 are (C, A, B, C) , (C, E, F, C) , (C, E, F, D) , and hence $\text{BMC}(\mathbf{F}\neg\text{ok}, 4)$ finally produces a counter-example that can serve as a plan, for example, the finite path (C, A, B, C) . This path cycles without reaching a state not in the set $\{A, B, C, E, F\}$ and hence is a plan for the original property $\mathbf{Gok}$.

8.6 Bounded vs. Unbounded Model Checking

Bounded model checking reduces the problem of checking the correctness of transition systems with infinitely many paths of infinite length to the problem of checking paths of finite length by introducing a certain upper bound for time. Now let us discuss the practical context. In practice, technical systems indeed do have limited lifetime. So the question is, of course, why to even bother with paths of infinite length? One could take this question even further: There is only a finite number of atoms on earth. So why bother with the infinite set of natural numbers, or even real numbers?

The answer becomes quite obvious, when considering the actual lifetime of technical devices in terms of the number of steps a microchip on such a device might make during its lifetime, or when considering the actual number of atoms in the universe. Those numbers are, of course, huge, and hence it is usually easier to assume an infinite number of steps than a finite, but huge number.

8.7 Efficient Checking of $\text{BMC}(\phi, n)$

Assume a transition system with 10 states and assume that we want to do bounded model checking for $n = 10$. This means that we have $10^{10} = 10000000000$

potential paths to check. Clearly this is a difficult problem. In Chapter 9 we will discuss, how to solve this problem efficiently, in practice.

Chapter 9

Boolean Satisfiability (SAT)

Figure 9.1 shows a digital circuit that implements a counter modulo 8. Its state is formed by Boolean vectors from $\mathbb{B}^3$, and its transitions follow the formula $v'_0 = \neg v_0 \wedge v'_1 = v_0 \oplus v_1 \wedge v'_2 = (v_0 \wedge v_1) \oplus v_2$.

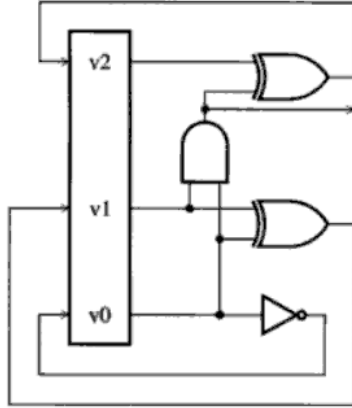


Figure 9.1: Circuit

Hence, we can describe the behavior of the circuit using the transition relation

$$R \doteq \left\{ ((v_0, v_1, v_2), (v'_0, v'_1, v'_2)) \mid \begin{array}{l} (v'_0 \Leftrightarrow \neg v_0) \wedge \\ (v'_1 \Leftrightarrow v_0 \oplus v_1) \wedge \\ (v'_2 \Leftrightarrow (v_0 \wedge v_1) \oplus v_2) \end{array} \right\}$$

As one can see, formulas in propositional logic (*Boolean formulas*) can be used to describe finite transition systems. We will use this for bounded model checking by first encoding the bounded model checking problem as a Boolean satisfiability problem, and to then solve this problem by a corresponding solver. The bad news is that checking satisfiability of Boolean formulas is NP-hard. The good news is that, in practice, algorithms can solve huge problems. Especially, for satisfiable problems, current algorithms can often find satisfying assignment

extremely quickly!

9.1 Boolean Encoding of BMC

Consider the transition system from Figure 9.2.

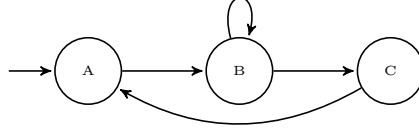


Figure 9.2: Transition System

Formally, this is the transition system

$$(\{A, B, C\}, \{A\}, \left\{ \begin{array}{l} (A, B), \\ (B, B), \\ (B, C), \\ (C, A) \end{array} \right\}).$$

The system has three states, hence we need two bits to represent them, for example as follows:

A	($\top$, $\top$)
B	($\top$, $\perp$)
C	($\perp$, $\top$)
	($\perp$, $\perp$)

The resulting, equivalent transition system is $(\mathbb{B}^2, \{(\top, \top)\}, \left\{ \begin{array}{l} ((\top, \top), (\top, \perp)), \\ ((\top, \perp), (\top, \perp)), \\ ((\top, \perp), (\perp, \top)), \\ ((\perp, \top), (\perp, \top)) \end{array} \right\})$.

This can be written using Boolean formulas, as follows:

$$\left(\mathbb{B}^2, \left\{ (v_0, v_1) \mid v_0 \wedge v_1 \right\}, \left\{ ((v_0, v_1), (v'_0, v'_1)) \mid \begin{array}{c} [v_0 \wedge v_1 \wedge v'_0 \wedge \neg v'_1] \\ \vee \\ [v_0 \wedge \neg v_1 \wedge v'_0 \wedge \neg v'_1] \\ \vee \\ [v_0 \wedge \neg v_1 \wedge \neg v'_0 \wedge v'_1] \\ \vee \\ [\neg v_0 \wedge v_1 \wedge v'_0 \wedge v'_1] \end{array} \right\} \right).$$

At this moment, this does not look like a very compact way of writing the transition system. However, when using Boolean formulas, one usually does not have to represent every individual transition individually. For example, observe that the first and second transition of the transition system are almost the same, they only differ in the second bit of the source state, as encoded by

the variable v_1 . Hence, we can describe both transitions using one and the same conjunction:

$$\left(\mathbb{B}^2, \left\{ (v_0, v_1) \mid v_0 \wedge v_1 \right\}, \left\{ ((v_0, v_1), (v'_0, v'_1)) \mid \begin{array}{c} [v_0 \wedge v'_0 \wedge \neg v'_1] \\ \vee \\ [v_0 \wedge \neg v_1 \wedge \neg v'_0 \wedge v'_1] \\ \vee \\ [\neg v_0 \wedge v_1 \wedge v'_0 \wedge v'_1] \end{array} \right\} \right)$$

In general, every finite transition systems (S, S_0, R) can be encoded as a Boolean one. Here one encodes the state using Boolean vectors, which needs $\lceil \log_2 |S| \rceil$ variables, and represents the set of initial states and transition relation as a Boolean formula.

In addition, when encoding transition systems using Boolean formulas, one can often exploit the structure of the given transition system. For example, assume

- n transition systems with m transitions each, and
- formulas $T_1, \dots, T_n$ describing their respective transition relations.

Then the composed system has m^n transitions, but the formula describing the composed transition relation is $T_1 \wedge \dots \wedge T_n$. So the number of transitions of the composition grows exponentially, but the size of the Boolean formula describing its transition relation grows only linearly.

9.2 Problem Statement and Applications

In this chapter we will study algorithms for solving the following problem (the *SAT problem*):

- Input: a Boolean formula
- Output:
 - a satisfying assignment, if it exists,
 - **unsat**, if the formula is not satisfiable.

We will call algorithms or software solving this problem *SAT solvers*.

As an example, consider the transition system

$$\left(\mathbb{B}^2, \left\{ (v_0, v_1) \mid v_0 \wedge v_1 \right\}, \left\{ ((v_0, v_1), (v'_0, v'_1)) \mid \begin{array}{c} [v_0 \wedge v'_0 \wedge \neg v'_1] \vee \\ [v_0 \wedge \neg v_1 \wedge \neg v'_0 \wedge v'_1] \vee \\ [\neg v_0 \wedge v_1 \wedge v'_0 \wedge v'_1] \end{array} \right\} \right)$$

whose state space is formed by pairs of Booleans, and whose initial states and transitions are given by Boolean formulas. We can use a SAT solver to answer questions on this transition system:

- Does the system have an initial state?
 - Input: $v_0 \wedge v_1$
 - Output: $\{v_0 \mapsto \top, v_1 \mapsto \top\}$

Hence the answer to the question is yes, with the example state A .

- May our system be in C after one step?

$$\begin{aligned} & - \text{Input: } v_0 \wedge v_1 \wedge \left[\begin{array}{c} [v_0 \wedge v'_0 \wedge \neg v'_1] \vee \\ [v_0 \wedge \neg v_1 \wedge \neg v'_0 \wedge v'_1] \vee \\ [\neg v_0 \wedge v_1 \wedge v'_0 \wedge v'_1] \end{array} \right] \wedge \neg v'_0 \wedge v'_1 \\ & - \text{Output: } \mathbf{unsat} \end{aligned}$$

So in this case the answer is no.

In a similar way, one can solve bounded model checking problems using SAT. For example, consider the problem $\text{BMC}(\mathbf{G ok}, n)$ which, according to the definition of BMC means to check whether

for all finite paths t of length n , $t \models \mathbf{G ok}$.

which, after expansion of $\models$ according to Definition 32 becomes

for all a finite paths t of length n ,
for all k s.t. $0 \leq k \leq n-1$, $t(k) \models \mathbf{ok}$.

The problem is that this expression contains universal quantifiers, whereas the Boolean satisfiability problem asks the question whether there *exists* a satisfying assignment. To resolve this mismatch, instead of checking the above bounded model checking problem, we check its negation $\neg \text{BMC}(\mathbf{G ok}, n)$, which, after moving the negation over the universal quantifier, is

there exists a finite path t of length n ,
there exists a k s.t. $0 \leq k \leq n-1$, $t(k) \not\models \mathbf{ok}$.

Replacing the existential quantifier by a disjunction we now check whether

there exists a finite path t s.t.
 $t(0) \not\models \mathbf{ok}$ or $\dots$ or $t(n-1) \not\models \mathbf{ok}$.

If we now apply this to the example above, with $\mathcal{I}(\mathbf{ok}) = \{A, B\}$, $n = 3$ we check whether

there exists a finite path t s.t.
 $t(0) \notin \{A, B\}$ or $t(1) \notin \{A, B\}$ or $t(2) \notin \{A, B\}$.

Representing the finite path using Boolean variables by

$$((v_0^0, v_1^0), (v_0^1, v_1^1), (v_0^2, v_1^2)),$$

we can now express the original bounded model checking problem as the problem of checking satisfiability of the formula

$$v_0^0 \wedge v_1^0 \wedge \left[\begin{array}{c} [v_0^0 \wedge v_1^0 \wedge \neg v_1^1] \\ \vee \\ [v_0^0 \wedge \neg v_1^0 \wedge \neg v_0^1 \wedge v_1^1] \\ \vee \\ [\neg v_0^0 \wedge v_1^0 \wedge v_0^1 \wedge v_1^1] \end{array} \right] \wedge \left[\begin{array}{c} [v_0^1 \wedge v_0^2 \wedge \neg v_1^2] \\ \vee \\ [v_0^1 \wedge \neg v_1^1 \wedge \neg v_0^2 \wedge v_1^2] \\ \vee \\ [\neg v_0^1 \wedge v_1^1 \wedge v_0^2 \wedge v_1^2] \end{array} \right] \wedge$$

$$\neg v_0^0 \vee \neg v_0^1 \vee \neg v_0^2.$$

In general, for checking $\text{BMC}(\phi, n)$, we check $\neg \text{BMC}(\phi, n)$, instead. This means to check whether

$$\text{not for all finite paths } t \text{ of length } n, t \models \phi$$

which can be rewritten to

$$\text{there exists a finite path } t \text{ of length } n, \text{ such that } t \not\models \phi.$$

Now we describe this finite path using a Boolean formula, and use a SAT solver to check its satisfiability. If the solver returns a satisfying assignment, this represents a counter-example to the LTL formula, and in the case of the property **Gok** a path to an unsafe state. If the solver returns **unsat** then the LTL formula holds. In the case of **Gok** this means that no path to an unsafe state of the given length exists.

In general, one can encode bounded model checking problems as SAT problems as follows. First one translates the given transition system into a transition system on Boolean vectors. Hence it has the form

$$(\mathbb{B}^k, \{\vec{v} \mid S_0\}, \{(\vec{v}, \vec{v}') \mid T\}),$$

where

- S_0 , T , and ok are Boolean formulas encoding the corresponding sets, and
- $\vec{v}$, $\vec{v}'$ are Boolean variables representing the states of the transition system.

Then one can represent a finite path t of length n as n Boolean vectors $(\vec{v}^0, \dots, \vec{v}^{n-1})$. Finally, one gets a representation of $\neg \text{BMC}(\mathbf{Gok}, n)$ as

$$S_0[\vec{v} \leftarrow \vec{v}^0] \wedge \bigwedge_{i=0}^{n-2} T[\vec{v} \leftarrow \vec{v}^i, \vec{v}' \leftarrow \vec{v}^{i+1}] \wedge \bigvee_{i \in \{0, \dots, n-1\}} \neg \text{ok}[\vec{v} \leftarrow \vec{v}^i]$$

The principle used here is called *symbolic model checking*. This means that one uses symbols (in our case logical formulas) for representing big sets. In the case of an infinite state space this is the only possibility.

Symbolic representations of infinite sets are used in many other areas of computer science, for example:

- representation of convex polyhedra using system of linear inequalities
- representation of regular languages using regular expressions

9.3 SAT Solving

Most algorithms for solving the SAT problem require the input to be in conjunctive normal form (CNF). There are special transformations for translating general formulas into CNF, for example Tseitin encoding [36].

During the time when I studied, the general wisdom was that there is no point in trying to solve NP-complete problems such as SAT unless somebody proves $P = NP$. But since then, the situation has changed completely, and today there is a race for more and more efficient SAT solvers and it is completely common to solve huge SAT problems. Today's solvers only have a few hundred lines of code, and they are often open-source, as in the example of MiniSAT¹. Moreover, there is a yearly SAT competition². But, of course, current solvers are still exponential in worst case, and the question $P = NP$ is still open!

So how do such SAT solvers work? First we will need some terminology. For this, consider the following formula in conjunctive normal form:

$$[\neg P \vee Q \vee R] \wedge [\neg Q \vee R] \wedge [\neg Q \vee \neg R] \wedge [P \vee Q]$$

We will call a Boolean variable or its negation *literal* (e.g., $\neg P$, Q) and a disjunction of literals a *clause* (e.g., $\neg P \vee Q \vee R$). Here we will not distinguish clauses with different order of the same literals.

Now everybody knows a very simple algorithm for checking satisfiability of such a formula: truth tables. For example, a truth table of the above formula can be seen in Figure 9.3.

P	Q	R	$\neg P \vee Q \vee R$	$\neg Q \vee R$	$\neg Q \vee \neg R$	$P \vee Q$	
$\perp$	$\perp$	$\perp$	$\top$	$\top$	$\top$	$\perp$	$\perp$
$\perp$	$\perp$	$\top$	$\top$	$\top$	$\top$	$\perp$	$\perp$
$\perp$	$\top$	$\perp$	$\top$	$\perp$	$\top$	$\top$	$\perp$
$\perp$	$\top$	$\top$	$\top$	$\top$	$\perp$	$\top$	$\perp$
$\top$	$\perp$	$\perp$	$\perp$	$\top$	$\top$	$\top$	$\perp$
$\top$	$\perp$	$\top$	$\top$	$\top$	$\top$	$\top$	$\top$
$\top$	$\top$	$\perp$	$\top$	$\perp$	$\top$	$\top$	$\perp$
$\top$	$\top$	$\top$	$\top$	$\top$	$\perp$	$\top$	$\perp$

Figure 9.3: Truth Table

In general, such a table has $2^{|V|}$ rows listing the possible assignments to the propositional variables. So we do not have a chance with this for formulas that contain many variables. How can we improve this? Observe that the table has some structure: All rows in the upper half use the same value for the variable P . Also the value for the variable Q stays the same in two consecutive rows. We make this explicit in Figure 9.4

As one can see from the table, the assignment $\{P \mapsto \top, Q \mapsto \perp, R \mapsto \top\}$ satisfies the formula. Also, the assignments form a tree. So we will use depth-first in this tree to search for a satisfying assignment.

The resulting algorithm can be seen in Figure 9.3. The simple version shown only returns $\top$ (for satisfiable input), or $\perp$ (unsatisfiable input), instead of computing a satisfiable assignment. For the original specification we only have to remember the substitutions performed by the algorithm.

The shown algorithm leaves several design decisions open. One such question is which variable and branch ($\perp/\top$) to choose first. This is an important

¹<http://minisat.se>

²<http://www.satcompetition.org>

P	Q	R	$\neg P \vee Q \vee R$	$\neg Q \vee R$	$\neg Q \vee \neg R$	$P \vee Q$	
$\perp$	$\perp$	$\perp$	$\top$	$\top$	$\top$	$\perp$	$\perp$
	$\perp$	$\top$	$\top$	$\top$	$\top$	$\perp$	$\perp$
	$\top$	$\perp$	$\top$	$\perp$	$\top$	$\top$	$\perp$
	$\top$	$\top$	$\top$	$\top$	$\perp$	$\top$	$\perp$
$\top$	$\perp$	$\perp$	$\perp$	$\top$	$\top$	$\top$	$\perp$
	$\perp$	$\top$	$\top$	$\top$	$\top$	$\top$	$\top$
	$\top$	$\perp$	$\top$	$\perp$	$\top$	$\top$	$\perp$
	$\top$	$\top$	$\top$	$\top$	$\perp$	$\top$	$\perp$

Figure 9.4: Structured Truth Table

```

SAT( $\phi$ )=
  let  $\phi' = \text{simplify}(\phi)$ 
  if  $\phi'$  is a Boolean constant then return  $\phi'$ 
  let  $v$  be a free variable of  $\phi'$ 
  return SAT( $\phi'[v \leftarrow \perp]$ ) or SAT( $\phi'[v \leftarrow \top]$ )      // short-circuit eval

```

Figure 9.5: SAT Algorithm

heuristic that has a decisive influence on the efficiency of the algorithm. Another question is which simplifications to use. Assuming input in conjunctive normal form, common SAT solvers use simplifications that can schematically be described as follows:

- $\neg \top \rightsquigarrow \perp$
- $\neg \perp \rightsquigarrow \top$
- $\dots \vee \top \vee \dots \rightsquigarrow \top$
- $\dots \vee \perp \vee \dots \rightsquigarrow \dots \vee \dots$
- $\dots \wedge \top \wedge \dots \rightsquigarrow \dots \wedge \dots$
- $\dots \wedge \perp \wedge \dots \rightsquigarrow \perp$

Assuming choice of variables in alphabetical order, and choice of $\perp$ before $\top$, the algorithm proceeds as follows for the input formula $[P \vee Q] \wedge [P \vee \neg Q \vee R]$:

call	simplification	further action
SAT($[P \vee Q] \wedge [P \vee \neg Q \vee R]$)		$P \leftarrow \perp$
SAT($[\perp \vee Q] \wedge [\perp \vee \neg Q \vee R]$)	$Q \wedge [\neg Q \vee R]$	$Q \leftarrow \perp$
SAT($\perp \wedge [\neg \perp \vee R]$)	$\perp$	backtrack, $Q \leftarrow \top$
SAT($\top \wedge [\neg \top \vee R]$)	R	$R \leftarrow \perp$
SAT($\perp$)		backtrack, $R \leftarrow \top$
SAT($\top$)		return $\top$

Here we observe that the choice $P \leftarrow \top$ would have resulted in immediate success!

Current algorithms use more simplifications. Consider the formula

$$[\neg Q \vee R] \wedge [\neg Q \vee \neg R] \wedge Q$$

for which it is clear that any satisfiable assignment must assign $\top$ to the variable Q . In general, whenever a clause appears that contains only one literal, we can immediately assign a value to corresponding variable. This simplification is called *unit propagation*.

Also, consider the following example:

$$[\neg Q \vee R \vee S \vee V] \wedge [\neg Q \vee \neg R \vee S \vee \neg V] \wedge [Q \vee V]$$

Here, the variable S only occurs positively. Using the fact that a variable assignment satisfies a conjunction of clauses iff it satisfies every clause one sees that a satisfying assignment must set this variable to $\top$. In general, if a variable occurs only positively or only negatively, we can immediately assign a truth value that evaluates all corresponding clauses to $\top$. This simplification is called *pure literal elimination*.

The resulting algorithm (i.e., the SAT algorithm from Figure 9.3, the basic simplifications, unit propagation and pure literal elimination) is called the Davis-Putnam-Logemann-Loveland algorithm (DPLL) [18].

Here is an example of its execution:

$$\text{SAT}([\neg P \vee Q \vee R] \wedge [\neg Q \vee R] \wedge [\neg Q \vee \neg R] \wedge [P \vee Q])$$

No simplification, recursive call with $[P \leftarrow \perp]$

$$\text{SAT}([\neg \perp \vee Q \vee R] \wedge [\neg Q \vee R] \wedge [\neg Q \vee \neg R] \wedge [\perp \vee Q])$$

Basic simplifications:

$$[\neg Q \vee R] \wedge [\neg Q \vee \neg R] \wedge Q$$

Unit propagation $[Q \leftarrow \top]$

$$[\neg \top \vee R] \wedge [\neg \top \vee \neg R] \wedge \top$$

Basic simplification

$$R \wedge \neg R$$

Unit propagation $[R \leftarrow \top]$:

$$\top \wedge \neg \top$$

Basic simplification

$$\perp$$

Backtrack, recursive call with $[P \leftarrow \top]$

$$\text{SAT}([\neg \top \vee Q \vee R] \wedge [\neg Q \vee R] \wedge [\neg Q \vee \neg R] \wedge [\top \vee Q])$$

Basic simplifications:

$$[Q \vee R] \wedge [\neg Q \vee R] \wedge [\neg Q \vee \neg R]$$

Recursive call with $[Q \leftarrow \perp]$

$$\text{SAT}([\perp \vee R] \wedge [\neg \perp \vee R] \wedge [\neg \perp \vee \neg R])$$

Basic simplification

$$R$$

Unit propagation [$R \leftarrow \top$]:

$$\top$$

Now, due to short-circuit evaluation, the algorithm terminates, returning $\top$. Hence the input formula is satisfiable by the assignment $\{P \mapsto \top, Q \mapsto \perp, R \mapsto \top\}$.

9.4 Conflict Driven Clause Learning (CDCL)

9.4.1 Basic Idea

Modern DPLL based SAT solvers contain many improvements. For example, they usually avoid pure clause learning, but add conflict driven clause learning (CDCL). The main idea can be observed in the example from above, checking satisfiability of the formula

$$[\neg P \vee Q \vee R] \wedge [\neg Q \vee R] \wedge [\neg Q \vee \neg R] \wedge [P \vee Q].$$

Our example showed that after the choice $P \leftarrow \perp$, DPLL uses unit propagation to choose values for the variables Q and R , which results in the formula $\perp$. In this situation we want to remember that the choice $P \leftarrow \perp$ cannot be successful, independent of any other choice. Hence we add the clause P to the input formula, resulting in

$$[\neg P \vee Q \vee R] \wedge [\neg Q \vee R] \wedge [\neg Q \vee \neg R] \wedge [P \vee Q] \wedge P.$$

For this formula, unit propagation will immediately choose $P \leftarrow \top$, avoiding the unsuccessful choice from before.

In general, after simplification to $\perp$ (which is called a *conflict*), CDCL adds a clause (called *conflict clause*) that excludes the assignments that lead to this conflict. For assignments of variables $v_1, \dots, v_j$ to $\perp$ and $v_{j+1}, \dots, v_n$ to $\top$, a possible conflict clause is

$$\neg[\neg v_1 \wedge \dots \wedge \neg v_j \wedge v_{j+1} \wedge \dots \wedge v_n]$$

that is

$$v_1 \vee \dots \vee v_j \vee \neg v_{j+1} \vee \dots \vee \neg v_n,$$

which excludes this assignment. There are more advanced methods for analyzing the causes of conflicts, and that result in stronger conflict clauses that exclude many more assignments than the basic conflict clause just mentioned.

Unlike basic DPLL, which always tries to assign both a Boolean constant and its negation, CDCL only uses one explicit assignment of Boolean constants to variables. The reason is that conflict clauses exclude the already chosen values, and as soon as a conflict clause is added, unit propagation on conflict clauses chooses alternative values.

9.4.2 Details

Consider an input formula containing the following clauses:

$$\begin{array}{ll}
 c_1: & v_{223} \vee \neg v_{355} \\
 c_2: & v_{343} \vee v_{355} \vee v_{634} \\
 c_3: & v_{343} \vee v_{355} \vee \neg v_{634} \\
 \dots & \\
 c_r: &
 \end{array}$$

Now assume that the solver assigns $\perp$ to the variables v_{223} which, after simplification, results in the following clauses:

$$\begin{array}{ll}
 c_2: & v_{343} \vee v_{634} \\
 c_3: & v_{343} \vee \neg v_{634} \\
 \dots & \\
 c_r: &
 \end{array}$$

If the solver further later assigns $\perp$ to v_{343} then it backtracks. The same situation occurs for all $2 \dots$ assignments with $v_{223} = \perp, v_{343} = \perp$.

CDCL avoids this, by maintaining a so-called *implication graph* (Figure 9.6) that records all assignments of truth values to variables together with the reasons for those assignments. From the implication graph one can see, that one can avoid the same situation, by adding the conflict clause $\neg [\neg v_{223} \wedge \neg v_{343}]$, that is $v_{223} \vee v_{343}$.

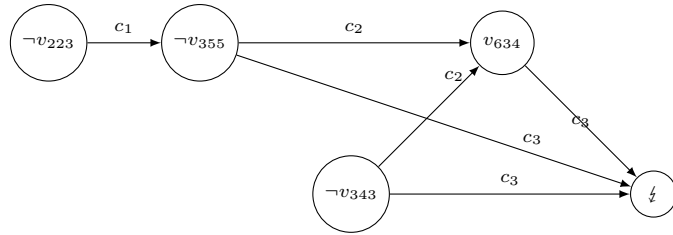


Figure 9.6: Implication Graph

As one can see, the vertices of the implication graph are formed by literals, and by the special vertex $\perp$. The edges are labeled with clauses (from the original problem, without simplification). During execution of the SAT algorithm in the corresponding implication graph,

- each assignment of $\top$ to a variable v corresponds to a vertex v ,
- each assignment of $\perp$ to a variable v corresponds to a vertex $\neg v$,
- each unit propagation of a literal l that belongs to a clause $l_1 \vee \dots \vee l_k \vee l$ of the original formula, corresponds, for each $i \in \{1, \dots, k\}$, to an edge from $\neg l_i$ to l labeled with $l_1 \vee \dots \vee l_k \vee l$, and
- each derivation of $\perp$ that belongs to a clause $l_1 \vee \dots \vee l_k$ of the original formula,

corresponds, for each $i \in \{1, \dots, k\}$, to
 an edge from $\neg l_i$ to $\bot$ labeled with $l_1 \vee \dots \vee l_k$.

Here, double negations cancel out.

Within the algorithm, the simplification step adds internal vertices, the conflict vertex $\bot$, and the corresponding edges to the graph, backtracking also backtracks the implication graph, and recursive calls that assign $\perp$ and $\top$ to a variable, add a new source vertex to the graph. For example, if the assignment $v_{343} \leftarrow \perp$ resulted in the conflict shown in Figure 9.6, then the graph can backtrack to the one in Figure 9.7, and we can continue with the next assignment $v_{343} \leftarrow \top$.

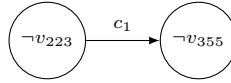


Figure 9.7: Implication Graph, Backtracked

From an implication graph, it is possible to extract not only one, but several conflict clauses. In the example of Figure 9.6 those are $\neg v_{634} \vee v_{355} \vee v_{343}$, $v_{355} \vee v_{343}$, $v_{223} \vee v_{343}$.

In general, cc is a *conflict clause* iff there exists a subgraph C of the implication graph s.t.

- C contains $\bot$
- there is a clause c s.t.
 C contains all edges labeled with c and leading to $\bot$,
- for each internal vertex l of C
 there is a clause c s.t.
 C contains all edges labeled with c and leading to l , and
- the negation of each root of C is a literal of cc .

9.5 Implementation Techniques

The SAT algorithm performs

- a substitution $\phi'[v \leftarrow \perp]$, then
- backtracks back to ϕ' , and then performs
- another substitution $\phi'[v \leftarrow \top]$.

So it has to store copies of those formulas in memory. One can do better by not changing formulas (neither by substitution, nor by simplification), but to just store newly computed information.

Also, practical implications replace the recursive algorithm by a loop.

9.6 Local Search

Local search was originally used for continuous problems, but can also be applied to solving the SAT problem. The intuition is to first choose a random assignment, and then to gradually change it such that the number of satisfied clauses increases. For example,

$GSAT(\phi)$:

```

for  $i \leftarrow 1$  to MAXTRIES do
   $V \leftarrow$  random assignment
  for  $i \leftarrow 1$  to MAXFLIPS do
    if  $V$  satisfies  $\phi$  then return  $\top$ 
    else
      Flip any variable in  $\phi$  that results in
        greatest decrease in the number of unsatisfied clauses

```

Of course, local search is incomplete and cannot prove unsatisfiability. But local search methods can often find satisfying assignments fast. There are many variants, for example Walksat ³

9.7 SAT Modulo Theory

SAT modulo theory problems do not only allow Boolean variables but also variables with other domains, for example $\mathbb{N}$, $\mathbb{R}$, arrays, lists, etc., and corresponding constraints (e.g., $2x + 3y^2 \leq 0$). For example, the solver iSAT [20] was the first SAT solver world-wide that was able to reliably handle non-linear equalities and inequalities over the real numbers.

9.8 Improvements for Bounded Model Checking

$BMC(\mathbf{G}p, n)$:

$$S_0(\vec{v}^0) \wedge \bigwedge_{i=0}^{n-2} T(\vec{v}^i, \vec{v}^{i+1}) \wedge \bigvee_{i=0}^{n-1} \neg p(\vec{v}^i)$$

unsat of $\bigwedge_{i=s}^t \hat{T}(\vec{v}^i, \vec{v}^{i+1})$ also holds for shifted version

$$\bigwedge_{i=s+1}^{t+1} \hat{T}(\vec{v}^i, \vec{v}^{i+1}), \bigwedge_{i=s+2}^{t+2} \hat{T}(\vec{v}^i, \vec{v}^{i+1}) \dots$$

So: shifted conflict clauses

Check for $BMC(\mathbf{G} \mathbf{p}, 0), BMC(\mathbf{G} \mathbf{p}, 1), BMC(\mathbf{G} \mathbf{p}, 2), \dots$

Re-use information: unsat implied by

$\hat{S}_0(\vec{v}^0) \wedge \bigwedge_{i=0}^k \hat{T}(\vec{v}^i, \vec{v}^{i+1})$ unsat also holds for higher k

So: re-use conflict clauses

³<http://www.cs.rochester.edu/u/kautz/walksat/>

9.9 Checking the Safety Verification Conditions

In general, SAT solvers can be used in many applications, for example for reasoning about sets. To understand why, let us return to the safety certificates introduced in Chapter 7. For example, somebody else might claim that a certain system is safe, and might substantiate this claim using a safety certificate. We now want to do an independent check, whether the claim is indeed correct.

So let us recall the safety verification conditions:

- $S_0 \subseteq V$
- $Post_R(V) \subseteq V$
- $V \subseteq \mathcal{I}(\text{ok})$

One immediately observes, that they mainly consist of set operations, especially the subset relation. Checking them is no problem for small sets, but may be difficult for large or infinite sets. Hence we will represent those sets symbolically, using logical formulas, and check them using a SAT solver. So assume that we have a transition system, the claimed safety certificate V , and the set of safe states given symbolically, using logical formulas:

$$(S, \{x \mid S_0(x)\}, \{(x, x') \mid R(x, x')\}), \{x \mid \text{ok}(x)\}, \{x \mid V(x)\}$$

For example, if the transition system is the one from Figure 7.2, then we can represent the state space $\{A, B, C, D, E\}$ using a vector of three bits, for example, as follows:

	x_1	x_2	x_3
A	$\top$	$\top$	$\top$
B	$\top$	$\top$	$\perp$
C	$\top$	$\perp$	$\top$
D	$\top$	$\perp$	$\perp$
E	$\perp$	$\perp$	$\perp$

Since the set of initial states S_0 is $\{A\}$, we can represent it using the formula $x_1 \wedge x_2 \wedge x_3$. The transition relation R is represented by a formula beginning with $[x_1 \wedge x_2 \wedge x_3 \wedge x'_1 \wedge x'_2 \wedge \neg x'_3] \vee \dots$. The set $\mathcal{I}(\text{ok})$ is represented by the formula x_1 , and the safety certificate $\{A, B, C\}$ by the formula $x_1 \wedge [x_2 \vee x_3]$.

Since we have logical formulas instead of sets now, we write the safety verification conditions equivalently in the form of logical formulas:

- $\forall x . S_0(x) \Rightarrow V(x)$
- $\forall x \forall x' . [V(x) \wedge R(x, x')] \Rightarrow V(x')$
- $\forall x . V(x) \Rightarrow \text{ok}(x)$

This just means that the formulas without the universal quantifiers have to be valid.

- $S_0(x) \Rightarrow V(x)$
- $[V(x) \wedge R(x, x')] \Rightarrow V(x')$
- $V(x) \Rightarrow \text{ok}(x)$

So we want to prove those formulas. For our concrete transition system this means that we need to prove

- $[x_1 \wedge x_2 \wedge x_3] \Rightarrow [x_1 \wedge [x_2 \vee x_3]]$
- $[x_1 \wedge [x_2 \vee x_3] \wedge R(x_1, x_2, x_3, x'_1, x'_2, x'_3)] \Rightarrow [x'_1 \wedge [x'_2 \vee x'_3]]$
- $[x_1 \wedge [x_2 \vee x_3]] \Rightarrow x_1$

But SAT solvers do not prove, they check satisfiability! Here we can use the fact that a formula is valid if its negation is unsatisfiable. Hence it suffices to check unsatisfiability of

- $\neg[S_0(x) \Rightarrow V(x)]$
- $\neg[[V(x) \wedge R(x, x')] \Rightarrow V(x')]$
- $\neg[V(x) \Rightarrow \text{ok}(x)]$

which is

- $S_0(x) \wedge \neg V(x)$
- $V(x) \wedge R(x, x') \wedge \neg V(x')$
- $V(x) \wedge \neg \text{ok}(x)$

9.10 General Usage of SAT Algorithms

SAT is *the* basic NP-complete problem, and many other NP-complete problems are often translated to SAT. The result is often more efficient than more specific algorithms. In general, SAT is one of the main workhorses for discrete reasoning, which are

- MILP,
- constraint programming, and
- SAT.

9.11 Example applications

- Intel, AMD, ... commonly use SAT solvers to check the correctness of their chips
- Many electronic design automation tools have SAT solvers built in
- Microsoft uses SAT for test generation (see MI-FME)
- Resolution of Linux package dependencies (ZYpp)
- Gene analysis [30]

Chapter 10

Timed Automata

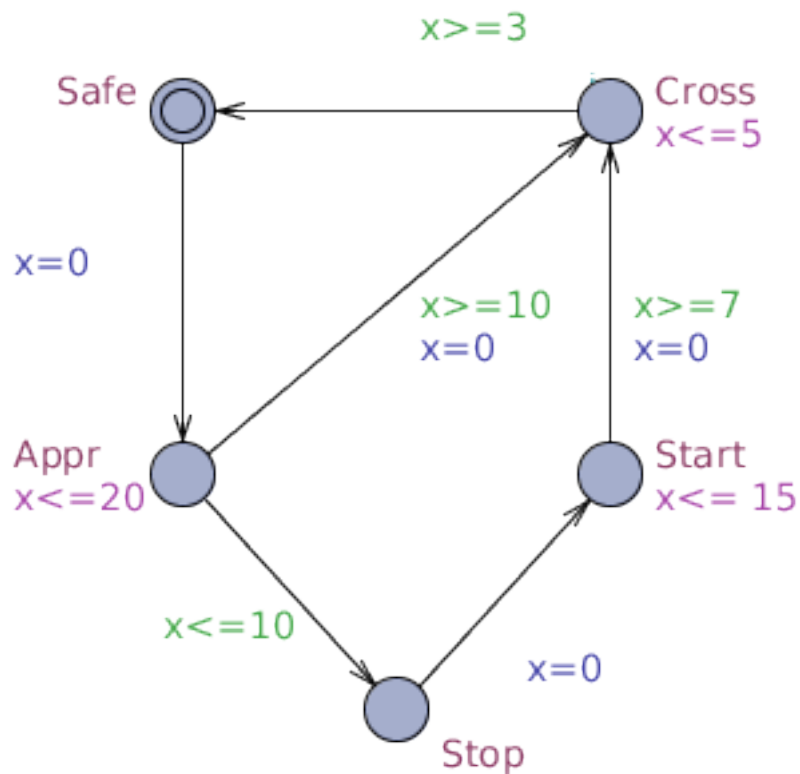


Figure 10.1: Timed Automaton

Figure 10.1 shows a model of a train crossing a bridge. Initially, the train is at a safe position, far away from the bridge. This is modeled by the vertex labelled “Safe”. For reasons that will become clear later, we will not call these vertices states, but locations. The train may then approach the bridge, which is

modeled by the location “Appr”. It may stay in this location for up to twenty time units. For measuring time, variables are used, that are called clocks. For example, here the variable x measures—among other things—the time the train is approaching: The transition from “Safe” to “Appr” resets this clock to zero, and then the condition $x \leq 20$, an invariant, has to hold while staying in location “Appr”. The train may then either change to location “Stop” or to location “Cross”. In the first case, the condition $x \leq 10$ has to hold, in the second case $x \geq 10$. These condition, called guards, express the fact that the train is allowed to stop only when having approached for maximally 10 time units. However, it can only cross the bridge after approaching for at least 10 time units.

The figure represents a so-called *timed automaton*. Concretely, the model stems from the tool UPPAAL that not only allows modeling of timed automata, but also

- composition,
- message passing, and
- finite data structures.

For arriving at a precise formal definition of such timed automata, we first define how the values of such clocks can be constrained:

Definition 33 *Given a set of clocks X , a clock constraint is a conjunction of strict or non-strict inequalities between variables from X and rationals, denoted by $\mathcal{C}(X)$.*

Based on this, we formally define timed automata as follows:

Definition 34 *A timed automaton is a 5-tuple $(L, L_0, X, \mathcal{I}, T)$ where*

- L is a finite set (locations)
- $L_0 \subseteq L$ (starting locations), where $L_0 \neq \emptyset$
- X is a finite set (clocks)
- $\mathcal{I} : L \rightarrow \mathcal{C}(X)$ (location invariants)
- $T \subseteq L \times \mathcal{C}(X) \times \mathcal{P}(X) \times L$ (transitions with guards and resets)

Attention: Here we meet the term “invariant” again, that denotes different, but often similar, things in many different contexts. The definition of a timed automaton does not define any inputs or outputs. However, one can easily extend it in such a way.

The example from the figure above corresponds to the following timed automaton (we do not list all transitions, denoting the missing transitions by dots):

$$\left(\begin{array}{l} \{Safe, Appr, Stop, Start, Cross\}, \\ \{Safe\}, \\ \{x\}, \\ \{Appr \mapsto x \leq 20, Start \mapsto x \leq 15, Cross \mapsto x \leq 5, Safe \mapsto \top, Stop \mapsto \top\} \\ \{(Safe, \top, \{x\}, Appr), (Appr, x \leq 10, \{\}, Stop), (Stop, \top, \{x\}, Start), \dots\} \end{array} \right)$$

10.1 State and Evolution of Timed Automata

We will now formalize the behavior of timed automata. Throughout we will assume a timed automaton $(L, L_0, X, \mathcal{I}, T)$.

The timed automaton in the example above might, for example, currently be

- in location *Appr* where
- 10 seconds have elapsed since resetting clock x .

We formalize this as follows:

Definition 35 A state of a timed automaton is a pair (l, v) , where

- $l \in L$ is a location, and
- $v : X \rightarrow \mathbb{R}_{\geq 0}$ (clock assignment).

Hence, the state described above is the pair $(\textit{Appr}, \{x \mapsto 10\})$. Another state is $(\textit{Appr}, \{x \mapsto 30\})$, however, the invariant of location *Appr*, which is $x \leq 20$, is not compatible with this state. To avoid such states, we call a state *consistent* iff its clock assignment satisfies the invariant of its location. Formally, a state (l, v) is consistent iff $v \models \mathcal{I}(l)$.

The set of *set of initial states* of a timed automaton is

$$\{(l, v) \mid l \in L_0, \text{ for all } x \in X, v(x) = 0\}.$$

A state can evolve in two different ways:

- Time can elapse while staying in the same location. For example, assuming that the timed automaton has a second clock y , the state may change from $(\textit{Appr}, \{x \mapsto 10, y \mapsto 7\})$ to $(\textit{Appr}, \{x \mapsto 15, y \mapsto 12\})$.
- The location can change, e.g., from $(\textit{Safe}, \{x \mapsto 355\})$ to $(\textit{Appr}, \{x \mapsto 0\})$.

For defining the evolution of time, we use the following notation: For a clock assignment v , $d \in \mathbb{R}_{\geq 0}$, $v + d$ is a clock assignment such that for all $x \in X$, $(v + d)(x) := v(x) + d$. In the example above, we add a delay of 5 to the clock assignment $\{x \mapsto 10, y \mapsto 7\}$, and we get the result of this delay by $\{x \mapsto 10, y \mapsto 7\} + 5 = \{x \mapsto 15, y \mapsto 12\}$.

Now we can formalize the two types of transitions as follows:

Definition 36 A delay transition has the form $(l, v) \xrightarrow{d} (l, v + d)$, where $d \in \mathbb{R}_{\geq 0}$, and for every $e \in [0, d]$, $v + e \models \mathcal{I}(l)$.

So, in a delay transition, the location l stays the same, the delay d is added to the clock assignment v , and throughout the invariant of the location has to hold.

When changing locations, certain clocks are reset to zero. So, for a clock assignment v and set of clocks λ , we use the notation $v[\lambda \leftarrow 0]$ which is such that for all $x \in X$,

$$v[\lambda \leftarrow 0](x) := \begin{cases} 0, & \text{if } x \in \lambda, \text{ and} \\ v(x), & \text{otherwise.} \end{cases}$$

Definition 37 An action transition has the form $(l, v) \dot{\rightarrow} (l', v')$, s.t. there is a transition $(l, \phi, \lambda, l') \in \mathcal{T}$ s.t. $v \models \phi$, and $v' = v[\lambda \leftarrow 0]$.

So, for an action transition, the guard ϕ has to hold, and the clocks are reset according to λ .

A transition of a timed automaton is a combination of delay and action transition:

Definition 38 $(l, v) \rightarrow (l', v')$ iff there is $d \in \mathbb{R}_{\geq 0}$ s.t.

$$(l, v) \xrightarrow{d} (l, v + d), (l, v + d) \dot{\rightarrow} (l', v').$$

For example, the timed automaton in Figure 10.1 allows a transition

$$(Cross, \{x \mapsto 0\}) \rightarrow (Safe, \{x \mapsto 4\})$$

because the choice $d \leftarrow 4$ results in the delay transition

$$(Cross, \{x \mapsto 0\}) \xrightarrow{4} (Cross, \{x \mapsto 4\})$$

whose combination with the action transition

$$(Cross, \{x \mapsto 4\}) \dot{\rightarrow} (Safe, \{x \mapsto 4\})$$

is the transition above.

Note that we have non-determinism here, since from some states several different transitions are possible that differ in the choice of d or the further location s' .

Now consider the timed automaton from Figure 10.2. This automaton cannot even do a single transition. The problem is that the invariant of location A does not allow any delay transition to reach a state that satisfies the transition to location B . We would like to avoid timed automata that have such deadlocks. One way of doing so would be to analyze all possibilities of how the timed automaton may evolve and check whether it can reach a deadlock. To

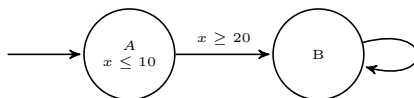


Figure 10.2: Timed Automaton with Deadlock

avoid such a complicated reachability analysis let us have a look at the definition of transition system (Definition 20). Here, the transition relation is required to be left-total, that is for each state s at least one transition must be possible. We can also enforce this for timed automata by requiring that for every consistent state s there is a consistent state s' s.t. $s \rightarrow s'$. If this is the case, we can also view timed automata as transition systems. More concretely, if

- S is the set of consistent states $\{(l, v) \in L \times [X \rightarrow \mathbb{R}_{\geq 0}] \mid v \models \mathcal{I}(l)\}$
- S_0 is the set of initial states $\{(l, v) \mid l \in L_0, \text{ for all } x \in X, v(x) = 0\}$

- R is the transition relation $\rightarrow$, and for all $s \in S$ there exists $s' \in S$ s.t. $(s, s') \in R$,

then (S, S_0, R) is a transition system.

As a consequence, we can apply everything we know about transition systems also to timed automata, for example, the notions of path, set of reachable states, safety verification conditions, invariant (of transition system), inductive invariant, ...

10.2 Simulation

Figure 10.3 shows a timed automaton modeling a process that is usually working, but sometimes writes something into a log file. Moreover, every 30 seconds, the process has to be restarted. Here is an example of a sequence of (delay and

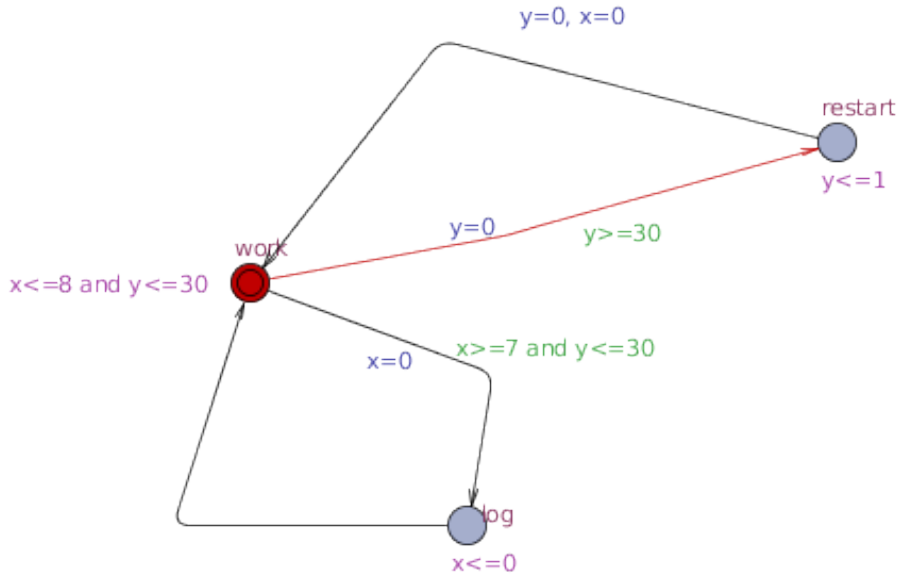


Figure 10.3: Timed Automaton—Server

action) transitions of this automaton:

$$\begin{array}{ll}
 (work, \{x \mapsto 0, y \mapsto 0\}) & \xrightarrow{7} (work, \{x \mapsto 7, y \mapsto 7\}) \hat{\rightarrow} \\
 (log, \{x \mapsto 0, y \mapsto 7\}) & \xrightarrow{0} (log, \{x \mapsto 0, y \mapsto 7\}) \hat{\rightarrow} \\
 (work, \{x \mapsto 0, y \mapsto 7\}) & \xrightarrow{8} (work, \{x \mapsto 8, y \mapsto 15\}) \hat{\rightarrow} \\
 (log, \{x \mapsto 0, y \mapsto 15\}) & \xrightarrow{0} (log, \{x \mapsto 0, y \mapsto 15\}) \hat{\rightarrow} \\
 \dots & \\
 (restart, \{x \mapsto 0, y \mapsto 0\}) & \xrightarrow{1} (restart, \{x \mapsto 1, y \mapsto 1\}) \hat{\rightarrow} \\
 \dots &
 \end{array}$$

Hence, one can simulate timed automata as follows:

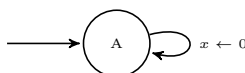
let (l, v) be s.t. $l \in L_0, \forall x \in X . v(x) = 0$
 loop
 choose $d \in \mathbb{R}_{\geq 0}$, transition $(l, \phi, \lambda, l') \in \mathcal{T}$ such that
 $\forall e \in [0, d] . v + e \models \mathcal{I}(l)$ // *current invariant not violated*
 $v + d \models \phi$ // *guard for transition to l' holds*
 $v + d \models \mathcal{I}(l')$ // *next invariant not violated*
 let (l, v) be result of the action trans. from $(l, v + d)$ using (l, ϕ, λ, l')

10.3 Specification

A timed automaton has a corresponding transition system, so we can use LTL to specify properties of timed automata, for example:

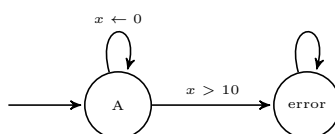
- **Gok**, where $\mathcal{I}(ok) = \{(l, v) \mid l = \text{restart} \Rightarrow v(y) \leq 20\}$
- **Fgoal** where $\mathcal{I}(goal) = \{(l, v) \mid l = \text{restart}\}$

Now consider the following timed automaton:



Somebody might expect that the answer to the question **Gok**, where $\mathcal{I}(ok) = \{(l, v) \mid v(x) \leq 10\}$ is false, since the clock x can reach arbitrarily high values in location A . However, this answer is *not* correct, due to the fact that a transition of a timed automaton always is a pair consisting of a delay and an action transition. The clock x can reach high values only using a delay transition alone, but not after such a pair, since the following action transition resets the clock x to zero.

If one wants to ask the question whether the x can reach a value higher than 10 after a delay transition alone, one can transform the above automaton to the following one:



Now the desired question can be simply formulated as **Gok**, where $\mathcal{I}(ok) = \{(l, v) \mid l = A\}$.

10.4 Testing

Here, one has a set of test cases, and wants to answer the questions whether theses test cases fulfill the specification. However, before answering this question, one has to generate such a set of test cases, and store them. The obvious candidate here, is to use the simulation algorithm from Section 10.2 for this. However, when analyzing this algorithm, one immediately hits upon the line

choose $d \in \mathbb{R}_{\geq 0}$, transition $(l, \phi, \lambda, l') \in \mathcal{T}$ such that

Here, the algorithm chooses from the set $\mathbb{R}_{\geq 0}$, which is not even infinite, but even uncountable. Hence we have one further occurrence of infinity:

- infinite length of paths
- infinite number of paths
- from each state, an infinite (even uncountable) number of successors

This is one of the reasons why testing of real-time system is so difficult.

It is also possible to do bounded model checking on timed automata. This is a straight-forward extension of bounded model checking for finite automata, using an extension of SAT with a constraint solver [3, 16] (cf. the notion of *difference logic*).

10.5 Symbolic Simulation

We observe that infinity in the form of an infinite number of successor states is the result of timed non-determinism. Infinite objects cannot be directly represented on computers, but they can be represented using a symbolic representation. Sets of real numbers can be represented symbolically using equalities and inequalities. The following is a sequence of delay and action transitions that does not use single states, but a symbolic representation of *all* states following a certain sequence of locations:

$$\begin{array}{ll}
 (work, x = 0 \wedge y = 0) & \xrightarrow{\sim} (work, x \leq 8 \wedge x = y) \xrightarrow{\sim} \\
 (log, x = 0 \wedge 7 \leq y \leq 8) & \xrightarrow{\sim} (log, x = 0 \wedge 7 \leq y \leq 8) \xrightarrow{\sim} \\
 (work, x = 0 \wedge 7 \leq y \leq 8) & \xrightarrow{\sim} (work, x \leq 8 \wedge 7 \leq y - x \leq 8) \xrightarrow{\sim} \\
 (log, x = 0 \wedge 14 \leq y \leq 16) & \xrightarrow{\sim} (log, x = 0 \wedge 14 \leq y \leq 16) \xrightarrow{\sim} \\
 (work, x = 0 \wedge 14 \leq y \leq 16) & \xrightarrow{\sim} (work, x \leq 8 \wedge 14 \leq y - x \leq 16) \xrightarrow{\sim} \\
 (log, x = 0 \wedge 21 \leq y \leq 24) & \xrightarrow{\sim} (log, x = 0 \wedge 21 \leq y \leq 24) \\
 \dots &
 \end{array}$$

Observe that here, delay transitions do not have a fixed length any more. Hence we use the symbol $\xrightarrow{\sim}$ to denote delay transitions of arbitrary length. Since the example has only two clocks, we can also draw those sets. The result can be seen in Figure 10.4, where the reachable states are drawn in dark blue, and the intermediate states between a timed and action transition in light blue.

To formalize this we define the elements of such a symbolic simulation:

Definition 39 A symbolic state has the form (l, ν) , where l is a location and ν is a conjunction of inequalities. A symbolic state (l, ν) represents the set of states

$$\llbracket (l, \nu) \rrbracket := \{(l, v) \mid v \models \nu\}.$$

However, to keep things as simple as possible, we will not always distinguish between symbolic states and the sets they represent.

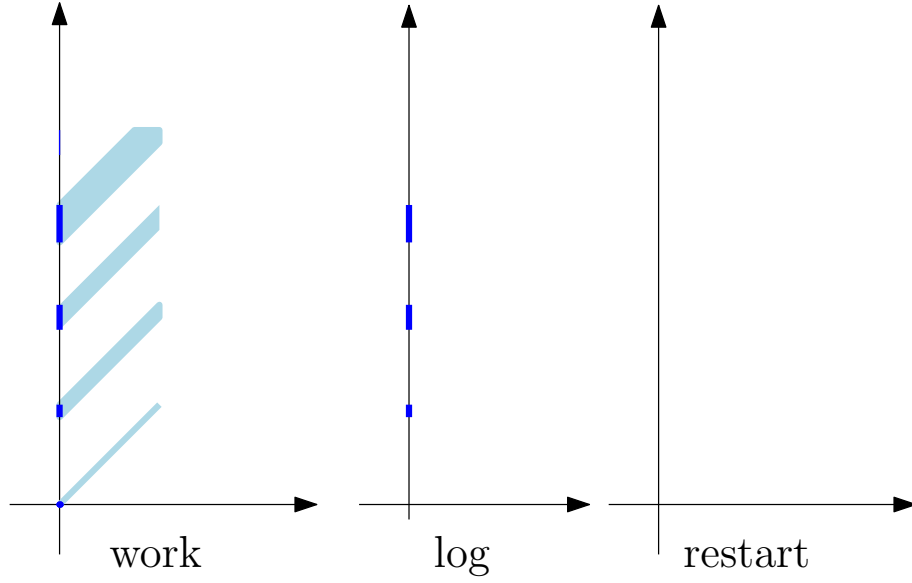


Figure 10.4: Symbolic Simulation

10.5.1 Computation of Symbolic Delay Transitions

Now let us analyze in more detail how to compute delay transitions on symbolic states, as denoted by

$$(l, \nu) \dot{\rightarrow} (l', \nu').$$

We want the set of states represented by the result (l', ν') to be the same as taking delay transitions on the set of states represented by (l, ν) , that is,

$$\llbracket (l', \nu') \rrbracket = \{(l', v') \mid (l, v) \in \llbracket (l, \nu) \rrbracket, (l, v) \xrightarrow{d} (l', v'), d \in \mathbb{R}_{\geq 0}\}.$$

In other words, the diagram

$$\begin{array}{ccc} (l, \nu) & \xrightarrow{\sim} & (l', \nu') \\ \downarrow \llbracket \cdot \rrbracket & & \downarrow \llbracket \cdot \rrbracket \\ \llbracket (l, \nu) \rrbracket & \xrightarrow{d \in \mathbb{R}_{\geq 0}} & \llbracket (l', \nu') \rrbracket \end{array}$$

should commute, that is, taking the top/right path should have the same result as the left/below path. This holds if

- $l' = l$ and
- $v' \models \nu'$ iff there exists a clock assignment v and $d \in \mathbb{R}_{\geq 0}$ s.t.
 - $v \models \nu$, (v satisfies the inequalities of the source)
 - $\forall e \in [0, d] . v + e \models \mathcal{I}(l)$, and (the invariant holds throughout)
 - $v' = v + d$. (v' is the result of the delay transition)

So we see that a symbolic state (l, ν) has a delay transition to precisely one symbolic state, and this symbolic state has the same location as the original state.

Now let us use this to compute the successor of the symbolic state $(work, x = 0 \wedge 21 \leq y \leq 24)$. Due to the first condition above, the location of the successor state will again be *work*. For computing the inequalities of the successor state, we use the second condition. Substituting the inequalities of the current symbolic state and the invariant of its location into those conditions, we get

$$\exists x, y, d . \left[\begin{array}{l} x = 0 \wedge 21 \leq y \leq 24 \wedge \\ x \leq 8 \wedge y \leq 30 \wedge x' \leq 8 \wedge y' \leq 30 \wedge \\ x' = x + d \wedge y' = y + d \end{array} \right]$$

Here, the variable x is quantified, and also occurs in the equality $x = 0$. So we can eliminate the quantified variable x by using this equality as a substitution. The result is

$$\exists y, d . \left[\begin{array}{l} 21 \leq y \leq 24 \wedge \\ 0 \leq 8 \wedge y \leq 30 \wedge x' \leq 8 \wedge y' \leq 30 \wedge \\ x' = d \wedge y' = y + d \end{array} \right]$$

Here we remove the trivially satisfied inequality $0 \leq 8$. We continue by using the equality $x' = d$ in a similar way as before for eliminating the quantified variable d . The result is

$$\exists y . \left[\begin{array}{l} 21 \leq y \leq 24 \wedge \\ y \leq 30 \wedge x' \leq 8 \wedge y' \leq 30 \wedge \\ y' = y + x' \end{array} \right]$$

Finally, using the equality $y' = y + x'$, which is equivalent to $y = y' - x'$, we eliminate the quantifier for y , and get

$$21 \leq y' - x' \leq 24 \wedge y' - x' \leq 30 \wedge x' \leq 8 \wedge y' \leq 30.$$

Now, the inequality $y' - x' \leq 30$ is redundant, and hence we can simplify this to

$$21 \leq y' - x' \leq 24 \wedge x' \leq 8 \wedge y' \leq 30$$

from which we get the result

$$21 \leq y - x \leq 24 \wedge x \leq 8 \wedge y \leq 30$$

by renaming the primed variable names to the original variable names.

Summarizing, the state

$$(work, x = 0 \wedge 21 \leq y \leq 24)$$

has the symbolic successor state:

$$(work, 21 \leq y - x \leq 24 \wedge x \leq 8 \wedge y \leq 30).$$

We see that for representing the resulting sets we need inequalities of the form $x - y \prec c$, $x \prec c$, and $c \prec x$, where $\prec$ is either $<$ or $\leq$. We call conjunctions of such inequalities *clock zones*.

10.5.2 Computation of Symbolic Action Transitions

Before defining action transitions from symbolic states, we observe that from a given symbolic state there might be more than one action transition. For example, from the symbolic state

$$(work, x \leq 8 \wedge 21 \leq y - x \leq 24 \wedge y \leq 30)$$

our example allows two possible action transition, one to the state

$$(log, x = 0 \wedge 28 \leq y - x \leq 30),$$

and the other to the state

$$(restart, y = 0 \wedge 6 \leq x \leq 8).$$

Symbolic simulation then chooses one of the two. In general, for every symbolic state (l, ν) , $(l, \nu) \xrightarrow{\alpha} (l', \nu')$ iff there is a transition $(l, \phi, \lambda, l') \in \mathcal{T}$ such that $\llbracket (l', \nu') \rrbracket$ contains the states reached by this transition from $\llbracket (l, \nu) \rrbracket$. This means that the diagram

$$\begin{array}{ccc} (l, \nu) & \xrightarrow{\alpha} & (l', \nu') \\ \downarrow \llbracket \cdot \rrbracket & & \downarrow \llbracket \cdot \rrbracket \\ \llbracket (l, \nu) \rrbracket & \xrightarrow{\alpha} & \llbracket (l', \nu') \rrbracket \end{array}$$

should commute. So we need to compute a clock zone ν' s.t. $v' \models \nu'$ iff there exists a clock assignment v s.t.

- $v \models \nu$, (v is in the source clock zone))
- $v \models \phi$, and (the guard of the transition holds)
- $v' = v[\lambda \leftarrow 0]$. (v' is the result resetting the clocks in λ)

The following example shows how this can be computed: Assume the symbolic state

$$(work, x \leq 8 \wedge 21 \leq y - x \leq 24 \wedge y \leq 30)$$

and the transition

$$(work, y \geq 30, \{y\}, restart).$$

The symbolic state contains the clock zone $x \leq 8 \wedge 21 \leq y - x \leq 24 \wedge y \leq 30$. Substituting this clock zone into the conditions above, and using the names x and y of the individual clocs instead of the clock assignment v , we get

$$\exists x, y . \left[\begin{array}{l} x \leq 8 \wedge y \leq 30 \wedge 21 \leq y - x \leq 24 \wedge \\ y \geq 30 \wedge \\ x' = x \wedge y' = 0 \end{array} \right].$$

Now we can use the equality $x' = x$ to eliminate the quantifier for x , which results in

$$\exists y . \left[\begin{array}{l} x' \leq 8 \wedge y \leq 30 \wedge 21 \leq y - x' \leq 24 \wedge \\ y \geq 30 \wedge y' = 0 \end{array} \right].$$

The two inequalities $y \leq 30$ and $y \geq 30$ are equivalent to the equality $y = 30$, which simplifies the formula to

$$\exists y . x' \leq 8 \wedge y = 30 \wedge 21 \leq y - x' \leq 24 \wedge y' = 0.$$

Now we use the equality $y = 30$ to eliminate the quantifier for y , resulting in

$$x' \leq 8 \wedge 21 \leq 30 - x' \leq 24 \wedge y' = 0.$$

We can simplify the formula as follows:

$$x' \leq 8 \wedge -9 \leq -x' \leq -6 \wedge y' = 0$$

$$x' \leq 8 \wedge 6 \leq x' \leq 9 \wedge y' = 0$$

$$6 \leq x' \leq 8 \wedge y' = 0$$

The result is again a clock zone. Renaming the variables back to x and y , we get the symbolic successor state

$$(restart, 6 \leq x \leq 8 \wedge y = 0)$$

as the result of the symbolic action transition.

10.6 Unbounded Model Checking

A given symbolic state may have several symbolic successor states. Still, this set of successor states is always finite. So we can compute all symbolic successor states, representing *all* states resulting from one transition. Hence we can compute the reach set of a given timed automata using the usual algorithm:

```

V ← S0
while there is a transition  $(x, x')$  such that  $x \in V, x' \notin V$  do
    V ← V ∪ {x' | (x, x') ∈ R, x ∈ V} // V ∪ PostR(V)
return V

```

Here we can represent the set V using a set of symbolic states containing clock zones. For a set of symbolic states V the corresponding set of states is

$$\bigcup_{(l, \nu) \in V} \llbracket (l, \nu) \rrbracket.$$

The symbolic states in V may represent overlapping sets. For example, it may contain the symbolic states

$$(work, 0 \leq x \leq 10) \text{ and } (work, 5 \leq x \leq 15)$$

If a symbolic state represents a subset of the set of states of a different symbolic state, we can remove it from V .

In the case of timed automata, this algorithm always terminates, due to the fact that by multiplying by denominators we can translate any timed automaton to an equivalent one that contains only integer constants, and due to the fact that outside of the biggest constant a timed automaton has uniform behavior.

Due to this, checking of **Gok** (and all of LTL) is decidable, even if timed automata have an infinite state space. In practice, an optimized representation of clock zones (*difference bounded matrices*), and various further optimizations are used.

10.7 Conclusions

- Timed automata provide the possibility of modeling behavior in time
- Infinite state space, still decidable
- Key: symbolic representation of infinite sets of states

[1, 5, 14]

Chapter 11

Petri Nets

11.1 Motivation

Assume a network protocol that merges two types of packets into one. If only packets of one of the two types are available, we have to wait for packets of the other type. In order to do this, we maintain two queues of packages that still have to be processed.

Such a situation cannot be reasonably modeled using a finite transition system, since we do not have a bound on the number of packets in the queue, and hence the number of needed states may grow over all bounds. Even if we would restrict the size of the queue to a finite, but large number, modeling of the situation using a transition system is tedious—at least if we insist on explicitly supplying each individual transition. One way around to the problem is to use variables to model the number of packages waiting in each queue, as allowed, for example, by the formalism of extended state machines introduced in Section 4.5. This would result in a model such as the one in Figure 11.1. In this section we will introduce an alternative way of modeling such situations.

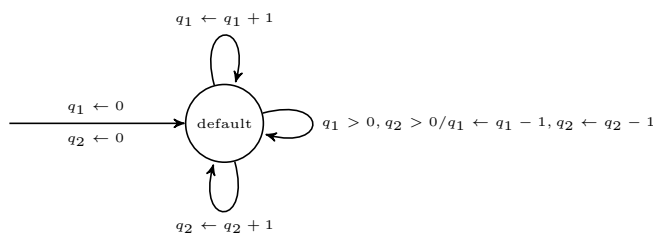


Figure 11.1: Extended State Machine Model

11.2 Examples

Petri nets are usually represented in graphical form. For example, Figure 11.2 models a protocol for communication between two processes [32, Fig. 9]. The

circles are called *places*. The black dots, called *tokens* move through the network using the rectangles, which are called *transitions*. We call a transition *enabled*, if each of its input places, that is, each place with an arrow leading to this transition, is marked with a token. A transition can *fire* iff it is enabled, which removes a token from each of its input places, and adds a token to each of its output places. The ordering of firings is non-deterministic, that is, in case of more enabled transitions, an arbitrary one is fired. Note that the number of tokens in the network can change. For example, in Figure 11.2, transition “send message” is enabled. After firing this transition, the token from the place “ready to send” is removed, and to each of the places “wait for ack” and “buffer” a token is added. So a firing of this transition removes one token and adds two tokens.

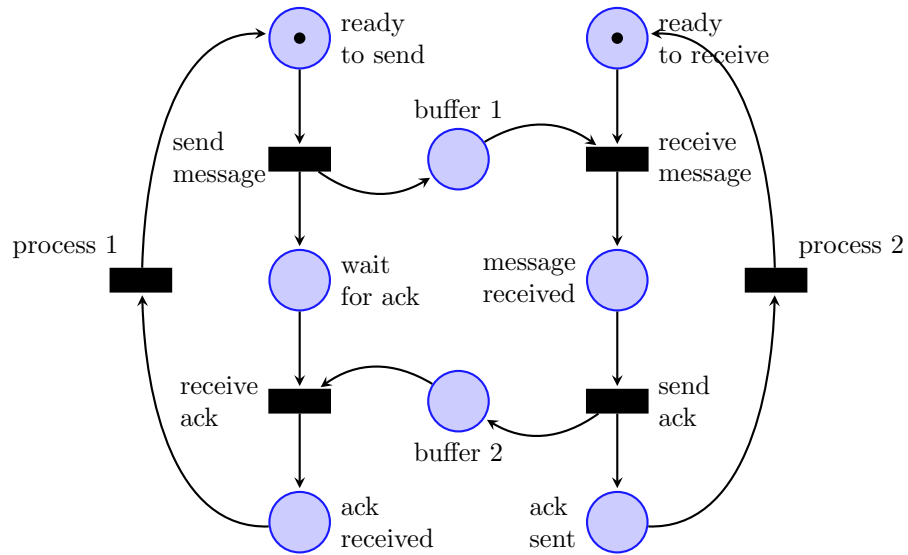


Figure 11.2: Protocol

This basic formalism can be extended by adding numbers to transitions. For example, the upper half of Figure 11.3 models the chemical reaction $2H_2 + O_2 \rightarrow 2H_2O$ [32, Fig. 1]. Here, the firing of the given transition needs two tokens in the input place H_2 , and adds two tokens to the output place H_2O . The lower half of the figure shows the result.

Petri nets may be useful in modeling many other phenomena (not only in technical applications, for example, also organizational ones). The following table lists typical usage scenarios:

Input Places	Transition	Output Places
Preconditions	Event	Results
Input data	Process	Output Data
Input signals	Signal processor	Output signals
Conditions	Deduction step	Conclusions
Buffers	Processor	Buffers

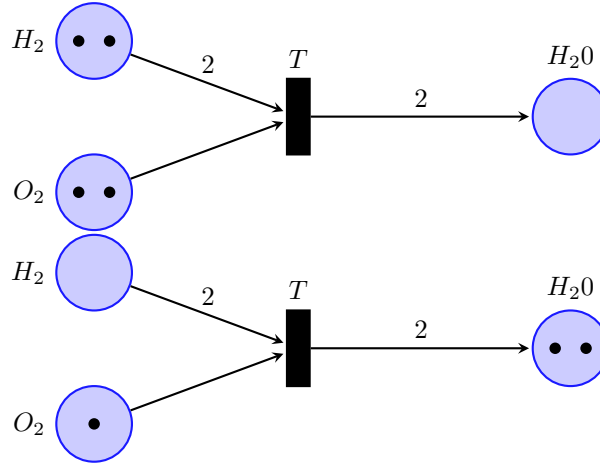


Figure 11.3: Chemical Reaction

11.3 Formal Definitions

Definition 40 A Petri net is a 5-tuple (P, T, F, w, M_0) where

- P is a finite set whose elements we call places,
- T is a finite set whose elements we call transitions,
- $F \subseteq (P \times T) \cup (T \times P)$ whose elements we call arcs,
- $w : F \rightarrow \mathbb{N}$ called weight function, and
- $M_0 : P \rightarrow \mathbb{N}_0$ called the initial marking.

Here $P \cap T = \emptyset$.

In the literature, one can find a few different, but equivalent definitions.

Note that a transition of a Petri net is something quite different from a transition of a transition system. Transitions of Petri nets are objects that are an integral part of every Petri net, while a transition of a transition system is part of the behavior of the transition system.

Definition 41 A source transition of a Petri net is a transition without inputs, and a sink transition a transition without outputs.

Definition 42 A marking (state) of a Petri net is a function $s : P \rightarrow \mathbb{N}_0$.

Here, for a state s , and a place p , $s(p) = n$ formalizes the intuition that p contains n tokens.

The state evolves, starting from the initial state, according to the following transition (or firing) rules:

- We call a transition t *enabled*, if each input place p of t is marked with at least $w(p, t)$ tokens.

- A transition t can *fire* iff it is enabled, which removes $w(p, t)$ tokens from each input place p of t , and adds $w(t, p)$ tokens to each output place p of t .
- Ordering of firings is non-deterministic, that is, in case of more enabled transitions, an arbitrary one is fired.

11.4 Properties of Petri Nets

11.4.1 Reachability

Definition 43 Given a Petri net (P, T, F, w, M_0) , a marking M' is reachable from a marking M iff there exists a sequence of firings from M to M' . A marking M is reachable iff it is reachable from the initial marking M_0 .

For modeling reachability of more states the literature on Petri nets does not use sets, but partial functions:

Definition 44 A partial marking is a partial function from $P \rightarrow \mathbb{N}_0$.

For example, for the Petri net in Figure 11.3, the partial marking $\{H_20 \mapsto 2\}$ requires the number of tokens in the place H_20 to be 2, but allows an arbitrary number of tokens in the other places.

Definition 45 A partial marking M_p is reachable iff there exists a reachable marking M that coincides with M_p on its defined elements

11.4.2 Boundedness

Definition 46 A Petri net is k -bounded iff the number of tokens in each place does not exceed k for any reachable marking. A Petri net is bounded iff there exists a k s.t. the net is k -bounded.

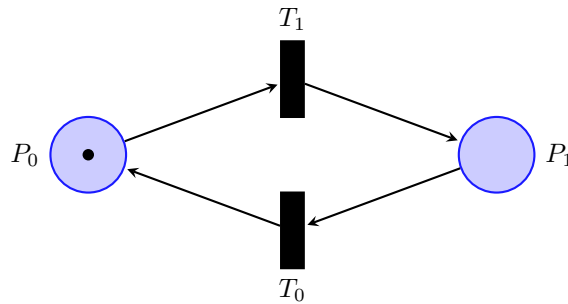


Figure 11.4: Bounded Petri Net

For example, the Petri net in Figure 11.4 is 7-bounded. In general, it is k -bounded, for every $k \in \mathbb{N}$. Hence it is also bounded.

In the Petri net in Figure 11.5, the number of tokens in place P_4 can grow over all bounds. Hence the net is not bounded (i.e., unbounded).

The notion of boundedness is useful in several situations. For example, this may ensure that a buffer of size k does not overflow. Also, every bounded Petri net corresponds to a transition system with finite state space.

11.4.3 Liveness

Petri nets model the absence of certain deadlocks as follows:

Definition 47 *A transition is live iff from every reachable marking one can reach a firing of this transition. A Petri net is live iff every transition is live.*

For example, in the Petri net of Figure 11.6, no transition is live since all tokens will eventually end up in P_3 . From that point on, no transitions will be able to fire, since every transition needs at least one token in a place different from P_3 . In the Petri net of Figure 11.7 transition T_1 will never fire since this needs a token both in place P_3 and in P_4 . This will never happen since transition T_2 adds a token to place P_4 but at the same time, removes one from P_3 . In addition, the Petri net in this figure is unbounded: The only outgoing arrow from place P_1 is leading to the transition T_4 , but firing this transition also returns a token back to P_1 . Hence transition T_4 cannot decrease the number of tokens in P_1 . However, transition T_3 will add a new token to P_1 and hence the number of tokens in P_1 can grow over all bounds.

11.5 Automatic Analysis of Petri Net Properties

Analyzing all reachable states is difficult. Due to this problem, one often restricts oneself to so-called *structural properties*. Those are properties, that only depend on the structure of a Petri net, but not on its initial state.

Consider the example of traffic lights, such as the ones on the left-hand side of Figure 11.8. A Petri net modeling such traffic lights can be seen on the right-hand side of this Figure. Now consider a junction controlled by a pair of such traffic lights, as in Figure 11.9.

The result will be a crash, since both lights may switch to green at the same time. In order to avoid this, we add an additional place: switching to green allowed. The result can be seen in Figure 11.10.

Now we assume that the Petri net has an arbitrary initial marking and study the question of which transitions lead any state back to itself.

Definition 48 *A T-invariant is a function $f : T \rightarrow \mathbb{N}_0$ s.t. for all markings M and M' s.t. when starting from M , the marking M' is the result of firing every transition $t \in T$ exactly $f(t)$ times, we have that $M = M'$.*

In our example, T -invariants have the form $f : \{T1a, T1b, T1c, T2a, T2b, T2c\} \rightarrow \mathbb{N}_0$ and every traffic light has a cycle of three firings. This corresponds to the T -invariants

- $\{T1a \mapsto 1, T1b \mapsto 1, T1c \mapsto 1, T2a \mapsto 0, T2b \mapsto 0, T2c \mapsto 0\}$

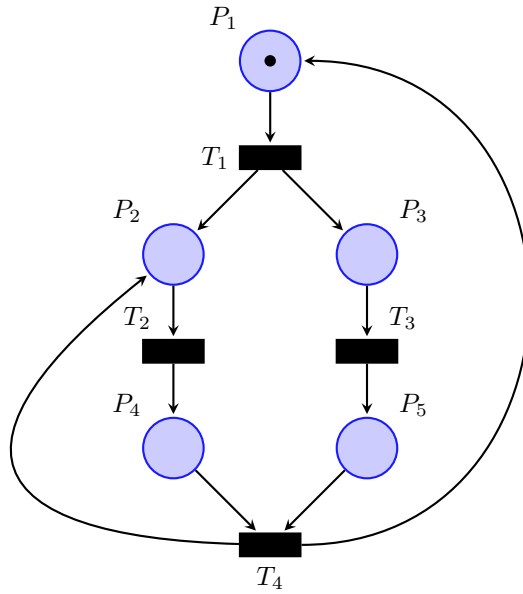


Figure 11.5: Unbounded Petri Net

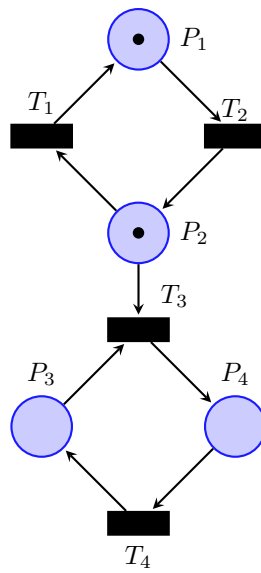


Figure 11.6: Example Petri Net

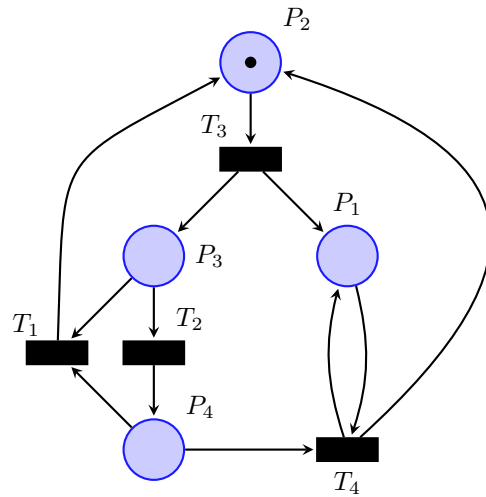


Figure 11.7: Liveness and Boundedness

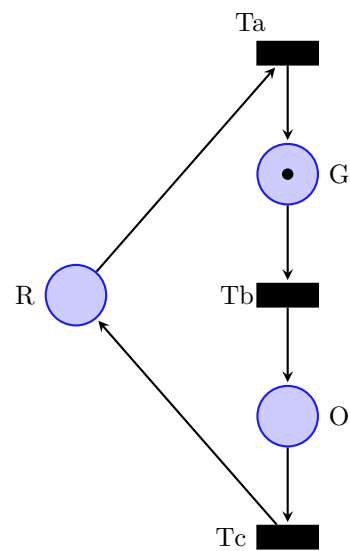
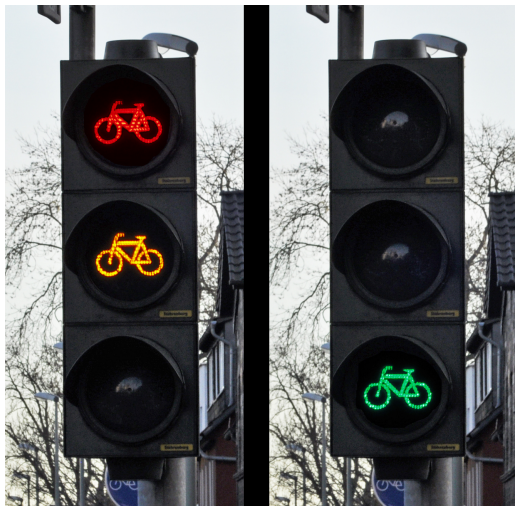


Figure 11.8: Traffic Lights

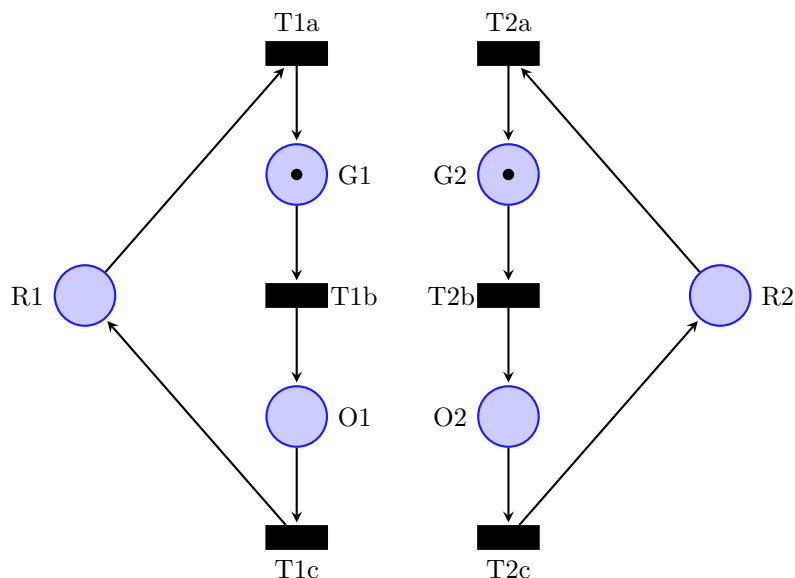


Figure 11.9: Junction

- $\{T1a \mapsto 0, T1b \mapsto 0, T1c \mapsto 0, T2a \mapsto 1, T2b \mapsto 1, T2c \mapsto 1\}$

Clearly, every positive linear combination of two T -invariants is again a T -invariant. For example, adding the two T -invariants above, one gets the new T -invariant

$$\{T1a \mapsto 1, T1b \mapsto 1, T1c \mapsto 1, T2a \mapsto 1, T2b \mapsto 1, T2c \mapsto 1\}.$$

The T -invariants that assigns to every transition the value 0 is called the *trivial* T -invariant.

T -invariants can, for example, be used in liveness verification (no deadlock) and verification of reversability (ability to return to the initial state M_0).

Now again assume that this Petri net has an arbitrary initial marking. What can we say about the numbers of tokens under arbitrary firings?

Definition 49 A P -invariant is a function $f : (P \rightarrow \mathbb{N}_0) \rightarrow \mathbb{Z}$ s.t. for every transition $t \in T$, for every marking M and M' s.t. M' is the result of firing t from M , $f(M) = f(M')$.

In the case of our junction, P -invariants are functions $f : (\{R1, G1, O1, R2, G2, O2\} \rightarrow \mathbb{N}_0) \rightarrow \mathbb{Z}$. In both traffic lights the number of tokens stays constants, and hence we have the following P -invariants:

- $f(M) = M(R1) + M(G1) + M(O1)$
- $f(M) = M(R2) + M(G2) + M(O2)$

Moreover, the tokens in the coordination place S are always added or removed with one red light, and so $f(M) = M(R1) + M(R2) - M(S)$ is another P -invariant.

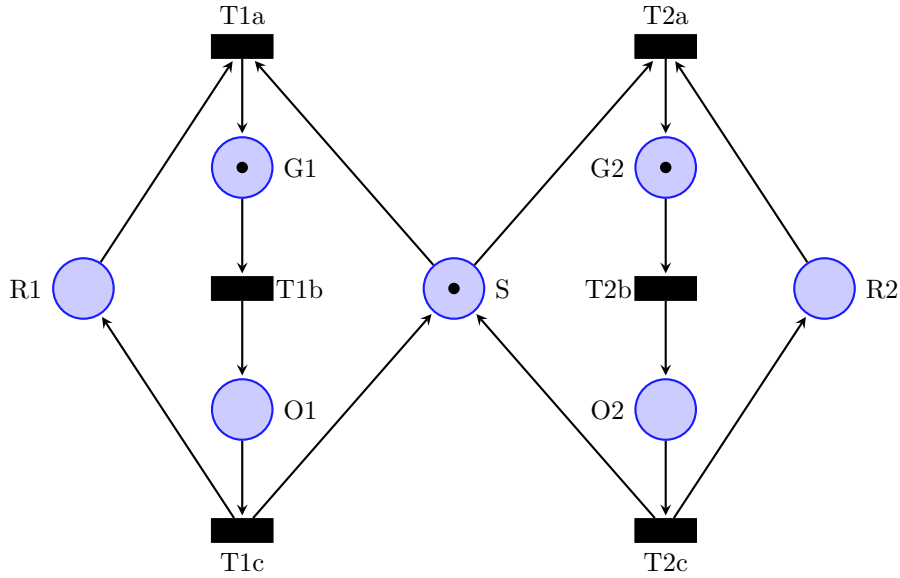


Figure 11.10: Coordinated Junction

Clearly, every linear combination of two P -invariants is again a P -invariant, and again, the trivial P -invariant is the P -invariant that assigns to every state the number 0.

Examples of situations where P -invariants are useful:

- Liveness verification: An invariant can ensure that there is always an enabled transition.
- Boundedness verification (no token is lost or created).

Further examples: <http://www.informatik.uni-hamburg.de/TGI/PetriNets/introductions/aalst>

We will delay the computation of invariants to Section 11.8, when we will have all the necessary theory available.

11.5.1 Petri Net Transformations

Certain simplifications preserve certain properties of Petri nets. For example, the transformations from Figure 11.11 preserve liveness, 1-boundedness, boundedness.

11.6 Translation to Transition System

Consider the Petri net from Figure 11.6, once more. This net is 1-bounded, so it has a finite number of reachable states. The states of a transition system describing this Petri net can be formed by the markings $S \subseteq [\{P_1, \dots, P_4\} \rightarrow \mathbb{N}_0]$. We will represent them as vectors from $\mathbb{N}_0^4$ in such way that the i -th entry of the vector, with $i \in \{1, \dots, 4\}$, represents the number of tokens in place P_i .

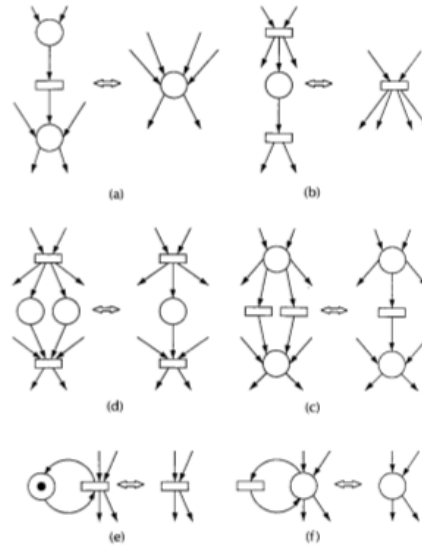


Figure 11.11: Petri Net Transformations [32, Fig. 22]

So the set of initial states is $\{(1,0,1,0)^T\}$. It is not difficult to see that the result must be something like the transition system of Figure 11.12.

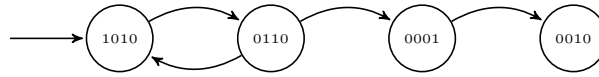


Figure 11.12: Transition System

Now it remains to define the transition relation of the resulting transition system. For a given Petri net transition, this

- transition has to be enabled, and
- firing changes the number of tokens.

For example, for the transition T_1 , if $s \geq (0,1,0,0)^T$, then the number of tokens in each location changes according to:

- $\Delta_1^- = (0,1,0,0)^T$
- $\Delta_1^+ = (1,0,0,0)^T$

Here, Δ_1^- is the number of tokens removed by transition T_1 , and Δ_1^+ the number of tokens added. Putting those two changes together, we get $\Delta_1 = \Delta_1^+ - \Delta_1^- = (1,-1,0,0)^T$. The corresponding transition of transition system

can be described by

$$s' = s + \begin{pmatrix} 1 \\ -1 \\ 0 \\ 0 \end{pmatrix}, \text{ if } s \geq \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix}.$$

As a result, we get the transition system $(\mathbb{N}_0^4, \{(1, 0, 1, 0)^T\}, T)$ where

$$T = \begin{aligned} & \{(s, s + (1, -1, 0, 0)^T) \in \mathbb{N}_0^4 \times \mathbb{N}_0^4 \mid s \geq (0, 1, 0, 0)^T\} \cup \\ & \{(s, s + (-1, 1, 0, 0)^T) \in \mathbb{N}_0^4 \times \mathbb{N}_0^4 \mid s \geq (1, 0, 0, 0)^T\} \cup \\ & \{(s, s + (0, -1, -1, 1)^T) \in \mathbb{N}_0^4 \times \mathbb{N}_0^4 \mid s \geq (0, 1, 1, 0)^T\} \cup \\ & \{(s, s + (0, 0, 1, -1)^T) \in \mathbb{N}_0^4 \times \mathbb{N}_0^4 \mid s \geq (0, 0, 0, 1)^T\} \end{aligned}$$

Summarizing this, we get:

Definition 50 *Given a Petri net $(\{P_1, \dots, P_n\}, \{T_1, \dots, T_m\}, F, w, M_0)$, its corresponding transition system is*

$$(\mathbb{N}_0^n, \{(M_0(P_1), \dots, M_0(P_n))^T\}, T),$$

where

$$T = \{(s, s + \Delta_j) \in S \times S \mid s \geq \Delta_j^-, j \in \{1, \dots, m\}\}$$

where $\Delta_j := \Delta_j^+ - \Delta_j^-$ and for every $j \in \{1, \dots, m\}$,

- $\Delta_j^- := (\delta_1, \dots, \delta_n)$, where

$$\delta_i = \begin{cases} w(P_i, T_j), & \text{if } (P_i, T_j) \in F, \text{ and} \\ 0, & \text{otherwise,} \end{cases}$$
- $\Delta_j^+ := (\delta_1, \dots, \delta_n)$, where

$$\delta_j = \begin{cases} w(T_j, P_i), & \text{if } (T_j, P_i) \in F, \text{ and} \\ 0, & \text{otherwise.} \end{cases}$$

This definition also works for unbounded nets! In the case of a k -bounded Petri net we can restrict ourselves to a finite set of states $P \rightarrow \{0, \dots, k\}$, that we represent as vectors $\{0, \dots, k\}^{|P|}$.

Our example is 1-bounded, and hence we can restrict ourselves to the set of states $\{0, 1\}^4$ which results in the transition relation

$$T = \begin{aligned} & \{(s, s + (1, -1, 0, 0)^T) \in \{0, 1\}^4 \times \{0, 1\}^4 \mid s \geq (0, 1, 0, 0)^T\} \cup \\ & \{(s, s + (-1, 1, 0, 0)^T) \in \{0, 1\}^4 \times \{0, 1\}^4 \mid s \geq (1, 0, 0, 0)^T\} \cup \\ & \{(s, s + (0, -1, -1, 1)^T) \in \{0, 1\}^4 \times \{0, 1\}^4 \mid s \geq (0, 1, 1, 0)^T\} \cup \\ & \{(s, s + (0, 0, 1, -1)^T) \in \{0, 1\}^4 \times \{0, 1\}^4 \mid s \geq (0, 0, 0, 1)^T\} \end{aligned}$$

In explicit form, this set is:

$$\left\{ \begin{array}{l} ((0, 1, 0, 0)^T, (1, 0, 0, 0)^T), \\ ((0, 1, 0, 1)^T, (1, 0, 0, 1)^T), \\ ((0, 1, 1, 0)^T, (1, 0, 1, 0)^T), \\ ((0, 1, 1, 1)^T, (1, 0, 1, 1)^T), \\ ((1, 0, 0, 0)^T, (0, 1, 0, 0)^T), \\ ((1, 0, 0, 1)^T, (0, 1, 0, 1)^T), \\ ((1, 0, 1, 0)^T, (0, 1, 1, 0)^T), \\ ((1, 0, 1, 1)^T, (0, 1, 1, 1)^T), \\ ((0, 1, 1, 0)^T, (0, 0, 0, 1)^T), \\ ((1, 1, 1, 0)^T, (1, 0, 0, 1)^T), \\ ((0, 0, 0, 1)^T, (0, 0, 1, 0)^T), \\ ((0, 1, 0, 1)^T, (0, 1, 1, 0)^T), \\ ((1, 0, 0, 1)^T, (1, 0, 1, 0)^T), \\ ((1, 1, 0, 1)^T, (1, 1, 1, 0)^T) \end{array} \right\}$$

One can immediately see, that this set contains more transitions than the number of arrows in the transition system from Figure 11.12. To analyze the situation, we also represent this transition relation graphically (Figure 11.13), ignoring states from $\{0, 1\}^4$ that are not involved in transitions. As one can see, the unreachable part corresponds to unreachable behavior of the Petri net which our translation includes into the transition relation.

In general, the translation from Definition 50 also includes transitions from or to unreachable states in $\mathbb{N}_0^n$. Figure 11.12 only contains the reachable part of the translation of the Petri net on the left-hand side to a transition system. However, the resulting transition system contains many more, even infinitely many unreachable transitions.

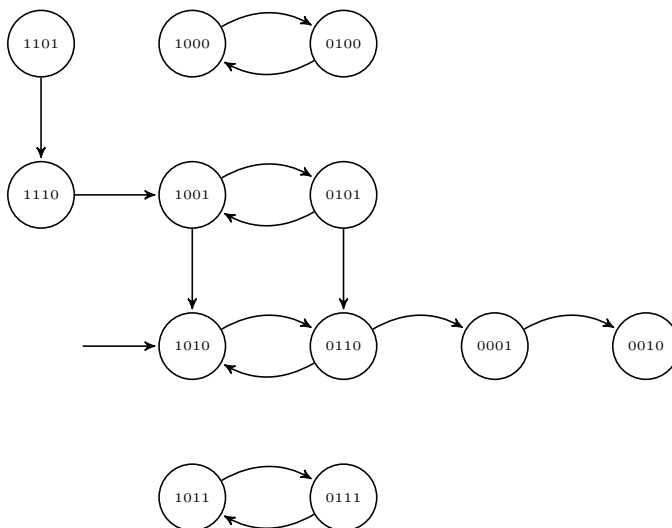


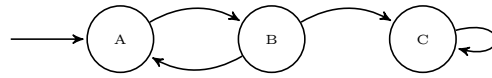
Figure 11.13:

When we compare the properties of the original Petri net and the corresponding transition system, we see that transition system states that do not

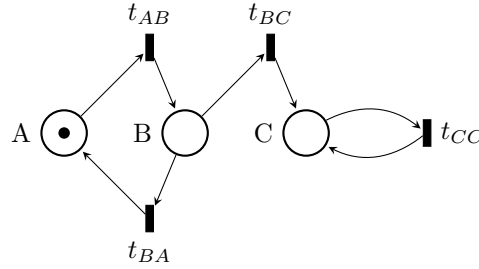
have any out-going transition correspond to Petri net states that do not have any enabled transition. In other words, the transition relation not being left-total corresponds to a Petri net that has a state with no enabled transition. A non-deterministic transition system corresponds to a Petri net that may have several enabled transitions.

11.7 Translation from (Finite) Transition System

We begin with an example. For the transition system



we have the following corresponding Petri net:



In general, we have:

Definition 51 Given a transition system $(S, \{s_0\}, R)$, the corresponding Petri net is (P, T, F, w, M_0) , with

- Places P : S
- Transitions T : $\{t_r \mid r \in R\}$
- Arcs F : $\{(s, t_{(s,s')}) \mid (s, s') \in R\} \cup \{(t_{(s,s')}, s') \mid (s, s') \in R\}$
- Weight function w : $w(x, y) = 1$
- M_0 : $M_0(s_0) = 1$, $M_0(p) = 0$, if $p \neq s_0$.

11.8 Invariant Computation

11.8.1 T -Invariants

The basis for computing T -invariants is the representation of Petri nets as transitions systems: We represent the function $f : T \rightarrow \mathbb{N}_0$ using a vector of firing

numbers $\begin{pmatrix} x_1 \\ \dots \\ x_m \end{pmatrix}$ that represents a function

$$\{T_1 \mapsto x_1, \dots, T_m \mapsto x_m\}.$$

Our goal now is to find values $x_1, \dots, x_m$ s.t. this function is a T -invariant. According to Definition 48, this holds iff for every state $s \in \mathbb{N}_0^n$ of the transition system, the state resulting from x_1 firings of $T_1, \dots, x_m$ firings of T_m is again the original state s . This means that

$$s + \Delta_1 x_1 + \dots + \Delta_m x_m = s$$

which holds iff

$$\Delta_1 x_1 + \dots + \Delta_m x_m = \vec{0}.$$

This is a homogeneous linear system of equations in the variables $x_1, \dots, x_m$. Each solution $(x_1, \dots, x_m)^T \in \mathbb{N}_0^m$ represents one T -invariant.

11.8.2 P -Invariants

Here we assume that the function $f : (P \rightarrow \mathbb{N}_0) \rightarrow \mathbb{Z}$ is linear in its coefficients. We will denote those coefficients by a vector $y = (y_1, \dots, y_n)$. We again work with the transition system corresponding to the Petri net. So we will compute a function $f : \mathbb{N}_0^n \rightarrow \mathbb{Z}$ that we will then translate back to the Petri net. For a state $s = (s_1, \dots, s_n)^T$ of the transition system, $f(s_1, \dots, s_n) = y_1 s_1 + \dots + y_n s_n$, that is, $f(s) = ys$.

For example, for a Petri net with four places, that is, for $n = 4$, we have:

- If $y = (1, 0, 0, 0)$, then $f(s) = (1, 0, 0, 0) \begin{pmatrix} s_1 \\ s_2 \\ s_3 \\ s_4 \end{pmatrix} = s_1$. Translating this

back to the Petri net we get a function $f : (P \rightarrow \mathbb{N}_0) \rightarrow \mathbb{Z}$ such that for a marking M , $f(M)$ is the number of tokens at the first place of M .

- If $y = (1, 1, 1, 1)$, then $f(s) = (1, 1, 1, 1) \begin{pmatrix} s_1 \\ s_2 \\ s_3 \\ s_4 \end{pmatrix} = s_1 + s_2 + s_3 + s_4$. Trans-

lating this back to the Petri net, for a marking M , $f(M)$ is the sum of all tokens at all four places.

Translating the whole definition of P -invariant to the corresponding transition system, we want to compute a function $f : \mathbb{N}_0^n \rightarrow \mathbb{Z}$ s.t. for every transition $t \in T$ for every state s and s' s.t. s' is the result of firing t from s , $f(s) = f(s')$, where $f(s) = ys$.

Applying this to the example from Figure 11.12, for example to the transition T_3 , we get:

$$f(s) = f(s + \Delta_3), \text{ where } \Delta_3 = (0, -1, -1, 1)^T$$

$$ys = y(s + \Delta_3), \text{ which holds if } y\Delta_3 = 0, \text{ i.e.,}$$

$$(y_1, \dots, y_4) \begin{pmatrix} 0 \\ -1 \\ -1 \\ 1 \end{pmatrix} = 0 \text{ i.e., } -y_2 - y_3 + y_4 = 0$$

This equation has infinitely many solutions, for example $y_2 = 1, y_3 = 1, y_4 = 2$, and hence the linear function $s_2 + s_3 + 2s_4$ does not change when firing transition T_3 .

The same property should hold for every transition, i.e., for all $i \in \{1, \dots, m\}$: $y(s + \Delta_i) = ys$. As a result we get

$$y\Delta_1 = 0, \dots, y\Delta_m = 0,$$

which is again a homogeneous linear system of equations.

11.8.3 Summary

Summarizing, for computing T -invariants, we have to solve the system

$$\Delta_1 x_1 + \dots + \Delta_m x_m = \vec{0}$$

where $\Delta_1, \dots, \Delta_m$ are column vectors, and for computing P -invariants we have to solve the system

$$y\Delta_1 = 0, \dots, y\Delta_m = 0$$

where $\Delta_1, \dots, \Delta_m$ are row vectors.

We can write this in more compact notation, by writing the vectors $\Delta_1, \dots, \Delta_m$ as a matrix, the so-called *incidence matrix*, which we define as $A := (\Delta_1, \dots, \Delta_m)$.

Then we have:

- Every solution $(x_1, \dots, x_m)$ of $Ax = \vec{0}$ in the natural numbers represents a T -invariant $f : \{T_1, \dots, T_m\} \rightarrow \mathbb{N}_0$ s.t. for all $i \in \{1, \dots, m\}$, $f(T_i) = x_i$.
- Every solution $(y_1, \dots, y_n)$ of $yA = \vec{0}$, i.e., $A^T y^T = \vec{0}$ represents a P -invariant $f : (\{P_1, \dots, P_n\} \rightarrow \mathbb{N}_0) \rightarrow \mathbb{Z}$ s.t. for all $M : \{P_1, \dots, P_n\} \rightarrow \mathbb{N}_0$, $f(M) = \sum_{i \in \{1, \dots, n\}} y_i M(P_i)$.

11.8.4 Example of T -Invariant Computation

$$\Delta_1 = \begin{pmatrix} 1 \\ -1 \\ 0 \\ 0 \end{pmatrix}, \Delta_2 = \begin{pmatrix} -1 \\ 1 \\ 0 \\ 0 \end{pmatrix}, \Delta_3 = \begin{pmatrix} 0 \\ -1 \\ -1 \\ 1 \end{pmatrix}, \Delta_4 = \begin{pmatrix} 0 \\ 0 \\ 1 \\ -1 \end{pmatrix}$$

The corresponding incidence matrix is

$$A = \begin{pmatrix} 1 & -1 & 0 & 0 \\ -1 & 1 & -1 & 0 \\ 0 & 0 & -1 & 1 \\ 0 & 0 & 1 & -1 \end{pmatrix}$$

For solving the resulting system $Ax = \vec{0}$ we try Gaussian elimination:

$$\begin{pmatrix} 1 & -1 & 0 & 0 \\ -1 & 1 & -1 & 0 \\ 0 & 0 & -1 & 1 \\ 0 & 0 & 1 & -1 \end{pmatrix}$$

after adding rows 1 and 2, 3 and 4:

$$\begin{pmatrix} 1 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & -1 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

after subtracting row 2 from 3:

$$\begin{pmatrix} 1 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

The result is

$$\begin{pmatrix} 1 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

which corresponds to the equations

$$x_1 = x_2, x_3 = 0, x_4 = 0$$

As a consequence, the set of T -invariants in vector representation is

$$\{(x, x, 0, 0) \mid x \in \mathbb{N}_0\}$$

which represents the functions

$$\{f : \{T_1, \dots, T_4\} \rightarrow \mathbb{N}_0 \mid f(T_1) = f(T_2), f(T_3) = 0, f(T_4) = 0\}.$$

11.8.5 Example of P -Invariant computation

The transposed incidence matrix A^T is

$$\begin{pmatrix} 1 & -1 & 0 & 0 \\ -1 & 1 & 0 & 0 \\ 0 & -1 & -1 & 1 \\ 0 & 0 & 1 & -1 \end{pmatrix}.$$

We again use Gaussian elimination for solving the corresponding system: Add rows 1 and 2, the result is a row with nulls only:

$$\begin{pmatrix} 1 & -1 & 0 & 0 \\ 0 & -1 & -1 & 1 \\ 0 & 0 & 1 & -1 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

add rows 2 and 3:

$$\begin{pmatrix} 1 & -1 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & -1 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

The result is

$$\begin{pmatrix} 1 & -1 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & -1 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

which corresponds to the equations

$$y_1 = y_2, y_2 = 0, y_3 = y_4$$

So the set of P -invariants in vector representation is:

$$\{(0, 0, y, y) \mid y \in \mathbb{Z}\}$$

which represents the functions

$$\{f : (\{P_1, \dots, P_4\} \rightarrow \mathbb{N}_0) \rightarrow \mathbb{Z} \mid f(M) = yM(P_3) + yM(P_4), y \in \mathbb{Z}\}$$

11.8.6 Solution of Equations

Gaussian elimination usually results in rational solutions. If the solution is not integer, from a rational solution

$$\frac{p_1}{q_1}, \dots, \frac{p_r}{q_r}$$

we obtain the integer solution:

$$\text{lcm}\{q_1, \dots, q_r\}p_1, \dots, \text{lcm}\{q_1, \dots, q_r\}p_r.$$

In the case of T -invariants the solution must be natural numbers. There are special algorithms for that but often it suffices to simply use one's own intelligence. For using software one can search for the keywords “null space”, “kernel” and “echelon form”.

11.9 Conclusion

Petri nets are a further possibility for modeling systems. Returning to the network protocol example from the beginning of this chapter, we can now model this example using a Petri net—as shown in Figure 11.14 instead of an extended state machine such as the own from Figure 11.1.

Whether it makes sense to use Petri nets or a different formalism partially depends on the type of problem, but partially also on culture (certain people see the world more through Petri net glasses, other people do not use them at all).

<http://www.informatik.uni-hamburg.de/TGI/PetriNets/>
 Simulator PIPE2: <http://pipe2.sourceforge.net>

g

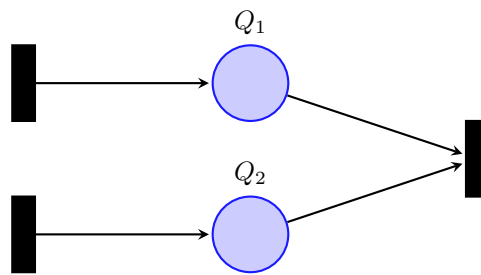


Figure 11.14: Petri Net Model

Chapter 12

Probabilistic Models

Consider the simple automaton from Figure 12.1. It can be used to model any system that switches between two modes, for example a door that can be open or closed, or an engine that can be on or off. Clearly the LTL formula $\mathbf{GF}b$, where $\mathcal{I}(b) = \{B\}$ holds. It ensures that at any time, the system will always eventually switch back to mode B .

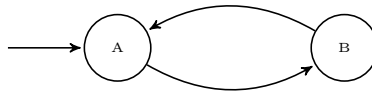


Figure 12.1: Two Mode System

Now imagine the situation where, due to some small technical problem, the system stays in state A for more than one step, but then quickly recovers normal functioning mode, switching back and forth between the two states again. One might be tempted to model such a situation by adding a self-loop to state A . However, as soon as state A has such a self-loop, the system may stay in state A forever, which does not correspond to the situation that we want to model. And especially, after adding such a loop, the property $\mathbf{GF}b$ does not hold any more.

A better way of modelling such a situation is to use probabilities, as shown in Figure 12.2. Now each transition has a label that denotes the probability for taking such a transition. Since the probability of staying in A is 0.1, the probability of staying in A forever is 0. Hence, we expect the probability of $\mathcal{P}(\mathbf{GF}b)$ to be 1 which is a more realistic way of modeling the given situation.

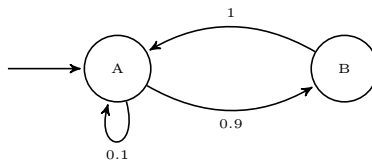


Figure 12.2: Two Mode System

The classical formalism that corresponds to such models are Markov Chains. Here, we will use a slightly different, but equivalent variant:

Definition 52 A probabilistic transition system is a triple (S, S_0, R) , where

- S is a countable set whose elements we call states,
- $S_0 : S \rightarrow [0, 1]$ s.t. $\sum_{s \in S} S_0(s) = 1$ (the initial distribution), and
- $R : S \times S \rightarrow [0, 1]$ s.t. for all $s \in S$, $\sum_{s' \in S} R(s, s') = 1$ (the transition probability matrix)

As can be seen in Figure 12.2, the graphical representation of probabilistic transition systems is straight-forward. Here, a missing edge between two state means, that the corresponding probability is zero. The Markov chain literature then defines notions such as distribution after n steps, stationary distribution, and so on.

For classical transition systems a given sequence of states either is a path or it is no path. In the probabilistic case we will use different terminology: A *path* is any sequence of states (i.e., an arbitrary element of Σ_S).

This immediately raises the question how, for a given path $s_0, \dots$, to determine its probability? One might be tempted to define it by multiplying the probability of all transitions that the path takes, that is, by

$$S_0(s_0) \prod_{i=0}^{\infty} R(s_i, s_{i+1}),$$

but the infinite product results in this probability being usually zero. For example, in the above example, the probability of the path $ABABA \dots$ would be $1 \times 0.9 \times 1 \times 0.9 \times \dots$ that is zero in the limit.

This problem does not arise in the case of finite paths where it is straight-forward to define the probability of a finite path $s_0, \dots, s_{n-1}$ in a probabilistic transition systems (S, S_0, R) as follows;

$$\begin{aligned} \mathcal{P}_{(S, S_0, R)}(s_0, \dots, s_{n-1}) &:= S_0(s_0) R(s_0, s_1) \dots R(s_{n-2}, s_{n-1}) \\ &= S_0(s_0) \prod_{i \in \{1, \dots, n-1\}} R(s_{i-1}, s_i) \end{aligned}$$

In the case of infinite paths we can build on probability theory. We assume that the reader is familiar with its basic concepts such as σ -algebra, measurability, or probability measure. The fundamental idea is to measure the probability that a system stays in a certain *set* of paths.

For certain sets of paths, determining the probability of the set is straight-forward:

Definition 53 The $(s_0, \dots, s_{n-1})$ -cylinder is the set of all paths that have the prefix $s_0, \dots, s_{n-1}$.

In other words, a cylinder is a set of paths that first follows a given finite sequence of states, but then can continue in an arbitrary way. We will denote the $(s_0, \dots, s_{n-1})$ -cylinder by $C_{s_0, \dots, s_{n-1}}$. Especially we will use the notation C for the $()$ -cylinder, that is, the cylinder following the empty sequence of states. This cylinder is equal to the set of all paths Σ_S .

Now we can define the probability of a cylinder as the probability of the corresponding finite path:

$$\mathcal{P}_{(S, S_0, R)}(C_{s_0, \dots, s_{n-1}}) := \mathcal{P}_{(S, S_0, R)}(s_0, \dots, s_{n-1})$$

Especially it holds that $\mathcal{P}_{(S, S_0, R)}(C.) = \mathcal{P}_{(S, S_0, R)}(\Sigma_S) = 1$. We use the shortcut $\mathcal{P}()$ for $\mathcal{P}_{(S, S_0, R)}()$.

It is not possible to assign probability in a reasonable way to all sets of paths (readers interested in details might google for the Banach-Tarski paradox that implies that for probability spaces of certain complexity, measurability of all sets is in contradiction with the axioms below). Still it is possible to generalize the probability of cylinders to a probability measure on all sets of paths that will be relevant for us. This implies:

- $\mathcal{P}(\emptyset) = 0$
- $\mathcal{P}(\Sigma_S) = 1$
- $\mathcal{P}(A \cup B) = \mathcal{P}(A) + \mathcal{P}(B)$, if $A \cap B = \emptyset$.

12.1 Specification of Properties of Probabilistic Transition Systems

There are various probabilistic variants of temporal logic. Here we will adapt LTL to the probabilistic case. We simply define the probability of an LTL formula for a given transition system as the probability of all paths that satisfy the formula:

$$\mathcal{P}_{(S, S_0, R)}(\phi) := \mathcal{P}_{(S, S_0, R)}(\{\pi \in \Sigma_S \mid \pi \models \phi\})$$

The set $\{\pi \in \Sigma_S \mid \pi \models \phi\}$ is always measurable [38], so this probability is well-defined.

12.2 Model Checking

12.2.1 Naive Model Checking of $\mathbf{F}p$

Figure 12.3 shows an example of a probabilistic transition system. It models a process that first is in a setup state, and then tries a certain task “try”. In some cases, the setup phase was not enough, and hence we have to return to this state. In other cases the task fails in such a way that we go into a “fail” state, and later back to the setup phase. And finally, the task might succeed. Now we want to compute the probability that at some point in the future we reach the state “fail”, that is, $\mathcal{P}(\mathbf{F}p)$, where $\mathcal{I}(p) = \{\text{fail}\}$.

For computing this probability, we observe that the system can reach this state either directly (setup, try, fail), or after cycling through setup and try an arbitrary number of times. Each of those possibilities, corresponds to a certain cylinder. Hence we have

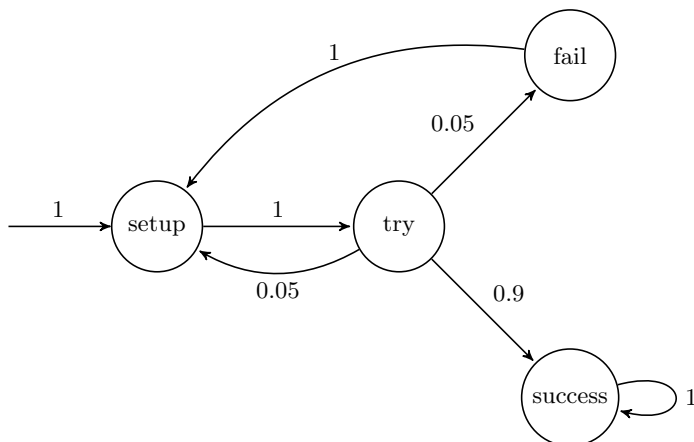


Figure 12.3: Probabilistic Transition System: Example

$$\begin{aligned}
 \mathcal{P}(\mathbf{F}p) &= \\
 \mathcal{P}(C_{\text{setup}, \text{try}, \text{fail}} \cup C_{\text{setup}, \text{try}, \text{setup}, \text{try}, \text{fail}} \cup \dots) &= \\
 \mathcal{P}(C_{\text{setup}, \text{try}, \text{fail}}) + \mathcal{P}(C_{\text{setup}, \text{try}, \text{setup}, \text{try}, \text{fail}}) + \dots &= \\
 \mathcal{P}(\text{setup}, \text{try}, \text{fail}) + \mathcal{P}(\text{setup}, \text{try}, \text{setup}, \text{try}, \text{fail}) + \dots &= \\
 1 \times 1 \times 0.05 + 1 \times 1 \times 0.05 \times 1 \times 0.05 \dots &= \sum_{n=1}^{\infty} 0.05^n
 \end{aligned}$$

This is quite simple in the case of this example. However, in general there may be a huge number of possibilities to reach the goal state. And hence, this method is inconvenient, in general.

12.2.2 Behavior from a Certain State

To arrive at a more convenient method that can also be generalized to more general temporal logic properties, we observe that temporal operators are defined based on operations on paths, such as suffixes. When taking such a suffix of a path, the system will already be in a non-initial state, and we will have to analyze the behavior from then on.

To be able to do this, we will introduce a simple formalism to move initial states. Informally, we will simply attach the arrow indicating an initial state in the graphical representation of transition systems to a different states. For example, Figure 12.4 shows three versions of the probabilistic transition system from Figure 12.3, but with moved initial state. We will denote those three versions by $(S, 1_{\text{setup}}, R)$, $(S, 1_{\text{try}}, R)$, and $(S, 1_{\text{fail}}, R)$, respectively.

To define this formally, we introduce the notation

$$1_s(t) := \begin{cases} 1, & \text{if } s = t \\ 0, & \text{otherwise} \end{cases}.$$

As a consequence, $(S, 1_s, R)$ denotes the changed probabilistic transition system that starts in the state s with probability 1.

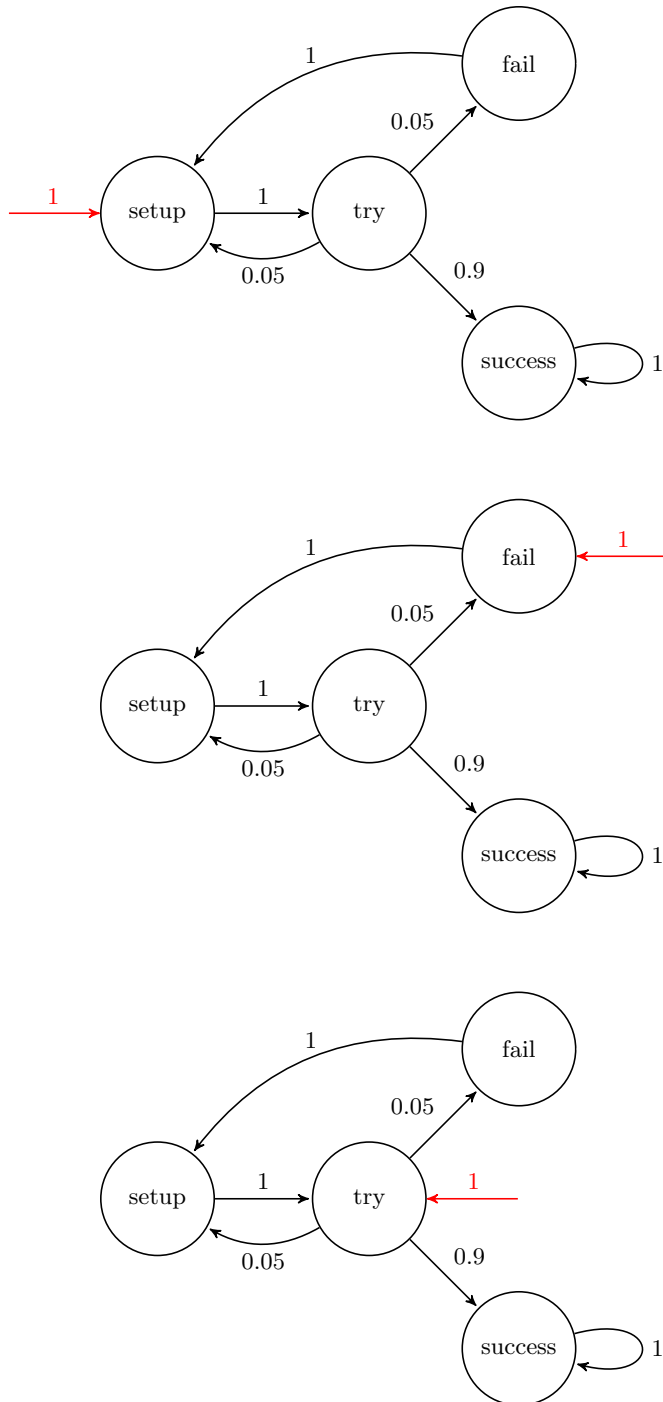


Figure 12.4: Moving the Initial State

12.2.3 Model Checking of $\mathbf{F}p$

For every state s , we will denote by x_s the probability $\mathcal{P}_{(S,1_s,R)}(\mathbf{F}p)$. Now we analyze the probabilistic transition system in Figure 12.3. The state fail is the goal, so we know that x_{fail} must be 1. From the state success it is not possible to reach the goal. Hence $x_{success}$ must be 0. From the state setup, there is only one transition, hence we know that x_{setup} is equal to $1x_{try}$. Finally, from the state try, there are three transitions, each with a certain probability. To reach the goal, we have to take one of those transitions, and then reach the goal from the resulting state. Hence x_{try} must be $0.05x_{fail} + 0.05x_{setup} + 0.9x_{success}$.

Writing down all that we know so far, we arrive at the following linear system of equations:

$$\begin{aligned} x_{fail} &= 1 \\ x_{success} &= 0 \\ x_{setup} &= 1x_{try} \\ x_{try} &= 0.05x_{fail} + 0.05x_{setup} + 0.9x_{success} \end{aligned}$$

The solution of this system is $x_{setup} = 0.05263\dots$

In general

$$\begin{aligned} \mathcal{P}_{(S,1_s,R)}(\mathbf{F}p) &= \mathcal{P}_{(S,1_s,R)}(p \vee [\neg p \wedge \mathbf{X}\mathbf{F}p]) \\ &= \mathcal{P}_{(S,1_s,R)}(p) + \mathcal{P}_{(S,1_s,R)}(\neg p \wedge \mathbf{X}\mathbf{F}p) \\ &= \mathcal{P}_{(S,1_s,R)}(p) + \mathcal{P}_{(S,1_s,R)}(\neg p)\mathcal{P}_{(S,1_s,R)}(\mathbf{X}\mathbf{F}p). \end{aligned}$$

In the special case where the initial state s is already the goal, that is, the case where $s \models p$, this evaluates to $1 + 0$ which is equal to 1. Otherwise, that is, in the case where $s \not\models p$, this is equal to

$$0 + \sum_{s' \in S} R(s, s')\mathcal{P}_{(S,1_{s'},R)}(\mathbf{F}p) = \sum_{s' \in S} R(s, s')\mathcal{P}_{(S,1_{s'},R)}(\mathbf{F}p).$$

As a consequence, we get a linear system of equations that for every $s \in S$ contains the equation

$$x_s = \begin{cases} 1, & \text{if } s \models p, \\ 0, & \text{if no state satisfying } p \text{ is reachable from } s, \\ \sum_{s' \in S} R(s, s')x_{s'}, & \text{otherwise.} \end{cases}$$

Here, the term “reachable” means reachable in the transition graph, ignoring non-zero probabilities.

12.2.4 Model Checking of Temporal Logic Formulas: Plan

We start with the observation that the property $\mathbf{G}\phi$ is equivalent to $\neg\mathbf{F}\neg\phi$. This means that if we can handle the temporal operators $\mathbf{F}$ and $\neg$, then we can also handle $\mathbf{G}$.

We will show how to compute the probability of general LTL properties by showing how to compute

1. $\mathcal{P}_{(S,1_s,R)}(\phi)$, $s \in S$ for a formula ϕ without temporal operators, and then recursively

2. $\mathcal{P}_{(S,1_s,R)}(\mathbf{F}\phi), s \in S$ from $\mathcal{P}_{(S,1_s,R)}(\phi), s \in S$, and $\mathcal{P}_{(S,1_s,R)}(\neg\phi), s \in S$
3. $\mathcal{P}_{(S,S_0,R)}(\phi)$ from $\mathcal{P}_{(S,1_s,R)}(\phi), s \in S$.

The general principle is to compute probabilities recursively, according to the structure of formulas. We will start with discussing this principle on the example from Figure 12.2. We assume $\mathcal{I}(b) = \{B\}$ and want to compute the probability of $\mathbf{GF}b$. For this, we first observe that this property is equivalent to $\neg\mathbf{F}\neg\mathbf{F}b$. Then we fill in the probabilities into the following table from top to bottom:

	$\mathcal{P}_{(S,1_A,R)}()$	$\mathcal{P}_{(S,1_B,R)}()$
b	0	1
$\mathbf{F}b$	1	1
$\neg\mathbf{F}b$	0	0
$\mathbf{F}\neg\mathbf{F}b$	0	0
$\neg\mathbf{F}\neg\mathbf{F}b$	1	1

We get the top row corresponding to the state property b directly from $\mathcal{I}(b) = \{B\}$. The numbers in the other rows follow from rules for recursively computing probabilities for the temporal operator $\mathbf{F}$, and for the negation symbol. We will discuss those rules below.

After having filled this table, we get the total probability from the fact that $\mathcal{P}_{(S,S_0,R)}(\mathbf{GF}b) = \mathcal{P}_{(S,1_A,R)}(\mathbf{GF}b)$ which, according to the table, is equal to 1.

12.2.5 LTL Model Checking: Formulas without Temporal Operators Properties

The base case of the recursion is the case of an LTL formula without temporal operators. Here, the computation of probabilities is straightforward:

$$\mathcal{P}_{(S,1_s,R)}(\phi) = \begin{cases} 1, & \text{if } (s, \dots) \models \phi \\ 0, & \text{otherwise} \end{cases}$$

The intuition is that ϕ holds on the initial state s .

In the special case, where p is a state property, that is, it does not even contain any logical operator, $(s, \dots) \models p$ if and only if $s \models p$. Now consider the example in Figure 12.5. Here, if $\mathcal{I}(p) = \{B, C\}$, then $A \not\models p$, and hence $\mathcal{P}_{(S,1_A,R)}(p) = 0$. But if $\mathcal{I}(p) = \{A, B\}$, then $A \models p$, and hence $\mathcal{P}_{(S,1_A,R)}(p) = 1$.

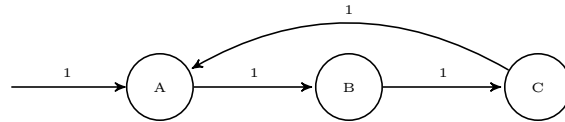


Figure 12.5: Probabilistic Transition System

12.2.6 LTL Model Checking: $\mathbf{X}\phi$

We return to the example of Figure 12.2 and assume that we already calculated the probabilities for ϕ . More specifically, we assume $\mathcal{P}_{(S,1_A,R)}(\phi) = 0$ and $\mathcal{P}_{(S,1_B,R)}(\phi) = 1$.

The probability $\mathcal{P}_{(S,1_A,R)}(\mathbf{X}\phi)$ is the probability of satisfying ϕ in the next state, when starting from the state A . The state A has two transitions with two corresponding probabilities. Hence we have:

$$\begin{aligned} \mathcal{P}_{(S,1_A,R)}(\mathbf{X}\phi) &= \sum_{s' \in S} R(A, s') \mathcal{P}_{(S,1_{s'},R)}(\phi) = \\ &= 0.1 \mathcal{P}_{(S,1_A,R)}(\phi) + 0.9 \mathcal{P}_{(S,1_B,R)}(\phi) = 0.9. \end{aligned}$$

In general

$$\begin{aligned} \mathcal{P}_{(S,1_s,R)}(\mathbf{X}\phi) &= \mathcal{P}_{(S,1_s,R)}(\{\pi \mid \pi \models \mathbf{X}\phi\}) = \\ &= \mathcal{P}_{(S,1_s,R)}(\{\pi \mid \pi^1 \models \phi\}) = \sum_{s' \in S} R(s, s') \mathcal{P}_{(S,1_{s'},R)}(\phi) \end{aligned}$$

12.2.7 LTL Model Checking: Negation

From the observation

$$\begin{aligned} \mathcal{P}_{(S,1_s,R)}(\neg\phi) + \mathcal{P}_{(S,1_s,R)}(\phi) &= \\ &= \mathcal{P}_{(S,1_s,R)}(\{\pi \in \Sigma_S \mid \pi \models \neg\phi\}) + \mathcal{P}_{(S,1_s,R)}(\{\pi \in \Sigma_S \mid \pi \models \phi\}) \\ &= \mathcal{P}_{(S,1_s,R)}(\{\pi \in \Sigma_S \mid \pi \models \neg\phi\} \cup \{\pi \in \Sigma_S \mid \pi \models \phi\}) \\ &= \mathcal{P}_{(S,1_s,R)}(\Sigma_S) \\ &= 1 \end{aligned}$$

we immediately get that

$$\mathcal{P}_{(S,1_s,R)}(\neg\phi) = 1 - \mathcal{P}_{(S,1_s,R)}(\phi)$$

and especially also

$$\mathcal{P}_{(S,1_s,R)}(\mathbf{G}\phi) = \mathcal{P}_{(S,1_s,R)}(\neg\mathbf{F}\neg\phi) = 1 - \mathcal{P}_{(S,1_s,R)}(\mathbf{F}\neg\phi)$$

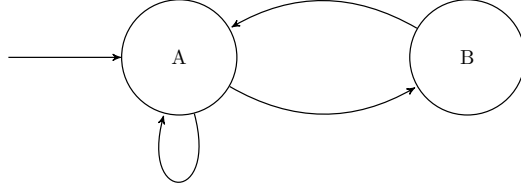
But, be careful to not to apply analogical reasoning in the non-probabilistic case! For non-probabilistic transition systems $\models \neg\phi$ is *not* equivalent to $\not\models \phi$:

$$\begin{aligned} \models \neg\phi &\iff \forall \pi . \pi \models \neg\phi \iff \forall \pi . \pi \not\models \phi \\ &\not\iff \\ \exists \pi . \pi \not\models \phi &\iff \neg \forall \pi . \pi \models \phi \iff \not\models \phi \end{aligned}$$

and especially

$$\models \mathbf{G}\phi \text{ iff } \models \neg\mathbf{F}\neg\phi \text{ iff } \not\models \mathbf{F}\neg\phi$$

For example, in the case of the transition system



and $\mathcal{I} = \{b\}$, neither $\models \neg \mathbf{F}p$ nor $\models \mathbf{F}p$ since for some paths π , $\pi \models \mathbf{F}p$ and for some, $\pi \models \mathbf{F}p$.

12.2.8 LTL Model Checking: $\mathbf{F} \phi$

When checking the $\mathbf{F}$ operator recursively, one can use a similar method as for the non-recursive case discussed in Section 12.2.3.

$$\begin{aligned}
 \mathcal{P}_{(S,1_s,R)}(\mathbf{F}\phi) &= \mathcal{P}_{(S,1_s,R)}(\phi \vee [\neg\phi \wedge \mathbf{X}\mathbf{F}\phi]) = \\
 &= \mathcal{P}_{(S,1_s,R)}(\phi) + \mathcal{P}_{(S,1_s,R)}([\neg\phi \wedge \mathbf{X}\mathbf{F}\phi]) = \\
 &= \mathcal{P}_{(S,1_s,R)}(\phi) + \mathcal{P}_{(S,1_s,R)}(\neg\phi) \mathcal{P}_{(S,1_s,R)}(\mathbf{X}\mathbf{F}\phi) = \\
 &= \mathcal{P}_{(S,1_s,R)}(\phi) + (1 - \mathcal{P}_{(S,1_s,R)}(\phi)) \mathcal{P}_{(S,1_s,R)}(\mathbf{X}\mathbf{F}\phi) = \\
 &= \mathcal{P}_{(S,1_s,R)}(\phi) + (1 - \mathcal{P}_{(S,1_s,R)}(\phi)) \sum_{s' \in S} R(s, s') \mathcal{P}_{(S,1_{s'},R)}(\mathbf{F}\phi)
 \end{aligned}$$

Hence we have:

$$\mathcal{P}_{(S,1_s,R)}(\mathbf{F}\phi) = \mathcal{P}_{(S,1_s,R)}(\phi) + (1 - \mathcal{P}_{(S,1_s,R)}(\phi)) \sum_{s' \in S} R(s, s') \mathcal{P}_{(S,1_{s'},R)}(\mathbf{F}\phi)$$

This results in a linear system of equations that contains for every $s \in S$, the equation

$$\mathcal{P}_{(S,1_s,R)}(\mathbf{F}\phi) = \begin{cases} 0, & \text{if for all states } t \text{ reachable from } s, \mathcal{P}_{(S,1_t,R)}(\phi) = 0, \\ \mathcal{P}_{(S,1_s,R)}(\phi) + (1 - \mathcal{P}_{(S,1_s,R)}(\phi)) \sum_{s' \in S} R(s, s') \mathcal{P}_{(S,1_{s'},R)}(\mathbf{F}\phi), & \text{otherwise.} \end{cases}$$

This system always has a unique solution [17, Lemma 3.1.1.1].

12.2.9 Combination

Now we can describe in detail, how to fill out the table from Section 12.2.4

To fill out the second row we set up the linear system of equations described in Section 12.2.8:

$$\begin{aligned}
 \mathcal{P}_{(S,1_A,R)}(\mathbf{F}b) &= 0.1\mathcal{P}_{(S,1_A,R)}(\mathbf{F}b) + 0.9\mathcal{P}_{(S,1_B,R)}(\mathbf{F}b) \\
 \mathcal{P}_{(S,1_B,R)}(\mathbf{F}b) &= 1.
 \end{aligned}$$

We can solve this system as follows:

$$\begin{aligned}
 \mathcal{P}_{(S,1_A,R)}(\mathbf{F}b) &= 0.1\mathcal{P}_{(S,1_A,R)}(\mathbf{F}b) + 0.9 \\
 0.9\mathcal{P}_{(S,1_A,R)}(\mathbf{F}b) &= 0.9
 \end{aligned}$$

$$\mathcal{P}_{(S,1_A,R)}(\mathbf{F}b) = 1$$

The next row results from applying the rule for negation:

$$\mathcal{P}_{(S,1_A,R)}(\neg \mathbf{F}b) = 1 - \mathcal{P}_{(S,1_A,R)}(\mathbf{F}b) = 0$$

$$\mathcal{P}_{(S,1_B,R)}(\neg \mathbf{F}b) = 1 - \mathcal{P}_{(S,1_B,R)}(\mathbf{F}b) = 0$$

The next row is again a linear system of equations resulting from the $\mathbf{F}$ operator:

$$\mathcal{P}_{(S,1_A,R)}(\mathbf{F}\neg \mathbf{F}b) = 0; \mathcal{P}_{(S,1_B,R)}(\mathbf{F}\neg \mathbf{F}b) = 0$$

The rule for negation results in the next row:

$$\mathcal{P}_{(S,1_A,R)}(\neg \mathbf{F}\neg \mathbf{F}b) = 1; \mathcal{P}_{(S,1_B,R)}(\neg \mathbf{F}\neg \mathbf{F}b) = 1$$

And from this we get the following overall probability:

$$\mathcal{P}_{(S,S_0,R)}(\neg \mathbf{F}\neg \mathbf{F}b) = 1\mathcal{P}_{(S,1_A,R)}(\neg \mathbf{F}\neg \mathbf{F}b) + 0\mathcal{P}_{(S,1_B,R)}(\neg \mathbf{F}\neg \mathbf{F}b) = 1$$

12.2.10 Overall Probability

Up to now, the final step of computing the probability of the initial distribution was very simple. To discuss the general case, consider Figure 12.6.

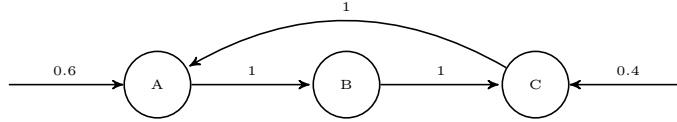


Figure 12.6: Non-Trivial Initial Distribution

In addition, assume

$$\mathcal{P}_{(S,1_A,R)}(\phi) = 0.2, \mathcal{P}_{(S,1_B,R)}(\phi) = 0.7, \mathcal{P}_{(S,1_C,R)}(\phi) = 0.1$$

Then:

$$\begin{aligned} \mathcal{P}_{(S,S_0,R)}(\phi) &= 0.6\mathcal{P}_{(S,1_A,R)}(\phi) + 0.4\mathcal{P}_{(S,1_C,R)}(\phi) \\ &= 0.6 \times 0.2 + 0.4 \times 0.1 \\ &= 0.16 \end{aligned}$$

In general, we have

$$\begin{aligned} \mathcal{P}_{(S,S_0,R)}(\phi) &= \mathcal{P}_{(S,S_0,R)}(\{\pi \in \Sigma_S \mid \pi \models \phi\}) \\ &= \mathcal{P}_{(S,S_0,R)}\left(\bigcup_{s \in S} \{\pi \in \Sigma_S \mid \pi(0) = s, \pi \models \phi\}\right) \\ &= \sum_{s \in S} \mathcal{P}_{(S,S_0,R)}(\{\pi \in \Sigma_S \mid \pi(0) = s, \pi \models \phi\}) \\ &= \sum_{s \in S} S_0(s) \mathcal{P}_{(S,1_s,R)}(\{\pi \in \Sigma_S \mid \pi(0) = s, \pi \models \phi\}) \\ &= \sum_{s \in S} S_0(s) \mathcal{P}_{(S,1_s,R)}(\{\pi \in \Sigma_S \mid \pi \models \phi\}) \\ &= \sum_{s \in S} S_0(s) \mathcal{P}_{(S,1_s,R)}(\phi) \end{aligned}$$

12.3 Nondeterminism versus Probability

At the beginning of the chapter, we already saw that in some cases, a probabilistic model makes more sense than a non-deterministic one. Especially, for the probabilistic transition system in Figure 12.1, $\mathcal{P}(\mathbf{GF}b) = 1$, but if we would ignore the probabilities, we would get $\not\models \mathbf{GF}b$.

Now we ask the same question in the opposite direction. Why do we need transition systems without probabilities at all? There are many situations, when this is the case. Especially we might not have any probabilities available. It might also be the case that our system may does have the Markov property that says that the probabilities of transition always depend on the current state only, not on the states visited before or the time when the state is visited.

12.4 Markov Decision Processes

Markov decision processes (MDP) are models that allow both non-deterministic and probabilistic transitions. They form the basis for reinforcement learning, where the non-deterministic transitions correspond to decisions that we can take, and the probabilistic transitions are probabilities on how the environment reacts to our decisions. Reinforcement learning learns those probabilities and optimal decisions while actually interacting with the environment.

Verification tool: <http://www.prismmodelchecker.org>

Demo: Synchronous Leader Election Protocol

http://www.prismmodelchecker.org/casestudies/synchronous_leader.php

12.5 Costs and Rewards

The classical view is that a computer program is either correct or incorrect. But in many cases this is a too crude viewpoint. For example, in some cases it is not possible to avoid all bugs, and hence one heads for a more modest goal, the smallest possible impact of bugs. For battery-powered devices it is important to have a design that uses as little energy as possible. When steering an aeroplane, its energy consumption is critical. And when writing software for trading at a stock exchange, the goal is to achieve the highest possible profit.

To model such a situation, we introduce a cost function $c : S \rightarrow \mathbb{N}_0$ that assigns to every state its cost. The cost of a finite path is the sum of the cost of all visited states:

$$c(s_0, \dots, s_n) := \sum_{i \in \{0, \dots, n-1\}} c(s_i)$$

Based on this, the expected cost is

$$E(\mathbf{F}p) := \sum_{\substack{(s_0, \dots, s_n) \in \Sigma_S, \\ s_0 \not\models p, \dots, s_{n-1} \not\models p, s_n \models p}} c(s_0, \dots, s_n) \mathcal{P}(s_0, \dots, s_n),$$

if $\mathcal{P}(\mathbf{F}p) = 1$ and ∞ , otherwise.

For demonstrating how to compute this expected cost, we return to the example in Figure 12.3. We will assume that the cost of each state is one, that is $c(\text{setup}) = c(\text{try}) = c(\text{fail}) = c(\text{success}) = 1$.

We define the variable $x_s := E(\mathbf{F}\text{success})$, for the probabilistic transition system $(S, 1_s, R)$, that is the transition system with initial probability 1 for the state s . This results in the linear system of equations

$$\begin{aligned} x_{\text{success}} &= 0 \\ x_{\text{try}} &= 1 + 0.05x_{\text{fail}} + 0.9x_{\text{success}} + 0.05x_{\text{setup}} \\ x_{\text{fail}} &= 1 + x_{\text{setup}} \\ x_{\text{setup}} &= 1 + x_{\text{try}} \end{aligned}$$

which has a solution with $x_{\text{setup}} = 2.05/0.9 \sim 2.278$.

In general, under the assumption that $\mathcal{P}(\mathbf{F}p) = 1$ one can compute the expected cost by solving the linear system

$$x_s = \begin{cases} 0, & \text{if } s \in \mathcal{I}(p), \\ c(s) + \sum_{s' \in S} R(s, s')x_{s'}, & \text{otherwise,} \end{cases}$$

from which we get:

$$E(\mathbf{F}p) = \sum_{s \in S} S_0(s)x_s$$

12.6 Further Possibilities

Various combinations and variations:

- Probabilistic timed automata
- Probabilistic hybrid systems
- Probabilistic Petri nets
- Continuous time Markov chains (viz. MI-SPI)
- atd.

12.7 Conclusion

Probability is another possibility of modeling uncertainty that behaves differently from non-determinism in the sense that have seen so far. Still, many of the notions that we have seen so far (transition system, LTL, etc.), have their probabilistic variants. The algorithms for analyzing such systems are different from the ones have hve seen so far. Especially, they tend to be based on linear algebra.

Chapter 13

Modeling of Physical Environment

13.1 Introduction

The models that we have studied up to now mainly were finite (e.g., using Boolean vectors as states), or infinite, but with simple evolution (clocks in timed automata). But this is not enough to model most real-world phenomena such as time, speed, acceleration, pressure, or temperature, in a realistic way. Timed automata at least allowed us to model time by clocks that can be set to 0, and run at exactly the same speed. But what about further physical phenomena such as speed, pressure, temperature etc.?

Of course, this may be relevant in *many* applications, where computational devices interact with their physical environment. Here are some examples:

- autonomous cars, drones, trains
- robotics
- computer games (physics engines)
- processing sensor data (mobile phones) to compute position/speed etc.

13.2 Continuous Modeling of Time

For introducing such continuous models, we will now introduce continuous versions of the definitions from Chapter 4. We will start by a continuous version of Definition 2:

Definition 54 *Given a set S , a (continuous time) signal over S is a function $\mathbb{R}_{\geq 0} \rightarrow S$.*

From now on we will write

- Σ_S^C for the set of continuous time signals over S , a
- Σ_S^D for the set of discrete time signals over S ,

and if clear from the context, simply Σ_S (or even Σ).

Independently from whether we use a discrete or continuous model of time, we can also choose the state of a model to be either

- continuous, especially vectors of real numbers $\mathbb{R}^n$, or
- discrete, for example, $\mathbb{N}$, or even finite.

All four combinations are useful, but especially the following two:

- *analog* signal: continuous-time signal with continuous state
- *digital* signal: discrete-time signal with discrete state

Especially, in cases where one wants to process analog signals on computers, one needs digital signals. Converting an analog signal to a digital one involves two steps:

- discretization of time: *sampling*
- discretization of state: *quantization*

For example, the digital signal that stores audio information on a classical CD has a sampling rate of 44100Hz , 2^{16} and states in the interval $[-2^{15}, 2^{15} - 1]$.

13.3 Continuous Component Models

We used Definition 3 for modeling system components in a discrete way. We introduce a continuous version of this definition, as well:

Definition 55 A continuous time system with input set I and output set O is a relation between continuous time signals over I and continuous time signals over O .

This means that a continuous time system is a subset of $\Sigma_I^C \times \Sigma_O^C$. Here are some examples: An amplifier component, often also called gain, multiplies the input signal by a constant:

$$\{(i, o) \mid \forall t \in \mathbb{R}_{\geq 0} . o(t) = 2i(t)\}$$

Figure 13.1 shows an example behavior of such a system. For a sine function as input (left-hand side), it produces the sine function multiplied by two as output (right-hand side). Another example is a system that outputs the running maximum of the input:

$$\{(i, o) \mid \forall t \in \mathbb{R}_{\geq 0} . o(t) = \max_{\tau \in [0, t]} i(\tau)\}$$

Figure 13.2 shows an example behavior of such a system.

An especially important component is the integrator:

$$\{(i, o) \mid \forall t \in \mathbb{R}_{\geq 0} . o(t) = s_o + \int_0^t i(\tau) d\tau\}$$

For example, for the step function shown on the left-hand side of Figure 13.3,

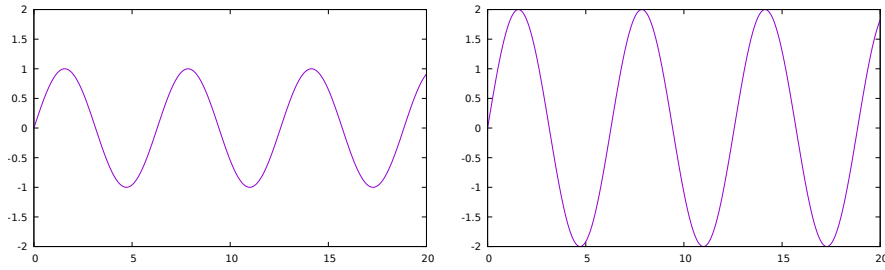


Figure 13.1: Example Behavior of Gain

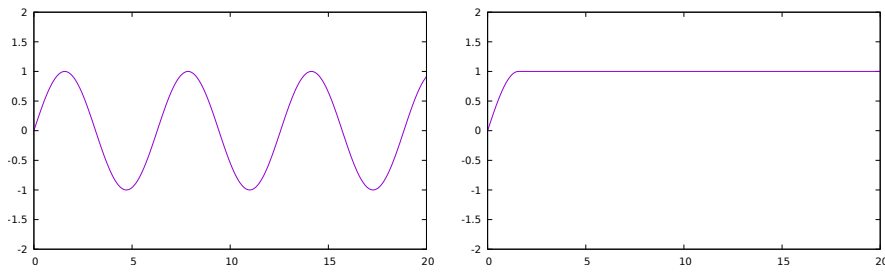


Figure 13.2: Example Behavior of Running Maximum

and integrator with $s_0 = 0$ outputs a function that decreases linearly as long as the input is -1 and then increases linearly. The integrator can, for example, model a system whose input is speed, and output is position. It could also model a system whose input is input is acceleration, and output is speed. Then, the example behavior in the figure corresponds to the situation, where acceleration is first negative then positive. On the right-hand side of the figure one then sees the corresponding speed that first decreases, and then increases, again.

In general, such an integrator, behaves in a similar way as a control lever: If the lever is at a negative position, it will decrease the controlled output value, and if it is at a positive position, it will increase the output value.

A system, that is mathematically almost equivalent to the integrator, is the the system

$$\{(i, o) \mid \forall t \in \mathbb{R}_{\geq 0} . \dot{o}(t) = i(t), o(0) = s_0\}.$$

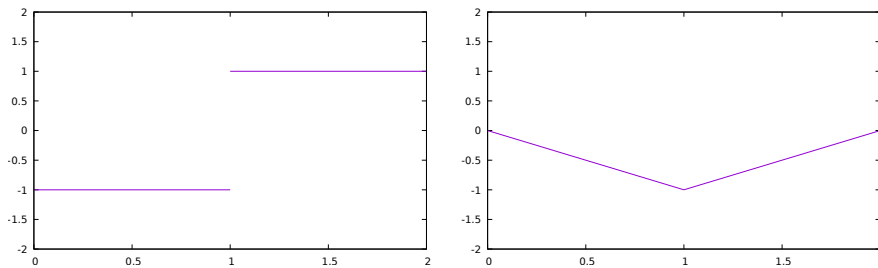


Figure 13.3: Example Behavior of Integrator

This is only slightly more restricted than the integrator since it requires the output to be differentiable which, for example, excludes the behavior from Figure 13.3. However, for any input that is a continuous function, the output of both systems will be precisely the same.

13.4 System Properties

The property of a system being receptive (Definition 5), causal (Definition 7), deterministic (Definition 6), and memory-less 8) can be directly adapted from discrete time systems to continuous time systems.

13.5 Modeling Real-World Phenomena

We will now discuss how one can model real-world phenomena using continuous time systems. Here we will concentrate on signals from $\Sigma_{\mathbb{R}^n}^C$, that is, continuous time signals with state space $\mathbb{R}^n$. In discrete time (e.g., in the case of automata or transition systems) the current state is usually a result of the previous one. But in continuous time we do not have a notion of “previous state”.

One way to get around this problem is to define signals directly as functions of time, for example, defining the signal $x : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}$ by the requirement

$$\forall x . x(t) = \sin t.$$

However, for describing most physical laws, this does not suffice. For example, consider a pendulum (Figure 13.4). The first step for creating a model of such a physical system is always to make certain simplifying assumptions. For example, we will ignore air friction, modeling the system in such way that its movements correspond to movement in vacuum. We will also assume that the pendulum itself is completely stiff and that all of its mass is concentrated at the end of the pendulum. Finally, we will assume that the distance of the pendulum to the center of gravity is so large that we can assume that gravity will always act in the vertical direction.

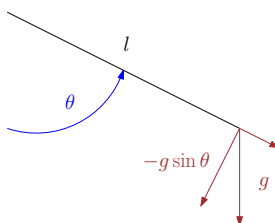


Figure 13.4: Pendulum

Now we can start to model. Clearly the angle θ changes according to the angular velocity v , resulting in the law $\dot{\theta} = v$. Moreover, gravity acts on the mass at the end of the pendulum, accelerating the pendulum into the direction of the center of gravity. This acceleration depends on the position of the pendulum, being zero when the pendulum is in vertical position and the gravitational constant g in horizontal position. In general, the acceleration corresponds to

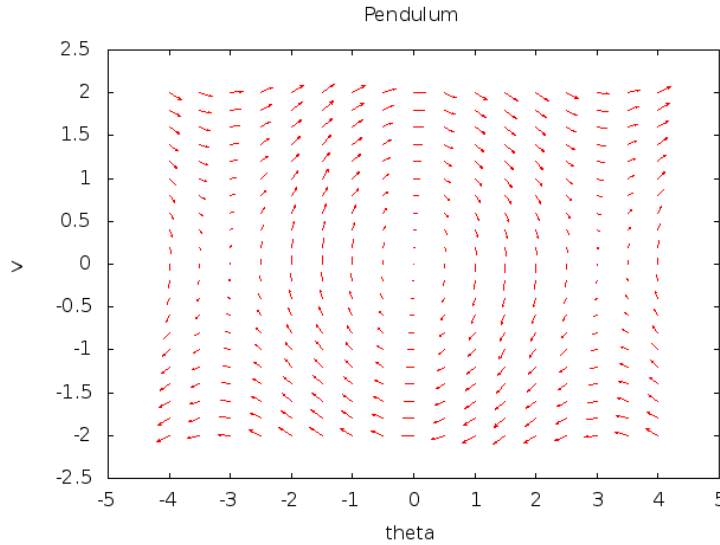


Figure 13.5: Vector Field of Pendulum

the component of the gravitational force that results in movement of the pendulum which is $-\frac{g}{l} \sin \theta$, where l is the length of the pendulum. The result is the system of ordinary differential equations

$$\begin{aligned}\dot{\theta} &= v \\ \dot{v} &= -\frac{g}{l} \sin \theta\end{aligned}$$

In general, for a function $f : S \rightarrow \mathbb{R}^n$, we call $\dot{x} = f(x)$ a *system of ordinary differential equations*, which we will often abbreviate to *ordinary differential equation* (ODE).

A function $f : S \rightarrow \mathbb{R}^n$, where $S \subseteq \mathbb{R}^n$ is called a *vector field*. The right-hand side of a system of ordinary differential equations forms such a vector field. More concretely, the function that assigns to every pair θ and v the corresponding right-hand side of our system forms such a vector field. The advantage of such vector fields is that one can easily visualize them in the 2-dimensional case. For example, Figure 13.5 is a visualization of the vector field of the pendulum. Figure 13.6 shows how to visualize such a vector field in 3D.

The intuition of such a vector field is that it assigns to each allowed value of real variables a direction into which the values will evolve. Based on this one can then follow the behavior of the system by following those direction in a similar way as shown in Figure 13.7. Wikipedia also contains a nice visualization: [http://en.wikipedia.org/wiki/Pendulum_\(mathematics\)](http://en.wikipedia.org/wiki/Pendulum_(mathematics)).

A point x where the vector field is the zero vector, that is, $f(x) = 0$ is called an *equilibrium*. This corresponds to the situation of a state of the system that the system will not leave, if already being there. Such an equilibrium can be *stable* or *unstable*, depending on whether the system tends to return to the equilibrium slight moved away from it, or tends to move away from the equilibrium, in this case. For example, the point $\theta = 0, v = 0$ is a stable equilibrium of the

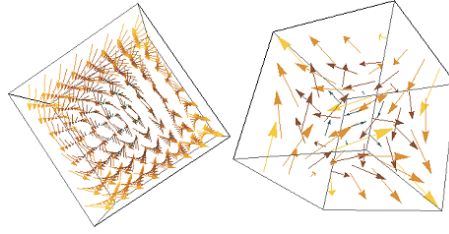


Figure 13.6: Vector Field of Pendulum

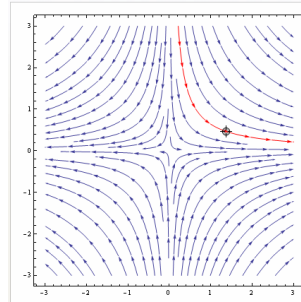


Figure 13.7: Following Vector Field

pendulum, and the point $\theta = \pi, v = 0$ an unstable equilibrium.

The analogon to a vector field in discrete time is the notion of deterministic transition systems and its visualization using a state diagram. This immediately opens the question about whether the notion of path of a transition has an analogon in continuous time.

We will discuss this on hand of a different model, the so-called Lotka-Volterra model that models the evolution of two kinds of biological species where one species is a predator (variable y) and the other one its prey (variable x).

$$\begin{aligned}\dot{x} &= \alpha x - \beta yx \\ \dot{y} &= -\gamma y + \delta xy\end{aligned}$$

Of course, biological species come in discrete numbers. For example, it does not makes sense to speak about two and a half (living) rabbits. However, instead of calculating the behavior of large populations, let us say, one million rabbits, it is often simpler to use a continuous abstraction, ignoring the fact that living rabbits cannot come in halves. To understand the behavior of such a differential equation one can visualize its vector field. However, we will now use a different possibility that can be more easily generalized to higher dimensions: We will visualize the behavior of the ODE over time from a given starting point using computer simulation. One can then visualize the result in a diagram whose x-axis is the time axis (see Figure 13.8).

In general, assuming an ordinary differential equation $\dot{x} = f(x)$, where f is a vector field $f : S \rightarrow \mathbb{R}^n$, we look for a function $x : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}^n$, that follows the vector field. In the representation with time axis, for all time $t \in \mathbb{R}_{\geq 0}$, every curve has a slope that corresponds to $f(x(t))$ (see the red lines in Figure 13.9).

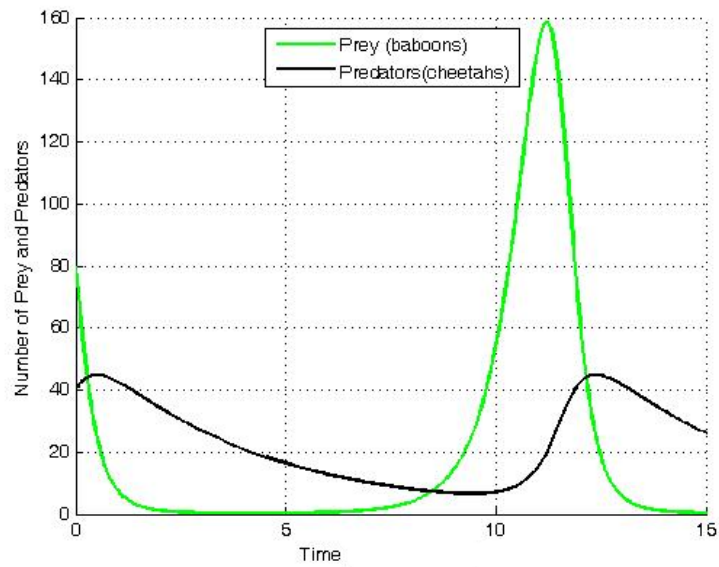


Figure 13.8: Simulation

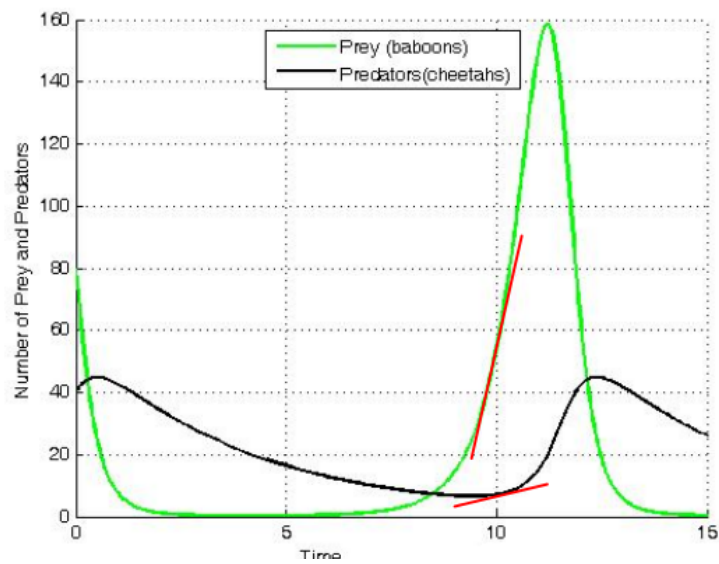


Figure 13.9: Slope

Definition 56 A solution of the equation $\dot{x} = f(x)$ is a function $x : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}^n$, s.t. for all $t \in \mathbb{R}_{\geq 0}$, $\dot{x}(t) = f(x(t))$

This is now our desired analogon of the notion of a path of a transition system. Such a solution is also called a *trajectory* of the differential equation.

Usually, in a similar way as for transition systems, we state initial conditions, calling the result *initial value problem (IVP)*.

13.6 Relationship to Timed Automata

Timed automata show both discrete and continuous behavior. Their locations correspond to discrete states and their clocks show continuous behavior. Action transitions are discrete and delay transitions continuous.

If we analyze delay transitions in more detail, for example using the delay transition

$$(work, \{x \mapsto 0, y \mapsto 4\}) \xrightarrow{7} (work, \{x \mapsto 7, y \mapsto 11\})$$

we see that the fact that every clock runs with the same speed corresponds to a very specific vector field (Figure 13.10).

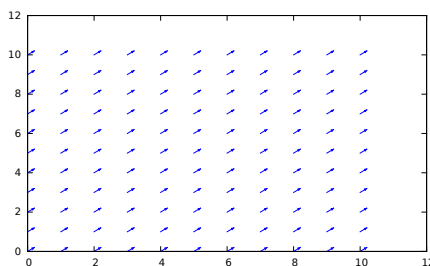


Figure 13.10: Vector Field of Delay Transitions

This corresponds to a system of ordinary differential equations with the equation $\dot{x} = 1$ for every clock $x \in X$. Here, timed automata represent only endpoints of solutions and ignore the trajectory leading to the endpoint.

13.7 Non-Determinism

Already in the case of discrete time systems we allowed for non-deterministic behavior coming from the system environment (e.g., user, weather), unknown details, or unmodeled details. How does this look like for differential equations?

For example, we might have a differential equation of the form $\dot{x} = 2x + 0.4$, where we do not know the constant 0.4 precisely. There are several common ways of denoting such a situation. For example, one can express the fact that $\dot{x}$ is any element of the set $\{2x + \delta \mid \delta \in [0.3, 0.5]\}$, as follows:

- $\dot{x} = 2x + 0.4 \pm 0.1$,
- $\dot{x} = 2x + [0.3, 0.5]$

- $2x + 0.3 \leq \dot{x} \leq 2x + 0.5$

In general, this means that we do not have a unique direction $f : S \rightarrow \mathbb{R}^n$, but either

- a set of possibilities $F : S \rightarrow 2^{\mathbb{R}^n}$, or
- a relation $r : S \times \mathbb{R}^n$.

The resulting mathematical object is called a:

- differential inclusion $\dot{u} \in F(u)$, or
- differential relation $r(u, \dot{u})$ (e.g., differential inequalities)

13.8 Input and Output

Now assume that we install a brake at the pendulum that applies some braking force i that can slow down its speed by subtracting iv from the force resulting from gravity:

$$\begin{aligned}\dot{\theta} &= v \\ \dot{v} &= -\frac{g}{l} \sin \theta - iv\end{aligned}$$

The braking force is an input to this system.

We could also have a system with a state that changes according to the input:

$$\dot{s} = i$$

This is a shortcut for

$$\forall t \in \mathbb{R}_{\geq 0} . \dot{s}(t) = i(t)$$

which can be (almost) equivalently written as

$$\forall t \in \mathbb{R}_{\geq 0} . s(t) = \int_0^t i(\tau) d\tau$$

So, this is a system whose state integrates the input. For example, the input could be the acceleration of some object that we can influence, and the current state its speed.

The example

$$\dot{s} = i, o = 2s$$

is a shortcut for

$$\forall t \in \mathbb{R}_{\geq 0} . \dot{s}(t) = i(t), o(t) = 2s(t).$$

which adds output to the previous system. For example, this could again model the situation, where we can accelerate some object, and the current state its speed, but the output is the speed translated from the physical unit km/h to some different physical unit.

In general, we can add input and output to differential equations by extending them as follows:

$$\dot{s} = f(s, i), o = g(i, s)$$

which is a short-cut for

$$\forall t \in \mathbb{R}_{\geq 0} . \dot{s}(t) = f(s(t), i(t)), o(t) = g(i(t), s(t))$$

Note that this defines a relation between the input signal i , the state s , the derivative of the state $\dot{s}$, and the output signal o . So, for defining general systems with input output, we need such a relation:

Definition 57 A continuous automaton is a quintuple (n, p, q, S_0, R) , where

- $n, p, q \in \mathbb{N}$ (then we call $\mathbb{R}^n$ state space, $\mathbb{R}^p$ input space, $\mathbb{R}^q$ output space)
- $S_0 \subseteq \mathbb{R}^n$ (set of initial states)
- $R \subseteq \mathbb{R}^p \times \mathbb{R}^n \times \mathbb{R}^n \times \mathbb{R}^q$ (transition relation) s.t.
for all $i \in \mathbb{R}^p$, $s \in \mathbb{R}^n$,
there exists $s' \in \mathbb{R}^n$, $o \in \mathbb{R}^q$ s.t. $(i, s, s', o) \in R$

The pair of signals $(i, o) \in \Sigma_I \times \Sigma_O$ is a *behavior* of the automaton iff there exists an $s \in \Sigma_S$ s.t.

- $s(0) \in S_0$,
- for all $t \in \mathbb{R}_{\geq 0}$, $(i(t), s(t), \dot{s}(t), o(t)) \in R$

An automaton T represents the system

$$\llbracket T \rrbracket := \{(i, o) \in \Sigma_I \times \Sigma_O \mid (i, o) \text{ is a behavior of } T\}$$

Here are some examples, where we view $\mathbb{R}^0$ as the set $\{()\}$, and we do not distinguish $\mathbb{R}^1$ and $\mathbb{R}$. The continuous automaton

$$(0, 0, 1, \{()\}, \{(((), (), (), 1))\})$$

which is another way of writing

$$(0, 0, 1, \{()\}, \{(i, s, s', o) \in \mathbb{R}^0 \times \mathbb{R}^0 \times \mathbb{R}^0 \times \mathbb{R}^1 \mid o = 1\})$$

represents a source with constant output. The system

$$(0, 1, 1, \{()\}, \{(i, (), (), 2i) \mid i \in \mathbb{R}\}),$$

which is another way of writing

$$(0, 1, 1, \{()\}, \{(i, s, s', o) \in \mathbb{R}^1 \times \mathbb{R}^0 \times \mathbb{R}^0 \times \mathbb{R}^1 \mid o = 2i\})$$

represents a gain (i.e., an amplifier). The system

$$(0, 2, 1, \{()\}, \{((i_1, i_2), (), (), i_1 + i_2) \mid i_1 \in \mathbb{R}, i_2 \in \mathbb{R}\}),$$

another way of writing

$$(0, 2, 1, \{()\}, \{(i, s, s', o) \in \mathbb{R}^2 \times \mathbb{R}^0 \times \mathbb{R}^0 \times \mathbb{R}^1 \mid i = (i_1, i_2), o = i_1 + i_2\})$$

represents an input adder input adder (see also table lookup). The system

$$(1, 0, 1, \mathbb{R}, \{(((), s, s^2 + 1, s) \mid s \in \mathbb{R}\})$$

which is another way of writing

$$(1, 0, 1, \mathbb{R}, \{(i, s, s', o) \in \mathbb{R}^0 \times \mathbb{R}^1 \times \mathbb{R}^1 \times \mathbb{R}^1 \mid s' = s^2 + 1, o = s\})$$

represents a system that is a source whose output comes from from the ordinary differential equation $\dot{s} = s^2 + 1$.

An especially interesting example is the system

$$(1, 1, 1, S_0, \{(i, o, i, o) \mid i \in \mathbb{R}, o \in \mathbb{R}\})$$

which is another way of writing

$$(1, 1, 1, S_0, \{(i, s, s', o) \mid s' = i, o = s\}).$$

To understand this system, we first observe that a pair $(i, o) \in \Sigma_{\mathbb{R}}^C \times \Sigma_{\mathbb{R}}^C$ is a behavior of this system iff there exists a signal $s \in \Sigma_S^C$ s.t.

- $s(0) \in S_0$,
- for all $t \in \mathbb{R}_{\geq 0}$. $\dot{s}(t) = i(t), o(t) = s(t)$.

The latter condition can be simplified to

$$\dot{o}(t) = i(t)$$

Hence, the automaton represents the integrator

$$\{(i, o) \mid \forall t \in \mathbb{R}_{\geq 0} . o(t) = s_0 + \int_0^t i(\tau) d\tau, s_0 \in S_0\}$$

Control theory/engineering uses the following terms for specific classes of continuous automata:

- SISO (single input, single output system): $p = 1, q = 1$
- MIMO (multiple input, multiple output system): $p > 1, q > 1$
- LTI (linear, time-invariant system): Transition relation is given in the form

$$\dot{x} = Ax + Bu, y = Cx + Du,$$

where x denotes state, u input, y output.

13.9 Choice of Model

One might be tempted to think that digital electronics always results in discrete models and their physical surrounding need continuous models. However, sometimes, already the physical system contains discrete aspects. For example:

- physical contact: bouncing ball
- technical device has discrete aspects: switches, car gears
- discrete modeling artifact: linearization

In the other direction, sometimes continuity occurs already in computer systems:

- real-time requirements: protocols (after 10 seconds, do this)
- computation of continuous output: music, simulation of continuous phenomena
- continuous abstraction of computer systems: data streams

And, of course, there is analogue circuits.

In general, the type of model used (continuous, discrete, probabilistic) is not an inherent property of the reality we are modeling, but dependent on the application and modeling level:

Very often, models of a system come in a hierarchy of abstractions. For example, for digital electronics may have a hierarchy such as the following:

- Programming languages
- Assembly language
- Hardware description languages
- Boolean Logic
- Transistor level description
- Electromagnetic field (partial differential equations: Maxwell equations)
- Particle (atomic)
- Quantum mechanics

For physical systems one can see a hierarchy such as the following

- Item database
- Newtonian mechanics
- Statistical thermodynamics
- Particle (atomic)
- Quantum mechanics

Here, discrete layers may be abstracted to continuous ones and vice versa, probabilistic to non-probabilistic ones, and vice versa.

13.10 Conclusion

In this chapter we introduced many continuous notions for which know we discrete analogies already before. The following table shows those analogies:

discrete	continuous
transition function $f : S \rightarrow S$ $\forall t \in \mathbb{N}_0 . s(t+1) = f(s(t))$	vector field $f : S \rightarrow \mathbb{R}^n$ dif. rovnice $\dot{x} = f(x)$ $\forall t \in \mathbb{R}_{\geq 0} . \dot{x}(t) = f(x(t))$
transition function $F : S \rightarrow 2^S$ $\forall t \in \mathbb{N}_0 . s(t+1) \in F(s(t))$	$F : S \rightarrow 2^{\mathbb{R}^n}$ dif. inkluze $\dot{x} \in F(x)$ $\forall t \in \mathbb{R}_{\geq 0} . \dot{x}(t) \in F(x(t))$
transition relation $T \subseteq S \times S$ $\forall t \in \mathbb{N}_0 . (s(t), s(t+1)) \in T$	$r \subseteq S \times \mathbb{R}^n$ dif. relace/nerovnice $\forall t \in \mathbb{R}_{\geq 0} . r(x(t), \dot{x}(t))$
state diagram path	vector field visualization solution
(discrete time) automaton $\forall t \in \mathbb{N}_0 . (i(t), s(t), s(t+1), o(t)) \in R$	(continuous time) automaton $\forall t \in \mathbb{R}_{\geq 0} . (i(t), s(t), \dot{s}(t), o(t)) \in R$
(discrete time) system	(continuous time) system
LTL BMC SAT unbounded model checking ...	

While this cannot be taken as a rule it is often the case the models of the computational world are discrete and models of the physical world continuous. For computer scientists it is more and more important to feel at home in both worlds.

Part III

Programs: Systems with Data-Structures

Chapter 14

Program Specification

Throughout this chapter, we will use data types introduced in Section 2. Hence, the symbols $\mathcal{N}$ will denote the natural numbers including zero, and $\mathcal{A}$ will denote arrays.

14.1 Input/Output Specifications

Let us start with the following example:

```
result:= "sorted"
for i:= 1 to 10 do
  if a[i]>a[i+1] then
    result:= "unsorted"
  endif
endfor
return result
```

Now try to think about the question whether this algorithm is correct. You might notice that the upper bound of the loop is 10, and that the body of the loop accesses the element $i + 1$. So one might think, that this algorithm is incorrect. But really? How can we know whether the algorithm is correct if we do not even know what the algorithm is supposed to do. In more details:

- If we want to know whether the elements from 1 to 11 of the array a are sorted, then the algorithm looks correct!
- If we want to know whether the elements from 1 to 10 of the array a are sorted, it has a bug!
- And if we want to know whether a contains the prime divisors of the number 12, then it is certainly incorrect.

So we conclude that correctness of algorithm only makes sense in combination with a specification. Let's try to write a specification that this algorithm satisfies:

- Input: array a of length 10

- Output: if for all i in $\{1, \dots, 9\}$, $a[i] \leq a[i + 1]$ then “sorted” else “unsorted”

Such a specification only restricts the input and output of the algorithm. However, it says nothing about side effects—anything else the algorithm might influence. For example, it might write something into a file, send something over the internet, delete files, and so on. So, whenever we will write such input/output specification, we will require that an algorithm implementing such a specification, does not have any side effects.

The specification above also does not restrict the run-time of an implementation. An implementation that takes thousands of years to produce the required output is considered to be correct! But, of course, we usually will not be satisfied with such an implementation.

We will consider input-output specifications to be well-formed only if they satisfy certain requirements. As an example, consider the following specification:

- In: natural number k
- Out: a prime factor of k

For example, if the input is the natural number 12, then both 2 and 3 are valid outputs. This is OK since I/O specifications may allow more than one correct output for one input. But the following requirement will be essential: An input/output specification must always allow at least one output for each input. The specification from above violates this rule, since the natural number 1 does not have a prime factor. The easiest way of correcting the problem is by strengthening the requirement on the input. For our concrete specification, we may require the number k to be a natural number bigger than one. Another solution is to weaken the requirement on the output. For our concrete specification, we may require the output to be a prime factor of the input only if such a prime factor exists. For any input that does not have a prime factor, we may return a special result, for example, the number 0.

Now we can return to the question of program correctness.

Definition 58 *A program satisfies an I/O specification iff for every input satisfying the input condition, the program returns an output satisfying the output condition.*

This does not say thing about *how* the user hands over inputs to the program, and *how* the program returns the corresponding output. If a program satisfies its I/O specification, and if this specification is clear from the context, we will also simply say that the program is *correct*. Later, in Section 15.1, we will also see another definition of program correctness that will take into account not only inputs and outputs, but internal requirements on program executions.

14.2 Specification Languages

Specifications are usually written in natural languages (Czech, English, etc.). This may sometimes result in problems. For example, the English word “eventually” has a different meaning than the Czech word “eventuálně” (which is the same as the meaning of the German word “eventuell”). This may easily result in misunderstandings.

In general, specifications written in natural languages may lack in precision and may result in ambiguous specifications. But they have an additional problem: They are difficult to understand for computers, and computer tools and much easier work with formal languages.

A solution to such problems is the usage of artificial languages. More concretely, we will use the first-order predicate language, which provides a precisely defined artificial language. More precisely, we will base our definitions on the first-order predicate language extended with set theory and further predicate logical theories, for example the theory of integers, lists, arrays, and so on.

14.3 Examples

14.3.1 Example 1: String Matching

We want to write a formal specification of the check whether one string matches another string containing the wildcard character '*' exactly once. We proceed in a top-down fashion, writing a specification that uses a string type that we will formally define later. We denote this string type by $\mathcal{S}$ and concatenation of strings by the infix operator $\#$.

Input:

- $P \in \mathcal{S}$ s.t. $\exists! i \in \{0, \dots, \text{len}(P) - 1\} . P[i] = '*'$
- $A \in \mathcal{S}$

Output:

- $\top$, if $\exists P_l, P_r, M \in \mathcal{S} . \begin{array}{l} P = P_l \# "*" \# P_r, \\ A = P_l \# M \# P_r, \end{array}$ and
- $\perp$, otherwise.

Now we define strings as pairs consisting of an array of characters and the length of the string. As defined in Section 2.2.2, in the whole course we assume that arrays have infinitely many elements, and cannot overflow. Hence the string length does *not* denote array length but simply marks the end of the text in an unbounded array.

Now we can define the needed operations on the resulting string type $\mathcal{S}$ as follows:

- $\forall (A, l) \in \mathcal{S} . \text{len}((A, l)) := l$
- $\forall (A, l) \in \mathcal{S} . (A, l)[i] := A[i]$
- $\forall S_1, S_2 \in \mathcal{S} . S_1 \# S_2 := T$ s.t.
 - $\text{len}(T) = \text{len}(S_1) + \text{len}(S_2)$, and
 - $\forall i \in \{0, \dots, \text{len}(T) - 1\} . T[i] = \begin{cases} S_1[i], & \text{if } i < \text{len}(S_1), \\ S_2[i - \text{len}(S_1)], & \text{otherwise.} \end{cases}$

In addition, we use the usual notation with quotation marks to denote string constants (e.g., "This is a string").

14.3.2 Example 2: String Search

In this example, we will re-use the string type that we already introduced in Section 14.3.1. Consider the following specification of string search:

- Input: strings S, S'
- Output: $\begin{cases} 1, & \text{if } \exists p \in \mathcal{N} . \text{substr}(S, p, S') \text{ and} \\ 0, & \text{otherwise.} \end{cases}$

where *substr* is a predicate that is defined as follows

$$\begin{aligned} \forall S \in \mathcal{S}, p \in \mathcal{N}, S' \in \mathcal{S} . \text{substr}(S, p, S') : \Leftrightarrow \\ & 0 \leq p < \text{len}(S) \\ & p + \text{len}(S') \leq \text{len}(S) \\ & \forall i \in \{0, \dots, \text{len}(S') - 1\} . S'[i] = S[p + i] \end{aligned}$$

Let us now extend this specification with a certificate. The case with the answer 1 is easy: We simply return, in addition to the result 1, the position p for which *substr*(S, p, S').

For the case with the answer 0, one can either try to come up with an intuitive idea, and then formalize this idea, or one can formally derive the certificate from the specification. Here, we follow the latter approach, and give an intuitive explanation of the result at the very end. So let us assume strings S and S' for which we should return 0, according to the specification. Hence $\neg \exists p \in \mathcal{N} . \text{substr}(S, p, S')$. So, according to the definition of the *substr* function,

$$\begin{aligned} & 0 \leq p < \text{len}(S), \\ \neg \exists p \in \mathcal{N} . & p + \text{len}(S') \leq \text{len}(S), \\ & \forall i \in \{0, \dots, \text{len}(S') - 1\} . S'[i] = S[p + i] \end{aligned}$$

This is equivalent to

$$\begin{aligned} & [0 \leq p < \text{len}(S) \wedge p + \text{len}(S') \leq \text{len}(S)] \\ \forall p \in \mathcal{N} . & \Rightarrow \\ & \neg \forall i \in \{0, \dots, \text{len}(S') - 1\} . S'[i] = S[p + i] \end{aligned}$$

that is,

$$\begin{aligned} & [0 \leq p < \text{len}(S) \wedge p + \text{len}(S') \leq \text{len}(S)] \\ \forall p \in \mathcal{N} . & \Rightarrow \\ & \exists i \in \{0, \dots, \text{len}(S') - 1\} . S'[i] \neq S[p + i] \end{aligned}$$

So, in this case, we return an array containing for every p with $0 \leq p < \text{len}(S)$ and $p + \text{len}(S') \leq \text{len}(S)$ an i with $S'[i] \neq S[p + i]$. The intuitive interpretation of this is, that the certificate contains for every potential position p of S' in S , the position i in string S' that shows that S' cannot occur in S at position p because S' and S differ at respective positions i and $p + i$.

14.3.3 Example 3

We will develop a formal specification of an algorithm that for a given text, given as an array of characters, extracts the set of words contained in the text. Based on this, the specification may look as follows:

Input: $a \in \mathcal{S}$

Output: $\{w \in \mathcal{S} \mid \text{word}(a, w)\}$

Here, the predicate *word* holds iff the text stored in the array a contains the word w at any position. To formalize this predicate, we realize that this is the case, if

- there is a position i such that w is a substring of a at position i , and
- at that position, a contains a word

So, using the predicate *substr* defined in the previous section, and denoting the second property by *wbound* we define the predicate *word* as follows:

$$\forall a \in \mathcal{S}, w \in \mathcal{S} . \text{word}(a, w) :\Leftrightarrow \exists i \in \mathcal{N} . \text{substr}(a, i, w) \wedge \text{wbound}(a, i, \text{len}(w))$$

Now we formalize the predicate *wbound*:

$$\begin{aligned} \forall a \in \mathcal{S}, p \in \mathcal{N}, l \in \mathcal{N} . \text{wbound}(a, p, l) :\Leftrightarrow & \\ & 0 \leq p \wedge p + l \leq \text{len}(a) \wedge \\ & [p = 0 \vee a[p - 1] = ' '] \wedge \\ & [p + l = \text{len}(a) \vee a[p + l] = ' '] \wedge \\ & \neg \exists k \in \{p, \dots, p + l - 1\} . a[k] = ' ' \end{aligned}$$

14.4 Shortcuts

Consider the following specification:

- Input: set of integers S
- Output: k s.t. $k \in S \wedge \text{even}(k)$, where $\forall i . \text{even}(i) :\Leftrightarrow \exists j \in \mathbb{Z} . 2j = i$

This specification does not fulfill the requirement that for every input exists at least one output. We might modify the output specification in such a way that we return -1 in such cases. In terms of a logical formula this means that the output will be a k such that

$$\begin{aligned} [\exists i . i \in S \wedge \text{even}(i)] &\Rightarrow k \in S \wedge \text{even}(k) \\ \wedge \\ \neg[\exists i . i \in S \wedge \text{even}(i)] &\Rightarrow k = -1 \end{aligned}$$

with the definition of the predicate *even* staying unchanged. Note the scope of the existential quantifier—if we would rename the existentially quantified variable i to k as follows

$$\begin{aligned} [\exists k . k \in S \wedge \text{even}(k)] &\Rightarrow k \in S \wedge \text{even}(k) \\ \wedge \\ \neg[\exists k . k \in S \wedge \text{even}(k)] &\Rightarrow k = -1 \end{aligned}$$

then the k outside of the brackets [...] is a *different* variable than the k inside of the brackets! In other words, such a renaming leaves the meaning of the formula unchanged.

Of course, in practice, we want a more comfortable way of writing such things. For example, we may allow the construction

if P then F else G

as a short-cut for the logical formula

$$[P \Rightarrow F] \wedge [\neg P \Rightarrow G]$$

which results in returning a k such that

$$\begin{array}{l} \mathbf{if} \ [\exists i . i \in S \wedge \text{even}(i)] \ \mathbf{then} \ k \in S \wedge \text{even}(k) \\ \qquad \mathbf{else} \ k = -1 \end{array}$$

We might even allow the short-cut

$$P, \text{ if such a } k \text{ exists, and } G, \text{ otherwise}$$

for

$$\begin{array}{c} [\exists k . P] \Rightarrow P \\ \wedge \\ \neg[\exists k . P] \Rightarrow G \end{array}$$

which results in the output being a k such that

$$k \in S \wedge \text{even}(k), \text{ if such a } k \text{ exists, and } k = -1, \text{ otherwise.}$$

A completely different approach would be to write the cases as a disjunction instead of a conjunction:

$$\begin{array}{c} [\exists i . i \in S \wedge \text{even}(i) \wedge k = i] \\ \vee \\ \neg[\exists i . i \in S \wedge \text{even}(i) \wedge k = -1] \end{array}$$

Chapter 15

Correctness of Programs Without Control Structures

15.1 Program Correctness and Assertions

Consider the following I/O specification:

- Input:
 - array a , natural number n s.t. $\forall i \in \{0, \dots, n-2\} . a[i] \leq a[i+1]$,
 - integer k
- Output: $p \in \{0, \dots, n-1\}$ s.t. $\begin{cases} a[p] = k, & \text{if such a } p \text{ exists, and} \\ -1, & \text{otherwise.} \end{cases}$

We will now write such specifications as assertions into programs by writing the input requirement as an assertion that forms the first line of the program, and the output requirement as an assertion before the final return statement:

```
assume  $\forall i \in \{0, \dots, n-2\} . a[i] \leq a[i+1]$ 
...
@  $p \in \{0, \dots, n-1\}$  s.t.  $a[p] = k$ , if such a  $p$  exists,  $-1$ , otherwise
return  $p$ 
```

Here, we use two different types of assertions:

- **assume**: to be ensured by the user, from the outside
- **@**: to be ensured by the given program

When writing the output requirement as an assertion, one has to be careful that the program before must not change any input variable that appears in that assertion:

```
assume  $\forall i \in \{0, \dots, n-2\} . a[i] \leq a[i+1]$ 
...
@  $p \in \{0, \dots, n-1\}$  s.t.  $a[p] = k$ , if such a  $p$  exists,  $-1$ , otherwise
return  $p$  // must not change  $a, k$ 
```

To avoid this problem, one can save the value of the changed input variables into an auxiliary variable, and rename it accordingly in the final assertion:

```

assume  $\forall i \in \{0, \dots, n-2\} . a[i] \leq a[i+1]$ 
 $a^* \leftarrow a; k^* \leftarrow k$                                 // auxiliary variables  $a^*, k^*$ 
...                                                         // may change  $a, k$ 
@  $p \in \{0, \dots, n-1\}$  s.t.  $a^*[p] = k$ , if such a  $p$  exists,  $-1$ , otherwise
return  $p$ 

```

Very often we have additional requirements that should hold during the execution of the program. For example, we want to avoid division by zero, or array overflows. We can also write such requirements as assertions into programs:

```

assume  $I$                                                     // input spec
@  $i \neq 0$ 
 $q \leftarrow 1000/i$ 
...
@  $l < 100$ 
 $m \leftarrow a[l]$                                            //  $a$  has length 100
...
@  $O$                                                         // output spec
return

```

We already mentioned that we assume that the user of a program ensures that all **assume**-assertions are satisfied by all program executions. The following definition denotes such executions:

Definition 59 *A program execution is regular iff it satisfies all **assume**-assertions.*

Based on this, we can define a program to be correct if all its executions satisfies the **@**-assertions:

Definition 60 *A program is (partially) correct iff every regular program execution from the first program line to a program line with an **@**-assertion, satisfies this **@**-assertion.*

Note that these definitions are not formal, since we did not formally define what we mean by the notion of program execution. In the following, we will use a tiny imperative programming language, that is so simple that the intuition that everybody has about the behavior of such a programming language is clear. So we can use the notion of program execution based on this intuition.

Also note that this is another, different definition of program correctness than the one we met earlier (Definition 58). The earlier definition related the program's behavior to its I/O specification, whereas Definition 60 assumes that the I/O specification has been encoded into program assertions, possibly along with further internal assertions such as the ones discussed above.

15.2 Program Correctness: Assignments

Assume the following program:

```

y ← x
z ← x + y
@ z ≥ 0

```

If it is clear, that this program is correct iff the formula

$$\forall x, y, z. [y = x \wedge z = x + y] \Rightarrow z \geq 0$$

holds (i.e., is valid). But actually, since a predicate logical formula is valid if and only iff its universal closure is valid, that is, for a formula ϕ with free variables v

$$\models \phi \text{ iff } \models \forall v. \phi,$$

we do not need to write the universal quantifiers and simply check the formula

$$[y = x \wedge z = x + y] \Rightarrow z \geq 0$$

for validity. This formula is invalid (i.e., not valid), and indeed the program is not correct. To ensure its correctness, we add an assume assertion at its beginning that excludes executions that raise the assertion at the end:

```

assume x ≥ 5
y ← x
z ← x + y
@ z ≥ 0

```

Now we can express the condition for program correctness using the formula

$$[x \geq 5 \wedge y = x \wedge z = x + y] \Rightarrow z \geq 0.$$

This simple translation from programs to logical formulas works, because the assignments to y and z do not change x . But which formula should we use if an assignment changes x in between?

```

assume x ≥ 5
y ← x
x ← -10
z ← x + y
@ z ≥ 0

```

This program clearly is incorrect. However, if we use the analogical translation to a logical formula, we get

$$[x \geq 5 \wedge y = x \wedge x = -10 \wedge z = x + y] \Rightarrow z \geq 0.$$

Here, the sub-formulas $x \geq 5$ and $x = -10$ contradict each other. Since the contradiction occurs on the left-hand side of an implication, the formula holds. Hence, if we would use such a formula, this would show an incorrect program to be correct. Where is the problem?

The following table shows the values of all variables during each step of program execution:

x	y	z
5		
5	5	
-10	5	
-10	5	-5

Here, we can see two *different* values in the column corresponding to the variable x . This results in the contradiction. To avoid this, we introduce two *different* variables names corresponding to the two values:

```

assume  $x \geq 5$ 
 $y \leftarrow x$ 
 $x_1 \leftarrow -10$ 
 $z \leftarrow x_1 + y$ 
@  $z \geq 0$ 

```

Now, every variable corresponds to only *one* value of a variables. This can also be seen in the changed table, where now no column shows more than one value:

x	x_1	y	z
5			
5		5	
5	-10	5	
5	-10	5	-5

The resulting formula is now

$$[x \geq 5 \wedge y = x \wedge x_1 = -10 \wedge z = x_1 + y] \Rightarrow z \geq 0$$

which does not hold. A counter-example is $\{x \mapsto 5, y \mapsto 5, x_1 \mapsto -10, z \mapsto -5\}$ that satisfies the left-hand side, but not the right-hand side of the implication and demonstrates how the assertion in the program can fail.

The general principle we use here, is to use a program transformation that is widely used in compiler construction, *static single assignment form (SSA)*. As soon as a program in the form we have seen above is in SSA, translation to a logical formula is straight-forward.

The same principle also applies to programs that contain several assignments to the same variable. Consider the example

```

 $x \leftarrow 1$ 
 $x \leftarrow x + 1$ 
@  $x \geq 10$ 

```

with SSA

```

 $x \leftarrow 1$ 
 $x_1 \leftarrow x + 1$ 
@  $x_1 \geq 10$ 

```

which results in the formula

$$[x = 1 \wedge x_1 = x + 1] \Rightarrow x_1 \geq 10.$$

Actually, programmers do similar things in their head when reasoning about programs, using intuitive names such as

- “the value of variable x in the first line”
- “the value of variable x from line two on”
- ...

However, it is clear that such long names are awkward to write down. This is one of the reasons why we use indices, instead.

15.3 Program Correctness: Array Assignments

Up to now, we have only considered assignments to variables. But now consider the following program:

```
a[i] ← 7
a[i] ← 5
...
```

This program does not assign values to variables, but to *entries* of an array. It is unclear, how the SSA of such a program can be formed. For doing so, recall from Chapter 2 that the logical theory of arrays provides the following function symbols:

- $.[\cdot] : \mathcal{A}[\mathcal{T}] \times \mathcal{N} \rightarrow \mathcal{T}$
- $\text{write} : \mathcal{A}[\mathcal{T}] \times \mathcal{N} \times \mathcal{T} \rightarrow \mathcal{A}[\mathcal{T}]$

The intuition behind this is that writing into an array results in a *new* array. We can reflect this in the program by writing it as follows

```
a ← write(a, i, 7)
a ← write(a, i, 5)
...
```

And now, forming the SSA is trivial:

```
a1 ← write(a, i, 7)
a2 ← write(a1, i, 5)
...
```

15.4 Program Correctness: User Input

Let us now extend our programming language with a command that asks the user for input. For example, consider this program:

```
assume x ≥ 5
input x
z ← x + y2
@ z ≥ 0
```

If we would ignore the input command, the resulting formula would be

$$[x \geq 5 \wedge z = x + y^2] \Rightarrow z \geq 0.$$

This does not make sense, since the **input** statement changes the value of the variables x , and hence the value of x in the assignment is *not* the same as its value after the **assume** statement. We can reflect this in SSA by introducing new variables also for **input** statements. Hence the resulting SSA is

```
assume x ≥ 5
input x1
z ← x1 + y2
@ z ≥ 0
```

which can be meaningfully translated to the formula

$$[x \geq 5 \wedge z = x_1 + y^2] \Rightarrow z \geq 0.$$

15.5 Correctness of Programs without Control Structures

We will now introduce a general method for checking correctness of programs without control structures. The examples we have considered above, all used programs of a certain form:

Definition 61 *A basic path is a program of the form*

$$\begin{array}{l} c_1 \\ \dots \\ c_n \\ @ \alpha \end{array}$$

where

- $c_1, \dots, c_n$ neither contains control structures nor $@$, and
- α is a logical formula.

Here we have allowed assignments, **assume**, and **input** statements. Static single assignment form now renames a variable to a new one, whenever it is assigned a new value (e.g., **input**). For example, the single static assignment form of the program

```
assume  $x \geq 0$ 
input  $x$ 
assume  $2x - 1 \geq 3$ 
 $x \leftarrow x - 2$ 
assume  $x \geq 0$ 
input  $x$ 
assume  $\neg 2x - 1 \geq 3$ 
 $x \leftarrow x - 1$ 
@  $x \neq 0$ 
```

is

```
assume  $x \geq 0$ 
input  $x_1$ 
assume  $2x_1 - 1 \geq 3$ 
 $x_2 \leftarrow x_1 - 2$ 
assume  $x_2 \geq 0$ 
input  $x_3$ 
assume  $\neg 2x_3 - 1 \geq 3$ 
 $x_4 \leftarrow x_3 - 1$ 
@  $x_4 \neq 0$ 
```

We will denote the single static assignment form of a basic path P by $SSA(P)$. Formally we can define this as follows:

Definition 62

$$SSA_V(@ \alpha) := @ \alpha$$

$$SSA_V(\phi; P) := \phi; SSA_V(P)$$

for all commands ϕ that do not change variable values

$$SSA_V(\mathbf{input} \ v; P) := \mathbf{input} \ v_{new}; SSA_{V \cup \{v_{new}\}}(P[v \leftarrow v_{new}])$$

where v_{new} is a variable s.t. $v_{new} \notin V$.

$$SSA_V(v \leftarrow t; P) := v_{new} \leftarrow t; SSA_{V \cup \{v_{new}\}}(P[v \leftarrow v_{new}])$$

where v_{new} is a variable s.t. $v_{new} \notin V$.

$$SSA(P) := SSA_{vars(P)}(P)$$

In this definition, the subscript V stores the set of variables that already have been assigned values, which means that the names of those variables should be avoided when introducing new variable names. The individual steps of applying this definition to the example above are the following:

$$\begin{aligned}
& SSA \left(\begin{array}{l} \mathbf{assume} \ x \geq 0 \\ \mathbf{input} \ x \\ \mathbf{assume} \ 2x - 1 \geq 3 \\ x \leftarrow x - 2 \\ \mathbf{assume} \ x \geq 0 \\ \mathbf{input} \ x \\ \mathbf{assume} \ \neg 2x - 1 \geq 3 \\ x \leftarrow x - 1 \\ @ \ x \neq 0 \end{array} \right) = SSA_{\{x\}} \left(\begin{array}{l} \mathbf{assume} \ x \geq 0 \\ \mathbf{input} \ x \\ \mathbf{assume} \ 2x - 1 \geq 3 \\ x \leftarrow x - 2 \\ \mathbf{assume} \ x \geq 0 \\ \mathbf{input} \ x \\ \mathbf{assume} \ \neg 2x - 1 \geq 3 \\ x \leftarrow x - 1 \\ @ \ x \neq 0 \end{array} \right) = \\
& \left(\mathbf{assume} \ x \geq 0 \right); SSA_{\{x\}} \left(\begin{array}{l} \mathbf{input} \ x \\ \mathbf{assume} \ 2x - 1 \geq 3 \\ x \leftarrow x - 2 \\ \mathbf{assume} \ x \geq 0 \\ \mathbf{input} \ x \\ \mathbf{assume} \ \neg 2x - 1 \geq 3 \\ x \leftarrow x - 1 \\ @ \ x \neq 0 \end{array} \right) = \\
& \left(\mathbf{assume} \ x \geq 0 \right); \mathbf{input} \ x_1; SSA_{\{x, x_1\}} \left(\begin{array}{l} \mathbf{assume} \ 2x_1 - 1 \geq 3 \\ x_1 \leftarrow x_1 - 2 \\ \mathbf{assume} \ x_1 \geq 0 \\ \mathbf{input} \ x_1 \\ \mathbf{assume} \ \neg 2x_1 - 1 \geq 3 \\ x_1 \leftarrow x_1 - 1 \\ @ \ x_1 \neq 0 \end{array} \right) = \\
& \left(\mathbf{assume} \ x \geq 0 \right); \mathbf{input} \ x_1; \mathbf{assume} \ 2x_1 - 1 \geq 3; SSA_{\{x, x_1\}} \left(\begin{array}{l} x_1 \leftarrow x_1 - 2 \\ \mathbf{assume} \ x_1 \geq 0 \\ \mathbf{input} \ x_1 \\ \mathbf{assume} \ \neg 2x_1 - 1 \geq 3 \\ x_1 \leftarrow x_1 - 1 \\ @ \ x_1 \neq 0 \end{array} \right) =
\end{aligned}$$

$$\begin{aligned}
& \left(\begin{array}{l} \text{assume } x \geq 0 \\ \text{input } x_1 \\ \text{assume } 2x_1 - 1 \geq 3 \\ x_2 \leftarrow x_1 - 2 \end{array} \right) ; SSA_{\{x, x_1, x_2\}} \left(\begin{array}{l} \text{assume } x_2 \geq 0 \\ \text{input } x_2 \\ \text{assume } \neg 2x_2 - 1 \geq 3 \\ x_2 \leftarrow x_2 - 1 \\ @ x_2 \neq 0 \end{array} \right) = \\
& \left(\begin{array}{l} \text{assume } x \geq 0 \\ \text{input } x_1 \\ \text{assume } 2x_1 - 1 \geq 3 \\ x_2 \leftarrow x_1 - 2 \\ \text{assume } x_2 \geq 0 \end{array} \right) ; SSA_{\{x, x_1, x_2\}} \left(\begin{array}{l} \text{input } x_2 \\ \text{assume } \neg 2x_2 - 1 \geq 3 \\ x_2 \leftarrow x_2 - 1 \\ @ x_2 \neq 0 \end{array} \right) = \\
& \left(\begin{array}{l} \text{assume } x \geq 0 \\ \text{input } x_1 \\ \text{assume } 2x_1 - 1 \geq 3 \\ x_2 \leftarrow x_1 - 2 \\ \text{assume } x_2 \geq 0 \\ \text{input } x_3 \end{array} \right) ; SSA_{\{x, x_1, x_2, x_3\}} \left(\begin{array}{l} \text{assume } \neg 2x_3 - 1 \geq 3 \\ x_3 \leftarrow x_3 - 1 \\ @ x_3 \neq 0 \end{array} \right) = \\
& \left(\begin{array}{l} \text{assume } x \geq 0 \\ \text{input } x_1 \\ \text{assume } 2x_1 - 1 \geq 3 \\ x_2 \leftarrow x_1 - 2 \\ \text{assume } x_2 \geq 0 \\ \text{input } x_3 \\ \text{assume } \neg 2x_3 - 1 \geq 3 \end{array} \right) ; SSA_{\{x, x_1, x_2, x_3\}} \left(\begin{array}{l} x_3 \leftarrow x_3 - 1 \\ @ x_3 \neq 0 \end{array} \right) = \\
& \left(\begin{array}{l} \text{assume } x \geq 0 \\ \text{input } x_1 \\ \text{assume } 2x_1 - 1 \geq 3 \\ x_2 \leftarrow x_1 - 2 \\ \text{assume } x_2 \geq 0 \\ \text{input } x_3 \\ \text{assume } \neg 2x_3 - 1 \geq 3 \\ x_4 \leftarrow x_3 - 1 \end{array} \right) ; SSA_{\{x, x_1, x_2, x_4\}} (@ x_4 \neq 0) = \left(\begin{array}{l} \text{assume } x \geq 0 \\ \text{input } x_1 \\ \text{assume } 2x_1 - 1 \geq 3 \\ x_2 \leftarrow x_1 - 2 \\ \text{assume } x_2 \geq 0 \\ \text{input } x_3 \\ \text{assume } \neg 2x_3 - 1 \geq 3 \\ x_4 \leftarrow x_3 - 1 \\ @ x_4 \neq 0 \end{array} \right)
\end{aligned}$$

Now we can extract a formula in first-order predicate language from the above program in static single assignment form:

$$x_1 \geq 0 \wedge 2x_2 - 1 \geq 3 \wedge x_3 = x_2 - 2 \wedge x_3 \geq 0 \wedge 2x_4 - 1 < 3$$

Formally, this means that we can translate individual program statements to logical formulas as follows:

- $F(\text{assume } \phi) := \phi$
- $F(\text{input } v) := \top$
- $F(v \leftarrow t) := v = t$

We can extend this to basic paths as follows:

$$F(c_1; \dots; c_n; @ \alpha) := \left[\bigwedge_{i \in \{1, \dots, n\}, F(c_i) \neq \top} F(c_i) \right] \Rightarrow \alpha$$

The condition $F(c_i) \neq \top$ is not strictly necessary. It just leaves out the tautologies created by the translation of the input statement.

Now we can put everything together and define the *verification condition* of a basic path as

$$VC(P) := F(SSA(P))$$

Then for every programming language that executes programs using the intuition we have used here, it should be true that a basic path P is *correct* (as a program) if and only if $\models VC(P)$.

15.6 Programs with Several Assertions @

The program

```
x ← 2
@ x ≥ 2
x ← 2x
@ x ≥ 4
```

is correct because both

```
x ← 2
@ x ≥ 2
```

and

```
assume x ≥ 2
x ← 2x
@ x ≥ 4
```

are correct. *But*

```
x ← 2
@ 0 = 0
x ← 2x
@ x ≥ 4
```

is correct and

```
assume 0 = 0
x ← 2x
@ x ≥ 4
```

is not. In other words, when decomposing a program with several @ assertions into several basic paths, correctness of the individual basic paths implies correctness of the original program, but not the other way round. In the example above, the assertion $0 = 0$ does not express enough information about the variable x to allow us to conclude correctness of the final assertion $x \geq 4$.

In general, a program of the form

...

@ α_1
 $\dots$
 @ α_2
 $\dots$
 $\dots$
 @ α_n

is correct if the following n basic paths are correct:

$\dots$ **assume** α_1 **assume** α_{n-1}
 @ α_1 $\dots$ @ α_n
 @ α_2 @ α_n

But since we only have an “if” here, not an “if and only if”, it might be necessary to strengthen the intermediate assertions $\alpha_1, \dots, \alpha_{n-1}$ to make the resulting basic paths correct. To return to the example from above, it suffices to add information to the intermediate assertion that x is greater or equal 2 which will allow us to conclude that the final assertion holds:

$x \leftarrow 2$
 @ $0 = 0 \wedge x \geq 2$
 $x \leftarrow 2x$
 @ $x \geq 4$

15.7 Proving Verification Conditions

We will first show, how such verification conditions can be proved manually. Since we now already have some experience with proving, we will proceed more quickly, summarizing several proof steps into one. The verification condition of the basic path

assume $x \leq -2$
 $y \leftarrow x$
 $x \leftarrow -2y + 1$
 $z \leftarrow x + y$
 @ $z \geq 0$

is

$$[x \leq -2 \wedge y = x \wedge x_1 = -2y + 1 \wedge z = x_1 + y] \Rightarrow z \geq 0.$$

To prove the formula, we assume $x \leq -2$, $y = x$, $x_1 = -2y + 1$, $z = x_1 + y$ and prove $z \geq 0$. Using the assumption $z = x_1 + y$, this means to prove $x_1 + y \geq 0$. Using the assumption $x_1 = -2y + 1$, this means to prove $-2y + 1 + y \geq 0$ which is $-y + 1 \geq 0$. Using the assumption $y = x$, this means to prove $-x + 1 \geq 0$ that is $x \leq 1$. This certainly holds under the assumption $x \leq -2$.

15.8 Verification Conditions: Alternative Form

Consider the program

assume $x \leq 5$

$y \leftarrow x$

$x \leftarrow -2y + 1$

$z \leftarrow x + y$

@ $z \geq 0$

with the verification condition

$$[x \leq 5 \wedge y = x \wedge x_1 = -2y + 1 \wedge z = x_1 + y] \Rightarrow z \geq 0.$$

The program is the same as the one from above except for a changed assumption at the first program line. The new assumption is weaker than the old one, which allows more program runs. As a consequence, the program has a bug. For example, if we start the program with x being 4, the execution of its individual lines result in the following assignments:

$x \leftarrow 4$

$y \leftarrow 4$

$x \leftarrow -7$

$z \leftarrow -3$

These assignments result in the counter-example

$$\{x \mapsto 4, y \mapsto 4, x \mapsto -7, z \mapsto -3\}$$

to the verification condition.

We can also replace the assertion in the last program line with its negation

assume $x \leq 5$

$y \leftarrow x$

$x \leftarrow -2y + 1$

$z \leftarrow x + y$

@ $z < 0$

which expresses an execution of the original program that results in a bug. In terms of logical formulas we can express such an execution by the formula

$$x \leq -5 \wedge y = x \wedge x_1 = -2y + 1 \wedge z = x_1 + y \wedge z < 0$$

which we call an *error condition*¹.

Now let us compare verification conditions and error conditions. The verification condition

$$[x \leq 5 \wedge y = x \wedge x_1 = -2y + 1 \wedge z = x_1 + y] \Rightarrow z \geq 0$$

is equivalent to

$$\neg[x \leq 5 \wedge y = x \wedge x_1 = -2y + 1 \wedge z = x_1 + y] \vee \neg z < 0,$$

which is equivalent to

$$\neg[x \leq 5 \wedge y = x \wedge x_1 = -2y + 1 \wedge z = x_1 + y \wedge z < 0].$$

¹This term is not commonly used. We use it here in order to be able to concisely express certain things.

So we see that verification conditions are equivalent to the negation of error conditions. The intuition is that a verification condition expresses that every execution satisfies all assertions, and the negation of the error condition expresses that no execution violates an assertion. Later, for example in Section 20, this will turn out to be very useful.

The fact that verification conditions are equivalent to negations of error conditions might also remind the reader of a basic fact from logic: a formula is invalid iff its negation is satisfiable. In our case, the verification condition was invalid and hence its negation—the error condition—satisfiable.

15.9 Summary

For proving correctness of a program without control structure we

1. translate the program to SSA, and
2. check the corresponding verification condition.

Of course, in practice one usually does not do these translations explicitly. But, training how to do this, also helps to be able to think more reliably about the correctness of programs. Moreover, many tools for program analysis are based on the fact that programs can be transformed to logical formulas in the way discussed in this chapter.

In the literature there are many different methods for constructing verification conditions, and not all of them use the static single assignment form. The approach that is closest to the one presented in Section 12.2 of a book by Kroening et. al. [26].

Chapter 16

Correctness of Programs with Control Structures

16.1 if-then-else

We will start with checking correctness of if-then-else statements. Consider the following example program:

```
if  $x > y$  then
   $r \leftarrow x$ 
else
   $r \leftarrow y$ 
@  $r = \max\{x, y\}$ 
```

This program executes either its **if** branch or its **else** branch. The **if** branch is executed under the condition $x > y$, and the **else** branch under its negation $\neg x > y$. This results in the two following basic paths:

assume $x > y$	assume $\neg x > y$
$r \leftarrow x$	$r \leftarrow y$
@ $r = \max\{x, y\}$	@ $r = \max\{x, y\}$

The corresponding verification conditions are:

$$[x > y \wedge r = x] \Rightarrow r = \max\{x, y\}$$

and

$$[\neg x > y \wedge r = y] \Rightarrow r = \max\{x, y\}.$$

In general, we can linearize control structures by extracting basic paths:

Definition 63 A *basic path* is a basic path of a program P iff the basic path

- corresponds to a sequence of program lines of P that can be executed in this order, and
- either begins at the beginning of P , or at an @-assertion that is replaced by the corresponding **assume**-assertion, and

- instead of control structures contains **assume** ϕ , where ϕ is the condition under which program execution follows this path.

Then we can check the correctness of a program containing control structure by checking the correctness of each of its basic paths:

Property 9 *If for all basic paths π of a program P , $\models VC(\pi)$, then the program is correct.*

We will use the notation $VC(P)$ to denote this set of basic paths of a program P .

In the case of an if-then-else statement of the form

```

if  $P$  then
    ...
else
    ...

```

when linearizing the statement we replace the control structure by the following statements:

- path following **if**: **assume** P
- path following **else**: **assume** $\neg P$

Now consider an **if-then-else** cascade of the following form:

```

if  $x_1 > y_1$  then
     $r_1 \leftarrow x_1$ 
else
     $r_1 \leftarrow y_1$ 
if  $x_2 > y_2$  then
     $r_2 \leftarrow x_2$ 
else
     $r_2 \leftarrow y_2$ 
if  $r_1 > r_2$  then
     $r \leftarrow r_1$ 
else
     $r \leftarrow r_2$ 
@  $r = \max\{x_1, x_2, y_1, y_2\}$ 

```

This program has 8 basic paths with corresponding verification conditions. For example, one of the eight paths, the one, taking the **if** branch of every **if-then-else**, is

```

assume  $x_1 > y_1$ 
 $r_1 = x_1$ 
assume  $x_2 > y_2$ 
 $r_2 = x_2$ 
assume  $r_1 > r_2$ 
 $r = r_1$ 
@  $r = \max\{x_1, x_2, y_1, y_2\}$ 

```

By adding further assertions to the **if-then-else** cascade, we can reduce the number of basic paths and verification conditions to $2 \times 3 = 6$:

```

if  $x_1 > y_1$  then
     $r_1 \leftarrow x_1$ 
else
     $r_1 \leftarrow y_1$ 
@  $r_1 = \max\{x_1, y_1\}$ 
if  $x_2 > y_2$  then
     $r_2 \leftarrow x_2$ 
else
     $r_2 \leftarrow y_2$ 
@  $r_1 = \max\{x_1, y_1\}, r_2 = \max\{x_2, y_2\}$ 
if  $r_1 > r_2$  then
     $r \leftarrow r_1$ 
else
     $r \leftarrow r_2$ 
@  $r = \max\{x_1, x_2, y_1, y_2\}$ 

```

Moreover, the basic paths are now shorter and have simpler verification conditions. One of those basic paths, the one leading from the first assertion to the second one, taking the **if** branch of the **if-then-else** in between, is

```

assume  $r_1 = \max\{x_1, y_1\}$ 
assume  $x_2 > y_2$ 
 $r_2 \leftarrow x_2$ 
@  $r_1 = \max\{x_1, y_1\}, r_2 = \max\{x_2, y_2\}$ 

```

Note the **assume** on the first line of this basic path that stems from an @ assertion in the program that, according to Definition 63, has been replaced by an **assume**.

In general, if we have an if-then-else cascade of the form

```

@
if  $P_1$  then
    ...
else
    ...
...
if  $P_n$  then
    ...
else
    ...
@

```

then this results in 2^n basic paths. This paths share parts and hence we can introduce intermediate assertions that reduce the number of basic paths. We arrive at

```

@
if  $P_1$  then
    ...
else
    ...
@

```

```

...
@
if  $P_n$  then
    ...
else
    ...
@

```

which has $2n$ basic paths.

16.2 Loops

We will now discuss, how to extract basic paths from programs containing loops. For this, we will again use Definition 63, and add details on how the introduced **assume** assertions should look like. Consider the program

```

 $i \leftarrow 0$ 
while  $a[i] \neq 0$  do
     $i \leftarrow i + 1$ 
@  $a[i] = 0 \wedge \forall k \in \{0, \dots, i - 1\} . a[k] \neq 0$ 

```

This program has the following basic paths:

```

 $s \leftarrow 0; i \leftarrow 0$ 
assume  $a[i] \neq 0$ 
 $i \leftarrow i + 1$ 
assume  $a[i] \neq 0$ 
 $i \leftarrow i + 1$ 
assume  $a[i] \neq 0$ 
 $i \leftarrow i + 1$ 
...
assume  $a[i] = 0$ 
@  $a[i] = 0 \wedge \forall k \in \{0, \dots, i - 1\} . a[k] \neq 0$ 

```

The dots indicate the problem: Such a loop has infinitely many basic paths! However, those basic paths have parts that repeat many times. For handling this situation, we will use a similar strategy as above: We introduce intermediate assertions:

```

 $i \leftarrow 0$ 
assume  $a[i] \neq 0$ 
@
 $i \leftarrow i + 1$ 
assume  $a[i] \neq 0$ 
@
 $i \leftarrow i + 1$ 
assume  $a[i] \neq 0$ 
@
 $i \leftarrow i + 1$ 
...
assume  $a[i] = 0$ 
@  $a[i] = 0 \wedge \forall k \in \{0, \dots, i - 1\} . a[k] \neq 0$ 

```


which corresponds to inserting one assertion into the loop:

```

i ← 0
while a[i] ≠ 0 do
  @
  i ← i + 1
  @ a[i] = 0 ∧ ∀k ∈ {0, ..., i - 1} . a[k] ≠ 0

```

Observe that the result contains infinitely many identical copies of the same basic path. When listing only one of those copies, only four basic paths are left:

```

i ← 0
assume a[i] = 0
  @ a[i] = 0 ∧ ∀k ∈ {0, ..., i - 1} . a[k] ≠ 0

i ← 0
assume a[i] ≠ 0
  @ ∀k ∈ {0, ..., i} . a[k] ≠ 0

assume ∀k ∈ {0, ..., i} . a[k] ≠ 0
i ← i + 1
assume a[i] ≠ 0
  @ ∀k ∈ {0, ..., i} . a[k] ≠ 0

assume ∀k ∈ {0, ..., i} . a[k] ≠ 0
i ← i + 1
assume a[i] = 0
  @ a[i] = 0 ∧ ∀k ∈ {0, ..., i - 1} . a[k] ≠ 0

```

Summarizing, we can linearize a while loop, if it contains at least one assertion @. In this case we have a loop of the form

```

while P do
  ...
  @
  ...

```

In the resulting basic paths, every basic path entering or staying in loop will have the assertion **assume** *P*, and every basic path leaving or jumping over loop the assertion **assume** ¬*P*. If the loop contains one assertion and no further control structures, we will have 4 basic paths as a result:

- The basic path entering the loop from the beginning of the program or the assertion directly before the loop, and ending in the assertion in the loop.
- The basic path skipping over the loop.
- The basic path starting at the assertion in the loop and taking one cycle of the loop, ending at the same assertion again.
- The basic path starting at the assertion in the loop, leaving the loop, and ending in the first assertion after the loop.

For loops can be linearized in a similar way. For example, the program

```

assume  $k \geq 5$ 
for  $i \leftarrow 1$  to 10 do
  @  $k \geq 3$ 
   $k \leftarrow k + i$ 
@  $k \geq 0$ 

```

has the following basic paths:

```

assume  $k \geq 5$ 
assume  $i = 1$ 
@  $k \geq 3$ 

assume  $1 \leq i \leq 10 \wedge k \geq 3$ 
 $k \leftarrow k + i$ 
 $i \leftarrow i + 1$ 
assume  $i \leq 10$ 
@  $k \geq 3$ 

assume  $k \geq 5$ 
assume  $\neg 1 \leq 10$ 
@  $k \geq 0$ 

assume  $1 \leq i \leq 10 \wedge k \geq 3$ 
 $k \leftarrow k + i$ 
 $i \leftarrow i + 1$ 
assume  $\neg i \leq 10$ 
@  $k \geq 0$ 

```

The verification condition of the second basic path can be calculated as follows:

```

assume  $1 \leq i \leq 10 \wedge k \geq 3$ 
 $k \leftarrow k + i$ 
 $i \leftarrow i + 1$ 
assume  $i \leq 10$ 
@  $k \geq 3$ 

```

In SSA this is

```

assume  $1 \leq i \leq 10 \wedge k \geq 3$ 
 $k_1 \leftarrow k + i$ 
 $i_1 \leftarrow i + 1$ 
assume  $i_1 \leq 10$ 
@  $k_1 \geq 3$ 

```

And the resulting verification condition is

$$[1 \leq i \wedge i \leq 10 \wedge k \geq 3 \wedge k_1 = k + i \wedge i_1 = i + 1 \wedge i_1 \leq 10] \Rightarrow k_1 \geq 3$$

In general, for linearizing a for loop, it

- should contain at least one assertion @, and it

- should not modify i .

If this holds, we can linearize a for loop of the form

```

for  $i \leftarrow l$  to  $u$  do
  ...
  @
  ...

```

by replacing the **for** statement by the following assumes:

- path jumping over the loop: **assume** $\neg l \leq u$
- path into the loop from outside: **assume** $i = l \wedge l \leq u$
- path staying in the loop: $i \leftarrow i + 1$; **assume** $i \leq u$
- path leaving the loop, depending on the programming language
 - $i \leftarrow i + 1$; **assume** $\neg i \leq u$ (e.g., C)
 - in some programming languages: **assume** $i = u$ (e.g., Pascal)

Moreover we can assume $l \leq i \leq u$ at the beginning of any path starting inside of the loop. Note here, that such assumptions are optional since they add additional knowledge. If an assumption is not need to prove correctness of the given path, it simply can be dropped.

Summarizing, a program with loops may have infinitely many basic paths and corresponding verification conditions. We can ensure a finite number of basic paths, by requiring that every loop contain at least one assertion. This assertion is called a *loop invariant*. Loop invariants can be at any position in the loop. In the case of nested loops, every loop needs an assertion, not just the innermost one!

So loop invariants ensure that a program has a finite number of basic paths. A finite number of basic paths in turn ensures that the program has finitely many verification conditions. And this allows us to check all of them.

16.3 General Program Linearization

Assume a program of the form

```

if  $C_1$  then
   $P_1 \dots$ 
else
   $P_2 \dots$ 
while  $C_2$  do
  @  $A_1$ 
   $P_3 \dots$ 
  while  $C_3$  do
    @  $A_2$ 
     $P_4 \dots$ 
   $P_5 \dots$ 
 $P_6 \dots$ 
@  $A_3$ 

```

To check correctness of this program, we have to check correctness of the following basic paths:

- **assume** C_1 ; P_1 ; **assume** C_2 ; @ A_1 (entering the outer loop through the if branch)
- **assume** C_1 ; P_1 ; **assume** $\neg C_2$; P_6 ; @ A_3 (entering the first if-branch and then skipping the while loop)
- **assume** $\neg C_1$; P_2 ; **assume** C_2 ; @ A_1 (entering the outer loop through the else branch)
- **assume** $\neg C_1$; P_2 ; **assume** $\neg C_2$; P_6 ; @ A_3 (entering the first else-branch and then skipping the while loop)
- **assume** A_1 ; P_3 ; **assume** C_3 ; @ A_2 (entering the inner loop from the loop invariant of the outer loop)
- **assume** A_1 ; P_3 ; **assume** $\neg C_3$; P_5 ; **assume** C_2 ; @ A_1 (one cycle of the outer loop, skipping the inner loop)
- **assume** A_1 ; P_3 ; **assume** $\neg C_3$; P_5 ; **assume** $\neg C_2$; P_6 ; @ A_3 (leaving the outer loop from its loop invariant, skipping the inner loop)
- **assume** A_2 ; P_4 ; **assume** C_3 ; @ A_2 (one cycle of the inner loop)
- **assume** A_2 ; P_4 ; **assume** $\neg C_3$; P_5 ; **assume** C_2 ; @ A_1 (entering the outer loop from the invariant of the inner loop)
- **assume** A_2 ; P_4 ; **assume** $\neg C_3$; P_5 ; **assume** $\neg C_2$; P_6 ; @ A_3 (leaving both inner and outer loop from the invariant of the inner loop)

In general, the difference between the fact that all assertions hold, and the fact that all verification conditions hold is the following:

- assertions hold: This is a *global* condition. To understand that this is really the case, one has to understand a large part, if not all of the program.
- verification conditions hold: This is a locally checkable condition. To check that this is really the case, it suffices to check the correctness of basic paths, which are usually just some very short program segments.

In practice, it is usually unrealistic to manually check all VCs. Still, it makes sense to write assertions, that make program correctness as locally checkable as possible.

16.4 Verification Conditions vs. Assertion Failures

It is important to understand that for proving correctness of a program by ensuring that all verification conditions hold, it does *not* suffice to ensure that no assertion failure occurs. To see this, consider the following program:

```

i ← 1
while ⊤ do
  @ i ≥ −1
  i ← 2i

```

It is clear, that no assertion failure will happen here. But still, we do *not* have a proof of this fact. Consider the basic path

```

assume i ≥ −1
i ← 2i
assume ⊤
@ i ≥ −1

```

which has the verification condition

$$[i \geq -1 \wedge i_1 = 2i] \Rightarrow i_1 \geq -1$$

This formula does not hold which can be seen from the counter-example $\{i \leftarrow -1; i_1 \leftarrow -2\}$.

To make this explicit, we will use the following terminology: We say that a loop invariant is *correct* iff it holds for every loop execution. The problem is that correctness of loop invariants depends on the global behavior of the given program and hence is difficult to check. Due to this, we ensure correctness of loops indirectly, based on verification conditions. We call a loop invariant *inductive* iff all verification conditions ending in this loop invariant are true. Clearly, inductivity of a loop invariant implies its correctness. However, the opposite direction does not hold: a loop invariant may be correct without being inductive.

16.5 Loop Invariants: Example

The most difficult part of checking unbounded program correctness is the development of loop invariants. At the end, this is a task that needs intuition and creativity. Consider the specification

- Input: array *a*
- Output: *r* s.t. $r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]$

and the program

```

r ← ⊥
for i ← 1 to n do
  @
  if a[i] = 7 then r ← ⊤
@  $r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]$ 
return r

```

The assertion with the missing loop invariant is involved in three verification conditions for which we must ensure the following three conditions:

- The loop invariant holds in the first loop iteration
- If it holds, and the loop is re-entered, then it must hold again
- If it holds, and the loop is left, then the assertion after the loop must hold

Let us first try the trivial guess $\top$, that is, a loop invariant that holds always. In this case, the first two of the mentioned conditions hold trivially. However, the third condition does not hold, that is, if the formula $\top$ holds, and we leave the loop then this does not ensure that the final assertion holds. The problem is that the tautology $\top$ is much too weak, and does not give us any information on what really happens in the loop.

So let us come up with a formula that captures what is going on in the loop, that is, the formula

$$r \Leftrightarrow [\exists k . 1 \leq k \leq i - 1 \wedge a[k] = 7]$$

Checking the two basic paths going through the loop (one for the if-branch, and one for the else-branch) we see that now they are correct:

assume $r \Leftrightarrow [\exists k . 1 \leq k \leq i - 1 \wedge a[k] = 7]$ assume $a[i] = 7$ $r \leftarrow \top$ $i \leftarrow i + 1$ assume $1 \leq i \leq n$ $@ r \Leftrightarrow [\exists k . 1 \leq k \leq i - 1 \wedge a[k] = 7]$	assume $r \Leftrightarrow [\exists k . 1 \leq k \leq i - 1 \wedge a[k] = 7]$ assume $a[i] \neq 7$ $i \leftarrow i + 1$ assume $1 \leq i \leq n$ $@ r \Leftrightarrow [\exists k . 1 \leq k \leq i - 1 \wedge a[k] = 7]$
--	--

The same holds for the basic paths leaving the loop:

assume $r \Leftrightarrow [\exists k . 1 \leq k \leq i - 1 \wedge a[k] = 7]$ assume $a[i] = 7$ $r \leftarrow \top$ assume $i = n$ $@ r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]$	assume $r \Leftrightarrow [\exists k . 1 \leq k \leq i - 1 \wedge a[k] = 7]$ assume $a[i] \neq 7$ assume $i = n$ $@ r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]$
--	--

A further question is: What happens, if we try to prove correctness of a program that has a bug? For example, assume that we replace the upper bound in the for loop of the program above by $n - 1$:

```

r ← ⊥
for i ← 1 to n - 1 do
  @ r ⇔ [∃k . 1 ≤ k ≤ i - 1 ∧ a[k] = 7]
  if a[i] = 7 then r ← ⊤
@ r ⇔ [∃k . 1 ≤ k ≤ n ∧ a[k] = 7]
return r

```

Now the final basic paths look as follows:

assume $r \Leftrightarrow [\exists k . 1 \leq k \leq i - 1 \wedge a[k] = 7]$ assume $a[i] = 7$ $r \leftarrow \top$ assume $i = n - 1$ $@ r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]$	assume $r \Leftrightarrow [\exists k . 1 \leq k \leq i - 1 \wedge a[k] = 7]$ assume $a[i] \neq 7$ assume $i = n - 1$ $@ r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]$
--	--

For the basic path on the left-hand side, that is, the basic path corresponding to the **if** branch, under the given assumptions, both sides of equivalence hold.

So this basic path is correct. The intuition is that after execution of the **if** branch the result is correct (independently of the initial state).

However the basic path on the right-hand side, that is, the basic path corresponding to the **else** branch, is not correct, since in this case, the given assumptions do not constrain $a[n]$ and hence we can choose a value for $a[n]$ that is in contradiction with the Boolean value of r .

16.6 Loop Invariant Positions

In Section 16.2, we introduced the requirement that each loop contain at least one assertion. This assertion can be at any position with the loop. The loop may even contain several assertions. So we have a lot of choice here, and the question is which choice to take. A typical position is at the beginning of the loop, as in the following program:

```

r ← ⊥
for i ← 1 to n do
  @ r ⇔ [∃k . 1 ≤ k ≤ i - 1 ∧ a[k] = 7]
  if a[i] = 7 then r ← ⊤
  @ r ⇔ [∃k . 1 ≤ k ≤ n ∧ a[k] = 7]
return r

```

The downside of this is that both the basic paths corresponding to one loop iterations, and the ones leaving the loop check the same loop body. The question is whether there is a way of avoiding this. Here is the same program, with an assertion at the beginning and at the end of the loop:

```

r ← ⊥
for i ← 1 to n do
  @ r ⇔ [∃k . 1 ≤ k ≤ i - 1 ∧ a[k] = 7]
  if a[i] = 7 then r ← ⊤
  @ r ⇔ [∃k . 1 ≤ k ≤ i ∧ a[k] = 7]
  @ r ⇔ [∃k . 1 ≤ k ≤ n ∧ a[k] = 7]
return r

```

In this case, the basic paths starting before the loop are the same as before. The basic paths through the loop body are the following:

```

assume r ⇔ [∃k . 1 ≤ k ≤ i - 1 ∧ a[k] = 7]
assume a[i] = 7
r ← ⊤
@ r ⇔ [∃k . 1 ≤ k ≤ i ∧ a[k] = 7]

assume r ⇔ [∃k . 1 ≤ k ≤ i - 1 ∧ a[k] = 7]
assume a[i] ≠ 7
@ r ⇔ [∃k . 1 ≤ k ≤ i ∧ a[k] = 7]

```

So they are slightly simpler than in the case with only one assertion at the beginning of the loop. How about the other basic paths? For considering this this, let us have a look at the following general situation:

```

for  $i \leftarrow l$  to  $u$  do
  @  $P$ 
  ...
  @  $P[i \leftarrow i + 1]$ 
@  $P[i \leftarrow u + 1]$ 

```

The basic path reentering the loop is:

```

assume  $P[i \leftarrow i + 1]$ 
 $i \leftarrow i + 1$ 
assume  $i \leq u$ 
@  $P$ 

```

And the basic path leaving the loop is:

```

assume  $P[i \leftarrow i + 1]$ 
assume  $i = u$ 
@  $P[i \leftarrow u + 1]$ 

```

Both hold automatically, independent of the precise form of P !

So this is a convenient way of writing loop invariants. Due to this, we will use the following shortcut:

```

for  $i \leftarrow l$  to  $u$  do
  @@  $P$ 
  ...

```

Indeed, in the literature, as well as in many tools, loop invariants are often used only in this form.

The situation is similar for **while**-loops:

```

while  $C$  do
  @  $P$ 
  ...
  @  $P$ 
@  $P$ 

```

Here the basic path reentering loop is:

```

assume  $P$ 
assume  $C$ 
@  $P$ 

```

And the basic path leaving loop is:

```

assume  $P$ 
assume  $\neg C$ 
@  $P$ 

```

Again both hold automatically. And again we will use the following shortcut for convenience:


```

while  $C$  do
  @@  $P$ 
  ...

```

16.7 Automation

The verification conditions can often be proved automatically, and explicit support for writing and checking assertions is growing, for example

- Eiffel
- Microsoft Code Contracts
- ANSI/ISO C Specification Language
- Java Modeling Language (JML), OpenJML (<http://www.openjml.org>)
- KeY (<http://www.key-project.org>)
- TLA+ (Microsoft, also used by Amazon)
- ...

In all cases the main problem is that we need loop invariants. The automatization of finding them

- is a difficult problem for programs (i.e., systems with infinite state space), and is
- current research, that is more and more used in industry

Examples of industrial tools:

- <http://www.absint.com/astree/>
- Mathworks Polyspace
- <https://www.imandra.ai>

Finite state systems: viz MI-TES

16.8 Conclusion

For proving correctness of a program we need

1. at least one assertion (i.e., a loop invariant) in every loop
2. a proof of the verification condition of every basic path of the program

In practice, usually no correctness proof is needed. Still, writing assertions has many advantages. Especially they localize the understanding of programs: Instead of thinking about the correctness of the whole program, one only needs to think about correctness from one assertion to the next. Hence assertions represent the essence of correctness of a program.

And assertions are useful, even if they are incomplete and unproved. In addition to localizing the understanding of programs they represent

- important documentation,
- they check correctness during program execution,
- and more and more it is even possible to do (semi)-automatic correctness proofs based on assertions.

Ideally, when developing programs, one would even develop loop invariants before the loop body. For example, when trying to write a program implementing the specification

- Input: array a
- Output: r s.t. $r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]$

the first step would be to write the output requirement as the last line of the program. After realizing that the implementation needs a loop, one can write the loop, and think of a corresponding loop invariant, even before implementing the body of the loop:

```

 $r \leftarrow \perp$ 
for  $i \leftarrow 1$  to  $n$  do
  @  $r \Leftrightarrow [\exists k . 1 \leq k \leq i \wedge a[k] = 7]$ 
  ???
@  $r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]$ 
return  $r$ 

```

Coming up with the loop body would then be the final step to finish the program.

16.9 Further Examples

16.9.1 Example 1

Consider the program listed in Figure 16.1 with α , α' , β , and β' being formulas that are still unknown.

Here, the following definition is used:

$$\forall a \in \mathcal{A}, n \in \mathcal{N}, s \in \mathcal{A}, k \in \mathcal{N}, p \in \mathcal{N} .$$

$$\text{substr}(a, n, s, k, p) :\Leftrightarrow \forall i \in \{0, \dots, k-1\} . p+i < n \wedge a[p+i] = s[i]$$

Now we will add logical formulas as assertions to the program lines starting with @, trying to ensure that all verification conditions hold. For understanding, which verification conditions, we list all basic paths of this program in Figure 16.2.

Maybe one would be tempted to set all assertions to the tautology $\top$. This would result in a program that certainly does not produce any assertion failure during run-time. But in this case, the final verification condition corresponding to leaving the outer loop certainly does not hold because the tautology $\top$ does not express any information needed to prove that the final result is correct. So which information would be needed to prove that the final result is correct? The final assertion on line 13 plays an important role in the verification condition

```

1:  found  $\leftarrow \perp$ 
2:  for  $i \leftarrow 0$  to  $n - k$  do
3:      @  $\alpha$ 
4:       $p \leftarrow \top$ 
5:      for  $j \leftarrow 0$  to  $k - 1$  do
6:          @  $\beta$ 
7:          if  $a[i + j] \neq s[j]$  then
8:               $p \leftarrow \perp$ 
9:          @  $\beta'$ 
10:     if  $p$  then
11:         found  $\leftarrow \top$ 
12:     @  $\alpha'$ 
13:  @ found  $\Leftrightarrow \exists r . \text{substr}(a, n, s, k, r)$ 

```

Figure 16.1: Program

lines	involved assertions	comment
1,2,3	from beginning to α	entering outer loop
1,2,13	from beginning to end	skipping outer loop
3,4,5,6	from α to β	entering inner loop
3,4,5,10,11,12	from α to α'	skipping inner loop, second if
3,4,5,10,12	from α to α'	skipping inner loop, second else
6,7,8,9	from β to β'	inner if
6,7,9	from β to β'	inner else
9,5,6	from β' to β	inner loop iteration
9,10,11,12	from β' to α'	second if
9,10,12	from β' to α'	second else
12,2,3	from α' to α	outer loop iteration
12,13	from α' to end	leaving outer loop

Figure 16.2: Basic Paths

leaving the outer loop. So we set α' to the information that the variable *found* already accumulated the result up to position i :

$$\text{found} \Leftrightarrow \exists r \in \{0, \dots, i\} . \text{substr}(a, n, s, k, r)$$

Clearly, this implies that we can set α to the same assertion, but constraining r to $\{0, \dots, i - 1\}$ instead of $\{0, \dots, i\}$, since the variable i has already been increased, here. Moreover, in order to be able to prove the final assertion in the outer loop, we need to add this formula also to the assertions β and β' in the inner loop.

Lines 10 and 11 update the variable *found* with the result for the current value of i . For being able to prove this, we need to add some additional information to the assertions β and β' of the inner loop. Here we need information about the variable p that is used to update *found*. What do we know about p

in the inner loop? At the end of the inner loop (assertion β') we know

$$p \Leftrightarrow \text{substr}(a, n, s, j+1, i),$$

and at the beginning the same formula, with $j+1$ replaced by j .

Let us now try to prove the verification conditions of the inner loop:

- $[[p \Leftrightarrow \text{substr}(a, n, s, j, i)] \wedge a[i+j] = s[j]] \Rightarrow [p \Leftrightarrow \text{substr}(a, n, s, j+1, i)]$
- $[[p \Leftrightarrow \text{substr}(a, n, s, j, i)] \wedge a[i+j] \neq s[j] \wedge \neg p] \Rightarrow [p \Leftrightarrow \text{substr}(a, n, s, j+1, i)]$

After substituting the definition of `substr` in the first condition, we get:

$$\begin{aligned} & [[p \Leftrightarrow [\forall i' \in \{0, \dots, j-1\} . i+i' < n \wedge a[i+i'] = s[i']]] \wedge a[i+j] = s[j]] \Rightarrow \\ & [p \Leftrightarrow [\forall i' \in \{0, \dots, j\} . i+i' < n \wedge a[i+i'] = s[i']]] \end{aligned}$$

This is easy to prove—with the exception of the condition $i+i' < n$. Indeed, in order to prove that condition, we also need the bounds $i \leq n-k$ and $j \leq k-1$ on the loop variables i and j . Those two inequalities imply $i+j \leq n-k+k-1 = n-1$, and hence $i+j < n$, which is what we need to prove $\forall i' \in \{0, \dots, j\} . i+i' < n$.

To summarize, we set the assertions to the following formulas:

$$\begin{aligned} \alpha: \quad & i \leq n-k \wedge \\ & \text{found} \Leftrightarrow \exists r \in \{0, \dots, i-1\} . \text{substr}(a, n, s, k, r) \\ \beta: \quad & i \leq n-k \wedge \\ & j \leq k-1 \wedge \\ & \text{found} \Leftrightarrow \exists r \in \{0, \dots, i-1\} . \text{substr}(a, n, s, k, r) \wedge \\ & p \Leftrightarrow \text{substr}(a, n, s, j, i) \\ \beta': \quad & i \leq n-k \wedge \\ & j \leq k-1 \wedge \\ & \text{found} \Leftrightarrow \exists r \in \{0, \dots, i-1\} . \text{substr}(a, n, s, k, r) \wedge \\ & p \Leftrightarrow \text{substr}(a, n, s, j+1, i) \\ \alpha': \quad & i \leq n-k \wedge \\ & \text{found} \Leftrightarrow \exists r \in \{0, \dots, i\} . \text{substr}(a, n, s, k, r) \end{aligned}$$

16.9.2 Example 2

Consider the algorithm

```

a* ← a
for i ← 0 to 9 do
  b[i] ← a[(i+1) mod 10] + 2
for i ← 0 to 9 do
  a[(i+1) mod 10] ← b[i] + 1
@ ∀i . 0 ≤ i ≤ 9 ⇒ a[i] = a*[i] + 3

```

We will prove the correctness of the algorithm. It overwrites the original array a with new values s.t. the relation expressed by the final assertion holds between the original array and the overwritten array. Here, it saves the original values of a in an auxiliary variables a^* . When writing the assertions, we will always use a^* for expressing properties of the original array, and a for expressing properties of the overwritten array.

We need at least one assertion in each loop. Here, we write the loop invariants at the end of the loops. Moreover, we add an assertion between the two loops in order to nicely summarize the role of the first loop.

```

 $a^* \leftarrow a$ 
for  $i \leftarrow 0$  to 9 do
   $b[i] \leftarrow a[(i + 1) \bmod 10] + 2$ 
  @  $a^* = a \wedge \forall i' \in \{0, \dots, i\} . b[i'] = a^*[(i' + 1) \bmod 10] + 2$ 
@  $\forall i' \in \{0, \dots, 9\} . b[i'] = a^*[(i' + 1) \bmod 10] + 2$ 
for  $i \leftarrow 0$  to 9 do
   $a[(i + 1) \bmod 10] \leftarrow b[i] + 1$ 
  @  $\forall i' \in \{0, \dots, 9\} . b[i'] = a^*[(i' + 1) \bmod 10] + 2,$ 
  @  $\forall i' \in \{0, \dots, i\} . a[(i' + 1) \bmod 10] = b[i'] + 1$ 
@  $\forall i . 0 \leq i \leq 9 \Rightarrow a[i] = a^*[i] + 3$ 

```

Note that our assertions need to contain more information than just the information reflecting the assignment statements in the loop:

- The first loop invariant contains the information $a^* = a$. The assignment in this loop uses a , and hence from the assignment alone, we are only able to prove a relationship between b and a . To prove the relationship between b and a^* we also need to know $a^* = a$.
- The second loop variant also contains information about the behavior of the first loop. The assignment in the loop uses a and b . To prove the final assertion of the program, we need to remember the relationship between b and a^* .

The verification conditions are all trivial, with one exception: the final condition ensuring that ending the second for loop implies the final assertion of the program:

$$\begin{aligned} & \forall i' \in \{0, \dots, 9\} . b[i'] = a^*[(i' + 1) \bmod 10] + 2, \\ & \forall i' \in \{0, \dots, 9\} . a[(i' + 1) \bmod 10] = b[i'] + 1 \quad \Rightarrow \quad \forall i . 0 \leq i \leq 9 \Rightarrow a[i] = a^*[i] + 3 \end{aligned}$$

We assume the left-hand side of the implication and, after a small equivalence transformation, we get:

$$\forall i' \in \{0, \dots, 9\} . b[i'] = a^*[(i' + 1) \bmod 10] + 2, a[(i' + 1) \bmod 10] = b[i'] + 1$$

Substituting the right-hand side of the first equality for $b[i']$ in the second equality, we get:

$$\forall i' \in \{0, \dots, 9\} . a[(i' + 1) \bmod 10] = a^*[(i' + 1) \bmod 10] + 2 + 1$$

that is,

$$\forall i' \in \{0, \dots, 9\} . a[(i' + 1) \bmod 10] = a^*[(i' + 1) \bmod 10] + 3.$$

This clearly implies what we want to prove since for a given $i \in \{0, \dots, 9\}$, the equality $(i' + 1) \bmod 10 = i$ always has a solution $i' \in \{0, \dots, 9\}$.

16.9.3 Example 3

Consider the following specification:

- Input: array a of length n
- Output: (k, l) s.t. $\text{eqseq}(a, n, k, l) \wedge \neg \exists l' . l' > l \wedge \text{eqseq}(a, n, k, l')$, where

$$\forall a, n, k, l . \text{eqseq}(a, n, k, l) :\Leftrightarrow \forall i \in \{k, \dots, k + l - 1\} . a[i] = a[k] \wedge i < n.$$

We will now write a program that fulfills this specification, together with assertions that ensure correctness of the program. We will develop the program in a top-down fashion, that is, at any time when we write a part of the program, we first write down assertions that ensure correctness of this part, and only then write the executable code of this part.

We start by writing the output specification as the final line of the program. Here, we pushed the negation inside of the existential quantifier, in order to be able to directly handle the resulting universal quantifier. We also observe that the first part of the specification, $\text{eqseq}(a, n, k, l)$, can be fulfilled trivially. So we add a corresponding assignment and assertion to the beginning of the program. The result is:

$k \leftarrow 0; l \leftarrow 1;$

@ $\text{eqseq}(a, n, k, l)$

@ $\text{eqseq}(a, n, k, l) \wedge \forall k', l' . \text{eqseq}(a, n, k', l') \Rightarrow l' \leq l$

It remains to fulfill the rest of the specification, that is, $\forall k', l' . \text{eqseq}(a, n, k', l') \Rightarrow l' \leq l$. We observe that the outer-most symbol is a universal quantifier. Our strategy is to fulfill it up to a certain k^* , with k^* increasing from 0 to $n - 1$. Hence we add a loop with such a loop invariant:

$k \leftarrow 0; l \leftarrow 1;$

@ $\text{eqseq}(a, n, k, l)$

for $k^* = 0$ **to** $n - 1$ **do**

@ $\text{eqseq}(a, n, k, l) \wedge \forall k', l' . [k' < k^* \wedge \text{eqseq}(a, n, k', l')] \Rightarrow l' \leq l$

@ $\text{eqseq}(a, n, k, l) \wedge \forall k', l' . \text{eqseq}(a, n, k', l') \Rightarrow l' \leq l$

In the body of the loop we should now ensure that k^* can be safely increased by one. Hence we should find the maximal length of a sequence of equal elements at starting at k^* , that is, an l^* such that $\text{eqseq}(a, n, k^*, l^*) \wedge \forall l' . \text{eqseq}(a, n, k^*, l') \Rightarrow l' \leq l^*$. We write this formula into the body of the for loop in the form of another assertion:

```

 $k \leftarrow 0; l \leftarrow 1;$ 
@ eqseq( $a, n, k, l$ )
for  $k^* = 0$  to  $n - 1$  do
  @ eqseq( $a, n, k, l$ )  $\wedge \forall k', l' . [k' < k^* \wedge \text{eqseq}(a, n, k', l')] \Rightarrow l' \leq l$ 

  @ eqseq( $a, n, k^*, l^*$ )  $\wedge \forall l' . \text{eqseq}(a, n, k^*, l') \Rightarrow l' \leq l^*$ 

@ eqseq( $a, n, k, l$ )  $\wedge \forall k', l' . \text{eqseq}(a, n, k', l') \Rightarrow l' \leq l$ 

```

Now, if $l^* > 0$, then we found a position that contains a longer sequence of equal elements than the current one stored in k . Hence, we update this information in this case:

```

 $k \leftarrow 0; l \leftarrow 1;$ 
@ eqseq( $a, n, k, l$ )
for  $k^* = 0$  to  $n - 1$  do
  @ eqseq( $a, n, k, l$ )  $\wedge \forall k', l' . [k' < k^* \wedge \text{eqseq}(a, n, k', l')] \Rightarrow l' \leq l$ 

  @ eqseq( $a, n, k^*, l^*$ )  $\wedge \forall l' . \text{eqseq}(a, n, k^*, l') \Rightarrow l' \leq l^*$ 
  if  $l^* > l$  then  $k \leftarrow k^*; l \leftarrow l^*$ 
@ eqseq( $a, n, k, l$ )  $\wedge \forall k', l' . \text{eqseq}(a, n, k', l') \Rightarrow l' \leq l$ 

```

It remains to find such an l^* . Certainly, $\text{eqseq}(a, n, k^*, 1)$, so we can use $l^* = 1$ as a first guess, and then check, whether there is a longer sequence. If we already know $\text{eqseq}(a, n, k^*, l^*)$ and learn that $a[k^* + l^* - 1] = a[k^* + l^*]$ where $k^* + l^*$ still points into the array, then $\text{eqseq}(a, n, k^*, l^* + 1)$. Hence we add a loop with such an assertion.

```

 $k \leftarrow 0; l \leftarrow 1;$ 
for  $k^* = 0$  to  $n - 1$  do
  @ eqseq( $a, n, k, l$ )  $\wedge \forall k', l' . [k' < k^* \wedge \text{eqseq}(a, n, k', l')] \Rightarrow l' \leq l$ 
   $l^* \leftarrow 1$ 
  while  $k^* + l^* < n \wedge a[k^* + l^* - 1] = a[k^* + l^*]$  do
    @ eqseq( $a, n, k^*, l^* + 1$ )
     $l^* \leftarrow l^* + 1$ 
  @ eqseq( $a, n, k^*, l^*$ )  $\wedge \forall l' . \text{eqseq}(a, n, k^*, l') \Rightarrow l' \leq l^*$ 
  if  $l^* > l$  then  $k \leftarrow k^*; l \leftarrow l^*$ 
@ eqseq( $a, n, k, l$ )  $\wedge \forall k', l' . \text{eqseq}(a, n, k', l') \Rightarrow l' \leq l$ 

```

Note that there is room for optimizing this algorithm. For example, we could observe that

$$\text{eqseq}(a, n, k, l) \wedge \forall l' . \text{eqseq}(a, n, k, l') \Rightarrow l' \leq l$$

implies

$$\forall k', l' . [k \leq k' < k + l \wedge \text{eqseq}(a, n, k', l')] \Rightarrow l' \leq l,$$

and hence we can replace the for loop by a while loop that increases k^* by larger values.

Chapter 17

Functions, Procedures

17.1 Correctness of Functions

For using a function, it suffices to know its interface. Such an interface description might, for example, look like this:

function `sort(a, n)`:
Input: $a \in \mathcal{A}, n \in \mathcal{N}$
Output: $b \in \mathcal{A}$ s.t. $\text{sorted}(b, n) \wedge \text{perm}(a, b, n)$

function `count(a, n, x)`:
Input: $a \in \mathcal{A}, n \in \mathcal{N}, x$ s.t. $\text{sorted}(a, n)$
Output: $|\{i \mid i \in \{1, \dots, n\}, a[i] < x\}|$

Such interfaces allow the usage of functions without knowing the underlying implementation, which corresponds to the important software engineering principle of information hiding. Many programming languages provide explicit support for such interface specifications, for example, the programming languages C and C++ in the form of header files.

Now assume some code that uses those functions:

```
s ← sort(a, n)
p ← count(s, n, v)
@ p = |\{i \mid i \in \{1, \dots, n\}, a[i] < v\}|
```

Intuitively, this basic path seems to be correct. To make this explicit, we can analyze the verification condition of the basic path:

$$[s = \text{sort}(a, n) \wedge p = \text{count}(s, n, v)] \Rightarrow p = |\{i \mid i \in \{1, \dots, n\}, a[i] < v\}|$$

Again, this seems to hold, intuitively. However, without any knowledge about the called functions, the above code is *not* correct, and the basic path does *not* hold. The missing knowledge is the one from interface descriptions above. But how can we use this knowledge? The strategy will simply be to represent those interfaces using logical formulas. Here we will use such formulas in two variants. The first variant uses variables to represent the output of the functions. This is straightforward in the case of the sort function:

$$\forall a \in \mathcal{A}, n \in \mathcal{N}, b \in \mathcal{A} . b = \text{sort}(a, n) \Rightarrow [\text{sorted}(b, n) \wedge \text{perm}(a, b, n)]$$

However, the interface description of the count function does not use a variable name to represent its output. Hence we first introduce such a variable name:

function count(a, n, x)

Input: $a \in \mathcal{A}, n \in \mathcal{N}, x$ s.t. sorted(a, n)

Output: r s.t. $r = |\{i \mid i \in \{1, \dots, n\}, a[i] < x\}|$

And now the representation using a logical formula is straightforward:

$$\begin{aligned} \forall a \in \mathcal{A}, n \in \mathcal{N}, x, r \in \mathcal{N} . \\ [\text{sorted}(a, n) \wedge r = \text{count}(a, n, x)] \Rightarrow \\ r = |\{i \mid i \in \{1, \dots, n\}, a[i] < x\}| \end{aligned}$$

The second variant represents the output using the name of the function itself. The result is the following:

$$\forall a \in \mathcal{A}, n \in \mathcal{N} . \text{sorted}(\text{sort}(a, n), n) \wedge \text{perm}(a, \text{sort}(a, n), n)$$

$$\begin{aligned} \forall a \in \mathcal{A}, n \in \mathcal{N}, x \in \mathcal{N} . \text{sorted}(a, n) \Rightarrow \\ \text{count}(a, n, x) = |\{i \mid i \in \{1, \dots, n\}, a[i] < x\}| \end{aligned}$$

To include the knowledge expressed by these formulas, we add further assume statements to the basic path from above. Using the version of the formulas that use variables to represent the output of the function, the results is

assume $\forall a \in \mathcal{A}, n \in \mathcal{N}, b \in \mathcal{A} . b = \text{sort}(a, n) \Rightarrow [\text{sorted}(b, n) \wedge \text{perm}(a, b, n)]$
assume $\forall a \in \mathcal{A}, n \in \mathcal{N}, x, r \in \mathcal{N} .$
 $[\text{sorted}(a, n) \wedge r = \text{count}(a, n, x)] \Rightarrow r = |\{i \mid i \in \{1, \dots, n\}, a[i] < x\}|$
 $s \leftarrow \text{sort}(a, n)$
 $p \leftarrow \text{count}(s, n, v)$
 $@ p = |\{i \mid i \in \{1, \dots, n\}, a[i] < v\}|$

Note that the added formulas have universal quantifiers as their outer-most symbol. Using the corresponding proof rule, one can then substitute terms for the quantified variables that correspond to the concrete usage of the functions.

Let us do this now for the verification condition of the basic path mentioned above. Here, we will not explicitly write down the verification condition, but immediately list what we assume and prove, using the fact that the verification condition is an implication with a big conjunction on its left-hand side. We will do two variants of the proof corresponding to the two variants of the formulas representing the interface of the used functions. Here is the variant using the formulas in the form using output variables:

Proof. We assume

$$\forall a \in \mathcal{A}, n \in \mathcal{N}, b \in \mathcal{A} . b = \text{sort}(a, n) \Rightarrow [\text{sorted}(b, n) \wedge \text{perm}(a, b, n)],$$

$$\begin{aligned} \forall a \in \mathcal{A}, n \in \mathcal{N}, x, r \in \mathcal{N} . [\text{sorted}(a, n) \wedge r = \text{count}(a, n, x)] \Rightarrow \\ r = |\{i \mid i \in \{1, \dots, n\}, a[i] < x\}|, \end{aligned}$$

$$s = \text{sort}(a, n),$$

and

$$p = \text{count}(s, n, v),$$

and prove

$$p = |\{i \mid i \in \{1, \dots, n\}, a[i] < v\}|.$$

We start by proving properties of the result of the first program line. From the assumption describing the function *sort*, we know $\text{sorted}(s, n)$ and $\text{perm}(a, s, n)$. From the assumption describing the function *count*, after the choices $a \leftarrow s$, $n \leftarrow n$, $x \leftarrow v$ we know:

$$[\text{sorted}(s, n) \wedge p = \text{count}(s, n, v)] \Rightarrow p = |\{i \mid i \in \{1, \dots, n\}, s[i] < v\}|.$$

Since we know both assumptions on the left-hand side, we also know the right-hand side. Since $\text{perm}(a, s, n)$, both a and s contain exactly the same elements, and hence

$$p = |\{i \mid i \in \{1, \dots, n\}, a[i] < v\}|,$$

which is what we wanted to prove. ■

And here is the version using the variant with function names:

Proof. We assume

$$\forall a \in \mathcal{A}, n \in \mathcal{N} . \text{sorted}(\text{sort}(a, n), n) \wedge \text{perm}(a, \text{sort}(a, n), n),$$

$$\forall a \in \mathcal{A}, n \in \mathcal{N}, x \in \mathcal{N} . \text{sorted}(a, n) \Rightarrow$$

$$\text{count}(a, n, x) = |\{i \mid i \in \{1, \dots, n\}, a[i] < x\}|,$$

$$s = \text{sort}(a, n),$$

and

$$p = \text{count}(s, n, v),$$

and prove

$$p = |\{i \mid i \in \{1, \dots, n\}, a[i] < v\}|.$$

Due to the substitutions resulting from the assumptions, it suffices to prove

$$\text{count}(\text{sort}(a, n), n, v) = |\{i \mid i \in \{1, \dots, n\}, a[i] < v\}|.$$

We start by proving properties of the result of the first program line $\text{sort}(a, n)$.

From the first assumption above, we know $\text{sorted}(\text{sort}(a, n), n)$ and $\text{perm}(a, \text{sort}(a, n), n)$. From the second assumption we know, after the choices $a \leftarrow \text{sort}(a, n)$, $n \leftarrow n$, $x \leftarrow v$:

$$\text{sorted}(\text{sort}(a, n), n) \Rightarrow \text{count}(\text{sort}(a, n), n, v) = |\{i \mid i \in \{1, \dots, n\}, \text{sort}(a, n)[i] < v\}|.$$

Since we know the assumption on the left-hand side, we also know the right-hand side. Since $\text{perm}(a, \text{sort}(a, n), n)$, both a and $\text{sort}(a, n)$ contain exactly the same elements, and hence

$$\text{count}(\text{sort}(a, n), n, v) = |\{i \mid i \in \{1, \dots, n\}, a[i] < v\}|.$$

which is what we wanted to prove. ■

Up to now we have just discussed usage of functions. Now we will have a look at their implementation. Here it is good practice to use assertions that check the I/O specification at the beginning and end of the implementation:

```

function sort(a, n):
Input:  $a \in \mathcal{A}, n \in \mathcal{N}$ 
Output:  $b \in \mathcal{A}$  s.t.  $\text{sorted}(b, n) \wedge \text{perm}(a, b, n)$ 
@  $a \in \mathcal{A}, n \in \mathcal{N}$ 
...
@  $\text{sorted}(b, n) \wedge \text{perm}(a, b, n)$ 
return b

function count(a, n, x):
Input:  $a \in \mathcal{A}, n \in \mathcal{N}, x$  s.t.  $\text{sorted}(a, n)$ 
Output:  $|\{a[i] < x \mid i \in \{1, \dots, n\}\}|$ 
@  $a \in \mathcal{A}, n \in \mathcal{N}, x$  s.t.  $\text{sorted}(a, n)$ 
...
@  $r = |\{i \mid i \in \{1, \dots, n\}, a[i] < x\}|$ 
return r

```

So let us summarize the thing we have discussed so far. Each function should have an input/output specification:

```

function  $f(v_1, \dots, v_n)$ 
Input:  $I(v_1, \dots, v_n)$ 
Output:  $v'$  s.t.  $O(v_1, \dots, v_n, v')$ 

```

For proving correctness of code that uses the function, we can add a logical formula corresponding to the I/O specification as an assumption. This corresponds to additional **assume** statements in the basic paths using these functions, which results in additional formulas on the left-hand side of the corresponding verification condition.

These logical formulas forming these assumptions come in two variants:

$$\begin{aligned} \forall v_1, \dots, v_n, v' . [I(v_1, \dots, v_n) \wedge v' = f(v_1, \dots, v_n)] &\Rightarrow O(v_1, \dots, v_n, v') \\ \forall v_1, \dots, v_n . I(v_1, \dots, v_n) &\Rightarrow O(v_1, \dots, v_n, f(v_1, \dots, v_n)) \end{aligned}$$

Here, we can fully ignore internal code of the function. It could have millions of lines of code, but when checking whether it is used correctly, it suffices to work with its input/output specification or the logical formula representing this input/output specification. When actually implementing the function, we can then add the formulas forming the I/O specification to assertions at the beginning and end of the implementation which will then check the correctness of the implementing every time the function is called.

17.2 What Is a Function?

Up to now, we ignored the question of what we actually mean by the term function. However, this question can be crucial, which can be seen on the following code fragment:

```

if(foo(1)==foo(1)) {
    ...                               // send the user 100 Euro
} else {
    ...                               // format all hard disks
}

```

Would you ever run such code? As long as we can rely on `foo()` being a function in the mathematical sense, applying the function twice to the same argument *must* result in the same result. However, C functions are, in general, no functions (in the mathematical sense). And hence, they might return two different result when called twice with the same argument. For example, the result of a function call may depend on the value of a global variable, on the content of some extern file, or on user input. All of this may change from one call to the next. This is different for pure functional languages, where functions really behave like mathematical functions.

And indeed, the discussion above depends on the assumption that functions are mathematical functions. Hence, they must not depend on user input, global variables, file system etc. Note however, that user output is o.k., since our method simply does not express anything about it.

Now consider the two following programs:

```

x= 1;                               y[0]= 1;
z= foo(x);                           z= foo(y);

if(x==1) {                           if(y[0]==1) {
    ... // send the user 100€         ... // send the user 100€
} else {                             } else {
    ... // format all hard disks     ... // format all hard disks
}                                    }

```

Here the question is, whether a function call can change the value of an argument. For functions in the sense of mathematics, the answer is of course no. In programming languages, the answer depends on the programming language. If we assume that the two code snippets are written in the programming language C and that the variable x is a C integer, and the variable y a C array, then the snippet on the left-hand side is safe to execute, but the one on the right-hand side not. The reason is that C uses the calling convention *call by value* for integers, which means that any C function receiving an integer variable as an argument, only receives a copy of the current value of this variable. However, C uses *call by reference* for arrays, which means that whenever a C array is used as an argument, the called function receives a pointer to this array, and hence is able to modify its content.

17.3 Recursive Function Calls

The greatest common divisor is usually defined as follows:

$$\forall r, x, y . r = \text{gcd}(x, y) :\Leftrightarrow r|x \wedge r|y \wedge \neg \exists r' . r'|x \wedge r'|y \wedge r' > r$$

Based on this, we will now implement a recursive function with the following specification:

function GCD(x, y)

Input: $x \in \mathcal{N}, y \in \mathcal{N}, x \geq y$

Output: $\text{gcd}(x, y)$

Writing this again as a mathematical formula, we get:

$$\forall x \in \mathcal{N}, y \in \mathcal{N} . [x \geq y \wedge r = \text{GCD}(x, y)] \Rightarrow r = \text{gcd}(x, y)$$

Note that here we use lower case letters to denote the mathematical notion of greatest common divisor, and upper case letters to denote our implementation. The implementation will be recursive:

function GCD(x, y)

@ $x, y \in \mathcal{N}, x \geq y$

if $y = 0$ **then**

$r \leftarrow x$

else

$r \leftarrow \text{GCD}(y, x \bmod y)$

@ $r = \text{gcd}(x, y)$

return r

This program has the following basic paths:

assume $x, y \in \mathcal{N}, x \geq y$

assume $y = 0$

$r \leftarrow x$

@ $r = \text{gcd}(x, y)$

assume $x, y \in \mathcal{N}, x \geq y$

assume $y \neq 0$

$r \leftarrow \text{GCD}(y, x \bmod y)$

@ $r = \text{gcd}(x, y)$

The first basic paths does not include any recursive call, and hence can be proved correct without additional information. For proving correctness of the second basic paths, which contains a recursive call, we include the formula from above as an assumption. The result is

assume $\forall x \in \mathcal{N}, y \in \mathcal{N} . [x \geq y \wedge r = \text{GCD}(x, y)] \Rightarrow r = \text{gcd}(x, y)$

assume $x, y \in \mathcal{N}, x \geq y$

assume $y \neq 0$

$r \leftarrow \text{GCD}(y, x \bmod y)$

@ $r = \text{gcd}(x, y)$

The proof of the verification condition corresponding to this basic path looks as follows:

Proof. We assume that the recursive call returns a correct result:

$$\forall x, y, r . [x \geq y \wedge r = \text{GCD}(x, y)] \Rightarrow r = \text{gcd}(x, y),$$

we assume $x \geq y$, $y \neq 0$, and $r = \text{GCD}(y, x \bmod y)$ and try to prove

$$r = \text{gcd}(x, y).$$

From the formula above, after choosing $x \leftarrow y, y \leftarrow x \bmod y$ we conclude

$$[y \geq x \bmod y \wedge r = \text{GCD}(y, x \bmod y)] \Rightarrow r = \text{gcd}(y, x \bmod y)$$

The part $y \geq x \bmod y$ is a mathematical fact, we already know $r = \text{GCD}(y, x \bmod y)$, and so we can conclude $r = \text{gcd}(y, x \bmod y)$. Now it suffices to prove $\text{gcd}(y, x \bmod y) = \text{gcd}(x, y)$, which is a well-known mathematical fact. ■

To summarize, for proving correctness of a recursive function, one can add the logical formula corresponding to the I/O specification of the function itself as an assumption. Here, the function must again be side-effect free. That is, it should not use any input or global variables to compute its result.

It may sound like a circular argument that for proving correctness of a recursive function we assume correctness of the function. However, this is not the case, since we assume correctness of the function only for the recursive call in the verification conditions.

For a more detailed argument, imagine a tree where the set of nodes is formed by the (infinite) set of all possible arguments to the function. The roots of the tree are the nodes corresponding to the base case of the recursion, that is, those arguments to the function, for which the function does not do any recursive call. The edges of the tree correspond to the recursive calls: For any pair of nodes (a, a') we have an edge from a to a' in the tree, if the function call with arguments a calls the function recursively with arguments a' .

Our proof method proceeds by (structural) induction over all nodes of this tree:

- It proves that for all roots of the tree, that is, all arguments to the function without any recursive call, the function is correct.
- It takes an arbitrary, but fixed node a of the tree, and proves correctness of the function for the arguments a , under the assumption that for all nodes a' with an edge (a, a') in the tree, the function call with arguments a' is correct.

Note however, that our argument does *not* ensure finiteness of this tree. In other words, the recursive function does not necessarily terminate. We will address this problem later, in Chapter 19.

17.4 Correctness of Procedures

Now we will relax the requirement of only handling functions in the mathematical sense: We will now allow procedures that do not return any value, but may change their arguments. This corresponds to the call-by-reference strategy of handling arguments. Here is an example:

procedure `reverse`(a, n)

Input: $a \in \mathcal{A}, n \in \mathcal{N}$

Output: $a^* \in \mathcal{A}$ s.t. $\forall i \in \{0, \dots, n-1\} . a^*[i] = a[n-1-i]$

Since this procedure should change its argument a , this specification uses a different name for its input and output value, and denotes the output value using a star. There are many alternative possibilities for doing so, for example, a^{in}/a^{out} or $a\downarrow/a\uparrow$.

We can also write the input/output specification as a logical formula. A first attempt, following the way of writing input/output specifications of functions as logical formulas, could be

$$\forall a, a^*, n . \text{reverse}(a, n) \Rightarrow \forall i \in \{0, \dots, n-1\} . a^*[i] = a[n-1-i]$$

However, this does not make sense, since the left-hand side of the implication only refer to the value a , whereas the right-hand side to both a and a^* . In order to introduce the difference between input and output value of an argument also to the notation of the procedure itself, we list input and output values separately:

$$\forall a, a^*, n . \text{reverse}(a, a^*, n) \Rightarrow \forall i \in \{0, \dots, n-1\} . a^*[i] = a[n-1-i]$$

Note that `reverse` now has three arguments. Also, within this formula, it is a *predicate symbol* instead of a function symbol.

Now we can again use this formula as an assumption when reasoning about usage of the procedure. Consider the following program:

```
assume  $\forall i \in \{0, \dots, n-1\} . a[i] \geq 0$ 
reverse( $a, n$ )
@  $\forall i \in \{0, \dots, n-1\} . a[i] \geq 0$ 
```

Adding the formula describing the behavior of the procedure as an assumption, we get

```
assume  $\forall a, a^*, n . \text{reverse}(a, a^*, n) \Rightarrow \forall i \in \{0, \dots, n-1\} . a^*[i] = a[n-1-i]$ 
assume  $\forall i \in \{0, \dots, n-1\} . a[i] \geq 0$ 
reverse( $a, n$ )
@  $\forall i \in \{0, \dots, n-1\} . a[i] \geq 0$ 
```

Here we observe a strange mismatch: The procedure `reverse` has two arguments, but in the new assumption, `reverse` has three arguments! Indeed, logical formulas cannot contain procedure calls. The usage of `reverse` in the assumption is something else: a predicate symbol describing the behavior of the procedure call.

The mismatch disappears when we convert the basic path to SSA. Since in SSA, we have to use different variable names for different values, we must use different names for the input values to `reverse`, and the output values. To do so, in SSA we handle `reverse` as a predicate throughout, replacing the procedure call with a corresponding assumption:

```
assume  $\forall a, a^*, n . \text{reverse}(a, a^*, n) \Rightarrow \forall i \in \{0, \dots, n-1\} . a^*[i] = a[n-1-i]$ 
assume  $\forall i \in \{0, \dots, n-1\} . a[i] \geq 0$ 
assume reverse( $a, a_1, n$ ) // reverse is a predicate now
@  $\forall i \in \{0, \dots, n-1\} . a_1[i] \geq 0$ 
```

We can prove the corresponding verification condition as follows:

Proof. We assume

- $\forall a, a^*, n . \text{reverse}(a, a^*, n) \Rightarrow \forall i \in \{0, \dots, n-1\} . a^*[i] = a[n-1-i]$,
- $\forall i \in \{0, \dots, n-1\} . a[i] \geq 0$, and
- $\text{reverse}(a, a_1, n)$.

We want to prove:

- $\forall i \in \{0, \dots, n-1\} . a_1[i] \geq 0$

The assumption $reverse(a, a_1, n)$ represents a concrete procedure call. To deduce something about the result of the call we apply the substitutions $a \leftarrow a_1, n \leftarrow n$ to the assumption

$$\forall a, a^*, n . reverse(a, a^*, n) \Rightarrow \forall i \in \{0, \dots, n-1\} . a^*[i] = a[n-1-i]$$

that represents the specification of the procedure. The result is

$$reverse(a, a_1, n) \Rightarrow \forall i \in \{0, \dots, n-1\} . a_1[i] = a[n-1-i]$$

Since we already know the left-hand side of this assumption we can conclude the right-hand side

$$\forall i \in \{0, \dots, n-1\} . a_1[i] = a[n-1-i].$$

So we know

- $\forall i \in \{0, \dots, n-1\} . a[i] \geq 0$
- $\forall i \in \{0, \dots, n-1\} . a_1[i] = a[n-1-i]$

and want to prove $\forall i \in \{0, \dots, n-1\} . a_1[i] \geq 0$. Now let i be arbitrary, but fixed. We prove $a_1[i] \geq 0$. Due to the second assumption this means to prove $a[n-1-i] \geq 0$, which we know after substituting $i \leftarrow n-1-i$ in the first assumption. ■

Now let us implement the reverse procedure. Again, our first step will be, to write corresponding assertions at the beginning and end of the implementation. However, the assertion ensuring correctness of the result must compare the final value of the array a with its value at the beginning of the procedure. In order to be able to do this, we remember the value at the beginning using an auxiliary variable.

procedure $reverse(a, n)$

@ $a \in \mathcal{A}, n \in \mathcal{N}$

$a^{in} \leftarrow a$

...

@ $\forall i \in \{0, \dots, n-1\} . a[i] = a^{in}[n-1-i]$

return

Another example that contains several arguments that the procedure may change is the following:

procedure $swap(x, y)$

Input:

Output: x^*, y^* s.t. $x^* = y, y^* = x$

$$\forall x, y, x^*, y^* . swap(x, x^*, y, y^*) \Rightarrow [x^* = y \wedge y^* = x]$$

This procedure could, for example, be used as follows:

assume $x \geq 0 \wedge y \geq 1$

swap(x, y)

@ $x \geq 1 \wedge y \geq 0$

Again we add information about the behavior of the procedure **swap** in the form of the formula from above:

assume $\forall x, y, x^*, y^* . \text{swap}(x, x^*, y, y^*) \Rightarrow [x^* = y \wedge y^* = x]$
assume $x \geq 0 \wedge y \geq 1$
swap(x, y)
@ $x \geq 1 \wedge y \geq 0$

Again we observe the mismatch, that **swap** is once mentioned with four arguments, and once with two arguments. And again, the two occurrences refer to two different things: The occurrence with four arguments is within a logical formula, and denotes a predicate (that describes the behavior of a procedure call), whereas the occurrence with two arguments refers to a procedure call. The mismatch again vanishes after converting the basic path to SSA:

assume $\forall x, y, x^*, y^* . \text{swap}(x, x^*, y, y^*) \Rightarrow [x^* = y \wedge y^* = x]$
assume $x \geq 0 \wedge y \geq 1$
assume $\text{swap}(x, x_1, y, y_1)$
@ $x_1 \geq 1 \wedge y_1 \geq 0$

Note that we again added an **assume** in order to replace the procedure call with a logical formula relating the input to the output values. This basic path in SSA can then be converted to a verification condition and proved, either manually, or automatically.

So let us summarize the handling of procedure calls. We use input/output specifications of the following form:

procedure $p(v_1, \dots, v_n)$
Input: $I(v_1, \dots, v_n)$
Output: $v_1^*, \dots, v_n^*$ s.t. $O(v_1, v_1^*, \dots, v_n, v_n^*)$

where the unstarred variables denote input values and starred variable the corresponding output values.

From such a specification we get a logical formula that we can add to our assumptions:

$$\forall v_1, v_1^*, \dots, v_n, v_n^* . \\ [I(v_1, \dots, v_n) \wedge p(v_1, v_1^*, \dots, v_n, v_n^*)] \Rightarrow O(v_1, v_1^*, \dots, v_n, v_n^*)$$

When transforming any code using this procedure to SSA, a procedure call $p(v_1, \dots, v_n)$ becomes **assume** $p(v_1, v_1^*, \dots, v_n, v_n^*)$ in SSA, with p now representing a predicate symbol.

17.5 Example: Recursive Binary Search

We will now have a look at another, more complex example of a recursive function. Throughout we will use arrays whose indices range from 1 to n . The specification of the function is the following:

function $\text{search}(a, l, u, x)$
Input: $a \in \mathcal{A}(\mathcal{N})$, $n \in \mathcal{N}$, **sorted**(a, n), $l, u \in \{1, \dots, n\}$, $x \in \mathcal{N}$
Output: r , s.t. $r = 0 \Rightarrow [\neg \exists i . l \leq i \leq u \wedge a[i] = x] \wedge$
 $r \neq 0 \Rightarrow [l \leq r \leq u \wedge a[r] = x]$

This corresponds to the logical formula

$$\forall a, l, u, x, r . r = \text{search}(a, l, u, x) \Rightarrow \left[\begin{array}{c} r = 0 \Rightarrow [\neg \exists i . l \leq i \leq u \wedge a[i] = x] \\ \wedge \\ r \neq 0 \Rightarrow [l \leq r \leq u \wedge a[r] = x] \end{array} \right]$$

Here is an implementation:

```

function search( $a, l, u, x$ )
assume  $a \in \mathcal{A}(\mathcal{N}), n \in \mathcal{N}, \text{sorted}(a, n), l, u \in \{1, \dots, n\}, x \in \mathcal{N}$ 
if  $u < l$  then
   $r \leftarrow 0$ 
else if  $a[(l + u)/2] = x$  then
   $r \leftarrow (l + u)/2$ 
else if  $a[(l + u)/2] < x$  then
   $r \leftarrow \text{search}(a, (l + u)/2 + 1, u, x)$ 
else
   $r \leftarrow \text{search}(a, l, (l + u)/2 - 1, x)$ 
@  $r = 0 \Rightarrow [\neg \exists i . l \leq i \leq u \wedge a[i] = x] \wedge$ 
   $r \neq 0 \Rightarrow [l \leq r \leq u \wedge a[r] = x]$ 
return  $r$ 

```

The requirements on the input are obviously preserved during recursive calls, so we will not further have a look at them. There are four basic paths leading through this functions. The two basic paths without a recursive call are obviously correct. We will have a more detailed look at the first basic path containing a recursive call.

```

assume  $a \in \mathcal{A}(\mathcal{N}), n \in \mathcal{N}, \text{sorted}(a, n), l, u \in \{1, \dots, n\}, x \in \mathcal{N}$ 
assume  $a[(l + u)/2] < x$ 
 $r \leftarrow \text{search}(a, (l + u)/2 + 1, u, x)$ 
@  $[r = 0 \Rightarrow [\neg \exists i . l \leq i \leq u \wedge a[i] = x]] \wedge [r \neq 0 \Rightarrow [l \leq r \leq u \wedge a[r] = x]]$ 

```

To show that this basic path is correct, we need information about the behavior of the recursive call. Hence we add an assumption at the beginning that contains the logical formula corresponding the interface of the function `search`:

```

assume  $\forall a, l, u, x, r . r = \text{search}(a, l, u, x) \Rightarrow$ 
   $\left[ \begin{array}{c} r = 0 \Rightarrow [\neg \exists i . l \leq i \leq u \wedge a[i] = x] \\ \wedge \\ r \neq 0 \Rightarrow [l \leq r \leq u \wedge a[r] = x] \end{array} \right]$ 
assume  $a \in \mathcal{A}(\mathcal{N}), n \in \mathcal{N}, \text{sorted}(a, n), l, u \in \{1, \dots, n\}, x \in \mathcal{N}$ 
assume  $a[(l + u)/2] < x$ 
 $r \leftarrow \text{search}(a, (l + u)/2 + 1, u, x)$ 
@  $[r = 0 \Rightarrow [\neg \exists i . l \leq i \leq u \wedge a[i] = x]] \wedge [r \neq 0 \Rightarrow [l \leq r \leq u \wedge a[r] = x]]$ 

```

We can now either do an informal check whether this basic path is correct, or—if we want to achieve higher certainty—we can translate the basic path to a verification condition, and check whether the verification condition is true.

17.6 Sorting Algorithm: Basic Definitions

Let us now look at the development of a sorting algorithm. Again, we will use array indices ranging from 1 to n . The specification of the sorting algorithms is the following:

procedure `sort`(a, n)

Input: array a

Output: a^* s.t. $\text{perm}(a, a^*) \wedge \text{sorted}(a^*, n)$ where

We will use the following auxiliary procedure:

procedure `swap`(a, n, i, j)

Input: array $a, i, j, n \in \mathcal{N}, 1 \leq i \leq n, 1 \leq j \leq n$

Output: a^* s.t. $\begin{cases} a^*[i] = a[j], a^*[j] = a[i], \\ \forall k. [1 \leq k \leq n, k \neq i, k \neq j] \Rightarrow a^*[k] = a[k] \end{cases}$

The first step to implement our sorting procedure is to write its specification as assertions:

procedure `sort`(a, n)

$a_{in} \leftarrow a$

...

@ $\text{perm}(a_{in}, a) \wedge \text{sorted}(a, n)$

return a

The idea for our implementation will be based on the definition

$$\text{minfirst}(a, n, i) :\Leftrightarrow a[i] = \min\{a[k] \mid k \in \{i, \dots, n\}\}$$

The intuition is that this predicate holds iff the element at position i in the array a is smaller or equal to all further elements. For example,

$$\text{minfirst}(\boxed{10 \mid 3 \mid 7 \mid 4 \mid 7}, 5, 2),$$

because the element $a[2] = 3$ is smaller than all further elements.

Our strategy to sort the array will be to incrementally ensure this property for parts of the array, starting from the end. Hence we will use a loop, as follows:

procedure `sort`(a, n)

$a_{in} \leftarrow a$

for $i \leftarrow 1$ **to** n **do**

 @ $\text{perm}(a_{in}, a) \wedge \forall k \in \{1, \dots, i-1\}. \text{minfirst}(a, n, k)$

 ...

 @ $\text{perm}(a_{in}, a) \wedge \forall k \in \{1, \dots, i\}. \text{minfirst}(a, n, k)$

@ $\text{perm}(a_{in}, a) \wedge \text{sorted}(a, n)$

return

As can be seen from the assertions, the missing part of the loop should ensure that at the end of the loop body $\text{minfirst}(a, n, i)$ will hold, in addition. To ensure this, we will again use a loop:

```

procedure sort( $a, n$ )
 $a_{in} \leftarrow a$ 
for  $i \leftarrow 1$  to  $n$  do
  @  $perm(a_{in}, a) \wedge \forall k \in \{1, \dots, i-1\} . minfirst(a, n, k)$ 
  for  $j \leftarrow n$  down to  $i+1$  do
    @  $perm(a_{in}, a) \wedge [\forall k \in \{1, \dots, i-1\} . minfirst(a, n, k)] \wedge minfirst(a, n, j)$ 
    ...
    @  $perm(a_{in}, a) \wedge [\forall k \in \{1, \dots, i-1\} . minfirst(a, n, k)] \wedge minfirst(a, n, j-1)$ 
  @  $perm(a_{in}, a) \wedge \forall k \in \{1, \dots, i\} . minfirst(a, n, k)$ 
@  $perm(a_{in}, a) \wedge sorted(a, n)$ 
return

```

Now we are left with body of the inner loop. Here we need to ensure the step from $minfirst(a, n, j)$ to $minfirst(a, n, j-1)$. Clearly, if $a[j-1]$ is already smaller or equal to $a[j]$, nothing needs to be done. If not, we simply swap the two elements. This finishes the sorting algorithm:

```

procedure sort( $a, n$ )
 $a_{in} \leftarrow a$ 
for  $i \leftarrow 1$  to  $n$  do
  @  $perm(a_{in}, a) \wedge \forall k \in \{1, \dots, i-1\} . minfirst(a, n, k)$ 
  for  $j \leftarrow n$  down to  $i+1$  do
    @  $perm(a_{in}, a) \wedge [\forall k \in \{1, \dots, i-1\} . minfirst(a, n, k)] \wedge minfirst(a, n, j)$ 
    if  $a[j-1] > a[j]$  then
      swap( $a, j-1, j$ )
    @  $perm(a_{in}, a) \wedge [\forall k \in \{1, \dots, i-1\} . minfirst(a, n, k)] \wedge minfirst(a, n, j-1)$ 
  @  $perm(a_{in}, a) \wedge \forall k \in \{1, \dots, i\} . minfirst(a, n, k)$ 
@  $perm(a_{in}, a) \wedge sorted(a, n)$ 
return

```

17.7 Conclusion

For proving correctness of function and procedure calls

- we assume that the function/procedure returns for every input the correct result, and
- prove correctness of the call under this assumption.

The function/procedure specification allows us to ignore its implementation!

Stepwise refinement:

1. What?
2. How?

Chapter 18

Modeling of Data Structures and Object-Oriented Programming

18.1 Object-Oriented Programming and Logic

A widely used term in computer science is the notion of an *abstract data type* which is usually defined as a data type with certain operations that is independent of any implementation. This is not a fully precise definition, since it does not formalize the notion of a data type. We can be a little more concrete, stating that an abstract data type is described by a signature and some axioms. Here is an example of a data type modeling a counter that can be initialized to a certain value that can be decreased, and whose current value can be retrieved.

Signature:

init: $\mathcal{N} \rightarrow \text{counter}$
val: $\text{counter} \rightarrow \mathcal{N}$
dec: $\text{counter} \rightarrow \text{counter}$

Axioms:

$$\forall n \in \mathcal{N} . \text{val}(\text{init}(n)) = n$$

$$\forall c \in \text{counter}, n \in \mathcal{N} . \left[\begin{array}{l} \text{val}(c) = 0 \Rightarrow \text{val}(\text{dec}(c)) = 0 \wedge \\ \text{val}(c) \neq 0 \Rightarrow \text{val}(\text{dec}(c)) = \text{val}(c) - 1 \end{array} \right]$$

At this point, we might remember, that we have already met something like this earlier. More concretely, in Definition 1 we defined a logical theory to be precisely a signature with some axioms. So logical theories describe nothing else than abstract data types! Another observation is the fact that many programming languages have some elements that were meant to describe abstract data types, for example, header files in C++ or interfaces in Java. However, these

language elements usually only define some sort of signature, without any axioms. Still, it is possible to add comments that either contain such axioms, or at least an informal description of the intended behavior of the abstract data type.

A programming paradigm that builds on abstract data types is object oriented (OO) programming. However, the notion of object oriented programming is even less precise. Roughly, it means a programming paradigm that provides some way of modeling abstract data types, together with dynamic binding. Moreover, it uses a specific notation that, from the point of view of logic, looks quite strange.

The notation for describing the signature from our example, might, for example, look like this:

```
class counter
constructor init( $n$ )
method val():  $\mathcal{N}$ 
method dec()
```

And when using a data type with such a signature, the assignment statement

$$i \leftarrow val(c)$$

might actually be written as

$$i \leftarrow c.val()$$

This opens the question of how one is supposed to write corresponding axioms. The axioms from above, that define the behavior of the counter may, for example, look as follows in OO notation:

$$\begin{aligned} & \forall n \in \mathcal{N} . init(n).val() = n \\ & \forall c . \left[\begin{array}{l} c.val() = 0 \Rightarrow c.dec().val() = 0 \wedge \\ c.val() \neq 0 \Rightarrow c.dec().val() = c.val() - 1 \end{array} \right] \end{aligned}$$

This, of course, does not follow the syntactical rules of the first-order predicate language. Still one might use such a notation when writing axioms as comments into OO programming. Moreover, when implementing such an abstract data type, it is again a good idea, to write assertions checking the implementation against the axioms, before actually starting with the implementation:

```
class counter
constructor init( $n$ )
  assume  $n \in \mathcal{N}$ 
  ...
  @ val() =  $n$ 
method val():  $\mathcal{N}$ 
method dec()
   $v \leftarrow val()$ 
  ...
  @ [ $v = 0 \Rightarrow val() = 0$ ]  $\wedge$  [ $v \neq 0 \Rightarrow val() = v - 1$ ]
```

The next question is how to write verification conditions corresponding to OO code. Consider the following code snippet, where the comments translate OO into traditional notation:

$c \leftarrow \text{init}(2)$	// $c \leftarrow \text{init}(2)$
$@\ c.\text{val}() = 2$	// $@\ \text{val}(c) = 2$
$v \leftarrow c.\text{val}()$	// $v \leftarrow \text{val}(c)$
$c.\text{dec}()$	// $c \leftarrow \text{dec}(c)$
$@\ c.\text{val}() = v - 1$	// $@\ \text{val}(c) = v - 1$

Then, in traditional notions, the verification conditions corresponding to the two basic paths of this code snippet, are the following:

- $c = \text{init}(2) \Rightarrow \text{val}(c) = 2$
- $[\text{val}(c) = 2 \wedge v = \text{val}(c) \wedge c_1 = \text{dec}(c)] \Rightarrow \text{val}(c_1) = v - 1$

In OO notation, this corresponds to the following:

- $c = \text{init}(2) \Rightarrow c.\text{val}() = 2$
- $[c.\text{val}() = 2 \wedge v = c.\text{val}() \wedge c_1 = c.\text{dec}()] \Rightarrow c_1.\text{val}() = v - 1$

Again, this is not correct syntax of the first order predicate language. Still, one might use it in a OO context for being consistent with OO notation.

When implementing data types, it is often necessary to preserve certain internal properties. For example, the implementation might use an internal variable x , not visible from the outside. And it might be the case, that a correct implementation will always ensure that this variable x is greater or equal zero. This would amount to an assertion at the beginning and end of every method. Some object oriented languages have explicit support for this. For example, in the programming language Eiffel, one would write

invariant $x \geq 0$

and as a result, the property $x \geq 0$ will be checked at the beginning and end of every method

All the things that we have seen in this chapter, can also be expressed (maybe even better) without OO constructions, and especially, without OO notation. The general paradigm that provides language constructs to ensure correctness of abstract data types is “design by contract” which is supported by more and more programming languages directly in the language (e.g., Eiffel, Racket), or based on language extensions (Microsoft Code Contracts, Java Modeling Language, ...).

18.2 Specification of Abstract Data Types

Consider this example of an specification of a FIFO Queue as a logical theory:

Signature:

emptyq: $\rightarrow \mathcal{Q}$
 enqueue: $\mathcal{N} \times \mathcal{Q} \rightarrow \mathcal{Q}$
 get: $\mathcal{Q} \rightarrow \mathcal{N}$
 dequeue: $\mathcal{Q} \rightarrow \mathcal{Q}$



Figure 18.1: Array Representation

Axioms:

$$\forall x \in \mathcal{N}, q \in \mathcal{Q} . \text{enqueue}(x, q) \neq \text{emptyq}()$$

$$\forall x \in \mathcal{N} . \text{get}(\text{enqueue}(x, \text{emptyq}())) = x$$

$$\forall x \in \mathcal{N}, q \in \mathcal{Q} . q \neq \text{emptyq}() \Rightarrow \text{get}(\text{enqueue}(x, q)) = \text{get}(q)$$

$$\forall x \in \mathcal{N} . \text{dequeue}(\text{enqueue}(x, \text{emptyq}())) = \text{emptyq}()$$

$$\begin{aligned} \forall x \in \mathcal{N}, q \in \mathcal{Q} . q \neq \text{emptyq}() \Rightarrow \\ \text{dequeue}(\text{enqueue}(x, q)) = \text{enqueue}(x, \text{dequeue}(q)) \end{aligned}$$

We first observe that these axioms leave the behavior of dequeue/get on an empty queue open. One possible way of using such axioms, is for simplifying expressions:

$$\text{get}(\text{enqueue}(7, \text{enqueue}(3, \text{emptyq}())))) = \text{get}(\text{enqueue}(3, \text{emptyq}())) = 3,$$

Axioms also allow further proofs of behavior of data type, independent of any implementation.

18.3 Implementation

One can now try to write an implementation of the data type. The terminology for this depends on the used programming language: For example in C++ or Java, the signature is fixed using an abstract class or interface. These languages do not provide explicit features for expressing the axioms that specify the behavior of an abstract data type. But, one can, of course, describe its behavior using comments. The implementation of an abstract data type is usually done using a concrete class, and the elements of a concrete type are then called objects.

We will now see how one can describe an implementation of an abstract data type in a way that is independent from a concrete programming language. By reducing the abstract data type to known data types, this allows for a correctness check with respect to the axioms.

We will discuss the principle again using the queue example, reducing it to the known data type of an array. The intuition will be, to represent, for example, the result of the operations

$$\text{enqueue}(1, \text{enqueue}(2, \text{enqueue}(3, \text{enqueue}(4, \text{enqueue}(5, \text{emptyq}())))))$$

by the array shown in Figure 18.1, where the length l of this array, is a global constant. For such a representation, we do not just need the array but, in addition, the numbers m and n from the figure, pointing to the beginning and end of the queue. Hence we define $\mathcal{Q} := \mathcal{A} \times \mathcal{N} \times \mathcal{N}$.

It is clear, that this representation is problematic: Our axioms do not restrict the number of enqueue operations in any way. However, in our representation we use the array elements from 0 to $l - 1$. We will come back to this problem later. However, there is another problem: In the figure, the choice of where to store the first element of the queue in the array is completely arbitrary. For example, we could also start at the beginning of the array, storing each element $i \in \{1, 2, 3, 4, 5\}$ at position $i - 1$. The problem is that this alternative representation is *different*, but represents the *same* queue. In other words, the equality predicate on triples from $\mathcal{A} \times \mathcal{N} \times \mathcal{N}$ will not work for comparing the queues that these triples represent. Hence we will extend our abstract data type with a type-specific equality predicate $=_{\mathcal{Q}}$:

Signature:

emptyq: $\rightarrow \mathcal{Q}$
 enqueue: $\mathcal{N} \times \mathcal{Q} \rightarrow \mathcal{Q}$
 get: $\mathcal{Q} \rightarrow \mathcal{N}$
 dequeue: $\mathcal{Q} \rightarrow \mathcal{Q}$
 $=_{\mathcal{Q}}$: $\mathcal{Q} \times \mathcal{Q}$

Axioms:

$$\begin{aligned} & \forall x \in \mathcal{N}, q \in \mathcal{Q} . \text{enqueue}(x, q) \neq_{\mathcal{Q}} \text{emptyq}() \\ & \forall x \in \mathcal{N} . \text{get}(\text{enqueue}(x, \text{emptyq}())) = x \\ & \forall x \in \mathcal{N}, q \in \mathcal{Q} . q \neq_{\mathcal{Q}} \text{emptyq}() \Rightarrow \text{get}(\text{enqueue}(x, q)) = \text{get}(q) \\ & \forall x \in \mathcal{N} . \text{dequeue}(\text{enqueue}(x, \text{emptyq}())) =_{\mathcal{Q}} \text{emptyq}() \\ & \forall x \in \mathcal{N}, q \in \mathcal{Q} . q \neq_{\mathcal{Q}} \text{emptyq}() \Rightarrow \\ & \quad \text{dequeue}(\text{enqueue}(x, q)) =_{\mathcal{Q}} \text{enqueue}(x, \text{dequeue}(q)) \end{aligned}$$

In order to ensure that this type-specific equality predicate will work in the same way as usual equality, we also add the usual axioms of reflexivity, symmetry, transitivity, and congruence for $=_{\mathcal{Q}}$.

Now finally, we can come up with an implementation. For this, we have to implement each function from the signature, and the equality predicate. Here, we can already exploit what we learned in the chapter on functions and procedures: To specify what these functions and the predicate should do, it suffices to specify their input/output behavior. Here is one possibility:

function emptyq : $\rightarrow \mathcal{Q}$

Output: $(a, 0, 0)$

function enqueue : $\mathcal{N} \times \mathcal{Q} \rightarrow \mathcal{Q}$

Input: $(x, (a, m, n))$

Output: $(\text{write}(a, (m - 1) \bmod l, x), (m - 1) \bmod l, n)$

function get : $\mathcal{Q} \rightarrow \mathcal{N}$

Input: (a, m, n)

Output: $a[(n - 1) \bmod l]$

function dequeue : $\mathcal{Q} \rightarrow \mathcal{Q}$

Input: (a, m, n)

Output: $(a, m, (n - 1) \bmod l)$

predicate `equal` : $\mathcal{Q} \times \mathcal{Q}$

Input: $(a_1, m_1, n_1), (a_2, m_2, n_2)$

Output: $(n_1 - m_1) \bmod l = (n_2 - m_2) \bmod l \wedge$
 $\forall i. 0 \leq i < (n_1 - m_1) \bmod l \Rightarrow a_1[(m_1 + i) \bmod l] = a_2[(m_2 + i) \bmod l]$

Most of this is self-explaining. The equality predicate compares whether the length of both queues is the same, and all elements are the same. This ensures that, for example, the empty queues $(a, 0, 0)$ and $(a, 3, 3)$ are the same.

Now we can implement these functions and predicates in an arbitrary programming language. We can use any language for this, and it might involve a huge number of lines of code. The advantage of our way of working with I/O specifications is that it makes checking correctness of the implementation at least partially independent from the concrete code.

The traditional way of ensuring correctness of the resulting implementation is testing. Here, we can use the axioms as background information for constructing test cases. For example, they can be used to extract test cases for unit testing by simply substituting specific constants (in our case, queues, and numbers) for the universally quantified variables.

Another way of ensuring correctness is an explicit correctness check. For this, we have to check the correctness of the two steps we have done:

1. Do the I/O specifications ensure the axioms?
2. Does the implementation ensure the I/O specifications?

The second item is nothing new for us—it can be done in the way discussed in the previous chapter. For the first item, we translate the function interfaces into logical formulas:

- `emptyq()` = $(a, 0, 0)$
- $\forall x \in \mathcal{N}, (a, m, n) \in \mathcal{Q} .$
 $\text{enqueue}(x, (a, m, n)) = (\text{write}(a, (m - 1) \bmod l, x), (m - 1) \bmod l, n)$
- $\forall (a, m, n) \in \mathcal{Q} . \text{get}((a, m, n)) = a[(n - 1) \bmod l]$
- $\forall (a, m, n) \in \mathcal{Q} . \text{dequeue}(a, m, n) = (a, m, (n - 1) \bmod l)$
- $\forall (a_1, m_1, n_1), (a_2, m_2, n_2) \in \mathcal{Q} \times \mathcal{Q} . (a_1, m_1, n_1) =_{\mathcal{Q}} (a_2, m_2, n_2) \Leftrightarrow$
 $(n_1 - m_1) \bmod l = (n_2 - m_2) \bmod l \wedge$
 $\forall i. 0 \leq i < (n_1 - m_1) \bmod l \Rightarrow a_1[(m_1 + i) \bmod l] = a_2[(m_2 + i) \bmod l]$

And now we can simply use these formulas as assumptions and prove the axioms based on those assumptions. We will not do this in all details, but in a semi-formal way. For example, we might want to check the axiom

$$\forall x \in \mathcal{N}, q \in \mathcal{Q} . \text{enqueue}(x, q) \neq_{\mathcal{Q}} \text{emptyq}().$$

Using the formulas describing the implementations of `enqueue` and `emptyq`, respectively, and handling the universal quantifiers in the obvious way, we get

$$(\text{write}(a, (m - 1) \bmod l, x), (m - 1) \bmod l, n) \neq_{\mathcal{Q}} (a, 0, 0).$$

Here, we immediately see a problem: If n is zero and m is one (which means that the array elements from 1 to $l - 1$ are used for storing elements), $(m - 1) \bmod l$

will also be zero, resulting in an empty queue. This is precisely the problem that we already observed when deciding about the array implementation: an overflow might occur. However, apart from this, everything seems to be OK, here.

Let us have a look at another axiom:

$$\forall x \in \mathcal{N} . \text{get}(\text{enqueue}(x, \text{emptyq}())) = x$$

We can rewrite this as follows:

$$\forall x \in \mathcal{N} . \text{get}(\text{enqueue}(x, (a, 0, 0))) = x$$

$$\forall x \in \mathcal{N} . \text{get}(\text{write}(a, -1 \bmod l, x), -1 \bmod l, 0) = x$$

$$\forall x \in \mathcal{N} . \text{write}(a, -1 \bmod l, x)[-1 \bmod l] = x$$

And this is true, as we know from the array axioms.

We did not check all queue axioms, but so far, everything looks OK. In general, the function interface formulas should imply the axioms from the abstract data type. In practice, one will usually not do this correctness check in a completely formally way. However, being able to do this check formally also improves the reliability of doing informal checks.

18.4 Logical Terminology

Above, we observed that abstract data types are more or less the same as the much older concept of a logical theory. But even the notion of an implementation of an abstract data type has a corresponding classical concept in logic! The first thing that an implementation must satisfy is that it implements the types and operations from the signature:

Definition 64 Let Σ be a signature over types $T_1, \dots, T_n$. A Σ -structure S is a tuple $(\Omega_1, \dots, \Omega_n, \mathcal{I})$ where

- $\Omega_1, \dots, \Omega_n$ are sets (the carrier sets of S)
- $\mathcal{I}$ assigns to
 - every function symbol $f : T_{i_1} \times \dots \times T_{i_a} \rightarrow T_j$ from Σ
a function $f : \Omega_{i_1} \times \dots \times \Omega_{i_a} \rightarrow \Omega_j$
 - every predicate symbol $p : T_{i_1} \times \dots \times T_{i_a}$ from Σ
a relation $r \subseteq \Omega_{i_1} \times \dots \times \Omega_{i_a}$

In the case of our queue example, misusing the symbol $\mathcal{Q}$ for the signature of our queue data type, our implementation corresponds to the $\mathcal{Q}$ -structure, $(\mathcal{A} \times \mathcal{N} \times \mathcal{N}, \{\text{emptyq} \mapsto \text{emptyq}, \text{enqueue} \mapsto \text{enqueue}, \text{get} \mapsto \text{get}, \text{dequeue} \mapsto \text{dequeue}, =_{\mathcal{Q}} \mapsto \text{equal}\})$. Note the two different fonts used here. We use teletype font to distinguish between the symbols from the signature and their corresponding implementation.

For people familiar with OO programming languages such as Java or C++, this has a simply intuition: In the case of just one carrier set (i.e., $n = 1$), a Σ -structure corresponds to a concrete class, where the elements of Ω_1 are the objects of this class.

The definition requires an implementation of both function and predicate symbols. Still, for simplicity, the examples in this chapter only have one single predicate, which is equality ($=$).

A given structure can satisfy certain properties. For example, in the structure $\mathbb{Z}_2$ that interprets the function symbol $+$ as addition modulo 2, the equality $1 + 1 = 0$ holds, whereas it does not hold in $\mathbb{Z}$ with $+$ interpreted as addition of integers. As another example, the rational numbers satisfy the axioms of group theory. However, rational numbers satisfy *more* properties than just the axioms of group theory, for example,

$$\forall x, y . x + y = y + x,$$

which is *not* an axiom of group theory. In logic, a structure that satisfies the axioms of a logical theory is called a *model*. The general intuition is this: An abstract data type defines some interface a data type has to satisfy, and a model is an implementation of such a data type.

software engineering	logic
interface (Java)	signature
(specification of) ADT	logical theory (signature+axioms)
concrete class	structure
object	element of carrier of structure
correct implementation	model
subtype	theory extension

Figure 18.2: Translation Table

Figure 18.2 gives a summarizing table for translating between terms from software engineering and corresponding terms from logic. Ignore the last line, for now, since it refers to something we will discuss below. While the definitions from logic are very useful, due to their precision, the usage of the word “model” is unfortunate. The reason is that this word means something completely different throughout science (a simplified abstract description of a complex object).

18.5 Theory Extensions

Sometimes one wants to add a function or predicate symbols to an existing specification, for example, one might want to

- add a length function to the queue specification, or
- add an error constant.

Sometimes we also want to add additional properties to an existing specification, for example, one might want to

- add the axiom `dequeue(emptyQ()) = emptyQ()` to the queue specification, or
- restrict groups to commutative groups

Such a situation is described by the following:

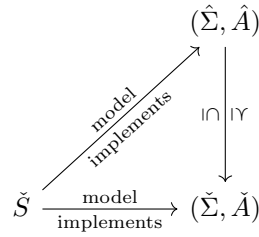


Figure 18.3: Theory Extensions and Their Implementation

Definition 65 Given two logical theories $(\check{\Sigma}, \check{A})$ and $(\hat{\Sigma}, \hat{A})$, $(\check{\Sigma}, \check{A})$ is an extension of $(\hat{\Sigma}, \hat{A})$ iff $\check{\Sigma} \supseteq \hat{\Sigma}$ and $\check{A} \supseteq \hat{A}$. In such a case, we will also write $(\check{\Sigma}, \check{A}) \preceq (\hat{\Sigma}, \hat{A})$.

Note that the symbol $\preceq$ has its open end on the other side of the superset relations $\supseteq$ used for defining it. The reason for this is that adding a function or predicate symbol to an abstract data type, or adding a new axiom, *restricts* the possibilities for implementing it. Intuitively, $(\check{\Sigma}, \check{A})$ is a sub-type of $(\hat{\Sigma}, \hat{A})$, which explains the last line in Figure 18.2.

Now assume two logical theories $(\check{\Sigma}, \check{A})$ and $(\hat{\Sigma}, \hat{A})$, with $(\check{\Sigma}, \check{A}) \preceq (\hat{\Sigma}, \hat{A})$ as on the right-hand side of Figure 18.3. Assume that we have a model $\check{S}$ of $(\check{\Sigma}, \check{A})$, that is, an implementation of the abstract data type described by $(\check{\Sigma}, \check{A})$. Then structure $\check{S}$ is also a model of $(\hat{\Sigma}, \hat{A})$, since $(\hat{\Sigma}, \hat{A})$ states requirements that are not as strong as $(\check{\Sigma}, \check{A})$ (strictly speaking, we have to restrict the operators of $\check{S}$ to the ones required by $(\hat{\Sigma}, \hat{A})$).

So to summarize, for two data types $(\check{\Sigma}, \check{A})$ and $(\hat{\Sigma}, \hat{A})$ with $(\check{\Sigma}, \check{A}) \preceq (\hat{\Sigma}, \hat{A})$, for every model $\check{S}$ of $(\check{\Sigma}, \check{A})$, the restriction of $\check{S}$ to $\hat{\Sigma}$ is a model of $(\hat{\Sigma}, \hat{A})$. This corresponds to the intuition that every implementation of $(\check{\Sigma}, \check{A})$ is also an implementation of $(\hat{\Sigma}, \hat{A})$.

We already mentioned, that theory extensions correspond to sub-types in object oriented programming. However, OO programming languages do not fully ensure Definition 65, since such programming languages do not use any axioms and hence do not know, which properties the programmer precisely expects for implemented data type.

Still, it is good programming style, to design types (i.e., classes) in such a way that subtypes should ensure the same properties as the supertype when used in the same way. Indeed this is nothing else than the famous Liskov substitution principle that was originally formulated as follows:

“Let $\phi(x)$ be a property provable about objects x of type T . Then $\phi(y)$ should be true for objects y of type S where S is a subtype of T ”. [29]

As already mentioned, OO programming languages ensure this programming languages only partially by checking certain syntactical requirements on subtypes (e.g., rules of co/contravariance). Theory extensions make all of this precise, with precise guarantees, but do not take into account usual phenomena of OO languages such as dynamic binding.

18.6 Definition of Abstract Data Types by Extension

Up to now we define abstract data types purely intensionally (based on their properties). But we already know many data types (i.e., logical theories). So why should we start from scratch each time we want to define a new data type? Would it not be easier to build new data type based on existing ones? For example, in Section 14.3.1, we defined strings as pairs of

- an array of characters, and
- the length of the string.

Let us look at another example here. We will define queues $\mathcal{Q}$ by extending lists $\mathcal{L}$.

Signature: $\mathcal{L}+$
 $\text{emptyq}: \rightarrow \mathcal{Q}$
 $\text{enqueue}: \mathcal{N} \times \mathcal{Q} \rightarrow \mathcal{Q}$
 $\text{get}: \mathcal{Q} \rightarrow \mathcal{N}$
 $\text{dequeue}: \mathcal{Q} \rightarrow \mathcal{Q}$

Axioms: $\mathcal{L}+$

$$\begin{aligned} \text{emptyq}() &= \text{empty} \\ \forall x \in \mathcal{N}, \forall q \in \mathcal{Q} . \text{enqueue}(x, q) &= \text{cons}(x, q) \\ \forall q \in \mathcal{Q} . q \neq \text{empty} &\Rightarrow \text{get}(q) = \text{last}(q) \\ \forall q \in \mathcal{Q} . q \neq \text{empty} &\Rightarrow \text{dequeue}(q) = \text{dropLast}(q) \end{aligned}$$

where last , dropLast are defined in $\mathcal{L}$, e.g. as follows:

$$\text{last}(\langle x \rangle) = x, l \neq \text{empty} \Rightarrow \text{last}(\text{cons}(x, l)) = \text{last}(l)$$

$$\text{dropLast}(\langle x \rangle) = \langle \rangle, l \neq \text{empty} \Rightarrow \text{dropLast}(\text{cons}(x, l)) = \text{cons}(x, \text{dropLast}(l))$$

For comparing this specification by theory extension with the intensional specification from above, we use the axioms of the respective specifications for simplifying expressions. In the specification by theory extension, we have:

$$\begin{aligned} \text{get}(\text{enqueue}(7, \text{enqueue}(3, \text{emptyq()}))) &= \\ \text{get}(\text{enqueue}(7, \text{enqueue}(3, \langle \rangle))) &= \\ \text{get}(\text{enqueue}(7, \langle 3 \rangle)) &= \\ \text{get}(\langle 7, 3 \rangle) &= 3 \end{aligned}$$

Using the intensional version, we get:

$$\begin{aligned} \text{get}(\text{enqueue}(7, \text{enqueue}(3, \text{emptyq()}))) &= \\ \text{get}(\text{enqueue}(3, \text{emptyq()})) &= 3 \end{aligned}$$

Another question we might ask is: Does the extending definition fulfill the intensional axioms? Let us look at the individual axioms. The first axioms is:

$$\forall x \in \mathcal{N}, q \in \mathcal{Q} . \text{enqueue}(x, q) \neq \text{emptyq}()$$

The axioms of the extending definition result in a simple proof:

$$\text{enqueue}(x, q) = \text{cons}(x, q) \neq \text{empty} = \text{emptyq}()$$

The second axiom

$$\forall x \in \mathcal{N} . \text{get}(\text{enqueue}(x, \text{emptyq}())) = x$$

has the following proof:

$$\begin{aligned} \text{get}(\text{enqueue}(x, \text{emptyq}())) &= \\ \text{get}(\text{enqueue}(x, <>)) &= \text{get}(<x>) = \text{last}(<x>) = x \end{aligned}$$

For proving the third axiom

$$\forall x \in \mathcal{N}, q \in \mathcal{Q} . q \neq \text{emptyq}() \Rightarrow \text{get}(\text{enqueue}(x, q)) = \text{get}(q)$$

we assume $q \neq \text{emptyq}()$ and prove $\text{get}(\text{enqueue}(x, q)) = \text{last}(\text{cons}(x, q)) = \text{last}(q) = \text{get}(q)$.

Finally, we will try to implement the queue. This immediately raises the question whether implementations must use lists, as well, since the extending specification was based on lists, where we implicitly assumed $\mathcal{Q}$ and $\mathcal{L}$ to be identical. Now we will drop this assumption and discuss an implementation without lists.

For this, we will use the array based implementation we already have met before. The intuition is to represent the list by an array, representing, for example, $\langle 1, 2, 3, 4, 5 \rangle$ by the array

4	5				1	2	3
		n				m	l-1

More concretely, we define the data type $\mathcal{Q}$ as $\mathcal{A} \times \mathcal{N} \times \mathcal{N}$, where (a, m, n) represents the list $\langle a[m], a[(m+1) \bmod l], \dots, a[n-1] \rangle$, for some constant l . Formally, a queue (a, m, n) corresponds to a list

$$\rho(a, m, n) := \begin{cases} \text{empty}, & \text{if } m = n, \text{ and} \\ \text{cons}(a[m], \rho(a, (m+1) \bmod l, n)), & \text{otherwise.} \end{cases}$$

Using the notation $i^- := (i-1) \bmod l$, we get the following implementation:

- $\text{emptyq}() := (a, 0, 0)$
- $\text{enqueue}(x, (a, m, n)) := (\text{write}(a, m^-, x), m^-, n)$
- $\text{get}(a, m, n) := a[n^-]$
- $\text{dequeue}(a, m, n) := (a, m, n^-)$

The question is whether this implementation is correct. The approach from above, checking whether the implementation using lists is a model of the logical theory of queues fails, since our definition of queues extends the signature of lists, and implicitly assumes each queue to be a list, whereas our implementation does not use lists. But is this relevant, at all? We use lists only as an auxiliary

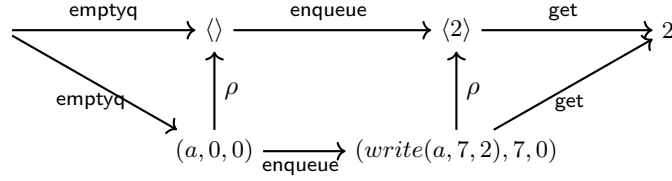


Figure 18.4: Expression Evaluation

structure that made it easy to define queues, but from the very beginning, we were interested in queues, not lists.

Let us look at an example. The top of Figure 18.4 shows the process of evaluating the term

$$\text{get}(\text{enqueue}(2, \text{emptyq}()))$$

using the extending specification, which results in

$$\text{get}(\text{enqueue}(2, \text{emptyq}())) = \text{get}(\text{enqueue}(2, \langle \rangle)) = \text{get}(\langle 2 \rangle) = 2.$$

Using our implementation (with $l = 8$), results in

$$\begin{aligned} \text{get}(\text{enqueue}(2, \text{emptyq}())) &= \text{get}(\text{enqueue}(2, (a, 0, 0))) = \\ &\text{get}(\text{write}(a, 7, 2), 7, 0) = \text{write}(a, 7, 2)[7] = 2 \end{aligned}$$

which corresponds to the path following the arrows through the bottom of Figure 18.4. The result is the same. Since we are not interested in the intermediate, but different computation steps, but only the final result, our implementing seems to be correct.

What is the general principle behind this? The key is in the representation function ρ . Each queue that is the result of one computation step in Figure 18.4 corresponds to a list assigned to the queue by ρ . As long as those corresponding lists satisfy the axioms of the specification, everything works as expected. Hence, instead of checking whether the implementation satisfies the axioms of the specification, we check whether the result of translating all queues in the implementation to the corresponding list, using the representation function, satisfies these axioms. So we check:

$$\rho(\text{emptyq}()) = \text{empty}$$

$$\forall x \in \mathcal{N}, q \in \mathcal{Q} . \rho(\text{enqueue}(x, q)) = \text{cons}(x, \rho(q))$$

$$\forall q \in \mathcal{Q} . \rho(q) \neq \text{empty} \Rightarrow \text{get}(q) = \text{last}(\rho(q))$$

$$\forall q \in \mathcal{Q} . \rho(q) \neq \text{empty} \Rightarrow \rho(\text{dequeue}(q)) = \text{dropLast}(\rho(q))$$

Clearly

$$\rho(\text{emptyq}()) = \rho(a, 0, 0) = \text{empty}$$

and hence the first axiom is o.k. For the second axiom, we check

$$\rho(\text{enqueue}(x, (a, m, n))) = \text{cons}(x, \rho(a, m, n)).$$

By definition,

$$\rho(\text{enqueue}(x, (a, m, n))) = \rho(\text{write}(a, m^-, x), m^-, n).$$

According to the definition of ρ , if $m^- \neq n$, this is

$$\text{cons}(\text{write}(a, m^-, x)[m^-], \rho(a, (m^- + 1) \bmod l, n)),$$

which, due to the array axioms and modular arithmetic is

$$\text{cons}(x, \rho(a, m, n)).$$

However, if $m^- = n$, we arrive at a problem. For example assume that a is

6	7		1	2	3	4	5
		n	m				

in which case $(\text{write}(a, m^-, x), m^-, n)$ is $(\text{write}(a, n, x), n, n)$ which, according to the definition of ρ , is the empty list, and certainly not the result of enqueue x . So we have bound a bug. Indeed this bug is to be expected, since the size of our array is finite, while we are using lists of unbounded length for specifying the behavior of queues. The bug is simply an overflow.

For proving

$$\rho(a, m, n) \neq \text{empty} \Rightarrow \text{get}(a, m, n) = \text{last}(\rho(a, m, n))$$

we assume $(a, m, n) \neq \text{empty}$, and prove $\text{get}(a, m, n) = \text{last}(\rho(a, m, n))$. By definition, $\text{get}(a, m, n) = a[n^-]$ and we prove that this is equal to $\text{last}(\rho(a, m, n))$.

If $m = n^-$, we have a queue containing one elements, that is

				m	n		

Then

$$\begin{aligned} \text{last}(\rho(a, m, n)) &= \text{last}(\text{cons}(a[m], \rho(a, n, n))) = \\ &= \text{last}(\text{cons}(a[m], \text{empty})) = a[m] = a[n^-]. \end{aligned}$$

Now we proceed by induction: We assume

$$\text{last}(\rho(a, m, n)) = a[n^-],$$

and prove

$$\text{last}(\rho(a, m^-, n)) = a[n^-].$$

We have:

$$\begin{aligned} \text{last}(\rho(a, m^-, n)) &= \text{last}(\text{cons}(a[m^-], \rho(a, m, n))) = \\ &= \text{last}(\rho(a, m, n)) = a[n^-], \end{aligned}$$

which is, what we wanted to prove, except if $m^- = n$ which again results in an overflow.

The proof of

$$\rho(a, m, n) \neq \text{empty} \Rightarrow \rho(\text{dequeue}(a, m, n)) = \text{dropLast}(\rho(a, m, n))$$

is similar.

18.7 Methods for Specification of Data Types

There are many more methods for specifying data types than the axiomatic method presented in this chapter. A general overview is the following:

- Property based:
 - axiomatic
 - algebraic (axioms restricted to equalities)

Example: Alloy (<https://alloytools.org>)

- Model based: list possible behavior instead of describing properties

Examples:

- Z notation
- Vienna Development Method (VDM)
- TLA+ (Amazon, Microsoft)

18.8 Conclusion

OO $\approx$ abstract data types = logical theories

Chapter 19

Program Termination

19.1 The Problem of Program Termination

Consider the following, innocent looking program:

```
assume  $x \geq 1$   
while  $x \neq 1$  do  
  if  $x \bmod 2 = 0$  then  
     $x \leftarrow x/2$   
  else  
     $x \leftarrow 3x + 1$ 
```

The if branch in the loop decreases the value of x , the else branch increases its value. The question is, whether for every initial value of x , its value will eventually decrease to one, which terminates the loop. This is the famous Collatz problem, and the answer to this question is unknown. If you either find an initial value of x for which you can show that the program will run forever, or show that the program will terminate for every input, you will be famous.

Indeed, the general problem of automatically checking whether a program terminates is the topic of a theorem by Alan Turing that set the foundations of theoretical computer science.

Theorem 3 ([37]) *The problem of checking termination of algorithms (the halting problem) is undecidable.*

Implicitly assuming that every program has a **return** statement after the very last program line, we define:

Definition 66 *A partially correct program P is terminating iff every regular execution of P reaches a program line with the statement **return**.*

One reason for why the halting problem is difficult is the fact that termination of a program does not imply that there is a constant upper bound on the number of loop iterations. For example, the following program terminates always, but the number of loop iterations can be arbitrarily high:

```
 $i \leftarrow 0$   
while  $i < n$  do  
   $i \leftarrow i + 1$ 
```

Before having a look at how to show that a given program terminates, we will discuss the relationship between program correctness and termination. For this, consider the two following programs:

<pre> assume $x = 564$ while $\top$ do $@ x = 564$ $@ x = 564$ return x </pre>	<pre> assume $x = 564$ while $\top$ do $@ x = 564$ $@ x = 632$ return x </pre>
---	---

Clearly, these program do not terminate. But are they correct? For this, we have to remind ourselves, what it means for a program to be correct, which is stated by Definition 60. Clearly, no execution of these programs results in the violation of an assertion. Hence both programs are partially correct. Moreover, our proof technique proves that, since all of the following verification conditions hold. Concretely, for the program on the left-hand side we have the verification conditions

- $[x = 564 \wedge \top] \Rightarrow x = 564$ (twice), and
- $[x = 564 \wedge \neg \top] \Rightarrow x = 564$ (twice),

and for the program on the right-hand side we have the conditions

- $[x = 564 \wedge \top] \Rightarrow x = 564$ (twice), and
- $[x = 564 \wedge \neg \top] \Rightarrow x = 632$ (twice).

Hence we want more than just partial correctness:

Definition 67 *A program is totally correct iff it is both partially correct and terminating.*

19.2 Termination Proofs

The technique for showing termination of programs that we will use hinges on relations of the following kind:

Definition 68 *A relation $\prec \subseteq \Omega \times \Omega$ is well-founded on $S \subseteq \Omega$ iff there is no infinite sequence $s_1 \in S, s_2 \in S, \dots$ such that $s_1 \succ s_2 \succ \dots$.*

As an example, consider the relation $<$ which is well founded on the natural numbers

$$5 > 4 > 3 > 2 > 1$$

but not on $\mathbb{R}^{\geq 0}$:

$$1 > \frac{1}{2} > \frac{1}{4} > \frac{1}{8} \dots$$

Usually such relations will also be orders, more concretely, strict partial orders.

Now let us look at an example. We first define: For $a \in \mathcal{A}$, $n \in \mathcal{N}$,

$$\text{sorted}(a, n) :\Leftrightarrow \forall i \in \{1, \dots, n-1\} . a[i] \leq a[i+1].$$

Now we will develop a program that searches an element x in a sorted array a . We will first write the program, and then analyze its termination. We start by writing the input/output specification as assertions:

```

assume  $a \in \mathcal{A}$ , sorted( $a, n$ ),  $x \in \mathcal{N}$ ,  $\exists i \in \{1, \dots, n\} . a[i] = x$ 
...
@  $a[r] = x$ 
return  $r$ 

```

Now the idea is to maintain a lower bound l and an upper bound u such that the element x is always contained in the array between those two indices, and then to decrease the gap between l and u :

```

assume  $a \in \mathcal{A}$ , sorted( $a, n$ ),  $x \in \mathcal{N}$ ,  $\exists i \in \{1, \dots, n\} . a[i] = x$ 
 $l \leftarrow 1$ ;  $u \leftarrow n$ 
while ... do
  @  $\exists i \in \{l, \dots, u\} . a[i] = x$ 
  ...
  @  $a[r] = x$ 
return  $r$ 

```

Within this process, we will always test one of the elements of a with index between l and u for whether it is equal to x . More concretely we will do this for the element $(l + u)/2$:

```

assume  $a \in \mathcal{A}$ , sorted( $a, n$ ),  $x \in \mathcal{N}$ ,  $\exists i \in \{1, \dots, n\} . a[i] = x$ 
 $l \leftarrow 1$ ;  $u \leftarrow n$ 
while  $a[(l + u)/2] \neq x$  do
  @  $\exists i \in \{l, \dots, u\} . a[i] = x$ 
  ...
  @  $a[(l + u)/2] = x$ 
   $r \leftarrow (l + u)/2$ 
  @  $a[r] = x$ 
return  $r$ 

```

Now it remains to update the bounds l and u within the loop in such a way that the loop invariant is always preserved and such that the loop eventually terminates.

```

assume  $a \in \mathcal{A}$ , sorted( $a, n$ ),  $x \in \mathcal{N}$ ,  $\exists i \in \{1, \dots, n\} . a[i] = x$ 
 $l \leftarrow 1$ ;  $u \leftarrow n$ 
while  $a[(l + u)/2] \neq x$  do
  @  $\exists i \in \{l, \dots, u\} . a[i] = x$ 
  if  $a[(l + u)/2] < x$  then  $l \leftarrow (l + u)/2 + 1$ 
  else  $u \leftarrow (l + u)/2 - 1$ 
  @  $a[(l + u)/2] = x$ 
   $r \leftarrow (l + u)/2$ 
  @  $a[r] = x$ 
return  $r$ 

```

The following basic path corresponds to one loop iteration from the invariant back to itself:

```

assume  $\exists i \in \{l, \dots, u\} . a[i] = x$ 
assume  $a[(l + u)/2] < x$ 

```

```

 $l \leftarrow (l + u)/2 + 1$ 
assume  $a[(l + u)/2] \neq x$ 
@  $\exists i \in \{l, \dots, u\} . a[i] = x$ 

```

The corresponding verification condition is

$$[\exists i \in \{l, \dots, u\} . a[i] = x \wedge a[(l + u)/2] < x \wedge l_1 = (l + u)/2 + 1 \wedge \dots] \Rightarrow \exists i \in \{l_1, \dots, u\} . a[i] = x$$

Does this verification condition hold? The answer is no. It only holds only under assumption of the array being sorted, that is, $\text{sorted}(a, n)$. Indeed, we have this condition as an assumption at the beginning of the program. Formally, we would have to add it to all assertions. However, this would clutter the assertions. And since the program does not change the array a , it is clear that this condition is preserved throughout the program, and we will not write it explicitly.

Let us now analyze termination of the program. A basic observation is that a program is terminating if and only each each loop has only finitely many iterations. Therefore, it will be enough for us, to study termination of loops. For this, let us return to our example program. Why does its loop terminate?

The reason is, of course, that the distance between l and u decrease in each iteration, but l cannot jump over u . More formally, $u - l$ decreases in each iterations, but always $u - l \geq 1$. This must terminate since $<$ is well founded on the natural numbers.

To prove that this is indeed the case, we will now write additional assertions into the loop that ensure the necessary properties.

```

 $v \leftarrow \infty$ 
while  $a[(l + u)/2] \neq x$  do
  @  $u - l < v \wedge u - l \geq 1 \wedge \exists i \in \{l, \dots, u\} . a[i] = x$ 
   $v \leftarrow u - l$ 
  if  $a[(l + u)/2] < x$  then  $l \leftarrow (l + u)/2 + 1$ 
  else  $u \leftarrow (l + u)/2 - 1$ 

```

In order to be able to compare the value of the term $u - v$ between two loop iterations, we remember its previous value using an auxiliary variable v . Initially we set its value to infinity, which makes the check $u - l < v$ succeed trivially.

The basic path corresponding to one loop iteration through the **if** branch now looks as follows:

```

assume  $u - l < v \wedge u - l \geq 1 \wedge \exists i \in \{l, \dots, u\} . a[i] = x$ 
 $v \leftarrow u - l$ 
assume  $a[(l + u)/2] < x$ 
 $l \leftarrow (l + u)/2 + 1$ 
assume  $a[(l + u)/2] \neq x$  do
  @  $u - l < v \wedge u - l \geq 1 \wedge \exists i \in \{l, \dots, u\} . a[i] = x$ 

```

We will not analyze the full verification condition, but only the parts that prove termination. The term $u - l$ decreases since

$$[u - l \geq 1 \wedge v = u - l \wedge l_1 = (l + u)/2 + 1] \Rightarrow u - l_1 < v$$

which can be proved automatically by tools such as the SMT solver CVC4. Still, we also provide a proof, here. However, we have to be careful, since division refers to integer division, here, which may result in a remainder.

Proof. We assume $u - l \geq 1$, $v = u - l$, $l_1 = \frac{l+u}{2} + 1$ and prove $u - l_1 < v$. For this, it suffices to prove $u - \frac{l+u}{2} + 1 < u - l$. Subtracting the upper bound u from both sides, and multiplying by -1 we get

$$\frac{l+u}{2} + 1 > l$$

Since $\frac{l+u}{2} + 1$ as a function in u monotonically increasing it suffices to prove this inequality for $u = l + 1$ in which case

$$\frac{l+u}{2} + 1 = \frac{2l+1}{2} + 1 = l + 1 > l.$$

■

For proving that $u - l \geq 1$ is an invariant, again in the case of the if branch, we will not bother to write down the verification condition, but argue directly on the basic path, instead:

- If $u - l = 1$, then the last **assume** fails (i.e., the loop terminates).
- If $u - l = 2$, then, at the final assertion $u - l = 1$.
- If $u - l > 2$, then the distance will be larger than in the previous case.

Summarizing, for proving termination of a program, we need for each loop

- a term t over program variables denoting a function to a set S , and
- a well-founded relation $\prec$ on S such that

the value of t decreases for each execution of the loop wrt. $\prec$. Such a term is called a *loop variant* (also *ranking function*).

The necessary conditions are ensured by adding the following assignments and assertions:

```

v ← ∞
while ... do
  @ v > t ∧ t ∈ S
  v ← t
  ...

```

where v is a new, auxiliary variables (for each loop a different one), and ∞ is a constant such that for every $x \in S$, $\infty > x$.

The same technique can be used for nested loops, which we will demonstrate on an example. Consider the problem

Input: $a \in \mathcal{A}, n \in \mathcal{N}, s \in \mathcal{A}, l \in \mathcal{N}$

Output: p , s.t. $\text{substr}(a, n, s, l, p)$, if such a p , exists, and 0, otherwise.

where

$$\text{substr}(a, n, s, l, p) :\Leftrightarrow \forall i \in \{1, \dots, l\} . p + i - 1 \leq n \wedge a[p + i - 1] = s[i].$$

Here is a program that solves this problem

```

@  $a \in \mathcal{A}, n \in \mathcal{N}, s \in \mathcal{A}, l \in \mathcal{N}$ 
 $\bar{p} \leftarrow 0; i \leftarrow 0$ 
while  $i \leq l \wedge \bar{p} + l \leq n$  do
  @  $\bar{p} \leq n - l \wedge \neg \exists p \in \{1, \dots, \bar{p}\} . \text{substr}(a, n, s, l, p)$ 
   $\bar{p} \leftarrow \bar{p} + 1; i \leftarrow 1$ 
  while  $i \leq l \wedge a[\bar{p} + i - 1] = s[i]$  do
    @  $i \leq l \wedge \text{substr}(a, n, s, i, \bar{p})$ 
     $i \leftarrow i + 1$ 
if  $i > l$  then
  @  $\text{substr}(a, n, s, l, \bar{p})$ 
  return  $\bar{p}$ 
else
  @  $\neg \exists p . \text{substr}(a, n, s, l, p)$ 
  return 0

```

To make sure that the program terminates, we use the technique from above, using an auxiliary variable w_1 for the outer loop, and w_2 for the inner loop. Different from above, we do not use the less than relation $<$ on $\mathbb{N}_0$, but the greater than relation $>$ on a set that is bounded from above. Hence we need to initialize the auxiliary variables with $-\infty$, instead of ∞ .

```

@  $a \in \mathcal{A}, n \in \mathcal{N}, s \in \mathcal{A}, l \in \mathcal{N}$ 
 $\bar{p} \leftarrow 0; i \leftarrow 0$ 
 $w_1 \leftarrow -\infty$ 
while  $i \leq l \wedge \bar{p} + l \leq n$  do
  @  $w_1 < \bar{p} \wedge \bar{p} \leq n - l \wedge \neg \exists p \in \{1, \dots, \bar{p}\} . \text{substr}(a, n, s, l, p)$ 
   $w_1 \leftarrow \bar{p}; \bar{p} \leftarrow \bar{p} + 1; i \leftarrow 1$ 
  while  $i \leq l \wedge a[\bar{p} + i - 1] = s[i]$  do
     $w_2 \leftarrow -\infty$ 
    @  $w_2 < i \wedge i \leq l \wedge \text{substr}(a, n, s, i, \bar{p})$ 
     $w_2 \leftarrow i; i \leftarrow i + 1$ 
if  $i > l$  then
  @  $\text{substr}(a, n, s, l, \bar{p})$ 
  return  $\bar{p}$ 
else
  @  $\neg \exists p . \text{substr}(a, n, s, l, p)$ 
  return 0

```

19.3 Connections to Different Areas

This proof technique is related to various similar ones in different areas. In temporal logic (see Section 6), termination of programs corresponds to a formula of the form $\mathbf{F} pc = r$, where r is the program line with the command **return**. And a similar technique can be used for proving such formulas (see Section 7.3).

For continuous systems an analogous object to loop variants is used, which is called a *Lyapunov function*.

Another question is the connection of loop variants to the computational complexity of algorithms. If we have a variant for the relation $<$ on natural numbers, we cannot execute the corresponding loop more often than the initial value of the variant. Hence this value is an upper bound on the run-time complexity of an algorithm.

Yet another question concerns the completeness of the method: Does there exist a variant for every terminating algorithm? In a certain sense, the answer is yes, since the number of loop iterations before termination satisfies the necessary conditions: This number decreases for every iteration, and always stays greater or equal zero. The problem, however, is that we do not know how to write this number as a simple term. For algorithms whose termination is not known, we do not know whether this number is finite.

19.4 Termination of Recursion

Now we will adapt the method to the termination of recursive programs. We already met the following program for computing the greatest common divisor.

```
GCD(x, y)
@  $x, y \in \mathcal{N}, x \geq y$ 
if  $y = 0$  then
     $r \leftarrow x$ 
else
     $r \leftarrow \text{GCD}(y, x \bmod y)$ 
return  $r$ 
```

We will again work with a well-founded relation, looking for a variant, a term, whose value decreases with every recursive call, but always stays in the set of natural numbers $\mathbb{N}_0$. Clearly, the term consisting only of the variable y satisfies this.

Now we again write assertions into the program that ensure that y is indeed a variant. For this, we remember the value of the variant at the beginning of the function, and compare it with its value at the point of the recursive call. Since the first argument to the recursive call is y (which the call assigns to x), and the second argument is $x \bmod y$ (which the call assigns to y), the value of the variant at the point of the recursive call is $y[x \leftarrow y, y \leftarrow x \bmod y]$, which is $x \bmod y$. Hence after adding assertions ensuring that y is a variant, the program looks as follows:

```
GCD(x, y)
@  $x, y \in \mathcal{N}, x \geq y$ 
 $v \leftarrow y$  // variant  $y$ 
if  $y = 0$  then
     $r \leftarrow x$ 
else
    @  $x \bmod y < v \wedge x \bmod y \geq 0$ 
     $r \leftarrow \text{GCD}(y, x \bmod y)$ 
return  $r$ 
```

The corresponding verification condition is

$$[x \geq y \wedge v = y \wedge y \neq 0] \Rightarrow [x \bmod y < v \wedge x \bmod y \geq 0]$$

19.5 Automation

Even though Theorem 3 states that a complete automation of checking termination is impossible, there are algorithms that can successfully check termination of many practical programs. If we have a variant, the corresponding verification conditions can often be proved automatically (e.g., using CVC4). So the remaining question is whether it is possible to find variants automatically. Here, we will have a look at a technique that is used by the T2 Temporal Prover¹ developed by Microsoft Research [15].

As an example, consider the following program:

```
while  $10 \leq x \wedge x \leq 20 \wedge 10 \leq y \wedge y \leq 20$  do
   $x \leftarrow x - y$ 
   $y \leftarrow x + 2y$ 
```

At this moment, we do not know whether the loop terminates. Hence we also do not know, whether it has a variant. But let us simply assume that it has a variant that is a linear function in x and y . Such a linear function can be denoted by a term of the form $ax + by$, where a and b are unknown constants. In mathematics, such a term is called an *ansatz* (a word of German origin), and in formal verification a *template*.

We will use the well-founded relation $\prec \subseteq \mathbb{R}^{\geq 0} \times \mathbb{R}^{\geq 0}$ s.t. $u \prec v$ iff $u \leq v - 1$. Writing corresponding assertions into the program results in

```
 $v \leftarrow \infty$ 
while  $10 \leq x \wedge x \leq 20 \wedge 10 \leq y \wedge y \leq 20$  do
   $@ v - 1 \geq ax + by \wedge ax + by \geq 0$ 
   $v \leftarrow ax + by$ 
   $x \leftarrow x - y$ 
   $y \leftarrow x + 2y$ 
```

The corresponding verification conditions, after some trivial simplifications, are

$$[10 \leq x \wedge x \leq 20 \wedge 10 \leq y \wedge y \leq 20] \Rightarrow ax + by \geq 0$$

and

$$\left[\begin{array}{l} ax + by \geq 0 \wedge \\ v = ax + by \wedge \\ x_1 = x - y \wedge y_1 = x + 2y \wedge \\ 10 \leq x_1 \wedge x_1 \leq 20 \wedge 10 \leq y_1 \wedge y_1 \leq 20 \end{array} \right] \Rightarrow [ax_1 + by_1 \leq v - 1 \wedge ax_1 + by_1 \geq 0]$$

We have to find a, b , such that for all x, y, v, x_1, y_1 the conditions hold. This is now a purely algebraic problem and there is mathematical software for solving these conditions.

¹<http://mmjb.github.io/T2/>

If the conditions do not have a solution, what does this imply? This does *not* imply non-termination: We only know that there is no variant of the given form, but there may be a variant of a different form. Hence we may try another template, for example increasing the degree of the polynomial, which results in $ax^2 + bcy + ex + fy$.

19.6 A Useful Well-Founded Relation

For termination proofs, one can use any well-founded relation. Sometimes, it is even convenient, to define a new well-founded relation. But there are also certain well-founded relations have shown to be very useful for termination proofs. We will demonstrate this on the following program.

```

while  $i \neq 0$  do
  if  $i > 0$  then
     $i \leftarrow -i$ 
  else
     $i \leftarrow i + 1$ 

```

For finding a loop variant, have a look at the following table that shows the values of the variable i , its sign, and absolute value for a certain execution of the program.

i	$sgn(i)$	$ i $
3	1	3
-3	-1	3
-2	-1	2
-1	-1	1

As we can see, none of the columns would work as a variant, on its own. However, we can combine several terms into a tuple and order the resulting tuples using the lexicographic order. Intuitively, this relation corresponds to the order used to order names of people, for example in phone books (in case anybody remembers). For given relations $\prec_1, \dots, \prec_n$ it is defined as

$$(s_1, \dots, s_n) \prec (t_1, \dots, t_n) :\Leftrightarrow \bigvee_{i=1}^n \left(\bigwedge_{j=1}^{i-1} s_j = t_j \wedge s_i \prec_i t_i \right)$$

Another, less anachronistic example, is the comparison of 8-bit numbers using individual bits, and the order $0 \prec_i 1$ on the individual bits. Then, for example,

$$(0, 1, 0, 1, 0, 1, 1, 1) < (0, 1, 0, 1, 1, 0, 0, 0)$$

since the first four bits are the same, and the fifth bit is smaller in $(0, 1, 0, 1, 0, 1, 1, 1)$ than in $(0, 1, 0, 1, 1, 0, 0, 0)$.

Well-foundedness of the relation used for the individual tuple entries implies well-foundedness of the resulting lexicographic relation:

Property 10 *If $\prec_1, \dots, \prec_n$ are well-founded relations on $S_1, \dots, S_n$, respectively then the corresponding lexicographic order $\prec$ is also a well-founded relation on $S_1 \times \dots \times S_n$.*

Now let us apply this to our example. Analyzing the table once more, we see that the values in the column $\text{sign}(i)$ never increase, and whenever they stay the same, the value in the column $|i|$ decreases. Hence we can use the pair $(\text{sign}(i), |i|)$ as a variant, together with the lexicographic relation wrt. $<$ and $<$, which is a well-founded relation over the set $\{(s, n) \mid s \in \{-1, 1\}, n \in \mathbb{N}_0\}$. Hence the program must terminate.

19.7 Conclusion

- Variants give evidence for the termination of programs
- The corresponding conditions can be checked at run-time using assertions
- Termination proofs can be partially automatized.
- However, in general, termination is undecidable

Chapter 20

Symbolic Execution

The motivation for the technique from this section is the desire to cover programs as completely as possible with test cases. In many applications, there are indeed certain rules for this, so called *coverage criteria*.

We now define an *execution path* as a sequence of program lines, and state the main question of this section as: How can we cover with test cases as many execution paths as possible?

Let us start with the following example of a program that we will denote by P :

```
1: while  $x \geq 0$  do  
2:   input  $x$   
3:   if  $2x - 1 \geq 3$  then  
4:      $x \leftarrow x - 2$   
5:   else  
6:      $x \leftarrow x - 1$ 
```

Let us assume that we want to follow the execution path 1, 2, 3, 4, 1, 2, 3, 6. A naive way of attacking this question would be to collect all assignments and all conditions that have to hold when following the execution path. This results in the formula

$$x \geq 0 \wedge 2x - 1 \geq 3 \wedge x = x - 2 \wedge x \geq 0 \wedge 2x - 1 < 3.$$

However, it can be immediately seen that this formula is not satisfiable, since there is no value for x such that $x = x - 2$ holds. And indeed, throughout execution of the program, the program variable x can take several different values. Hence, we use different variable names for all different values the program variable x may take, resulting in the formula

$$x_1 \geq 0 \wedge 2x_2 - 1 \geq 3 \wedge x_3 = x_2 - 2 \wedge x_3 \geq 0 \wedge 2x_4 - 1 < 3.$$

A solver can then find a test case or prove that none exists.

In order to formalize this approach at least partially, consider the basic path resulting from the program lines 1, 2, 3, 4, 1, 2, 3, 6 of the execution path under consideration, and a final assertion @ $\perp$:

```

1: assume  $x \geq 0$ 
2: input  $x$ 
3: assume  $2x - 1 \geq 3$ 
4:  $x \leftarrow x - 2$ 
1: assume  $x \geq 0$ 
2: input  $x$ 
3: assume  $\neg 2x - 1 \geq 3$ 
  @  $\perp$ 

```

Denoting this basic path by $BP_{1,2,3,4,1,2,3,6}(P)$ we observe that whenever it is possible to execute $BP_{1,2,3,4,1,2,3,6}(P)$, the assertion at its last line will fail. Hence, the program P can follow the execution path 1, 2, 3, 4, 1, 2, 3, 6 iff the basic path $BP_{1,2,3,4,1,2,3,6}(P)$ is incorrect. Since the correctness of basic paths is expressed by verification conditions, the corresponding verification condition $VC(BP_{1,2,3,4,1,2,3,6}(P))$ which is

$$[x_1 \geq 0 \wedge 2x_2 - 1 \geq 3 \wedge x_3 = x_2 - 2 \wedge x_3 \geq 0 \wedge 2x_4 - 1 < 3] \Rightarrow \perp$$

must not hold. This means that its negation $\neg VC(BP_{1,2,3,4,1,2,3,6}(P))$ that is

$$\neg[[x_1 \geq 0 \wedge 2x_2 - 1 \geq 3 \wedge x_3 = x_2 - 2 \wedge x_3 \geq 0 \wedge 2x_4 - 1 < 3] \Rightarrow \perp]$$

must be satisfiable. Now we observe that this is equivalent to

$$x_1 \geq 0 \wedge 2x_2 - 1 \geq 3 \wedge x_3 = x_2 - 2 \wedge x_3 \geq 0 \wedge 2x_4 - 1 < 3$$

which indeed is the formula from the beginning of this section. If this formula is satisfiable, then any satisfying assignment results in a test case.

Summarizing, for a program P , we define $BP_{l_1, \dots, l_n}(P)$ as the basic path that

- consists of the program lines $l_1, \dots, l_n$, with all control structures replaced by **assume**-assertions with conditions that must be satisfied when following the execution path $l_1, \dots, l_n$, and that
- has @ $\perp$ as its final assertion.

Then a program P can follow the execution path $l_1, \dots, l_n$ iff

$$\neg VC(BP_{l_1, \dots, l_n l}(P))$$

is satisfiable. We also observed that $\neg VC(BP_{l_1, \dots, l_n l}(P))$ can be written equivalently as a conjunction (see also the notion of “error condition” discussed in Section 15.8). We will use the notation $SE_{l_1, \dots, l_n}(BP_{l_1, \dots, l_n l}(P))$ for the resulting formula.

To use this technique for covering program with test cases, we can proceed as follows:

- Choose a set of execution paths
- For every execution path $l_1, \dots, l_n$, find the corresponding test case by computing a satisfiable assignment of $SE_{l_1, \dots, l_n}(BP_{l_1, \dots, l_n l}(P))$.

Here, the two following problems might turn up:

- The formula $SE(BP_{l_1, \dots, l_n}(P))$ might belong to an undecidable theory (e.g., due to non-linear operations on integers)
- The program may use external functions with unknown source code (e.g., from the operating system)

For solving these problems, let us consider another example:

```

...
x ← 2x + 1
r ← ext(x)
assume r > 7
y ← xy + z
assume y > 3
...

```

The assume assertions in this basic path might, for example, correspond to one branch of an **if – then – else** statement. A condition for executing this basic path would be

$$x_2 = 2x_1 + 1 \wedge r = \text{ext}(x_2) \wedge r > 7 \wedge y_2 = x_2y_1 + z \wedge y_2 > 3$$

where $\text{ext}()$ can, for example read a sensor in a nuclear power plant, call a function on super-computer, install some software on 2000 computers etc. Clearly no solver can handle this. The only way to obtain some information in this situation is to use some form of approximation of the behavior of this function. The first attempt will be to use over-approximation, for example simply replacing the resulting atomic formulas by the tautology $\top$:

$$x_2 = 2x_1 + 1 \wedge \top \wedge r > 7 \wedge y_2 = x_2y_1 + z \wedge y_2 > 3$$

Clearly, the resulting constraint is weaker than the original one which means that if it is unsatisfiable, this implies unsatisfiability of the original formula. Moreover, any satisfying assignment (i.e., a solution) is not necessarily a satisfying assignment of the original formula. If our goal would be verification, where unsatisfiability means absence of an error, this is what we want. However, we want to find test cases, and the satisfying assignment probably does not result in a test cases that follows the wanted program lines. Hence over-approximation is not what we want here.

Now let us try under-approximation. Replacing the atomic formula $r = \text{ext}(x_2)$ by $\perp$ results in

$$x_2 = 2x_1 + 1 \wedge \perp \wedge r > 7 \wedge y_2 = x_2y_1 + z \wedge y_2 > 3$$

which is an unsatisfiable formula. But maybe we can come up with a better under-approximation? For this we observe that we can execute the external function ext for arbitrary inputs! So let us compute the result for some random input, e.g., $x_2 = 0$. Assuming that the result is zero, we can replace the external function call by $x_2 = 0 \wedge r = 0$, which results in

$$x_2 = 2x_1 + 1 \wedge x_2 = 0 \wedge r = 0 \wedge r > 7 \wedge y_2 = x_2y_1 + z \wedge y_2 > 3.$$

Even though $x_2 = 0 \wedge r = 0$ is a better approximation to the external function call than $\perp$, this is still not good enough. The problem is that—under the

condition $x_2 = 0$ —the part $x_2 = 2x_1 + 1$ does not have a solution (if x_1 is an integer).

Thus we search for a solution of $x_2 = 2x_1 + 1$ *first*, and only then we compute $\text{ext}(x_2)$ and so on.

Now let us demonstrate the overall process. We follow the given program through the desired program lines, noting the necessary conditions as we go. We first note $x_2 = 2x_1 + 1$, but on the next line we encounter the external function $\text{ext}()$. For its execution we need input, a value for the program variable x_2 . So we solve $x_2 = 2x_1 + 1$. An example solution is $x_1 \mapsto 2, x_2 \mapsto 5$. Now we can execute the external function for the given value of x_2 , that is, we execute $\text{ext}(5)$. Let us assume that the result is 13. Now, instead of $r = \text{ext}(x_2)$ we add the conditions $x_2 = 5$ and $r = 13$ which results in $x_2 = 2x_1 + 1 \wedge x_2 = 5 \wedge r = 13$. Now we can continue with symbolic execution, step by step building up the formula

$$x_2 = 2x_1 + 1 \wedge x_2 = 5 \wedge r = 13 \wedge r > 7 \wedge y_2 = x_2 y_1 + z \wedge y_2 > 3.$$

We will now formalize this a little bit more. Our goal is a function satisfying the following specification:

Input: basic path P (may contain external function calls)

Output: formula ϕ such that every solution of ϕ corresponds to a regular program execution of P

Here we will assume, that external function calls do not have any side effects. In other words, they do not use any other information than the information passed to them using arguments, and they do not influence P in any other way than over the returned result.

We implement the specification by calling for a given program P a function $\text{trans}(L, \top)$, where L is a list of pairs (c, c') where

- c corresponds to the program lines in P , and
- c' is the corresponding atomic sub-formula of $SE(P)$.

The function trans is defined as follows:

- $\text{trans}(\langle \rangle, \phi) := \phi$
- $\text{trans}(\text{cons}((v \leftarrow \text{ext}(x), c'), r), \phi) := \text{trans}(r, x = \nu_x \wedge v = \nu_v \wedge \phi)$, where
 - ν_x is the value of x for a solution of ϕ , and
 - ν_v result of the executing $\text{ext}(\nu_x)$
- $\text{trans}(\text{cons}((c, c'), r), \phi) := \text{trans}(r, c' \wedge \phi)$

This can be extended to a method that goes both under the name “Dynamic Test Generation” and “Concolic Testing”. The main difference to our formalization is that concolic testing also allows side effects. For this, it executes the program both concretely and symbolically, and creates the logical formula $SE(BP_{l_1, \dots, l_n}(P))$ line by line. As soon as formula creation arrives at a function that the solver cannot handle, it runs the program until it receives the result of the function. If we need user inputs for this, the method computes them by

solving the symbolic formula (the part, that corresponds to the program before the external function). Then it uses the result of the concrete execution to replace the external function call by corresponding equalities

An interesting question is, whether such a method is complete: Does the method always find a test case following a given path, if it exists? The answer is, of course, no, since we compute the result only approximately. If in the above example, the result of executing `ext(5)` is 6, and of `ext(7)` it is 13, and we again choose $x_2 = 5$ as a solution of $x_2 = 2x_1 + 1$ we add $x_2 = 5, r = 6$, and the resulting formula does not have a solution, although the original formula had one. We would have found the solution, if we would have chosen $x_2 = 7$ instead of $x_2 = 5$.

When using symbolic execution for test case generation, one possible strategy is to cover a tree of paths (either if or else, either we stay in loop, or we leave it) up to a certain depth. This may result in the so-called path explosion problem.

For example, the program

```
@  $\forall i \in \{1, \dots, 1000\} . a[i] \in \{0, 1\}$ 
for  $i \leftarrow 1$  to 1000 do
  if  $a[i] = 0$  then
    ...
  else
    ...
```

has 2^{1000} execution paths. In such a case the coverage of the full tree of execution paths up to a practically relevant depth is unrealistic. Possible solutions to this problem is to only choose certain paths, or to merge some paths.

Note that if we would have loop invariants, the whole problem would not arise, since we would only have to analyze basic paths from one loop iteration to the next. In other words, loop invariants localize reasoning to single loop iterations. Since we do rely on loop invariants here, we have to cover a huge number of possibilities.

Another strategy, introduced by Microsoft under the name SAGE: White-box Fuzzing [21], is to replace depth-first search of execution paths by local search. The basic observation is that often some database of test cases is already available. The method cover neighboring paths by negating the individual conditions. The method also handles several technological issues in a clever way. For example, it is independent of a certain programming language, since it works on machine code.

This method regularly finds security bugs (e.g., buffer overflow), which saves security updates, that Microsoft would have to send to millions of computers, worldwide (<https://en.wikipedia.org/wiki/OneFuzz>).

Many further symbolic execution tools are available. Some for specific programming languages, some for LLVM intermediate representation, see the Wikipedia articles “Symbolic execution”, “Concolic testing” for more details. There are also several survey articles [10, 4].

A related technique is symbolic execution of timed automata, where a program line corresponds to a location of a timed automaton (see Section 10.5).

Chapter 21

Bounded Software Model Checking

The basic idea of bounded software model checking is, that loops can be unrolled. Consider the following example program:

```
s ← 0
i ← 0
while i ≤ n do
  s ← s + a[i]
  i ← i + 1
i ← 0
while i ≤ n do
  s ← s - a[i]
  i ← i + 1
@ s ≤ 0
```

If we replace each loop by an **assume** statement that ensures the behavior corresponding to skipping the loop, we arrive at

```
s ← 0
i ← 0
assume ¬i ≤ n
i ← 0
assume ¬i ≤ n
@ s ≤ 0
```

If we replace each loop by a single **if-then** statement, and add an **assume** statement that ensures the behavior corresponding to leaving the loop, we arrive at

```
s ← 0
i ← 0
if i ≤ n then
  s ← s + a[i]
  i ← i + 1
  assume ¬i ≤ n
```

```

 $i \leftarrow 0$ 
if  $i \leq n$  then
   $s \leftarrow s - a[i]$ 
   $i \leftarrow i + 1$ 
  assume  $\neg i \leq n$ 
@  $s \leq 0$ 

```

This program does *not* have the same behavior as the original program. But we still can observe a certain relationship between the correctness of the original and of the changed program: Every execution of the changed program also corresponds to an execution of the original one, but not vice versa. As a consequence, if the original program is correct, the changed one certainly will be correct as well. This also means that incorrectness of the changed program implies that also the original program was incorrect. However, the changed program will miss all bugs that occur after more than one iteration of the loops. In order to also catch bugs that occur after two loop iterations, we can unroll the loops twice. The result is

```

 $s \leftarrow 0$ 
 $i \leftarrow 0$ 
if  $i \leq n$  then
   $s \leftarrow s + a[i]$ 
   $i \leftarrow i + 1$ 
  if  $i \leq n$  then
     $s \leftarrow s + a[i]$ 
     $i \leftarrow i + 1$ 
    assume  $\neg i \leq n$ 
 $i \leftarrow 0$ 
if  $i \leq n$  then
   $s \leftarrow s - a[i]$ 
   $i \leftarrow i + 1$ 
  if  $i \leq n$  then
     $s \leftarrow s - a[i]$ 
     $i \leftarrow i + 1$ 
    assume  $\neg i \leq n$ 
@  $s \leq 0$ 

```

We can continue with this progress, unrolling the loops further, but we have a problem: If we want to check the correctness of program with n **if-then** statements we have to check the correctness of 2^n basic paths. So the question is whether it is possible to translate **if** statements into logical formulas without creating exponentially many basic paths?

So let us try to translate the following program into a logical formula.

```

 $s \leftarrow 0$ 
 $i \leftarrow 0$ 
if  $i \leq n$  then
   $s \leftarrow s + a[i]$ 
   $i \leftarrow i + 1$ 
  if  $i \leq n$  then
     $s \leftarrow s + a[i]$ 

```

$i \leftarrow i + 1$
 @ $s \leq 0$

We start by bringing the program into SSA. However, we encounter a problem: there are several execution paths that lead into the final assertion, and hence the variable s occurring in the final assertion may refer to the values resulting from different execution paths. Those different values correspond to different indexed variables, and it is not clear which one to use:

$s \leftarrow 0$
 $i \leftarrow 0$
if $i \leq n$ **then**
 $s_1 \leftarrow s + a[i]$
 $i_1 \leftarrow i + 1$
 if $i_1 \leq n$ **then**
 $s_2 \leftarrow s_1 + a[i_1]$
 $i_2 \leftarrow i_1 + 1$
 @ $s_? \leq 0$

The solution is to remember which branch is chosen using Boolean variables, and then using those Boolean variables to decide which value should be used after the branch:

$s \leftarrow 0$
 $i \leftarrow 0$
 $\gamma_1 \leftarrow [i \leq n]$
 $s_1 \leftarrow s + a[i]$
 $i_1 \leftarrow i + 1$
 $\gamma_2 \leftarrow [i_1 \leq n]$
 $s_2 \leftarrow s_1 + a[i_1]$
 $s_3 \leftarrow \gamma_2 ? s_2 : s_1$
 $s_4 \leftarrow \gamma_1 ? s_3 : s$
 @ $s_4 \leq 0$

Here

- $\gamma_i \in \{\top, \perp\}$, $i \in \mathbb{N}$
- $[\phi] := \begin{cases} \top, & \text{if } \phi \text{ is true for the current values of program variables, and} \\ \perp, & \text{otherwise} \end{cases}$
- $? :$ behaves in the same way as the corresponding C-construct

In the case of an **if-then-else** statement, the situation is similar. The example

if $p(x)$ **then**
 $k \leftarrow 0$
else
 $k \leftarrow 1$
 @ $k \geq 0$

can be translated to

```

 $\gamma_1 \leftarrow [p(x)]$ 
 $k_1 \leftarrow 0$ 
 $k_2 \leftarrow 1$ 
 $k_3 \leftarrow \gamma_1 ? k_1 : k_2$ 
 $@ k_3 \geq 0$ 

```

Translation to a logical formula is now easy. The original program is correct iff

$$\models [\gamma_1 \Leftrightarrow p(x) \wedge k_1 = 0 \wedge k_2 = 1 \wedge k_3 = \gamma_1 ? k_1 : k_2] \Rightarrow k_3 \geq 0.$$

where $s = \phi ? t_1 : t_2$ stands for

$$[\phi \Rightarrow s = t_1] \wedge [\neg\phi \Rightarrow s = t_2]$$

Up to now we assumed that unrolled programs only have one assertion at the end. However, we also would like to check assertions occurring within programs. Consider the program

```

if  $p(x)$  then
   $k \leftarrow 0$ 
   $@ k = 0$ 
else
   $k \leftarrow 1$ 
 $@ k \geq 0$ 

```

Here we need to check the assertion $k = 0$ only in the if branch. As soon we have Boolean variables that remember which branch was taken, it is easy to take this into account as follows:

```

 $\gamma \leftarrow [p(x)]$ 
 $k_1 \leftarrow 0$ 
 $k_2 \leftarrow 1$ 
 $k_3 \leftarrow \gamma ? k_1 : k_2$ 
 $@ [\gamma \Rightarrow k_1 = 0] \wedge k_3 \geq 0$ 

```

And again we can easily translate the result to a logical formula. The original program is correct iff

$$\models [\gamma_1 \Leftrightarrow p(x) \wedge k_1 = 0 \wedge k_2 = 1 \wedge k_3 = \gamma_1 ? k_1 : k_2] \Rightarrow [[\gamma \Rightarrow k_1 = 0] \wedge k_3 \geq 0]$$

Summarizing, bounded model checking consists of the following four steps:

1. Unroll the loops a certain number of times (the resulting program does not contain loops any more)
2. Translate the program to SSA (each program variable corresponds to precisely one value)
3. Translate SSA to a logical formula
4. Check the resulting formula

Putting this together, we define

$$BMC_P(n) := F(SSA(unroll_P(n))),$$

where $unroll_P(n)$ denotes the result of n -times unrolling the loops of P . If the resulting formula $BMC_P(n)$ only uses data types from decidable theories, we can check $BMC_P(n)$ automatically. If $\not\models BMC_P(n)$, then $\neg BMC_P(n)$ is satisfiable and the satisfying assignments represents variable values leading to a bug. This is called *counter-example* or *error trace*.

If $\models BMC_P(n)$ then we did not find a bug. However, this does not mean that the program is correct since the result of a finite number of loop unrollings is not equivalent to the original program.

To increase the trust in a given program, we can now check its correctness after 1, 2, ... loop unrollings:

$$\begin{array}{c} BMC(1) \\ BMC(2) \\ BMC(3) \\ \dots \end{array}$$

For $i = 1, \dots$, if $BMC(i)$ finds a bug, then $BMC(i + 1)$, as well, since $BMC(i + 1)$ implies $BMC(i)$.

21.1 Consequences

Every bug occurs after a certain finite number of steps. Due to this, counter-examples always have a certain length. So we can always find them, if we have enough time: For every incorrect program P there is a k s.t. $\neg BMC_P(k)$. Due to this, finding errors in programs with data types in decidable theories is semi-decidable.

In practice, however, programs can do a huge number of steps, and so we have to check $BMC_P(n)$ for huge n . Still, in certain applications this usually does not happen. An important example are embedded systems where the reaction to a certain event may take only a short amount of time.

21.2 Application: Equivalence Checking

Consider two functions, one being the optimized version of the other one. Bounded model checking can be used to check whether the two versions are equivalent using a program such as the following:

```
function foo(x)
function foo_optimized(x)

input x
assert foo(x)=foo_optimized(x)
```

21.3 Further Application: Combination with Testing

For software in safety critical applications, there are standards that require completeness of tests according to a certain criteria. Usually those criteria require that tests cover program code in a certain sense (*coverage criteria*). An example of such a coverage criterion is that tests have to execute each program line at least once.

As a result, one needs to solve problems of the following kind: How to find a test that executes a certain program line l ?

A solution may be to check $BMC_{P_l}(1), BMC_{P_l}(2), \dots$, where P_l is the original program with line l replaced by $@ \perp$. This has, for example, been applied to checking correctness of the European Train Control System (ETCS) [2].

21.4 Literature and Tools

Bounded model checking has been popular in hardware verification for many years. Its application to software was introduced later [13], resulting in tools such as CBMC (<http://www.cprover.org/cbmc/>). Today, software model checking finds more and more usage of BMC in industry. For example, Amazon Web Services uses CBMC to check the memory safety of the initial boot code running in their data centers, and the Germany company BTC embedded systems (<http://www.btc-es.de>) has expanded to the whole world as a startup applying bounded model checking in the automotive industry.

Chapter 22

Decision Procedures

The methods introduced in the previous chapters depend on verification conditions that need to be checked. Such checks can, to a large extent, be done automatically using decision procedure for logical theories, as introduced in Section 2.2.6. Today, there are powerful solvers implementing such decision procedures (e.g., CVC4), which—to a large extent—are the reason why such methods are nowadays used more and more. In this section we will have a closer look at how such decision procedures work.

22.1 Prelude: Validity vs. Satisfiability

As an example, assume that we are working with real numbers. Then

$$\models x + 1 \geq x, \text{ that is } \models \neg x + 1 < x.$$

Here the symbols $\models$ has the usual meaning from logic, validity. In other words, the given formula is true *for all* values of the free variable x .

Equivalent to the above is the fact that

$$\neg x + 1 \geq x, \text{ that is } x + 1 < x \text{ is not satisfiable,}$$

which means that *there is* no value for the free variable x for which the formula holds.

In general, we have that

$$\begin{array}{c} \models \phi \\ \text{iff} \\ \text{the formula } \neg\phi \text{ is not satisfiable.} \end{array}$$

Moreover, if it turns out that $\neg\phi$ is satisfiable, then the satisfying assignment gives us a counter-example to $\models \phi$.

Now assume that ϕ is of the form $\neg\psi$. Then $\neg\phi$ is equivalent to $\neg\neg\psi$ which is equivalent to ψ . Hence we can apply the above also after swapping ϕ and $\neg\phi$. Hence,

$$\begin{array}{c} \models \neg\phi \\ \text{iff} \\ \text{the formula } \phi \text{ is not satisfiable.} \end{array}$$

As a consequence, we can arbitrarily switch between validity and satisfiability by adding and removing negations.

22.2 Abstraction

Consider the following example:

```

Q ← ⊥
if 200 + 314x ≥ 2 ∧ end_of_the_world > 2073 then
  Q ← ⊤
if [¬[end_of_the_world > 2073] ∨ 200 + 314x < 2] ∧ Q then
  explosion!

```

The example program has two execution paths that lead to the explosion, one skipping the body of the first **if**, and one executing it. To check whether those execution paths are executable (see Section 20), we can check whether the two following conditions are satisfiable:

$$\begin{aligned}
& \neg Q \wedge \neg[200 + 314x \geq 2 \wedge \text{end_of_the_world} > 2073] \wedge \\
& [\neg[\text{end_of_the_world} > 2073] \vee 200 + 314x < 2] \wedge Q \\
\\
& \neg Q \wedge 200 + 314x \geq 2 \wedge \text{end_of_the_world} > 2073 \wedge Q_1 \wedge \\
& [\neg[\text{end_of_the_world} > 2073] \vee 200 + 314x < 2] \wedge Q_1
\end{aligned}$$

Everything will be o.k., if both formulas are not satisfiable (*unsat*). To prove that a formula is *unsat*, we use the observation from above that $\models \neg\phi$ iff the formula ϕ is not satisfiable.

So: To check satisfiability of these formulas, we try to prove their negation. The resulting formula has a negation as its outer-most symbol, and hence we assume ϕ , and try to find a contradiction. If we will not succeed in finding a contradiction, we will look for an assignment satisfying ϕ , which will be a counterexample to $\models \neg\phi$, and at the same time documents how to arrive at the bug.

Let us now apply this to our concrete verification conditions. At first sight, this seems to be difficult, since they contain unknown values, such as `end_of_the_world`. Ignoring this difficulty for now, let us try to show that the first verification condition is *unsat*. For this, we assume it, and try to find a contradiction. The contradiction immediately appears, since the formula contains both $\neg Q$ and Q . Hence we can conclude that the formula is not satisfiable, and the execution path cannot lead to the explosion, independent of which value the variable `end_of_the_world` has. For arriving at this conclusion, we only used the Boolean variables appearing in the formula, but could ignore everything else.

Now let us have a look at the second verification condition. Again, we assume it, and try to find a contradiction. However, now we do not get a contradiction from properties of Boolean variables alone. Still, it is not necessary to analyze the formula in details. Instead, we just use the information that two atomic formulas that are syntactically equal, will always have the same truth value. This results in the following formula in propositional logic, where the two occurrences of the atomic formula `end_of_the_world > 2073` are represented by the same propositional variable R :

$$\neg Q \wedge S \wedge R \wedge Q_1 \wedge [\neg R \vee T] \wedge Q_1.$$

However, this formula does not lead to a contradiction. An example of a satisfying assignment is

$$\{Q \mapsto \perp, S \mapsto \top, R \mapsto \top, Q_1 \mapsto \top, T \mapsto \top\}$$

Having a look at the atomic formulas underlying these Boolean variables, we see that this assigns $\top$ to the atomic formulas $200 + 314x \geq 2$, `end_of_the_world` > 2073 , and $200 + 314x < 2$. And, of course, it is not consistent to assign $\top$ to both $200 + 314x \geq 2$ and $200 + 314x < 2$. In other words, the abstraction that we made ignored some details that are necessary for excluding the inconsistent assignment. Here is a more precise abstraction that takes the relationship between the two mentioned atomic formulas into account by modeling $200 + 314x < 2$ as the negation of $200 + 314x \geq 2$ instead of introducing a new Boolean variable T .

$$\neg Q \wedge S \wedge R \wedge Q_1 \wedge [\neg R \vee \neg S] \wedge Q_1$$

It is easy to see that this formula results in a contradiction which proves both this formula, and the original formula to be unsat.

So, for proving the second verification condition to be unsat, we again ignored some of its details. Still, it was necessary to take into account more information from the original formula than in the case of the first verification condition.

Summarizing, the general principle was to start by ignoring almost all details of the original formula, and to incrementally add more and more of its properties:

1. properties of Boolean variables
2. syntactic equality of atomic formulas
3. the fact that $<$ is the negation of $\geq$

Both the principle of ignoring details that are not relevant for the given problem, and the resulting formula are called *abstraction*. If the abstraction is unsat then original formula is unsat, but not the other way round. If the abstraction is satisfiable, and we want to prove unsatisfiability, we have to add additional properties. The same holds analogically, if we want to prove validity of a formula.

22.3 Program Abstraction

We can apply the same principle directly to programs. More specifically, we can abstract our program, for example, by replacing atomic formulas by Boolean variables. A possible result is:

```

Q ← ⊥
if S ∧ R then
  Q ← ⊤
if [¬R ∨ T] ∧ Q then
  explosion!
```

Every execution of the original program has a corresponding execution of the abstracted program, but not the other way round. For example, $\{S \mapsto \top, R \mapsto \top, T \mapsto \top\}$ leads to explosion that cannot happen for the original program. Now we can refine the abstraction analogically to the way we did it above.

22.4 Decision Procedures

Decision procedures are the algorithms behind SMT solvers. The check formula in specific logical theories (e.g., arrays, integers, real numbers) for satisfiability. SMT solver then combine many such decision procedures. We sometimes also simply use the word *solver* if we do not explicitly want to make the difference between a decision procedure and an SMT solver.

Decision procedures usually do not prove, but check satisfiability. The main reason for this is the following:

- The separation into assumptions and things to prove is beneficial for human intuition.
- Working only with assumptions suits algorithms since this allows a uniform handling of formulas).

22.4.1 Solvers for Propositional Logic

Here, we solve the so-called SAT problem, where SAT is a short-cut for satisfiability. This problem is the following:

- Input: a propositional logical formula, usually in conjunctive normal form
- Output:
 - satisfying assignment, if it exists,
 - **unsat**, if the formula is not satisfiable.

An example of allowed input formula (in conjunctive normal form) is

$$[\neg P \vee Q \vee R] \wedge [\neg Q \vee R] \wedge [\neg Q \vee \neg R] \wedge [P \vee Q].$$

As every computer scientist knows, solving this problem is NP-hard. However, practical solvers have seen huge efficiency improvements in recent years and today's solvers can often solve huge formulas coming from practical problems.

22.4.2 Equality and Function Calls

Consider this example:

```

if x=1 then
  y ← 1
  if adfxg7(x) ≠ adfxg7(y) then
    @ ⊥
  
```

Again, we want to know whether it is possible to reach the line with the assertion. So we ask the question whether the formula

$$x = 1 \wedge y = 1 \wedge \text{adfxg7}(x) \neq \text{adfxg7}(y)$$

is satisfiable.

Let us first try a Boolean abstraction. It is

$$P \wedge Q \wedge R$$

which is satisfied by the assignment

$$\{P \mapsto \top, Q \mapsto \top, R \mapsto \top\}.$$

However, it is certainly not possible that $x = 1$, $y = 1$, and $\text{adfxg7}(x) \neq \text{adfxg7}(y)$ are all true, at least not, if we assume adfxg7 to be a function in the mathematical sense. The reason is that $x = 1$, $y = 1$, and $\text{adfxg7}(x) \neq \text{adfxg7}(y)$ imply that $\text{adfxg7}(1) \neq \text{adfxg7}(1)$, which is in contradiction with the law of reflexivity.

And note that this argument only uses knowledge about the behavior of equality, but does not use (i.e., abstracts from) the fact that 1 is a number, and also does not use any knowledge about adfxg7 beyond the knowledge that it is a function in the mathematical sense. Indeed, the very same argument would have worked equally well for the formula

$$a = c \wedge b = c \wedge f(a) \neq f(b),$$

that is, after replacing the number 1 by any other constant and the function adfxg7 by any other function.

To generalize this sort of reasoning, let us write down the usual properties of equality:

- $\forall x . x = x$ (reflexivity)
- $\forall x, y . x = y \Rightarrow y = x$ (symmetry)
- $\forall x, y, z . [x = y \wedge y = z] \Rightarrow x = z$ (transitivity)
- for every function symbol f of arity n ,

$$\begin{aligned} \forall x_1, \dots, x_n, y_1, \dots, y_n . [x_1 = y_1, \dots, x_n = y_n] \Rightarrow \\ f(x_1, \dots, x_n) = f(y_1, \dots, y_n) \end{aligned}$$

The first three axioms define an equivalence relation and all axioms together define a congruence relation. Together, these axioms define the *theory of free function symbols*.

A decision procedure for this theory, proceeds as above: To check satisfiability, we assume the axioms of the theory of free function symbols, and the given formula. If we succeed in deriving a contradiction, we know that the given formula is unsatisfiable. More specifically, we deduce equalities using

1. the equivalence axioms
 - $\forall x . x = x$ (reflexivity)
 - $\forall x, y . x = y \Rightarrow y = x$ (symmetry)
 - $\forall x, y, z . [x = y \wedge y = z] \Rightarrow x = z$ (transitivity),
2. the equalities occurring in the input formula, and
3. the congruence axioms

$$\begin{aligned} \forall x_1, \dots, x_n, y_1, \dots, y_n . \\ [x_1 = y_1, \dots, x_n = y_n] \Rightarrow f(x_1, \dots, x_n) = f(y_1, \dots, y_n) \end{aligned}$$

until we derive a contradiction between a derived equality and a disequality occurring in the input formula.

The basic idea is based on the observation that the equivalence axioms express the fact that equality is an equivalence relation. This allows us to directly work with the corresponding equivalence classes of this equivalence relation, instead of individual equalities.

Consider the following example formula [8]:

$$f(a, b) = a \wedge f(f(a, b), b) \neq a.$$

This formula contains the subterms a , b , $f(a, b)$, and $f(f(a, b), b) \neq a$. We can divide the set of this subterms into equivalence classes. For example, the equivalence classes $\{a, b, f(a, b)\}$ and $f(f(a, b), b) \neq a$ express the information that any two elements from $\{a, b, f(a, b)\}$ are equal. At the beginning of the algorithm, we do not have any information about equality between subterms, and hence we begin by putting every sub-term in a separate class:

$$\{\{a\}, \{b\}, \{f(a, b)\}, \{f(f(a, b), b)\}\}.$$

Now we can use the equalities occurring in the input formula to deduce further information about equality between subterms. The quality $f(a, b) = a$ from our example tells us that we can merge the equivalence classes $\{f(a, b)\}$ and $\{a\}$. The result is

$$\{\{a, f(a, b)\}, \{b\}, \{f(f(a, b), b)\}\}$$

Now, since a and $f(a, b)$ are in the same equivalence class, the congruence axiom tells us that, $f(a, b)$ and $f(f(a, b), b)$ also have to be in the same class. This information allows us to do so-called *congruence propagation*, merging the classes $\{a, f(a, b)\}$ and $\{f(f(a, b), b)\}$. The result is

$$\{\{a, f(a, b), f(f(a, b), b)\}, \{b\}\}$$

This is in contradiction with the disequality $f(f(a, b), b) \neq a$ from the formula, which implies its unsatisfiability.

This illustrated the workings of the so-called congruence-closure algorithm [34] which works as follows, in general:

- Input: formula $s_1 = t_1 \wedge \dots \wedge s_m = t_m \wedge s_{m+1} \neq t_{m+1} \wedge \dots \wedge s_n \neq t_n$
- Output: **unsatisfiable**, **satisfiable**
(in the theory of free function symbols)

Let τ be the set of sub-terms of

$$s_1 = t_1 \wedge \dots \wedge s_m = t_m \wedge s_{m+1} \neq t_{m+1} \wedge \dots \wedge s_n \neq t_n$$

$\sim \leftarrow$ equivalence relation on τ s.t. for all $u, v \in \tau$, $u \neq v$ implies $u \not\sim v$

while there is $i \in \{1 \dots m\}$ s.t. $s_i \not\sim t_i$ **do**

merge equivalence classes of s_i and t_i in $\sim$

while there is $p, q \in \tau$, $u, v \in \tau$ s.t.

$p \sim q$, $[u[p \leftarrow q] = v$ or $u[q \leftarrow p] = v]$, $u \not\sim v$ **do**

merge equivalence classes of u and v in $\sim$

if there is an $i \in \{m+1, \dots, n\}$ s.t. $s_i \sim t_i$ **then**

return unsatisfiable

else

return satisfiable

The loops of this algorithm preserve the following loop invariant: If $u \sim v$ then $u = v$ follows from axioms of the theory of uninterpreted function symbols plus the input formula. Due to this, if two subterms from the same equivalence class must be different due to a disequality from the input formula, we have a contradiction, and the answer **unsatisfiable** is correct, in this case. Proving that the answer **satisfiable** is correct in the case where no such contradiction is derived, is more involved [34].

22.4.3 General Formulas: Disjunctions, Quantifiers

Let us consider a formula with a disjunction. Is the formula

$$a \neq a \vee [a = b \wedge f(a) \neq f(b)]$$

satisfiable? The formula consists of two parts, $a \neq a$, and $a = b \wedge f(a) \neq f(b)$, and clearly, both are unsatisfiable. Hence, also the original formula is unsatisfiable. This is consistent with our proof rules that handle a disjunction in the assumptions using a case split.

In general, a formula $\phi_1 \vee \dots \vee \phi_n$ is satisfiable iff one of the formulas $\phi_1, \dots, \phi_n$ is satisfiable and hence we can decide formulas with conjunctions and disjunctions by first putting them into disjunctive normal form, and then applying a decision procedure on each part of the resulting disjunction. This works, but has the disadvantage of blowing up the size of the formula. Hence, in practice, a different approach is used, so called SMT solvers. Those combine a SAT solver with solvers for various logical theories.

The general case combining free function symbols with quantifiers is undecidable.

22.4.4 Application

Before discussing further logical theories, let us have a look at how the theory of free function symbols can be applied to the analysis of computer programs. Assume the unsatisfiable formula

$$a[x] = 0 \wedge x = y \wedge a[y] = 1$$

which speaks about arrays and integers. However, since we do not yet have decision procedures for these theories available, we abstract them to free function symbols. The result is

$$f(a, x) = c \wedge x = y \wedge f(a, y) = d$$

which has the sub-terms: $\{a, c, d, x, y, f(a, x), f(a, y)\}$. Applying the congruence closure algorithm, we get the answer “satisfiable” based on the equivalence classes $\{a\}, \{x, y\}, \{c, d, f(a, x), f(a, y)\}$. Since we know that the original formula was unsatisfiable, we clearly abstracted too much. Analyzing the equivalence classes, we see that the constants c and d are in the same equivalence class, although the corresponding original constants 0 and 1 are *not* the same. To take this information into account in the abstracted formula, we refine the abstraction to the formula

$$f(a, x) = c \wedge x = y \wedge f(a, y) = d \wedge c \neq d$$

for which the congruence closure algorithm returns the correct answer “unsat”.

A further example is the abstraction of pointers to free function symbols: We can abstract every pointer dereference $*x$ to $f(x)$. This takes all properties of the theory of free function symbols into account, for example the fact that dereferencing two pointers that are the same, results in the same dereferenced object.

In general, we can apply the theory of free function symbols to data structures by abstracting from detailed properties, and trying a proof just using equality, that is, in the theory of free function symbols. Here it sometimes helps to add additional knowledge (e.g., $0 \neq 1$, commutativity).

22.4.5 Arrays

We already observed, that sometimes it is possible to prove something about arrays by abstracting the array read and write operations to free function symbols and using a decision procedure for the theory of free function symbols. However, this does not always work. Consider the example $\text{write}(a, i, 7)[i] \neq 7$. Clearly, this formula is not satisfiable, since the result of writing the number 7 to position i and again reading from the same position is again 7 which cannot be different from 7. However, if we abstract the read and write operations to free function symbols, we get, for example, $r(w(a, i, 7), i) \neq 7$ which is satisfiable. The problem is that the abstracted formula, does not contain any information about the behavior of arrays. Especially it does not contain information about the interaction of the functions r and w that is expressed by the array axioms

Read-same (RS): $\forall a, v, i, j. i = j \Rightarrow \text{write}(a, i, v)[j] = v$, and

Read-different (RD): $\forall a, v, i, j. i \neq j \Rightarrow \text{write}(a, i, v)[j] = a[j]$.

So let us have a look how to take these properties into account. Consider the example

$$i_1 = j \wedge i_1 \neq i_2 \wedge a[j] = v_1 \wedge \text{write}(\text{write}(a, i_1, v_1), i_2, v_2)[j] \neq a[j].$$

To check the satisfiability of this formula, we consider it an assumption and try to find a contradiction. For this, we observe that the term $\text{write}(\text{write}(a, i_1, v_1), i_2, v_2)[j]$, writes the value v_2 into the position i_2 , and then reads from the position j . The result of this combination of writing and reading depends on whether i_2 is equal to j . If $i_2 = j$, we can apply the axiom RS, if $i_2 \neq j$ the axioms RD.

Since we do not know which of the two case apply, we try both, after adding the case distinction $i_2 = j \vee i_2 \neq j$. The application of the proof rule for disjunctions in assumptions splits the proof into two branches, one where $i_2 = j$, and another one where $i_2 \neq j$.

In the case $i_2 = j$, the formula with the added assumptions is

$$i_2 = j \wedge i_1 = j \wedge i_1 \neq i_2 \wedge a[j] = v_1 \wedge \text{write}(\text{write}(a, i_1, v_1), i_2, v_2)[j] \neq a[j]$$

where due to the axiom RS we can simplify $\text{write}(\text{write}(a, i_1, v_1), i_2, v_2)[j]$ to v_2 . However, $i_2 = j \wedge i_1 = j \wedge i_1 \neq i_2$ is not satisfiable and hence the whole formula in the case $i_2 = j$.

In the case $i_2 \neq j$, the formula with the added assumptions is

$$i_2 \neq j \wedge i_1 = j \wedge i_1 \neq i_2 \wedge a[j] = v_1 \wedge \text{write}(\text{write}(a, i_1, v_1), i_2, v_2)[j] \neq a[j]$$

Due to RD, $\text{write}(\text{write}(a, i_1, v_1), i_2, v_2)[j]$ simplifies to $\text{write}(a, i_1, v_1)[j]$ and hence we have

$$i_2 \neq j \wedge i_1 = j \wedge i_1 \neq i_2 \wedge a[j] = v_1 \wedge \text{write}(a, i_1, v_1)[j] \neq a[j].$$

Now, since $i_1 = j$ we can apply the axioms RS, which simplifies $\text{write}(a, i_1, v_1)[j]$ to v_1 which results in

$$i_2 \neq j \wedge i_1 = j \wedge i_1 \neq i_2 \wedge a[j] = v_1 \wedge v_1 \neq a[j]$$

However, $a[j] = v_1 \wedge v_1 \neq a[j]$ is not satisfiable, and hence the whole formula in the case $i_2 \neq j$.

To summarize, the original formula is neither satisfiable in the case $i_2 = j$, nor in the case $i_2 \neq j$, and hence the whole formula is not satisfiable.

A general decision procedure for the theory of arrays is the following

- Input: a conjunctive quantifier-free formula ϕ
- Output: **unsatisfiable**, **satisfiable**
(in the theory of arrays (with equality))

```

function satA( $\phi$ )
if  $\phi$  is satisfiable in the theory of free function symbols then
  choose a term of the form  $\text{write}(a, i, v)[j]$  in  $\phi$ 
  if  $\phi$  does not contain such a term then
    return satisfiable
  if satA( $F[\text{write}(a, i, v)[j] \leftarrow v] \wedge i = j$ ) then
    return satisfiable
  if satA( $F[\text{write}(a, i, v)[j] \leftarrow a[j]] \wedge i \neq j$ ) then
    return satisfiable
return unsatisfiable

```

The axiomatization we introduced in Section 2.2.2 consists of three axioms not two. In addition to the two axioms mentioned above, it also contains an axioms expressing that two arrays are the same iff they contain the same elements. The decision procedure we introduce here, does not take this axiom into account. However, there are extensions for also handling this third axiom, and also some limited handling of quantifiers [9, 8]. However, the general case with arbitrary quantification is undecidable.

Figure 22.1 contains a list of logical theories, together with information their decidability, where QFF stands for “quantifier free formulas”. Formulas coming from analyzing computer programs usually combine elements from various logical theories. There is special algorithms that combain individual decision procedures to decision procedures of combined theories [33].

22.4.6 Witnesses and Their Usage

Some decision procedures also return satisfying assignments which serve as a witness for satisfiability, and a counter-example to validity of the negation. For example, in the theory of integers, the input formula

$$x + y \geq 0 \wedge x - 2y \leq 1$$

Theory	Full	QFF
equality	no	yes
Peano arithmetic (i.e., $\mathbb{N}$ without $\times$)	no	no
Presburger arithmetic (i.e., $\mathbb{N}$ without $\times$)	yes	yes
real numbers (with $\times$)	yes	yes
real numbers (with $\sin$)	no	no
recursive data structure (e.g., lists)	no	yes
arrays	no	yes

Figure 22.1: Decidability of Logical Theories

is satisfiable, and a decision procedure cannot only return this information but, in addition, a satisfying assignment such as $\{x \mapsto -1, y \mapsto 1\}$.

This allows automatic execution of I/O specifications: For a specification of the form

- Input: $I(x)$
- Output: y s.t. $O(x, y)$

and given input a , one can ask a solver for the corresponding output by checking the satisfiability of $I(a) \wedge O(a, y)$. For a well-formed specification, this formula is always satisfiable, independent of the input a , and the corresponding satisfying assignment provides a value for y that is correct output.

22.4.7 Real Numbers

In the case of decision procedures for the real numbers, there are some well-known algorithms for special cases:

- conjunctions of linear equations: Gaussian elimination
- conjunctions of linear equations and inequalities: simplex algorithm

Sometimes it is even possible to compute something in undecidable cases (<http://rsolver.sourceforge.net>).

22.5 Summary and Conclusion

Decision procedures for logical theories can attack the check for satisfiability at several levels

- Boolean: SAT solvers
- equality between function symbols: congruence closure
- individual theories: decision procedures for arrays, real numbers etc.

Chapter 23

Automated Formal Software Verification

In Chapter 16 we saw how to ensure correctness of programs with control structures. The method assumes the existence of loop invariants: for every loop at least one assertion with a logical formula that summarizes the essential behavior of the loop. If such loop invariants exist, one can automatically check program correctness by checking the resulting verification conditions using decision procedures for the relevant logical theories.

However, real world programs often do not have loop invariants: There may be loops that do not contain assertions at all, or the assertions do not capture enough information about the behavior of the loop to imply correctness of the overall program. We have already seen two ways of getting around this problem:

- Symbolic execution: Avoids loop invariants by only checking individual execution paths.
- Bounded software model checking: Avoids loop invariants by only checking programs for a certain number of steps.

If we do not want to make such compromises, we need a method for synthesizing loop invariants. This is an active research area, but there are already commercial tools that support this:

- <http://www.absint.com/astree/>
- Mathworks Polyspace
- <https://www.imandra.ai>

Chapter 7 discusses some techniques that are widely used for verifying hardware, which is usually modeled as a finite state systems. In this chapter, we will have a look at some basic techniques for software.

23.1 Weakest Preconditions

Consider the following example:

```

assume  $I(x, y, z)$ 
 $y \leftarrow x$ 
 $x \leftarrow -10$ 
 $z \leftarrow x + y$ 
 $@ \ z \geq 0$ 

```

Let us consider the following question: For which condition I is the program P correct? What if $I(x, y, z)$ is the formula $\perp$, that is false in any case? For this, we have to return to the question of what it means for a program to be correct. Definition 60 roughly says, that this is the case, if for all program executions that satisfy all **assume**-assertions, all $@$ -assertions are true. If I is the formula $\perp$, then *no* program execution satisfies all **assume**-assertions, and hence no assertion failure can happen. This means that in this case, the program is correct.

However, this formula is the strongest possible one. Since it excludes all program executions, it does not provide any information about the program, at all. Let us now consider the formula $x \geq 70$, which is weaker, allowing certain program executions. A simple check of the corresponding verification condition shows that this formula also results in a correct program. In order to obtain as much information about the program as possible, it may be useful, to have the weakest possible formula.

Definition 69 *A formula I is a precondition of a program P and a formula O iff*

- *every free variable of I is a program variable of P and*
- *the program **assume** I ; P ; $@O$ is correct*

Every precondition I of a program P and a formula O is a weakest precondition of P and O iff for every precondition I' of P and O , $\models I' \Rightarrow I$.

But is it possible to compute weakest preconditions automatically? If we take the basic path from above, ignoring the initial **assume** statement, we arrive at the following verification condition:

$$[y_1 = x \wedge x_1 = -10 \wedge z_1 = x_1 + y_1] \Rightarrow z_1 \geq 0$$

We know that the basic path is correct if and only if this verification condition is holds. The problem is, that is formula contains the variables x_1 , y_1 , and z_1 that do not correspond to initial values of variables. We want the program to be correct in any way, for whatever intermediate values these variables take. So we add a universal quantifier, resulting in

$$\forall x_1, y_1, z_1. [y_1 = x \wedge x_1 = -10 \wedge z_1 = x_1 + y_1] \Rightarrow z_1 \geq 0$$

This is already a weakest precondition: If the variable x has a value that satisfies this formula, the execution of the basic path will be correct. And moreover, if it takes a values that does not satisfy this formula, it will be incorrect.

However, the fact that the formula contains a quantifier is not nice. Hence we will simplify it. First we will eliminate the quantified variable z_1 using the equation $z_1 = x_1 + y_1$ that allows us to substitute $x_1 + y_1$ for z_1 . The result is

$$\forall x_1, y_1. [y_1 = x \wedge x_1 = -10] \Rightarrow x_1 + y_1 \geq 0$$

Now we use the equation $x_1 = -10$ to eliminate the variable x_1 , resulting in

$$\forall y_1 . y_1 = x \Rightarrow -10 + y_1 \geq 0.$$

And finally, we eliminate y_1 , which results in

$$-10 + x \geq 0$$

which can also be written as

$$x \geq 10.$$

This is a weakest precondition, and moreover, it has a very simple form. This can be easily checked by proving the verification condition of

assume $x \geq 10$

$y \leftarrow x$

$x \leftarrow -10$

$z \leftarrow x + y$

@ $z \geq 0$

which is valid, while replacing the number 10 by a smaller one, results in the verification condition to be invalid.

So sum up, if we have a specification of the form

Input:

Output: O

where the input condition is left open, and a program P without control structures, a weakest precondition is

$$\forall \vec{v} . VC(P; @O)$$

where $\vec{v}$ is the tuple of auxiliary variables of $VC(P; @O)$. Here the verification condition must use the original variable names for variables corresponding to initial values.

The problem is, that this formula contains a quantifier, which we would like to eliminate. This corresponds to the following problem of quantifier elimination:

- Given: formula ϕ
- Find: formula ϕ' s.t. $\models \phi \Leftrightarrow \phi'$, but ϕ' is quantifier free

Quantifier elimination for variables introduced by assignments is easy. However, for variables introduced by user input or procedure calls, it might be very difficult. Here is another example:

input x

@ $x^2 + a \geq 1$

The weakest precondition introduced above is

$$\forall x . x^2 + a \geq 1$$

which can be written as follows:

$$\forall x . x^2 \geq 1 - a$$

Now we observe that the term x^2 is always non-negative, and hence we can replace it by 0, resulting in

$$0 \geq 1 - a$$

which is equivalently

$$a \geq 1.$$

This already needed some more intricate consideration. And in general, quantifier elimination is a very difficult problem.

23.2 Strongest Postconditions

Now we will have a look at the dual question of a specification with an input condition, but where now the output condition is left open:

Input: I

Output:

The goal is now, for a program P , to find an output condition such that the **assume** $I; P; @O$ is correct. This problem again has a trivial solution: The tautology $\top$ makes the program trivially correct. However, it does not provide any interesting information about the behavior of the program P , and we would like to have a stronger condition, expressing more information. We arrive at a dual of Definition 69

Definition 70 *A formula O is a postcondition of a program P and a formula I iff*

- *every free variable of O is a program variable of P , and*
- *the program **assume** $I; P; @O$ is correct*

Every postcondition O of a program P and a formula I is a strongest postcondition of P and I iff for every postcondition O' of P and I , $\models O \Rightarrow O'$.

To see how a strongest postcondition can be computed, we will again have a look at an example. To ensure that the basic path

```

assume  $x \geq 10$ 
 $y \leftarrow x$ 
 $x \leftarrow -10$ 
 $z \leftarrow x + y$ 
@  $O(x, y, z)$ 

```

is correct, the verification condition

$$[x \geq 10 \wedge y_1 = x \wedge x_1 = -10 \wedge z_1 = x_1 + y_1] \Rightarrow O(x_1, y_1, z_1)$$

should be valid. The strongest formula, for which this is the case, is the left-hand side of the implication. However, this left-hand side does not only contain the

free variables x_1 , y_1 , and z_1 , but also the variable x . However, we can rewrite the verification condition equivalently to

$$\models \forall x . [x \geq 10 \wedge y_1 = x \wedge x_1 = -10 \wedge z_1 = x_1 + y_1] \Rightarrow O(x_1, y_1, z_1)$$

and

$$\models [\exists x . x \geq 10 \wedge y_1 = x \wedge x_1 = -10 \wedge z_1 = x_1 + y_1] \Rightarrow O(x_1, y_1, z_1),$$

and now the left-hand side only contains the desired variables. So the formula

$$\exists x . x \geq 10 \wedge y_1 = x \wedge x_1 = -10 \wedge z_1 = x_1 + y_1$$

is a strongest postcondition.

To see the general principle, we observe that this corresponds to the formula constructed by symbolic execution (see Chapter 20), with the variables corresponding to non-final values quantified away:

$$\exists x . SE(\textbf{assume } x \geq 10; y \leftarrow x; x \leftarrow -10; z \leftarrow x + y)$$

The intuition is that we want a formula that corresponds to the set of program states that are reachable by the program. If the formula would exclude such a state, it would not be a postcondition. And if it would contain a non-reachable program state, it would not be a strongest postcondition.

In general, for a specification of the form

Input: I

Output:

and program P without control structures, we get a strongest postcondition in quantified form, with indexed variables from

$$\exists \vec{v} . SE(\textbf{assume } I; P)$$

where $\vec{v}$ is a tuple of the variables in $SE(\textbf{assume } I; P)$ that do *not* correspond to a final value.

To relate to the resulting formula to the program variables, one usually needs to rename variables then. For example, one could first rename the quantified variable x to x' , and then rename x_1 , y_1 , and z_1 to the corresponding names without indices. For the example above, this results in

$$\exists x' . x' \geq 10 \wedge y = x' \wedge x = -10 \wedge z = x + y,$$

which only has x , y , and z as free variables.

In general, there is no simple way of eliminating the quantifiers in the resulting formulas. There is special quantifier elimination software, that can sometimes help. However, practical software verification tools usually work around this problem by approximating quantifier elimination in certain ways.

23.3 Pre- or Postcondition Based Software Verification

Now assume that we have a given verification problem:

- **Input:** I
- Spec: **Output:** O
- Program: P

If we have a pre- or postcondition available, it can be used for proving correctness of the program. More specifically, if we have a precondition I' available, and this precondition is weak enough, that is, $\models I \Rightarrow I'$, then this implies correctness of the program. Dually, if we have a postcondition O' available, and this postcondition is strong enough, that is $\models O' \Rightarrow O$, then this also implies correctness of the program.

If the program P also has internal assertions that we want to satisfy, then we can use pre- or postconditions of the part of the program that leads to the internal assertion, or that starts in the internal assertion.

We have already seen above how to compute pre- and postconditions for basic paths, which do not contain any loops. However, programs usually contain loops, and so we will now discuss how to compute pre- and postconditions for programs with loops.

23.4 Computing Pre- and Postconditions for Programs with Loops

The most difficult part of checking unbounded program correctness is the development of loop invariants. Algorithms for software verification use certain strategies, that may also be useful when devising loop invariants manually. However, such algorithms use certain heuristics, that need to need intuition and creativity. However, there are some strategies that may help in this process. In the section, we explain such a strategy on an example. Consider the specification

- Input: array a
- Output: r s.t. $r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]$

and the program

```

 $r \leftarrow \perp$ 
for  $i \leftarrow 1$  to  $n$  do
  if  $a[i] = 7$  then  $r \leftarrow \top$ 
return  $r$ 

```

The first step is to add assertions to every loop, and at the end. A first attempt will be, to make the assertions as strong as possible, which corresponds to the formula $\perp$ that cannot be satisfied by anything.

```

 $r \leftarrow \perp$ 
for  $i \leftarrow 1$  to  $n$  do

```

```

    @  $\perp$ 
    if  $a[i] = 7$  then  $r \leftarrow \top$ 
@  $\perp$ 
return  $r$ 

```

Since the final assertion is so strong, it implies the output condition $r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]$. But even more, the contradiction $\perp$ implies *any* formula! The problem is somewhere else: To ensure correctness of the program, all basic paths should be correct. However, with these assertions already the basic path leading from the beginning of the program to its (attempted) loop invariant

```

 $r \leftarrow \perp$ 
assume  $i = 1$ 
@  $\perp$ 

```

is incorrect. The problem is that the formula $\perp$ is too strong—it is not a postcondition. To correct the problem, we replace it by the strongest postcondition of this basic path, which corresponds the state of the program after entering the loop. This strongest post-condition is $i = 1 \wedge \neg r$, and hence we get:

```

 $r \leftarrow \perp$ 
for  $i \leftarrow 1$  to  $n$  do
    @  $i = 1 \wedge \neg r$ 
    if  $a[i] = 7$  then  $r \leftarrow \top$ 
@  $\perp$ 
return  $r$ 

```

Now the basic path leading into the loop is correct. But this is not enough. Within the loop we have two basic paths, one corresponding to the if branch, and the other to the else branch:

<pre> assume $i = 1$ assume $a[i] = 7 \wedge \neg r$ $r \leftarrow \top$ $i \leftarrow i + 1$ @ $i = 1 \wedge \neg r$ </pre>	<pre> assume $i = 1 \wedge \neg r$ assume $a[i] \neq 7$ $i \leftarrow i + 1$ @ $i = 1 \wedge \neg r$ </pre>
---	---

Clearly, it is not possible to prove correctness of the assertion $i = 1 \wedge \neg r$ that closes both basic paths. To improve the situation, we observe that after execution of the first basic path we have $a[1] = 7 \wedge r \wedge i = 2$, and after execution of the second basic path $a[1] \neq 7 \wedge \neg r \wedge i = 2$. We do not know which branch of the if-then-else the program will take, so we take into account both possibilities, in addition to the original assertion $i = 1$:

$$[i = 1 \wedge \neg r] \vee [a[1] = 7 \wedge r \wedge i = 2] \vee [a[1] \neq 7 \wedge \neg r \wedge i = 2]$$

which is equivalent to

$$[i = 1 \wedge \neg r] \vee [i = 2 \wedge [r \Leftrightarrow a[1] = 7]].$$

If we want to use this formula as a loop invariant, the result is

```

 $r \leftarrow \perp$ 
for  $i \leftarrow 1$  to  $n$  do
  @  $[i = 1 \wedge \neg r] \vee [i = 2 \wedge [r \Leftrightarrow a[1] = 7]]$ 
  if  $a[i] = 7$  then  $r \leftarrow \top$ 
@  $\perp$ 
return  $r$ 

```

Now the two basic paths corresponding to a loop iteration look as follows:

assume $[i = 1 \wedge \neg r] \vee [i = 2 \wedge [r \Leftrightarrow a[1] = 7]]$	assume $[i = 1 \wedge \neg r] \vee [i = 2 \wedge [r \Leftrightarrow a[1] = 7]]$
assume $a[i] = 7$	assume $a[i] \neq 7$
$r \leftarrow \top$	$i \leftarrow i + 1$
$i \leftarrow i + 1$	@ $[i = 1 \wedge \neg r] \vee [i = 2 \wedge [r \Leftrightarrow a[1] = 7]]$
@ $[i = 1 \wedge \neg r] \vee [i = 2 \wedge [r \Leftrightarrow a[1] = 7]]$	

Again, it is not possible to prove correctness of the assertion that closes both basic paths. And again, we add the possibilities that we get by one more execution of the loop, arriving at

$$[i = 1 \wedge \neg r] \vee [i = 2 \wedge [r \Leftrightarrow a[1] = 7]] \vee [i = 3 \wedge r \Leftrightarrow [a[1] = 7 \vee a[2] = 7]].$$

Now the principle is becoming clear. Also this formula will not be a correct loop invariant. We will have to generalize the invariant to take into account an arbitrary number of loop executions. The result is the formula

$$r \Leftrightarrow [\exists k . 1 \leq k \leq i - 1 \wedge a[k] = 7]$$

and the program

```

 $r \leftarrow \perp$ 
for  $i \leftarrow 1$  to  $n$  do
  @  $r \Leftrightarrow [\exists k . 1 \leq k \leq i - 1 \wedge a[k] = 7]$ 
  if  $a[i] = 7$  then  $r \leftarrow \top$ 
@  $\perp$ 
return  $r$ 

```

Checking the two basic paths again, we see that now they are correct:

assume $r \Leftrightarrow [\exists k . 1 \leq k \leq i - 1 \wedge a[k] = 7]$	assume $r \Leftrightarrow [\exists k . 1 \leq k \leq i - 1 \wedge a[k] = 7]$
assume $a[i] = 7$	assume $a[i] \neq 7$
$r \leftarrow \top$	$i \leftarrow i + 1$
$i \leftarrow i + 1$	assume $1 \leq i \leq n$
assume $1 \leq i \leq n$	@ $r \Leftrightarrow [\exists k . 1 \leq k \leq i - 1 \wedge a[k] = 7]$
@ $r \Leftrightarrow [\exists k . 1 \leq k \leq i - 1 \wedge a[k] = 7]$	

However, we still have to ensure correctness of the program when leaving the loop. For this we could again use strongest postconditions, now the ones corresponding to the two basic paths leaving the loop. However, to cut a long story short, we simply use the insight that after terminating the loop, i has the final value n , and hence we can use the formula

$$r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]$$

in the final assertion. This results in the two final paths

<pre> assume $r \Leftrightarrow [\exists k . 1 \leq k \leq i - 1 \wedge a[k] = 7]$ assume $a[i] = 7$ $r \leftarrow \top$ assume $i = n$ @ $r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]$ </pre>	<pre> assume $r \Leftrightarrow [\exists k . 1 \leq k \leq i - 1 \wedge a[k] = 7]$ assume $a[i] \neq 7$ assume $i = n$ @ $r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]$ </pre>
--	--

which are correct.

In general, postconditions of programs with loops can be computed as follows: We assume as input a program P such that

- every loop has an assertion (i.e., an invariant),
- the last line is an assertion, and
- all assertions are of the form @ $\perp$.

We compute a program P' such that

- P' is identical to P except for formulas at assertions, and
- the formula of the final assertion is a postcondition of the program (hopefully strong enough).

This program is computed by the following recursive function:

```

postcond(P)=
  if all basic paths of P are correct then  $P$ 
  else postcond( $P_w$ ), where  $P_w$  is equal to  $P$  except for
    some assertions @  $\alpha$  replaced by @ simplify( $\alpha \vee \pi$ ) where
     $\pi$  is a postcondition of a basic path ending with @  $\alpha$ 

```

As shown above, this does not converge in general. To make it converge, we may need some way of generalizing loop invariants over arbitrary many iterations. This generalization step is a different problem, and there are various ingenious ways of doing this, but this is out of the scope of this text.

Let us now have a look at the dual problem, computation of preconditions. Here we proceed in the opposite direction, backward from the requirement on the output. We start by initializing all assertions except the final one with the formula $\top$:

```

assume  $\top$ 
 $r \leftarrow \perp$ 
for  $i \leftarrow 1$  to  $n$  do
  if  $a[i] = 7$  then  $r \leftarrow \top$ 
  @  $\top$ 
@  $r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]$ 
return  $r$ 

```

As a result, not all basic paths of the program are correct. We will try to make them correct by computing weakest preconditions for the individual basic paths, working ourselves back from the final assertion. So we start with the weakest precondition ensuring that the result is correct when leaving the loop:

assume $i = n$
@ $r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]$

The corresponding verification conditions is

$$i = n \Rightarrow [r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]]$$

which already is the weakest pre-condition of the basic path. Adding it to the loop invariant, we get

$r \leftarrow \perp$
for $i \leftarrow 1$ **to** n **do**
 if $a[i] = 7$ **then** $r \leftarrow \top$
 @ $i = n \Rightarrow [r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]]$
@ $r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]$
return r

As a result, the basic path leaving the loop is correct. Now we will try to make the basic paths corresponding to loop iterations correct. Without the initial assume, these are

$i \leftarrow i + 1$
assume $i \leq n$
assume $a[i] = 7$
 $r \leftarrow \top$
@ $i = n \Rightarrow [r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]]$

and

$i \leftarrow i + 1$
assume $i \leq n$
assume $a[i] \neq 7$
@ $i = n \Rightarrow [r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]]$

Interestingly enough, the first of those two basic paths, which corresponds to execution of the if branch, is already correct. This corresponds to the weakest precondition $\top$. The basic path correspond to skipping the if has the verification conditions

$$\forall i_1 . \left[\begin{array}{l} [i_1 = i + 1 \wedge i_1 \leq n \wedge a[i_1] \neq 7 \wedge i_1 = n] \Rightarrow \\ r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7] \end{array} \right]$$

which, after doing quantifier elimination, is

$$[a[n] \neq 7 \wedge i + 1 = n] \Rightarrow [r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]]$$

Adding this information, to the loop invariant, we get

$r \leftarrow \perp$
for $i \leftarrow 1$ **to** n **do**
 if $a[i] = 7$ **then** $r \leftarrow \top$
 @ $i = n \Rightarrow [r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]] \wedge$
 $[a[n] \neq 7 \wedge i + 1 = n] \Rightarrow [r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]]$
@ $r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]$
return r

Again we could continue doing this, and again this would not converge. To finish the process, we somehow need to generalize the process to a loop invariant, expressing the information for arbitrary many iterations. Here is an attempt:

$$[\forall k \in \{i+1, \dots, n\} . a[k] \neq 7] \Rightarrow [r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]]$$

This formula makes the basic paths corresponding to loop iterations correct, that is, it is an inductive loop invariant. Even more, it also makes the initial basic path leading into the loop correct. What is interesting, is that it looks quite different from the loop invariant that we arrived at above by computing postconditions. Intuitively, it says that if at iteration i no further values of 7 will occur at positions $i+1$ to n , then r already must be the correct result. But if further values of 7 will occur, this does not constraint r in any way. Especially, in this situation r does not have to express anything about whether we already encountered the values 7 before.

In general, preconditions of programs with loops can be computed as follows: We assume as input a program P such that

- every loop has an assertion
- the first line is an **assume**-assertion
- all assertions are of the form $@ \top$

We compute a program P' such that

- P' is identical to P except for formulas at assertions
- the formula of the first assertion is a precondition of the program (hopefully weak enough)

This result is computed by the following recursive function:

```
precond(P)=
  if all basic paths of P are correct then P
  else precond( $P_s$ ), where  $P_s$  is equal to  $P$  except for
    some assertions  $@ \alpha$  replaced by  $@ \text{ simplify } (\alpha \wedge \pi)$  where
       $\pi$  is a precondition of a basic path starting with  $@ \alpha$ 
```

Again, to make the process converge, one may need some way of generalizing loop invariants over arbitrary many iterations.

Here is a summary of the main idea for compute postconditions and preconditions over loops. For computing postconditions, we

- proceed in forward direction, we
- initialize assertions with $\perp$, and then
- iteratively weaken.

For computing preconditions, we

- proceed in backward computation, we
- initialize assertions with $\top$, and then

- iteratively strengthen.

In general, convergence of the process is not guaranteed, and we need a way of generalizing loop variants over arbitrarily many iterations. When applying the method manually, this needs a good idea. Algorithms for automated software verification apply some ingenious algorithms to do this. Many research groups world-wide work on this topic, and there is a yearly software verification competition where the resulting tools compete: <https://sv-comp.sosy-lab.org/>

23.5 Conclusion

Automated formal software verification is difficult

Industrial use in specific applications

Mostly for simple properties (no division by zero etc.)

Huge progress each year

Bibliography

- [1] R. Alur and D. L. Dill. A theory of timed automata. *Theoretical Computer Science*, 126:183–235, 1994.
- [2] D. Angeletti, E. Giunchiglia, M. Narizzano, A. Puddu, and S. Sabina. Using bounded model checking for coverage analysis of safety-critical software in an industrial setting. *Journal of Automated Reasoning*, 45:397–414, 2010.
- [3] G. Audemard, A. Cimatti, A. Kornilowicz, and R. Sebastiani. Bounded model checking for timed systems. In D. A. Peled and M. Y. Vardi, editors, *Formal Techniques for Networked and Distributed Systems—FORTE 2002*, volume 2529 of *Lecture Notes in Computer Science*, pages 243–259. Springer Berlin Heidelberg, 2002.
- [4] R. Baldoni, E. Coppa, D. C. D’elia, C. Demetrescu, and I. Finocchi. A survey of symbolic execution techniques. *ACM Comput. Surv.*, 51(3):50:1–50:39, May 2018.
- [5] J. Bengtsson and W. Yi. Timed automata: Semantics, algorithms and tools. In *Lectures on Concurrency and Petri Nets*, volume 3098 of *LNCS*, pages 87–124. Springer, 2004.
- [6] A. Biere, A. Cimatti, E. M. Clarke, O. Strichman, and Y. Zhu. Bounded model checking. *Advances in Computers*, 58:117 – 148, 2003.
- [7] A. Biere, M. J. H. Heule, H. van Maaren, and T. Walsh, editors. *Handbook of Satisfiability*, volume 185 of *Frontiers in Artificial Intelligence and Applications*. IOS Press, February 2009.
- [8] A. Bradley and Z. Manna. *The Calculus of Computation*. Springer, 2007.
- [9] A. Bradley, Z. Manna, and H. Sipma. What’s decidable about arrays? In *Verification, Model Checking, and Abstract Interpretation*, volume 3855 of *Lecture Notes in Computer Science*, pages 427–442. Springer Berlin / Heidelberg, 2006.
- [10] C. Cadar and K. Sen. Symbolic execution for software testing: Three decades later. *Commun. ACM*, 56(2):82–90, Feb. 2013.
- [11] A. Chaigne and A. Askenfelt. Numerical simulations of piano strings. i. a physical model for a struck string using finite difference methods. *The Journal of the Acoustical Society of America*, 95:1112, 1994.

- [12] A. Church. An unsolvable problem of elementary number theory. *Am. J. Math.*, 58:345–363, 1936.
- [13] E. Clarke, D. Kroening, and F. Lerda. A tool for checking ANSI-C programs. In K. Jensen and A. Podelski, editors, *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2004)*, volume 2988 of *Lecture Notes in Computer Science*, pages 168–176. Springer, 2004.
- [14] E. M. Clarke, O. Grumberg, and D. A. Peled. *Model Checking*. MIT Press, 1999.
- [15] B. Cook, A. Podelski, and A. Rybalchenko. Proving program termination. *Commun. ACM*, 54(5):88–98, May 2011.
- [16] S. Cotton and O. Maler. Fast and flexible difference constraint propagation for DPLL(T). In *International Conference on Theory and Applications of Satisfiability Testing*, pages 170–183. Springer, 2006.
- [17] C. Courcoubetis and M. Yannakakis. The complexity of probabilistic verification. *Journal of the ACM (JACM)*, 42(4):857–907, 1995.
- [18] M. Davis, G. Logemann, and D. Loveland. A machine program for theorem-proving. *Commun. ACM*, 5(7):394–397, July 1962.
- [19] H.-D. Ebbinghaus, J. Flum, and W. Thomas. *Mathematical Logic*. Springer Verlag, 1984.
- [20] M. Fränzle, C. Herde, S. Ratschan, T. Schubert, and T. Teige. Efficient solving of large non-linear arithmetic constraint systems with complex boolean structure. *JSAT—Journal on Satisfiability, Boolean Modeling and Computation, Special Issue on SAT/CP Integration*, 1:209–236, 2007.
- [21] P. Godefroid, M. Y. Levin, and D. Molnar. SAGE: whitebox fuzzing for security testing. *Commun. ACM*, 55(3):40–44, Mar. 2012.
- [22] K. Gödel. *Über die Vollständigkeit des Logikkalküls*. PhD thesis, Universität Wien, 1929.
- [23] K. Gödel. Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I. *Monatshefte für Mathematik*, 38:173–198, 1931.
- [24] D. Harel. Statecharts: a visual formalism for complex systems. *Science of Computer Programming*, 8(3):231 – 274, 1987.
- [25] J. E. Hopcroft and J. D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. Addison Wesley, 1979.
- [26] D. Kroening and O. Strichman. *Decision Procedures*. Springer, 2nd edition, 2016.
- [27] C. Kuratowski. Sur la notion de l’ordre dans la théorie des ensembles. *Fundamenta mathematicae*, 2(1):161–171, 1921.
- [28] T. Latvala, A. Biere, K. Heljanko, and T. Junttila. Simple bounded LTL model checking. In *FMCAD*, volume 4, pages 186–200. Springer, 2004.

- [29] B. H. Liskov and J. M. Wing. A behavioral notion of subtyping. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 16(6):1811–1841, 1994.
- [30] I. Lynce and J. Marques-Silva. Efficient haplotype inference with Boolean satisfiability. In *Proceedings of the National Conference on Artificial Intelligence*, volume 21, page 104. Menlo Park, CA; Cambridge, MA; London; AAAI Press; MIT Press; 1999, 2006.
- [31] M. Mohri. Minimization algorithms for sequential transducers. *Theoretical Computer Science*, 234(1–2):177 – 201, 2000.
- [32] T. Murata. Petri nets: Properties, analysis and applications. *Proceedings of the IEEE*, 77(4):541 –580, Apr. 1989.
- [33] G. Nelson and D. C. Oppen. Simplification by cooperating decision procedures. *ACM Trans. Program. Lang. Syst.*, 1(2):245–257, 1979.
- [34] G. Nelson and D. C. Oppen. Fast decision procedures based on congruence closure. *J. ACM*, 27(2):356–364, Apr. 1980.
- [35] G. Peano. *Arithmetices principia: nova methodo*. Turino, 1889.
- [36] S. Prestwich. CNF encodings. In Biere et al. [7], pages 75–97.
- [37] A. M. Turing. On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, s2-42(1):230–265, 1937.
- [38] M. Y. Vardi. Automatic verification of probabilistic concurrent finite state programs. In *26th Annual Symposium on Foundations of Computer Science (sfcs 1985)*, pages 327–338. IEEE, 1985.